

# HTML

## 1. html 语义化

意义：根据内容的结构化（内容语义化），选择合适的标签（代码语义化）便于开发者阅读和写出更优雅的代码的同时让浏览器的爬虫和机器很好地解析。

注意：

- 尽可能少的使用无语义的标签 `div` 和 `span`；
- 在语义不明显时，既可以使用 `div` 或者 `p` 时，尽量用 `p`，因为 `p` 在默认情况下有上下间距，对兼容特殊终端有利；
- 不要使用纯样式标签，如：`b`、`font`、`u` 等，改用 `css` 设置。
- 需要强调的文本，可以包含在 `strong` 或者 `em` 标签中（浏览器预设样式，能用 `CSS` 指定就不用他们），`strong` 默认样式是加粗（不要用 `b`），`em` 是斜体（不用 `i`）；
- 使用表格时，标题要用 `caption`，表头用 `thead`，主体部分用 `tbody` 包围，尾部用 `tfoot` 包围。表头和一般单元格要区分开，表头用 `th`，单元格用 `td`；
- 表单域要用 `fieldset` 标签包起来，并用 `legend` 标签说明表单的用途；
- 每个 `input` 标签对应的说明文本都需要使用 `label` 标签，并且通过为 `input` 设置 `id` 属性，在 `label` 标签中设置 `for=someld` 来让说明文本和相对应的 `input` 关联起来。

新标签：

|                                 |                                          |
|---------------------------------|------------------------------------------|
| <code>&lt;article&gt;</code>    | 定义文档内的文章。                                |
| <code>&lt;aside&gt;</code>      | 定义页面内容之外的内容。                             |
| <code>&lt;bdi&gt;</code>        | 定义与其他文本不同的文本方向。                          |
| <code>&lt;details&gt;</code>    | 定义用户可查看或隐藏的额外细节。                         |
| <code>&lt;dialog&gt;</code>     | 定义对话框或窗口。                                |
| <code>&lt;figcaption&gt;</code> | 定义 <code>&lt;figure&gt;</code> 元素的标题。    |
| <code>&lt;figure&gt;</code>     | 定义自包含内容，比如图示、图表、照片、代码清单等等。               |
| <code>&lt;footer&gt;</code>     | 定义文档或节的页脚。                               |
| <code>&lt;header&gt;</code>     | 定义文档或节的页眉。                               |
| <code>&lt;main&gt;</code>       | 定义文档的主内容。                                |
| <code>&lt;mark&gt;</code>       | 定义重要或强调的内容。                              |
| <code>&lt;menuitem&gt;</code>   | 定义用户能够从弹出菜单调用的命令/菜单项目。                   |
| <code>&lt;meter&gt;</code>      | 定义已知范围（尺度）内的标量测量。                        |
| <code>&lt;nav&gt;</code>        | 定义文档内的导航链接。                              |
| <code>&lt;progress&gt;</code>   | 定义任务进度。                                  |
| <code>&lt;rp&gt;</code>         | 定义在不支持 <code>ruby</code> 注释的浏览器中显示什么。    |
| <code>&lt;rt&gt;</code>         | 定义关于字符的解释/发音（用于东亚字体）。                    |
| <code>&lt;ruby&gt;</code>       | 定义 <code>ruby</code> 注释（用于东亚字体）。         |
| <code>&lt;section&gt;</code>    | 定义文档中的节。                                 |
| <code>&lt;summary&gt;</code>    | 定义 <code>&lt;details&gt;</code> 元素的可见标题。 |
| <code>&lt;time&gt;</code>       | 定义日期/时间。                                 |
| <code>&lt;wbr&gt;</code>        | 定义可能的折行（ <code>line-break</code> ）。      |

## 2. canvas 相关

使用前需要获得上下文环境，暂不支持 3 d

常用 api:

- fillRect(x,y,width,height)实心矩形
- strokeRect(x,y,width,height)空心矩形
- fillText( "Hello world" , 200 , 200 );实心文字
- strokeText( "Hello world" , 200 , 300 )空心文字

新标签兼容低版本

- ie9 之前版本通过 createElement 创建 html5 新标签
- 引入 html5shiv.js

## 3. svg和canvas的区别?

- canvas是h5提供的新的绘图方法；svg已经有了十多年的历史
- canvas画图基于像素点，是位图，如果进行放大或缩小会失真；svg基于图形，用html标签描绘形状，放大缩小不会失真

状，放大缩小不会失真

- canvas需要在js中绘制；svg在html绘制
- canvas支持颜色比svg多
- canvas无法对已经绘制的图像进行修改、操作；svg可以获取到标签进行操作

## 4. html5有哪些新特性?

HTML5主要是关于图像，位置，存储，多任务等功能的增加。

- 拖拽释放(Drag and drop) API
- 语义化更好的内容标签 (header,nav,footer,aside,article,section)
- 音频、视频API(audio,video)
- 画布(Canvas) API
- 地理(Geolocation) API
- 本地离线存储 localStorage 长期存储数据，浏览器关闭后数据不丢失；
- sessionStorage 的数据在浏览器关闭后自动删除
- 表单控件，calendar、date、time、email、url、search

## 5. 如何处理HTML5新标签的浏览器兼容问题?

IE8/IE7/IE6支持通过document.createElement方法产生的标签，可以利用这一特性让这些浏览器支持HTML5新标签，当然最好的方式是直接使用成熟的框架、使用最多的是html5shim框架

## 6. 说说 title 和 alt 属性

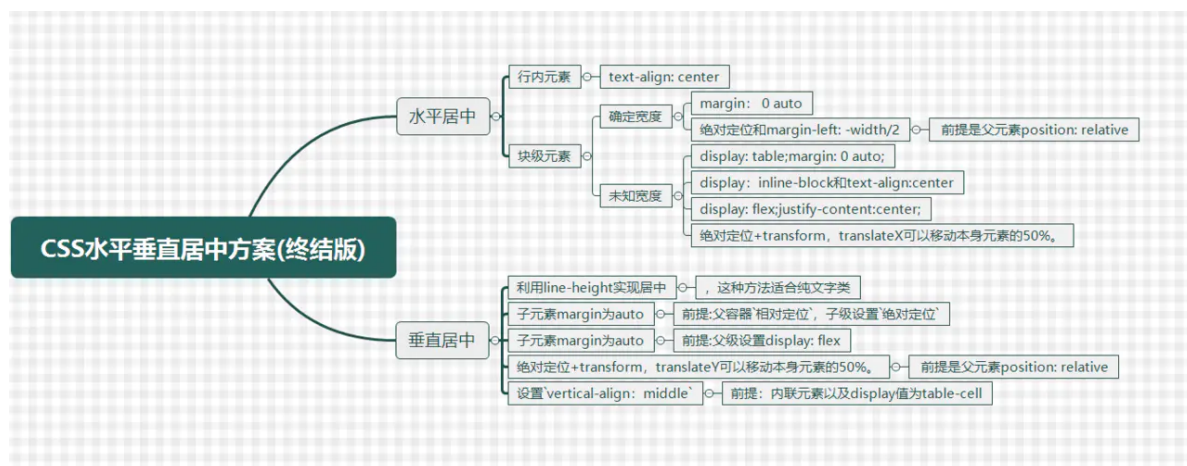
- 两个属性都是当鼠标滑动到元素上的时候显示
- `alt` 是 `img` 的特有属性，是图片内容的等价描述，图片无法正常显示时候的替代文字。
- `title` 属性可以用在除了`base`, `basefont`, `head`, `html`, `meta`, `param`, `script`和`title`之外的所有标签，是对dom元素的一种类似注释说明

## 7. HTML全局属性(global attribute)有哪些

- `class` :为元素设置类标识
- `data-*` : 为元素增加自定义属性
- `draggable` : 设置元素是否可拖拽
- `id` : 元素 id , 文档内唯一
- `lang` : 元素内容的的语言
- `style` : 行内 css 样式
- `title` : 元素相关的建议信息

## CSS

### 1. 让一个元素水平垂直居中，到底有多少种方案？



### 水平居中

- 对于 行内元素 : `text-align: center;`
- 对于确定宽度的块级元素:

(1) `width`和`margin`实现。 `margin: 0 auto;`

(2) 绝对定位和`margin-left: -width/2`, 前提是父元素`position: relative`

- 对于宽度未知的块级元素

(1) `table`标签配合`margin`左右`auto`实现水平居中。使用`table`标签（或直接将块级元素设置为`display: table`），再通过给该标签添加左右`margin`为`auto`。

(2) `inline-block`实现水平居中方法。 `display: inline-block`和`text-align: center`实现水平居中。

(3) 绝对定位+transform, translateX可以移动本身元素的50%。

(4) flex布局使用justify-content:center

## 垂直居中

1. 利用 line-height 实现居中, 这种方法适合纯文字类
2. 通过设置父容器 相对定位, 子级设置 绝对定位, 标签通过margin实现自适应居中
3. 弹性布局 flex: 父级设置display: flex; 子级设置margin为auto实现自适应居中
4. 父级设置相对定位, 子级设置绝对定位, 并且通过位移 transform 实现
5. table 布局, 父级通过转换成表格形式, 然后子级设置 vertical-align 实现。(需要注意的是: vertical-align: middle使用的前提条件是内联元素以及display值为table-cell的元素)。

## 2. 浮动布局的优点? 有什么缺点? 清除浮动有哪些方式?

浮动布局简介: 当元素浮动以后可以向左或向右移动, 直到它的外边缘碰到包含它的框或者另外一个浮动元素的边框为止。元素浮动以后会脱离正常的文档流, 所以文档的普通流中的框就变的好像浮动元素不存在一样。

### 优点

这样做的优点就是在图文混排的时候可以很好的使文字环绕在图片周围。另外当元素浮动了起来之后, 它有着块级元素的一些性质例如可以设置宽高等, 但它与inline-block还是有一些区别的, 第一个就是关于横向排序的时候, float可以设置方向而inline-block方向是固定的; 还有一个就是inline-block在使用时有时会有空白间隙的问题

### 缺点

最明显的缺点就是浮动元素一旦脱离了文档流, 就无法撑起父元素, 会造成父级元素高度塌陷。

### 清除浮动的方式

- 添加额外标签

```
<div class="parent">
  //添加额外标签并且添加clear属性
  <div style="clear:both"></div>
  //也可以加一个br标签
</div>
```

- 父级添加overflow属性, 或者设置高度

```
<div class="parent" style="overflow:hidden"> //auto 也可以
  //将父元素的overflow设置为hidden
  <div class="f"></div>
</div>
```

- 建立伪类选择器清除浮动 (推荐)



```
//在css中添加:after伪元素
.parent:after{
    /* 设置添加子元素的内容是空 */
    content: '';
    /* 设置添加子元素为块级元素 */
    display: block;
    /* 设置添加的子元素的高度0 */
    height: 0;
    /* 设置添加子元素看不见 */
    visibility: hidden;
    /* 设置clear: both */
    clear: both;
}
<div class="parent">
    <div class="f"></div>
</div>
```

### 3. 使用display:inline-block会产生什么问题？解决方法？

#### 问题复现

问题: 两个display: inline-block元素放到一起会产生一段空白。

如代码:

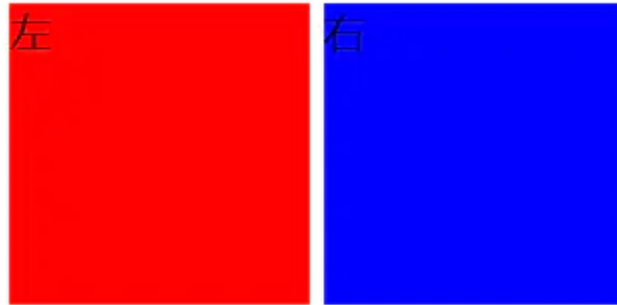
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
    <style>
      .container {
        width: 800px;
        height: 200px;
      }

      .left {
        font-size: 14px;
        background: red;
        display: inline-block;
        width: 100px;
        height: 100px;
      }

      .right {
        font-size: 14px;
        background: blue;
        display: inline-block;
        width: 100px;
        height: 100px;
      }
    </style>
  </head>
  <body>
```

```
<div class="container">
  <div class="left">
    左
  </div>
  <div class="right">
    右
  </div>
</div>
</body>
</html>
```

效果如下:



### 产生空白的原因

元素被当成行内元素排版的时候，元素之间的空白符（空格、回车换行等）都会被浏览器处理，根据CSS中white-space属性的处理方式（默认是normal，合并多余空白），原来HTML代码中的回车换行被转成一个空白符，在字体不为0的情况下，空白符占据一定宽度，所以inline-block的元素之间就出现了空隙。

### 解决办法

#### (1) 将子元素标签的结束符和下一个标签的开始符写在同一行或把所有子标签写在同一行

```
<div class="container">
  <div class="left">
    左
  </div><div class="right">
    右
  </div>
</div>
```

#### (2) 父元素中设置font-size: 0，在子元素上重置正确的font-size

```
.container{
  width:800px;
  height:200px;
  font-size: 0;
}
```

### (3) 为子元素设置float:left

```
.left{
  float: left;
  font-size: 14px;
  background: red;
  display: inline-block;
  width: 100px;
  height: 100px;
}
//right是同理
```

## 4. 布局题：div垂直居中，左右10px，高度始终为宽度一半

问题描述: 实现一个div垂直居中, 其距离屏幕左右两边各10px, 其高度始终是宽度的50%。同时div中有一个文字A, 文字需要水平垂直居中。

思路一：利用height:0; padding-bottom: 50%;

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
    <style>
      *{
        margin: 0;
        padding: 0;
      }
      html, body {
        height: 100%;
        width: 100%;
      }
      .outer_wrapper {
        margin: 0 10px;
        height: 100%;
        /* flex布局让块垂直居中 */
        display: flex;
        align-items: center;
      }
      .inner_wrapper{
        background: red;
        position: relative;
        width: 100%;
        height: 0;
        padding-bottom: 50%;
      }
      .box{
        position: absolute;
        width: 100%;
        height: 100%;
```

```

        display: flex;
        justify-content: center;
        align-items: center;
        font-size: 20px;
    }
</style>
</head>
<body>
    <div class="outer_wrapper">
        <div class="inner_wrapper">
            <div class="box">A</div>
        </div>
    </div>
</body>
</html>

```

强调两点:

- padding-bottom究竟是相对于谁的?

答案是相对于 父元素的width值。

那么对于这个out\_wrapper的用意就很好理解了。CSS呈流式布局，div默认宽度填满，即100%大小，给out\_wrapper设置margin: 0 10px;相当于让左右分别减少了10px。

- 父元素相对定位，那绝对定位下的子元素宽高若设为百分比，是相对谁而言的?

相对于父元素的(content + padding)值, 注意不含border

延伸：如果子元素不是绝对定位，那宽高设为百分比是相对于父元素的宽高，标准盒模型下是content, IE盒模型是content+padding+border。

## 思路二: 利用calc和vw

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
    <style>
        * {
            padding: 0;
            margin: 0;
        }

        html,
        body {
            width: 100%;
            height: 100%;
        }

        .wrapper {
            position: relative;
            width: 100%;
            height: 100%;
        }
    </style>

```

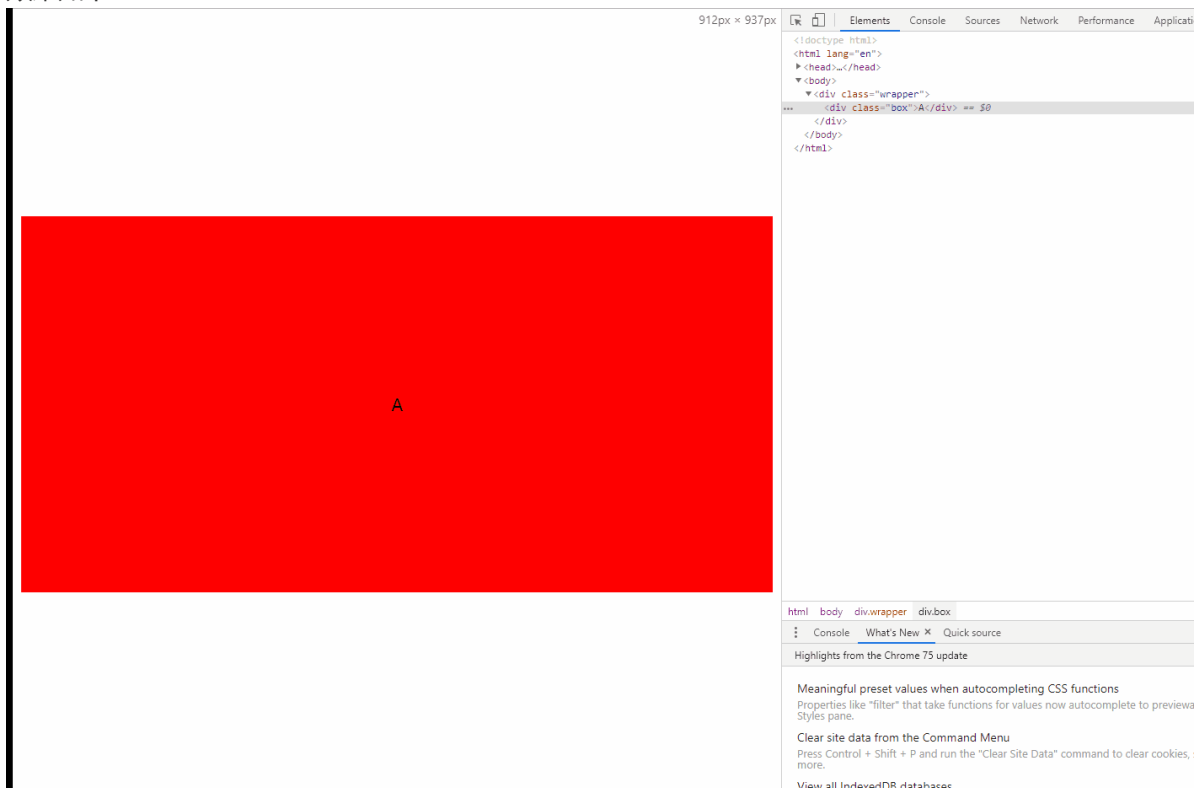
```

}

.box {
  margin-left: 10px;
  /* vw是视口的宽度， 1vw代表1%的视口宽度 */
  width: calc(100vw - 20px);
  /* 宽度的一半 */
  height: calc(50vw - 10px);
  position: absolute;
  background: red;
  /* 下面两行让块垂直居中 */
  top: 50%;
  transform: translateY(-50%);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 20px;
}
</style>
</head>
<body>
  <div class="wrapper">
    <div class="box">A</div>
  </div>
</body>
</html>

```

效果如下:



## 5. 盒模型

```
/* 红色区域的大小是多少? 200 - 20*2 - 20*2 = 120 */
.box {
width: 200px;
height: 200px;
padding: 20px;
margin: 20px;
background: red;
border: 20px solid black;
box-sizing: border-box; }

/* 标准模型 */
box-sizing:content-box;

/*IE模型*/
box-sizing:border-box;
```

## 6. CSS如何进行品字布局?

### 第一种

```
<!doctype html>
<html>

<head>
  <meta charset="utf-8">
  <title>品字布局</title>
  <style>
    * {
      margin: 0;
      padding: 0;
    }
    body {
      overflow: hidden;
    }
    div {
      margin: auto 0;
      width: 100px;
      height: 100px;
      background: red;
      font-size: 40px;
      line-height: 100px;
      color: #fff;
      text-align: center;
    }

    .div1 {
      margin: 100px auto 0;
    }

    .div2 {
      margin-left: 50%;
      background: green;
      float: left;
```



```

        transform: translateX(-100%);
    }

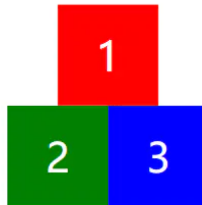
    .div3 {
        background: blue;
        float: left;
        transform: translateX(-100%);
    }
</style>
</head>

<body>
    <div class="div1">1</div>
    <div class="div2">2</div>
    <div class="div3">3</div>
</body>

</html>

```

效果:



## 第二种(全屏版)

```

<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <title>品字布局</title>
    <style>
        * {
            margin: 0;
            padding: 0;
        }

        div {
            width: 100%;
            height: 100px;
            background: red;
            font-size: 40px;
            line-height: 100px;
            color: #fff;
            text-align: center;
        }

        .div1 {

```

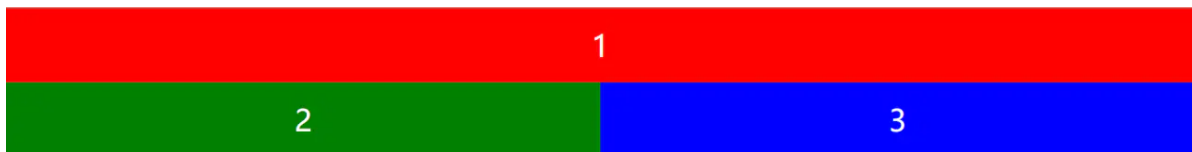
```
margin: 0 auto 0;
}

.div2 {
  background: green;
  float: left;
  width: 50%;
}

.div3 {
  background: blue;
  float: left;
  width: 50%;
}
</style>
</head>

<body>
  <div class="div1">1</div>
  <div class="div2">2</div>
  <div class="div3">3</div>
</body>
</html>
```

效果:



## 7. CSS如何进行圣杯布局

圣杯布局如图:



而且要做到左右宽度固定，中间宽度自适应。

### 利用flex布局

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <style>
    *{
      margin: 0;
      padding: 0;
    }
    .header,.footer{
      height:40px;
      width:100%;
```

```

        background:red;
    }
    .container{
        display: flex;
    }
    .middle{
        flex: 1;
        background:yellow;
    }
    .left{
        width:200px;
        background:pink;
    }
    .right{
        background: aqua;
        width:300px;
    }
</style>
</head>
<body>
    <div class="header">这里是头部</div>
    <div class="container">
        <div class="left">左边</div>
        <div class="middle">中间部分</div>
        <div class="right">右边</div>
    </div>
    <div class="footer">这里是底部</div>
</body>
</html>

```

## float布局(全部float:left)

```

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
    <style>
        *{
            margin: 0;
            padding: 0;
        }
        .header,
        .footer {
            height: 40px;
            width: 100%;
            background: red;
        }

        .footer {
            clear: both;
        }
    </style>
</head>
<body>
    <div class="header">这里是头部</div>
    <div class="container">
        <div class="left">左边</div>
        <div class="middle">中间部分</div>
        <div class="right">右边</div>
    </div>
    <div class="footer">这里是底部</div>
</body>
</html>

```

```

.container {
  padding-left: 200px;
  padding-right: 250px;
}

.container div {
  position: relative;
  float: left;
}

.middle {
  width: 100%;
  background: yellow;
}

.left {
  width: 200px;
  background: pink;
  margin-left: -100%;
  left: -200px;
}

.right {
  width: 250px;
  background: aqua;
  margin-left: -250px;
  left: 250px;
}
</style>
</head>

<body>
  <div class="header">这里是头部</div>
  <div class="container">
    <div class="middle">中间部分</div>
    <div class="left">左边</div>
    <div class="right">右边</div>
  </div>
  <div class="footer">这里是底部</div>
</body>

</html>

```

这种float布局是最难理解的，主要是浮动后的负margin操作，这里重点强调一下。

设置负margin和left值之前是这样子：



左边的盒子设置margin-left: -100%是将盒子拉上去，效果：

```
.left{
  /* ... */
  margin-left: -100%;
}
```



然后向左移动200px来填充空下来的padding-left部分

```
.left{
  /* ... */
  margin-left: -100%;
  left: -200px;
}
```

效果呈现:



右边的盒子设置margin-left: -250px后, 盒子在该行所占空间为0, 因此直接到上面的middle块中,效果:

```
.right{
  /* ... */
  margin-left: -250px;
}
```



然后向右移动250px, 填充父容器的padding-right部分:

```
.right{
  /* ... */
  margin-left: -250px;
  left: 250px;
}
```

现在就达到最后的效果了:



**float布局(左边float: left, 右边float: right)**

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    *{
      margin: 0;
      padding: 0;
    }
    .header,
    .footer {
      height: 40px;
      width: 100%;
      background: red;
    }
    .container{
      overflow: hidden;
    }

    .middle {
      background: yellow;
    }

    .left {
      float: left;
      width: 200px;
      background: pink;
    }

    .right {
      float: right;
      width: 250px;
      background: aqua;
    }
  </style>
</head>

<body>
  <div class="header">这里是头部</div>
  <div class="container">
    <div class="left">左边</div>
    <div class="right">右边</div>
    <div class="middle">中间部分</div>
  </div>
  <div class="footer">这里是底部</div>
</body>

</html>
```

## 绝对定位

```
<!DOCTYPE html>
<html lang="en">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    *{
      margin: 0;
      padding: 0;
    }
    .header,
    .footer {
      height: 40px;
      width: 100%;
      background: red;
    }
    .container{
      min-height: 1.2em;
      position: relative;
    }

    .container>div {
      position: absolute;
    }

    .middle {
      left: 200px;
      right: 250px;
      background: yellow;
    }

    .left {
      left: 0;
      width: 200px;
      background: pink;
    }

    .right {
      right: 0;
      width: 250px;
      background: aqua;
    }
  </style>
</head>

<body>
  <div class="header">这里是头部</div>
  <div class="container">
    <div class="left">左边</div>
    <div class="right">右边</div>
    <div class="middle">中间部分</div>
  </div>
  <div class="footer">这里是底部</div>
</body>

</html>

```

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    body{
      display: grid;
    }
    #header{
      background: red;
      grid-row:1;
      grid-column:1/5;
    }

    #left{
      grid-row:2;
      grid-column:1/2;
      background: orange;
    }
    #right{
      grid-row:2;
      grid-column:4/5;
      background: cadetblue;
    }
    #middle{
      grid-row:2;
      grid-column:2/4;
      background: rebeccapurple
    }
    #footer{
      background: gold;
      grid-row:3;
      grid-column:1/5;
    }
  </style>
</head>

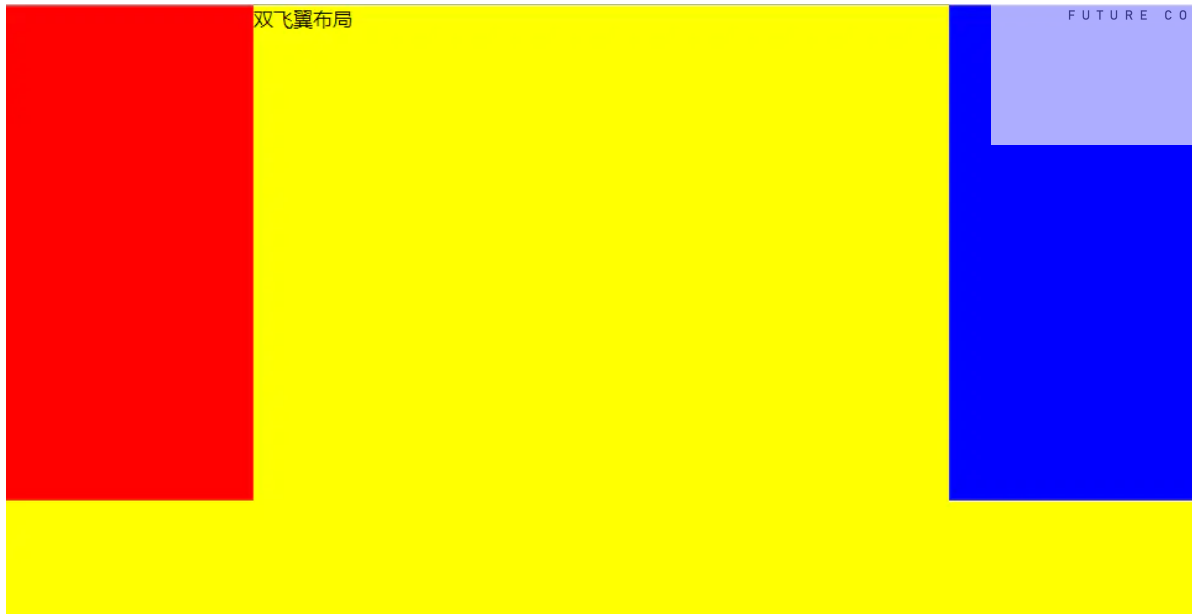
<body>
  <div id="header">header</div>
  <div id="left">left</div>
  <div id="middle">middle</div>
  <div id="right">right</div>
  <div id="footer">footer</div>

</body>

</html>
```



## 8. CSS如何实现双飞翼布局?



有了圣杯布局的铺垫，双飞翼布局也就问题不大啦。这里采用经典的float布局来完成。

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    *{
      margin: 0;
      padding: 0;
    }
    .container {
      min-width: 600px;
    }
    .left {
      float: left;
      width: 200px;
      height: 400px;
      background: red;
      margin-left: -100%;
    }
    .center {
      float: left;
      width: 100%;
      height: 500px;
      background: yellow;
    }
    .center .inner {
      margin: 0 200px;
    }
    .right {
      float: left;
      width: 200px;
      height: 400px;
```

```
        background: blue;
        margin-left: -200px;
    }
</style>
</head>

<body>
    <article class="container">
        <div class="center">
            <div class="inner">双飞翼布局</div>
        </div>
        <div class="left"></div>
        <div class="right"></div>
    </article>
</body>

</html>
```

## 9. 什么是BFC?

W3C对BFC的定义如下：浮动元素和绝对定位元素，非块级盒子的块级容器（例如 inline-blocks, table-cells, 和 table-captions），以及overflow值不为"visible"的块级盒子，都会为他们的内容创建新的BFC（Block Formatting Context，即块级格式上下文）。

## 10. 触发条件

一个HTML元素要创建BFC，则满足下列的任意一个或多个条件即可：下列方式会创建块格式化上下文：

- 根元素()
- 浮动元素（元素的 float 不是 none）
- 绝对定位元素（元素的 position 为 absolute 或 fixed）
- 行内块元素（元素的 display 为 inline-block）
- 表格单元格（元素的 display 为 table-cell，HTML表格单元格默认为该值）
- 表格标题（元素的 display 为 table-caption，HTML表格标题默认为该值）
- 匿名表格单元格元素（元素的 display 为 table、table-row、table-row-group、table-header-group、table-footer-group（分别是HTML table、row、tbody、thead、tfoot的默认属性）或 inline-table）
- overflow 值不为 visible 的块元素 -弹性元素（display 为 flex 或 inline-flex 元素的直接子元素）
- 网格元素（display 为 grid 或 inline-grid 元素的直接子元素）等等。

## 11. BFC渲染规则

- (1) BFC垂直方向边距重叠
- (2) BFC的区域不会与浮动元素的box重叠
- (3) BFC是一个独立的容器，外面的元素不会影响里面的元素
- (4) 计算BFC高度的时候浮动元素也会参与计算

## 12. 应用场景

### (1) 防止浮动导致父元素高度塌陷

现有如下页面代码:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
    <style>
      .container {
        border: 10px solid red;
      }
      .inner {
        background: #08BDEB;
        height: 100px;
        width: 100px;
      }
    </style>
  </head>
  <body>
    <div class="container">
      <div class="inner"></div>
    </div>
  </body>
</html>
```



接下来将inner元素设为浮动:

```
.inner {
  float: left;
  background: #08BDEB;
  height: 100px;
  width: 100px;
}
```

会产生这样的塌陷效果:



但如果我们对父元素设置BFC后, 这样的问题就解决了:

```
.container {
  border: 10px solid red;
  overflow: hidden;
}
```

同时这也是清除浮动的一种方式。

## (2) 避免外边距折叠

两个块同一个BFC会造成外边距折叠，但如果对这两个块分别设置BFC，那么边距重叠的问题就不存在了。

现有代码如下：

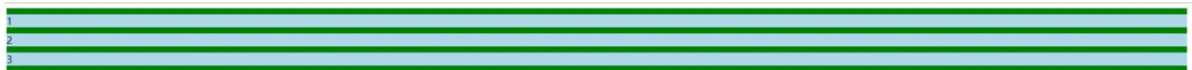
```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    .container {
      background-color: green;
      overflow: hidden;
    }

    .inner {
      background-color: lightblue;
      margin: 10px 0;
    }
  </style>
</head>

<body>
  <div class="container">
    <div class="inner">1</div>
    <div class="inner">2</div>
    <div class="inner">3</div>
  </div>
</body>

</html>
```



此时三个元素的上下间隔都是10px，因为三个元素同属于一个BFC。现在我们做如下操作：

```
<div class="container">
  <div class="inner">1</div>
  <div class="bfc">
    <div class="inner">2</div>
  </div>
  <div class="inner">3</div>
</div>
```

style增加：

```
.bfc{
  overflow: hidden;
}
```

效果如下:



可以明显地看到间隔变大了，而且是原来的两倍，符合我们的预期。

### 13. 画一个对话框

要画一个对话框，首先来学习做一个三角形。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    .triangle{
      width: 0;
      height: 0;
      border: 50px solid;
      border-color: #f00 #0f0 #ccc #00f;
    }
  </style>
</head>
<body>
  <div class="triangle"></div>
</body>
</html>
```

出现如下效果:



已经知道border的构成原理，然后只需改一行代码：

```
// 四个参数对应：上 右 下 左
border-color: transparent transparent #ccc transparent;
```

于是就只剩下下面的三角形部分啦！

现在来利用三角形技术做对话框：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    .dialog {
      position: relative;
      margin-top: 50px;
      margin-left: 50px;
      padding-left: 20px;
      width: 300px;
      line-height: 2;
      background: lightblue;
      color: #fff;
    }
    .dialog::before {
      content: '';
      position: absolute;
      border: 8px solid;
      border-color: transparent lightblue transparent transparent;
      left: -16px;
      top: 8px;
    }
  </style>
</head>
<body>
  <div class="dialog">这是一个对话框鸭！ </div>
</body>
</html>
```

效果如下：



这是一个对话框鸭！

## 14. 画一个平行四边形

利用skew特性，第一个参数为x轴倾斜的角度，第二个参数为y轴倾斜的角度。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>Document</title>
<style>
  .parallel {
    margin-top: 50px;
    margin-left: 50px;
    width: 200px;
    height: 100px;
    background: lightblue;
    transform: skew(-20deg, 0);
  }
</style>
</head>
<body>
  <div class="parallel"></div>
</body>
</html>
```

效果如下：



## 15. 用一个div画五角星



对于这个五角星而言，我们可以拆分成三个部分，想一想是不是这样？



那我们现在就来实现这三个部分。对于最上面的三角，我们在第一个部分已经实现了三角形，这个不难。但是下面的两个如何实现呢？

其实也非常的简单，想一想，下面这两个是不是就是一个向上的三角形旋转而来呢？明白了这一点，就可以动手实现啦！

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <style>
    #star {
      position: relative;
      margin: 200px auto;
      width: 0;
      height: 0;
      border-style: solid;
      border-color: transparent transparent red transparent;
      border-width: 70px 100px;
      transform: rotate(35deg);
    }
    #star::before {
      position: absolute;
      content: '';
      width: 0;
      height: 0;
      top: -128px;
      left: -95px;
      border-style: solid;
      border-color: transparent transparent red transparent;
      border-width: 80px 30px;
      transform: rotate(-35deg);
    }
    #star::after {
      position: absolute;
      content: '';
      width: 0;
      height: 0;
      top: -45px;
      left: -140px;
      border-style: solid;
```



```
border-color: transparent transparent red transparent;
border-width: 70px 100px;
transform: rotate(-70deg);
}
</style>
</head>
<body>
  <div id="star"></div>
</body>
</html>
```

## JavaScript

### 1. JS原始数据类型有哪些？引用数据类型有哪些？

在JS中，存在着7种原始值，分别是：

- boolean
- null
- undefined
- number
- string
- symbol
- bigint

引用数据类型: 对象Object（包含普通对象-Object，数组对象-Array，正则对象-RegExp，日期对象-Date，数学函数-Math，函数对象-Function）

### 2. 说出下面运行的结果，解释原因。

```
function test(person) {
  person.age = 26
  person = {
    name: 'hzj',
    age: 18
  }
  return person
}
const p1 = {
  name: 'fyq',
  age: 19
}
const p2 = test(p1)
console.log(p1) // -> ?
console.log(p2) // -> ?
```

结果:

```
p1: {name: "fyq", age: 26}
p2: {name: "hzj", age: 18}
```

原因: 在函数传参的时候传递的是对象在堆中的内存地址值, test函数中的实参person是p1对象的内存地址, 通过调用person.age = 26确实改变了p1的值, 但随后person变成了另一块内存空间的地址, 并且在最后将这另外一份内存空间的地址返回, 赋给了p2。

### 3. null是对象吗? 为什么?

结论: null不是对象。

解释: 虽然 `typeof null` 会输出 `object`, 但是这只是 JS 存在的一个悠久 Bug。在 JS 的最初版本中使用的是 32 位系统, 为了性能考虑使用低位存储变量的类型信息, 000 开头代表是对象然而 `null` 表示为零, 所以将它错误的判断为 `object`。

### 4. '1'.toString()为什么可以调用?

其实在这个语句运行的过程中做了这样几件事情:

```
var s = new Object('1');  
s.toString();  
s = null;
```

第一步: 创建Object类实例。注意为什么不是String? 由于Symbol和BigInt的出现, 对它们调用new都会报错, 目前ES6规范也不建议用new来创建基本类型的包装类。

第二步: 调用实例方法。

第三步: 执行完方法立即销毁这个实例。

整个过程体现了 基本包装类型 的性质, 而基本包装类型恰恰属于基本数据类型, 包括Boolean, Number和String。

### 5. 0.1+0.2为什么不等于0.3?

0.1和0.2在转换成二进制后会无限循环, 由于标准位数的限制后面多余的位数会被截掉, 此时就已经出现了精度的损失, 相加后因浮点数小数位的限制而截断的二进制数字在转换为十进制就会变成0.30000000000000004。

### 6. 什么是BigInt?

BigInt是一种新的数据类型, 用于当整数值大于Number数据类型支持的范围时。这种数据类型允许我们安全地对 大整数 执行算术操作, 表示高分辨率的时间戳, 使用大整数id, 等等, 而不需要使用库。

### 7. 为什么需要BigInt?

在JS中, 所有的数字都以双精度64位浮点格式表示, 那这会带来什么问题呢?

这导致JS中的Number无法精确表示非常大的整数, 它会将非常大的整数四舍五入, 确切地说, JS中的Number类型只能安全地表示-9007199254740991(-(2<sup>53</sup>-1))和9007199254740991 ((2<sup>53</sup>-1)), 任何超出此范围的整数值都可能失去精度。

```
console.log(9999999999999999); //=>10000000000000000
```

同时也会有一定的安全性问题:

```
9007199254740992 === 9007199254740993; // → true 居然是true!
```

## 8. 如何创建并使用BigInt?

要创建BigInt, 只需要在数字末尾追加n即可。

```
console.log( 9007199254740995n ); // → 9007199254740995n  
console.log( 9007199254740995 ); // → 9007199254740996
```

另一种创建BigInt的方法是用BigInt()构造函数、

```
BigInt("9007199254740995"); // → 9007199254740995n
```

简单使用如下:

```
10n + 20n; // → 30n  
10n - 20n; // → -10n  
+10n; // → TypeError: Cannot convert a BigInt value to a number  
-10n; // → -10n  
10n * 20n; // → 200n  
20n / 10n; // → 2n  
23n % 10n; // → 3n  
10n ** 3n; // → 1000n  
  
const x = 10n;  
++x; // → 11n  
--x; // → 9n  
console.log(typeof x); //"bigint"
```

### 值得警惕的点

- 1) BigInt不支持一元加号运算符, 这可能是某些程序可能依赖于 + 始终生成 Number 的不变量, 或者抛出异常。另外, 更改 + 的行为也会破坏 asm.js代码。
- 2) 因为隐式类型转换可能丢失信息, 所以不允许在bigint和 Number 之间进行混合操作。当混合使用大整数和浮点数时, 结果值可能无法由BigInt或Number精确表示。

```
10 + 10n; // → TypeError
```

- 3) 不能将BigInt传递给Web api和内置的JS 函数, 这些函数需要一个 Number 类型的数字。尝试这样做会报TypeError错误。

```
Math.max(2n, 4n, 6n); // → TypeError
```

4) 当 Boolean 类型与 BigInt 类型相遇时, BigInt 的处理方式与 Number 类似, 换句话说, 只要不是 0n, BigInt 就被视为 truthy 的值。

```
if(0n){//条件判断为false  
  
}  
if(3n){//条件为true  
  
}
```

5) 元素都为 BigInt 的数组可以进行 sort。

6) BigInt 可以正常地进行位运算, 如 |、&、<<、>> 和 ^

## 浏览器兼容性

caniuse 的结果:



其实现在的兼容性并不怎么好, 只有 chrome67、firefox、Opera 这些主流实现, 要正式成为规范, 其实还有很长的路要走。

## 9. typeof 是否能正确判断类型?

对于原始类型来说, 除了 null 都可以调用 typeof 显示正确的类型。

```
typeof 1 // 'number'  
typeof '1' // 'string'  
typeof undefined // 'undefined'  
typeof true // 'boolean'  
typeof Symbol() // 'symbol'
```

但对于引用数据类型, 除了函数之外, 都会显示 "object"。

```
typeof [] // 'object'  
typeof {} // 'object'  
typeof console.log // 'function'
```

因此采用typeof判断对象数据类型是不合适的，采用instanceof会更好，instanceof的原理是基于原型链的查询，只要处于原型链中，判断永远为true

```
const Person = function() {}  
const p1 = new Person()  
p1 instanceof Person // true  
  
var str1 = 'hello world'  
str1 instanceof String // false  
  
var str2 = new String('hello world')  
str2 instanceof String // true
```

## 10. instanceof能否判断基本数据类型？

能。比如下面这种方式：

```
class PrimitiveNumber {  
  static [Symbol.hasInstance](x) {  
    return typeof x === 'number'  
  }  
}  
console.log(111 instanceof PrimitiveNumber) // true
```

其实就是自定义instanceof行为的一种方式，这里将原有的instanceof方法重定义，换成了typeof，因此能够判断基本数据类型。

## 11. 能不能手动实现一下instanceof的功能？

核心：原型链的向上查找。

```
function myInstanceOf(left, right) {  
  //基本数据类型直接返回false  
  if(typeof left !== 'object' || left === null) return false;  
  //getPrototypeOf是Object对象自带的一个方法，能够拿到参数的原型对象  
  let proto = Object.getPrototypeOf(left);  
  while(true) {  
    //查找到尽头，还没找到  
    if(proto == null) return false;  
    //找到相同的原型对象  
    if(proto == right.prototype) return true;  
    proto = Object.getPrototypeOf(proto);  
  }  
}
```

测试：

```
console.log(myInstanceOf("111", String)); //false  
console.log(myInstanceOf(new String("111"), String)); //true
```

## 12. Object.is和===的区别？

Object在严格等于的基础上修复了一些特殊情况下的失误，具体来说就是+0和-0，NaN和NaN。源码如下：

```
function is(x, y) {
  if (x === y) {
    //运行到1/x === 1/y的时候x和y都为0，但是1/+0 = +Infinity， 1/-0 = -Infinity，是不一样的
    return x !== 0 || y !== 0 || 1 / x === 1 / y;
  } else {
    //NaN===NaN是false,这是不对的，我们在这里做一个拦截，x !== x，那么一定是 NaN，y 同理
    //两个都是NaN的时候返回true
    return x !== x && y !== y;
  }
}
```

## 13. [] == ![]结果是什么？为什么？

解析：

== 中，左右两边都需要转换为数字然后进行比较。

[]转换为数字为0。

![] 首先是转换为布尔值，由于[]作为一个引用类型转换为布尔值为true，

因此![]为false，进而在转换成数字，变为0。

0 == 0， 结果为true

## 14. JS中类型转换有哪几种？

JS中，类型转换只有三种：

- 转换成数字
- 转换成布尔值
- 转换成字符串

转换具体规则如下：

注意"Boolean 转字符串"这行结果指的是 true 转字符串的例子

| 原始值               | 转换目标 | 结果                             |
|-------------------|------|--------------------------------|
| number            | 布尔值  | 除了 0、-0、NaN 都为 true            |
| string            | 布尔值  | 除了空串都为 true                    |
| undefined、null    | 布尔值  | FALSE                          |
| 引用类型              | 布尔值  | TRUE                           |
| number            | 字符串  | 5 => '5'                       |
| Boolean、函数、Symbol | 字符串  | 'true'                         |
| 数组                | 字符串  | [1,2] => '1,2'                 |
| 对象                | 字符串  | '[object Object]'              |
| string            | 数字   | '1' => 1, 'a' => NaN           |
| 数组                | 数字   | 空数组为0, 存在一个元素且为数字转数字, 其他情况 NaN |
| null              | 数字   | 0                              |
| 除了数组的引用类型         | 数字   | NaN                            |
| Symbol            | 数字   | 抛错                             |

## 15. == 和 ===有什么区别？

===叫做严格相等，是指：左右两边不仅值要相等，类型也要相等，例如'1'===1的结果是false，因为一边是string，另一边是number。

==不像===那样严格，对于一般情况，只要值相等，就返回true，但==还涉及一些类型转换，它的转换规则如下：

- 两边的类型是否相同，相同的话就比较值的大小，例如1==2，返回false
- 判断的是否是null和undefined，是的话就返回true
- 判断的类型是否是String和Number，是的话，把String类型转换成Number，再进行比较
- 判断其中一方是否是Boolean，是的话就把Boolean转换成Number，再进行比较
- 如果其中一方为Object，且另一方为String、Number或者Symbol，会将Object转换成字符串，再进行比较

```
console.log({a: 1} == true); // false
console.log({a: 1} == "[object Object]"); // true
```

## 16. 对象转原始类型是根据什么流程运行的？

对象转原始类型，会调用内置的[ToPrimitive]函数，对于该函数而言，其逻辑如下：

- 1) 如果Symbol.toPrimitive()方法，优先调用再返回
- 2) 调用valueOf()，如果转换为原始类型，则返回
- 3) 调用toString()，如果转换为原始类型，则返回
- 4) 如果都没有返回原始类型，会报错

```
var obj = {
  value: 3,
  valueOf() {
    return 4;
  },
  toString() {
    return '5'
  },
  [Symbol.toPrimitive]() {
    return 6
  }
}
console.log(obj + 1); // 输出7
```

## 17. 如何让if(a == 1 && a == 2)条件成立?

```
var a = {
  value: 0,
  valueOf: function() {
    this.value++;
    return this.value;
  }
};
console.log(a == 1 && a == 2); // true
```

## 18. 什么是闭包?

红宝书上对于闭包的定义：闭包是指有权访问另外一个函数作用域中的变量的函数，

MDN 对闭包的定义为：闭包是指那些能够访问自由变量的函数。

(其中自由变量，指在函数中使用的，但既不是函数参数arguments也不是函数的局部变量的变量，其实就是另外一个函数作用域中的变量。)

## 19. 闭包产生的原因?

首先要明白作用域链的概念，其实很简单，在ES5中只存在两种作用域——全局作用域和函数作用域，当访问一个变量时，解释器会首先在当前作用域查找标示符，如果没有找到，就去父作用域找，直到找到该变量的标示符或者不在父作用域中，这就是作用域链，值得注意的是，每一个子函数都会拷贝上级的作用域，形成一个作用域的链条。比如：

```
var a = 1;
function f1() {
  var a = 2
  function f2() {
    var a = 3;
    console.log(a); // 3
  }
}
```



在这段代码中，f1的作用域指向有全局作用域(window)和它本身，而f2的作用域指向全局作用域(window)、f1和它本身。而且作用域是从最底层向上找，直到找到全局作用域window为止，如果全局还没有的话就会报错。就这么简单一事情！

闭包产生的本质就是，当前环境中存在指向父级作用域的引用。还是举上面的例子：

```
function f1() {  
  var a = 2  
  function f2() {  
    console.log(a); // 2  
  }  
  return f2;  
}  
var x = f1();  
x();
```

这里x会拿到父级作用域中的变量，输出2。因为在当前环境中，含有对f2的引用，f2恰恰引用了window、f1和f2的作用域。因此f2可以访问到f1的作用域的变量。

那是不是只有返回函数才算是产生了闭包呢？

回到闭包的本质，我们只需要让父级作用域的引用存在即可，因此我们还可以这么做：

```
var f3;  
function f1() {  
  var a = 2  
  f3 = function() {  
    console.log(a);  
  }  
}  
f1();  
f3();
```

让f1执行，给f3赋值后，等于说现在f3拥有了window、f1和f3本身这几个作用域的访问权限，还是自底向上查找，最近是在f1中找到了a,因此输出2。

在这里是外面的变量f3存在着父级作用域的引用，因此产生了闭包，形式变了，本质没有改变。

## 20. 闭包有哪些表现形式？

在真实的场景中，究竟在哪些地方能体现闭包的存在？

- 1) 返回一个函数。刚刚已经举例。
- 2) 作为函数参数传递

```
var a = 1;  
function foo(){  
  var a = 2;  
  function baz(){  
    console.log(a);  
  }  
  bar(baz);  
}  
function bar(fn){  
  // 这就是闭包
```

```
fn();
}
// 输出2, 而不是1
foo();
```

3) 在定时器、事件监听、Ajax请求、跨窗口通信、Web Workers或者任何异步中，只要使用了回调函数，实际上就是在使用闭包。

以下的闭包保存的仅仅是window和当前作用域。

```
// 定时器
setTimeout(function timeHandler(){
  console.log('111');
}, 100)

// 事件监听
$('#app').click(function(){
  console.log('DOM Listener');
})
```

4) IIFE(立即执行函数表达式)创建闭包, 保存了全局作用域window和当前函数的作用域，因此可以全局的变量。

```
var a = 2;
(function IIFE(){
  // 输出2
  console.log(a);
})();
```

## 21. 如何解决下面的循环输出问题？

```
for(var i = 1; i <= 5; i ++){
  setTimeout(function timer(){
    console.log(i)
  }, 0)
}
```

为什么会全部输出6？如何改进，让它输出1，2，3，4，5？(方法越多越好)

因为setTimeout为宏任务，由于JS中单线程eventLoop机制，在主线程同步任务执行完后才去执行宏任务，因此循环结束后setTimeout中的回调才依次执行，但输出i的时候当前作用域没有，往上一级再找，发现了i,此时循环已经结束，i变成了6。因此会全部输出6。

解决方法：

1、利用IIFE(立即执行函数表达式)当每次for循环时，把此时的i变量传递到定时器中

```
for(var i = 1; i <= 5; i++){
  (function(j){
    setTimeout(function timer(){
      console.log(j)
    }, 0)
  })(i)
}
```

2、给定时器传入第三个参数, 作为timer函数的第一个函数参数

```
for(var i=1;i<=5;i++){
  setTimeout(function timer(j){
    console.log(j)
  }, 0, i)
}
```

3、使用ES6中的let

```
for(let i = 1; i <= 5; i++){
  setTimeout(function timer(){
    console.log(i)
  },0)
}
```

let使JS发生革命性的变化, 让JS有函数作用域变为了块级作用域, 用let后作用域链不复存在。代码的作用域以块级为单位, 以上面代码为例:

```
// i = 1
{
  setTimeout(function timer(){
    console.log(1)
  },0)
}
// i = 2
{
  setTimeout(function timer(){
    console.log(2)
  },0)
}
// i = 3
...

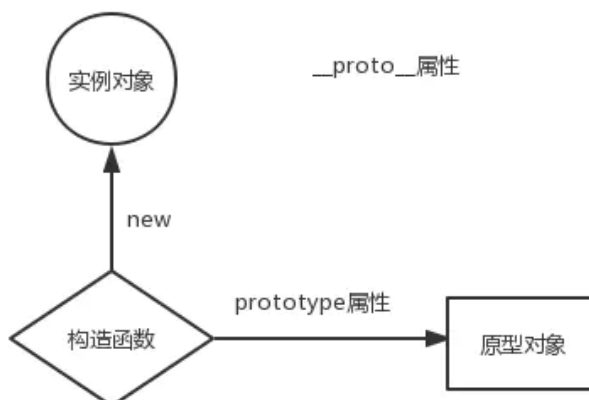
```

因此能输出正确的结果。

## 22. 原型对象和构造函数有何关系?

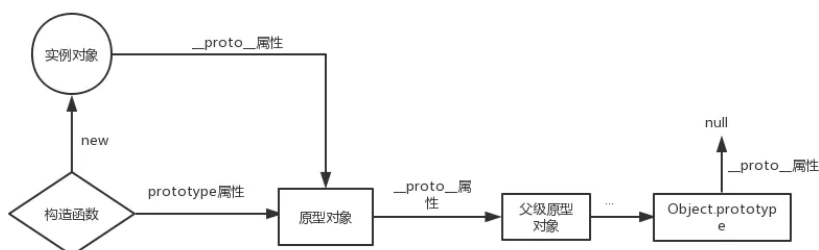
在JavaScript中, 每当定义一个函数数据类型(普通函数、类)时候, 都会天生自带一个prototype属性, 这个属性指向函数的原型对象。

当函数经过new调用时，这个函数就成为了构造函数，返回一个全新的实例对象，这个实例对象有一个 **proto** 属性，指向构造函数的原型对象。



## 23. 能不能描述一下原型链？

JavaScript对象通过**proto** 指向父类对象，直到指向Object对象为止，这样就形成了一个原型指向的链条, 即原型链。



- 对象的 `hasOwnProperty()` 来检查对象自身中是否含有该属性
- 使用 `in` 检查对象中是否含有某个属性时，如果对象中没有但是原型链中有，也会返回 `true`

## 24. JS如何实现继承？

第一种: 借助call

```
function Parent1(){
  this.name = 'parent1';
}
function Child1(){
  Parent1.call(this);
  this.type = 'child1'
}
console.log(new Child1);
```

这样写的时候子类虽然能够拿到父类的属性值，但是问题是父类原型对象中一旦存在方法那么子类无法继承。那么引出下面的方法。

## 第二种: 借助原型链

```
function Parent2() {  
  this.name = 'parent2';  
  this.play = [1, 2, 3]  
}  
function Child2() {  
  this.type = 'child2';  
}  
Child2.prototype = new Parent2();  
  
console.log(new Child2());
```

看似没有问题，父类的方法和属性都能够访问，但实际上有一个潜在的不足。举个例子：

```
var s1 = new Child2();  
var s2 = new Child2();  
s1.play.push(4);  
console.log(s1.play, s2.play);
```

可以看到控制台：

```
▶ (4) [1, 2, 3, 4] ▶ (4) [1, 2, 3, 4]
```

明明我只改变了s1的play属性，为什么s2也跟着变了呢？很简单，因为两个实例使用的是同一个原型对象。

那么还有更好的方式么？

## 第三种: 将前两种组合

```
function Parent3 () {  
  this.name = 'parent3';  
  this.play = [1, 2, 3];  
}  
function Child3() {  
  Parent3.call(this);  
  this.type = 'child3';  
}  
Child3.prototype = new Parent3();  
var s3 = new Child3();  
var s4 = new Child3();  
s3.play.push(4);  
console.log(s3.play, s4.play);
```

可以看到控制台：

```
▶ (4) [1, 2, 3, 4] ▶ (3) [1, 2, 3]
```

之前的问题都得以解决。但是这里又徒增了一个新问题，那就是Parent3的构造函数会多执行了一次 (Child3.prototype = new Parent3();)。这是我们不愿看到的。那么如何解决这个问题？

#### 第四种: 组合继承的优化1

```
function Parent4 () {  
  this.name = 'parent4';  
  this.play = [1, 2, 3];  
}  
function Child4() {  
  Parent4.call(this);  
  this.type = 'child4';  
}  
Child4.prototype = Parent4.prototype;
```

这里让将父类原型对象直接给到子类，父类构造函数只执行一次，而且父类属性和方法均能访问，但是我们来测试一下：

```
var s3 = new Child4();  
var s4 = new Child4();  
console.log(s3)
```

```
▼ Child4 {name: "parent4", play: Array(3), type: "child4"}  
  name: "parent4"  
  ► play: (3) [1, 2, 3]  
  type: "child4"  
  ▼ __proto__:  
    ► constructor: f Parent4()  
    ► __proto__: Object
```

子类实例的构造函数是Parent4，显然这是不对的，应该是Child4。

#### 第五种(最推荐使用): 组合继承的优化1

```
function Parent5 () {  
  this.name = 'parent5';  
  this.play = [1, 2, 3];  
}  
function Child5() {  
  Parent5.call(this);  
  this.type = 'child5';  
}  
Child5.prototype = Object.create(Parent5.prototype);  
Child5.prototype.constructor = Child5;
```

这是最推荐的一种方式，接近完美的继承，它的名字也叫做寄生组合继承。

#### ES6的extends被编译后的JavaScript代码

ES6的代码最后都是要在浏览器上能够跑起来的，这中间就利用了babel这个编译工具，将ES6的代码编译成ES5让一些不支持新语法的浏览器也能运行。

那最后编译成了什么样子呢？

```
function _possibleConstructorReturn(self, call) {
  // ...
  return call && (typeof call === 'object' || typeof call === 'function') ?
  call : self;
}

function _inherits(subClass, superClass) {
  // ...
  //看到没有
  subClass.prototype = Object.create(superClass && superClass.prototype, {
    constructor: {
      value: subClass,
      enumerable: false,
      writable: true,
      configurable: true
    }
  });
  if (superClass) Object.setPrototypeOf ? Object.setPrototypeOf(subClass,
  superClass) : subClass.__proto__ = superClass;
}

var Parent = function Parent() {
  // 验证是否是 Parent 构造出来的 this
  _classCallCheck(this, Parent);
};

var Child = (function (_Parent) {
  _inherits(Child, _Parent);

  function Child() {
    _classCallCheck(this, Child);

    return _possibleConstructorReturn(this, (Child.__proto__ ||
    Object.getPrototypeOf(Child)).apply(this, arguments));
  }

  return Child;
}(Parent));
```

核心是\_inherits函数，可以看到它采用的依然是第五种方式——寄生组合继承方式，同时证明了这种方式的成功。不过这里加了一个Object.setPrototypeOf(subClass, superClass)，这是用来干啥的呢？

答案是用来继承父类的静态方法。这也是原来的继承方式疏忽掉的地方。

追问：面向对象的设计一定是好的设计吗？

不一定。从继承的角度说，这一设计是存在巨大隐患的。

假如现在有不同品牌的车，每辆车都有drive、music、addOil这三个方法。

```
class Car{
  constructor(id) {
    this.id = id;
  }
  drive(){
    console.log("wuwuwu!");
  }
  music(){
    console.log("la la la!")
  }
  addOil(){
    console.log("哦哟！")
  }
}
class otherCar extends Car{}
```

现在可以实现车的功能，并且以此去扩展不同的车。

但是问题来了，新能源汽车也是车，但是它并不需要addOil(加油)。

如果让新能源汽车的类继承Car的话，也是有问题的，俗称“大猩猩和香蕉”的问题。大猩猩手里有香蕉，但是我现在明明只需要香蕉，却拿到了一只大猩猩。也就是说加油这个方法，我现在是不需要的，但是由于继承的原因，也给到子类了。

继承的最大问题在于：无法决定继承哪些属性，所有属性都得继承。

当然你可能会说，可以再创建一个父类啊，把加油的方法给去掉，但是这也是有问题的，一方面父类是无法描述所有子类的细节情况的，为了不同的子类特性去增加不同的父类，代码势必会大量重复，另一方面一旦子类有所变动，父类也要进行相应的更新，代码的耦合性太高，维护性不好。

那如何解决继承的诸多问题呢？

用组合，这也是当今编程语法发展的趋势，比如golang完全采用的是面向组合的设计方式。

顾名思义，面向组合就是先设计一系列零件，然后将这些零件进行拼装，来形成不同的实例或者类。

```
function drive(){
  console.log("wuwuwu!");
}
function music(){
  console.log("la la la!")
}
function addOil(){
  console.log("哦哟！")
}

let car = compose(drive, music, addOil);
let newEnergyCar = compose(drive, music);
```

代码干净，复用性也很好。这就是面向组合的设计方式。



## 25. 函数的arguments为什么不是数组？如何转化成数组？

因为arguments本身并不能调用数组方法，它是一个另外一种对象类型，只不过属性从0开始排，依次为0, 1, 2...最后还有callee和length属性。我们也把这样的对象称为类数组。

常见的类数组还有：

- 用getElementsByTagName/ClassName()获得的HTMLCollection
- 用querySelector获得的nodeList

那这导致很多数组的方法就不能用了，必要时需要我们将它们转换成数组，有哪些方法呢？

### Array.prototype.slice.call()

```
function sum(a, b) {  
  let args = Array.prototype.slice.call(arguments);  
  console.log(args.reduce((sum, cur) => sum + cur)); //args可以调用数组原生的方法啦  
}  
sum(1, 2); //3
```

### Array.from()

```
function sum(a, b) {  
  let args = Array.from(arguments);  
  console.log(args.reduce((sum, cur) => sum + cur)); //args可以调用数组原生的方法啦  
}  
sum(1, 2); //3
```

这种方法也可以用来转换Set和Map哦！

### ES6展开运算符

```
function sum(a, b) {  
  let args = [...arguments];  
  console.log(args.reduce((sum, cur) => sum + cur)); //args可以调用数组原生的方法啦  
}  
sum(1, 2); //3
```

### 利用concat+apply

```
function sum(a, b) {  
  let args = Array.prototype.concat.apply([], arguments); //apply方法会把第二个参数展开  
  console.log(args.reduce((sum, cur) => sum + cur)); //args可以调用数组原生的方法啦  
}  
sum(1, 2); //3
```

当然，最原始的方法就是再创建一个数组，用for循环把类数组的每个属性值放在里面，过于简单，就不浪费篇幅了。

## 26. forEach中return有效果吗？如何中断forEach循环？

在forEach中用return不会返回，函数会继续执行。

```
let nums = [1, 2, 3];
nums.forEach((item, index) => {
  return; //无效
})
```

中断方法：

- 1) 使用try监视代码块，在需要中断的地方抛出异常。
- 2) 官方推荐方法（替换方法）：用every和some替代forEach函数。every在碰到return false的时候，中止循环。some在碰到return true的时候，中止循环

## 27. JS判断数组中是否包含某个值

### 方法一：array.indexOf

此方法判断数组中是否存在某个值，如果存在，则返回数组元素的下标，否则返回-1。

```
var arr=[1,2,3,4];
var index=arr.indexOf(3);
console.log(index);
```

### 方法二：array.includes(searchElement[,fromIndex])

此方法判断数组中是否存在某个值，如果存在返回true，否则返回false

```
var arr=[1,2,3,4];
if(arr.includes(3))
  console.log("存在");
else
  console.log("不存在");
```

### 方法三：array.find(callback[,thisArg])

返回数组中满足条件的第一个元素的值，如果没有，返回undefined

```
var arr=[1,2,3,4];
var result = arr.find(item =>{
  return item > 3
});
console.log(result);
```

### 方法四：array.findIndex(callback[,thisArg])

返回数组中满足条件的第一个元素的下标，如果没有找到，返回 -1]

```
var arr=[1,2,3,4];
var result = arr.findIndex(item =>{
    return item > 3
});
console.log(result);
```

当然，for循环当然是没有问题的，这里讨论的是数组方法，就不再展开了。

## 28. JS中flat---数组扁平化

对于前端项目开发过程中，偶尔会出现层叠数据结构的数组，我们需要将多层级数组转化为一维数组（即提取嵌套数组元素最终合并为一个数组），使其内容合并并且展开。那么该如何去实现呢？

需求:多维数组=>一维数组

```
let ary = [1, [2, [3, [4, 5]]], 6]; // -> [1, 2, 3, 4, 5, 6]
let str = JSON.stringify(ary);
```

### 调用ES6中的flat方法

```
ary = ary.flat(Infinity);
```

### replace + split

```
ary = str.replace(/(\[|\])/g, '').split(',')
```

### replace + JSON.parse

```
str = str.replace(/(\[|\])/g, '');
str = '[' + str + ']';
ary = JSON.parse(str);
```

### 普通递归

```
let result = [];
let fn = function(ary) {
  for(let i = 0; i < ary.length; i++) {
    let item = ary[i];
    if (Array.isArray(ary[i])){
      fn(item);
    } else {
      result.push(item);
    }
  }
}
```

## 利用reduce函数迭代

```
function flatten(ary) {
  return ary.reduce((pre, cur) => {
    return pre.concat(Array.isArray(cur) ? flatten(cur) : cur);
  }, []);
}
let ary = [1, 2, [3, 4], [5, [6, 7]]]
console.log(flatten(ary))
```

## 扩展运算符

```
//只要有一个元素有数组，那么循环继续
while (ary.some(Array.isArray)) {
  ary = [].concat(...ary);
}
```

这是一个比较实用而且很容易被问到的问题，欢迎大家交流补充。

## 29. 什么是高阶函数

概念非常简单，如下：

一个函数 就可以接收另一个函数作为参数或者返回值为一个函数，这种函数 就称之为高阶函数。

## 30. 数组中的高阶函数

### map

- 参数:接受两个参数，一个是回调函数，一个是回调函数的this值(可选)。

其中，回调函数被默认传入三个值，依次为当前元素、当前索引、整个数组。

- 创建一个新数组，其结果是该数组中的每个元素都调用一个提供的函数后返回的结果
- 对原来的数组没有影响

```
let nums = [1, 2, 3];
let obj = {val: 5};
let newNums = nums.map(function(item, index, array) {
  return item + index + array[index] + this.val;
  //对第一个元素, 1 + 0 + 1 + 5 = 7
  //对第二个元素, 2 + 1 + 2 + 5 = 10
  //对第三个元素, 3 + 2 + 3 + 5 = 13
}, obj);
console.log(newNums); //[7, 10, 13]
```

当然，后面的参数都是可选的，不用的话可以省略。

## reduce

- 参数: 接收两个参数，一个为回调函数，另一个为初始值。回调函数中四个默认参数，依次为积累值、当前值、当前索引和整个数组。

```
let nums = [1, 2, 3];
// 多个数的加和
let newNums = nums.reduce(function(preSum, curVal, currentIndex, array) {
  return preSum + curVal;
}, 0);
console.log(newNums); //6
```

不传默认值会怎样？

不传默认值会自动以第一个元素为初始值，然后从第二个元素开始依次累计。

## filter

参数: 一个函数参数。这个函数接受一个默认参数，就是当前元素。这个作为参数的函数返回值为一个布尔类型，决定元素是否保留。

filter方法返回值为一个新的数组，这个数组里面包含参数里面所有被保留的项。

```
let nums = [1, 2, 3];
// 保留奇数项
let oddNums = nums.filter(item => item % 2);
console.log(oddNums);
```

## sort

参数: 一个用于比较的函数，它有两个默认参数，分别是代表比较的两个元素。

举个例子:

```
let nums = [2, 3, 1];  
//两个比较的元素分别为a, b  
nums.sort(function(a, b) {  
  if(a > b) return 1;  
  else if(a < b) return -1;  
  else if(a == b) return 0;  
})
```

当比较函数返回值大于0，则 a 在 b 的后面，即a的下标应该比b大。

反之，则 a 在 b 的前面，即 a 的下标比 b 小。

整个过程就完成了一次升序的排列。

当然还有一个需要注意的情况，就是比较函数不传的时候，是如何进行排序的？

答案是将数字转换为字符串，然后根据字母unicode值进行升序排序，也就是根据字符串的比较规则进行升序排序。

### 31. 能不能实现数组map方法？

依照 ecma262 草案，实现的map的规范如下：

When the **map** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **LengthOfArrayLike**(*O*).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *thisArg* is present, let *T* be *thisArg*; else let *T* be **undefined**.
5. Let *A* be ? **ArraySpeciesCreate**(*O*, *len*).
6. Let *k* be 0.
7. Repeat, while *k* < *len*
  - a. Let *Pk* be ! **ToString**(*k*).
  - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
  - c. If *kPresent* is **true**, then
    - i. Let *kValue* be ? **Get**(*O*, *Pk*).
    - ii. Let *mappedValue* be ? **Call**(*callbackfn*, *T*, « *kValue*, *k*, *O* »).
    - iii. Perform ? **CreateDataPropertyOrThrow**(*A*, *Pk*, *mappedValue*).
  - d. Set *k* to *k* + 1.
8. Return *A*.

下面根据草案的规定一步步来模拟实现map函数：

```
Array.prototype.map = function(callbackFn, thisArg) {  
  // 处理数组类型异常  
  if (this === null || this === undefined) {  
    throw new TypeError("Cannot read property 'map' of null or undefined");  
  }  
  // 处理回调类型异常  
  if (Object.prototype.toString.call(callbackFn) !== "[object Function]") {  
    throw new TypeError(callbackFn + ' is not a function')  
  }  
  // 草案中提到要先转换为对象
```

```

let O = Object(this);
let T = thisArg;

let len = O.length >>> 0;
let A = new Array(len);
for(let k = 0; k < len; k++) {
  // 还记得原型链那一节提到的 in 吗? in 表示在原型链查找
  // 如果用 hasOwnProperty 是有问题的, 它只能找私有属性
  if (k in O) {
    let kvalue = O[k];
    // 依次传入this, 当前项, 当前索引, 整个数组
    let mappedValue = callbackfn.call(T, kvalue, k, O);
    A[k] = mappedValue;
  }
}
return A;
}

```

这里解释一下, `length >>> 0`, 字面意思是指"右移 0 位", 但实际上是把前面的空位用 0 填充, 这里的作用是保证 `len` 为数字且为整数。

举几个特例:

```

null >>> 0    //0

undefined >>> 0 //0

void(0) >>> 0 //0

function a (){}; a >>> 0 //0

[] >>> 0 //0

var a = {}; a >>> 0 //0

123123 >>> 0 //123123

45.2 >>> 0 //45

0 >>> 0 //0

-0 >>> 0 //0

-1 >>> 0 //4294967295

-1212 >>> 0 //4294966084

```

总体实现起来并没那么难, 需要注意的就是使用 `in` 来进行原型链查找。同时, 如果没有找到就不处理, 能有效处理稀疏数组的情况。

最后给大家奉上 V8 源码, 参照源码检查一下, 其实还是实现得很完整了。

```

function ArrayMap(f, receiver) {
  CHECK_OBJECT_COERCIBLE(this, "Array.prototype.map");

  // Pull out the length so that modifications to the length in the

```

```
// loop will not affect the looping and side effects are visible.
var array = TO_OBJECT(this);
var length = TO_LENGTH(array.length);
if (!IS_CALLABLE(f)) throw %make_type_error(kCalledNonCallable, f);
var result = ArraySpeciesCreate(array, length);
for (var i = 0; i < length; i++) {
  if (i in array) {
    var element = array[i];
    %CreateDataProperty(result, i, %_call(f, receiver, element, i, array));
  }
}
return result;
}
```

## 32. 能不能实现数组reduce方法？

依照 ecma262 草案，实现的reduce的规范如下：

When the **reduce** method is called with one or two arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(**this** value).
2. Let *len* be ? **LengthOfArrayLike**(*O*).
3. If **IsCallable**(*callbackfn*) is **false**, throw a **TypeError** exception.
4. If *len* is 0 and *initialValue* is not present, throw a **TypeError** exception.
5. Let *k* be 0.
6. Let *accumulator* be **undefined**.
7. If *initialValue* is present, then
  - a. Set *accumulator* to *initialValue*.
8. Else,
  - a. Let *kPresent* be **false**.
  - b. Repeat, while *kPresent* is **false** and *k* < *len*
    - i. Let *Pk* be ! **ToString**(*k*).
    - ii. Set *kPresent* to ? **HasProperty**(*O*, *Pk*).
    - iii. If *kPresent* is **true**, then
      1. Set *accumulator* to ? **Get**(*O*, *Pk*).
    - iv. Set *k* to *k* + 1.
  - c. If *kPresent* is **false**, throw a **TypeError** exception.
9. Repeat, while *k* < *len*
  - a. Let *Pk* be ! **ToString**(*k*).
  - b. Let *kPresent* be ? **HasProperty**(*O*, *Pk*).
  - c. If *kPresent* is **true**, then
    - i. Let *kValue* be ? **Get**(*O*, *Pk*).
    - ii. Set *accumulator* to ? **Call**(*callbackfn*, **undefined**, « *accumulator*, *kValue*, *k*, *O* »).
  - d. Set *k* to *k* + 1.
10. Return *accumulator*.

其中有几个核心要点：

- 1) 初始值不传怎么处理
- 2) 回调函数的参数有哪些，返回值如何处理。

```
Array.prototype.reduce = function(callbackfn, initialValue) {
```



```
// 异常处理, 和 map 一样
// 处理数组类型异常
if (this === null || this === undefined) {
  throw new TypeError("Cannot read property 'reduce' of null or undefined");
}
// 处理回调类型异常
if (Object.prototype.toString.call(callbackfn) !== "[object Function]") {
  throw new TypeError(callbackfn + ' is not a function')
}
let o = Object(this);
let len = o.length >>> 0;
let k = 0;
let accumulator = initialValue;
if (accumulator === undefined) {
  for(; k < len ; k++) {
    // 查找原型链
    if (k in o) {
      accumulator = o[k];
      k++;
      break;
    }
  }
}
// 表示数组全为空
if(k === len && accumulator === undefined)
  throw new Error('Each element of the array is empty');
for(;k < len; k++) {
  if (k in o) {
    // 注意, 核心!
    accumulator = callbackfn.call(undefined, accumulator, o[k], k, o);
  }
}
return accumulator;
}
```

其实是从最后一项开始遍历, 通过原型链查找跳过空项。

最后给大家奉上V8源码, 以供大家检查:

```
function ArrayReduce(callback, current) {
  CHECK_OBJECT_COERCIBLE(this, "Array.prototype.reduce");

  // Pull out the length so that modifications to the length in the
  // loop will not affect the looping and side effects are visible.
  var array = TO_OBJECT(this);
  var length = TO_LENGTH(array.length);
  return InnerArrayReduce(callback, current, array, length,
    arguments.length);
}

function InnerArrayReduce(callback, current, array, length, argumentsLength) {
  if (!IS_CALLABLE(callback)) {
    throw %make_type_error(kCalledNonCallable, callback);
  }

  var i = 0;
  find_initial: if (argumentsLength < 2) {
```

```

    for (; i < length; i++) {
        if (i in array) {
            current = array[i++];
            break find_initial;
        }
    }
    throw %make_type_error(kReduceNoInitial);
}

for (; i < length; i++) {
    if (i in array) {
        var element = array[i];
        current = callback(current, element, i, array);
    }
}
return current;
}

```

### 33. 能不能实现数组 push、pop 方法？

参照 ecma262 草案的规定，关于 push 和 pop 的规范如下图所示：

#### 22.1.3.20 Array.prototype.push ( ...items )

NOTE 1 The arguments are appended to the end of the array, in the order in which they appear. The new length of the array is returned as the result of the call.

When the **push** method is called with zero or more arguments, the following steps are taken:

1. Let *O* be ? **ToObject**(this value).
2. Let *len* be ? **LengthOfArrayLike**(*O*).
3. Let *items* be a List whose elements are, in left to right order, the arguments that were passed to this function invocation.
4. Let *argCount* be the number of elements in *items*.
5. If *len* + *argCount* > 2<sup>53</sup> - 1, throw a **TypeError** exception.
6. Repeat, while *items* is not empty
  - a. Remove the first element from *items* and let *E* be the value of the element.
  - b. Perform ? **Set**(*O*, ! **ToString**(*len*), *E*, **true**).
  - c. Set *len* to *len* + 1.
7. Perform ? **Set**(*O*, "length", *len*, **true**).
8. Return *len*.

### 22.1.3.19 Array.prototype.pop ()

NOTE 1 The last element of the array is removed from the array and returned.

When the **pop** method is called, the following steps are taken:

1. Let *O* be ? ToObject(**this** value).
2. Let *len* be ? LengthOfArrayLike(*O*).
3. If *len* is zero, then
  - a. Perform ? Set(*O*, "length", 0, true).
  - b. Return **undefined**.
4. Else,
  - a. Assert: *len* > 0.
  - b. Let *newLen* be *len* - 1.
  - c. Let *index* be ! ToString(*newLen*).
  - d. Let *element* be ? Get(*O*, *index*).
  - e. Perform ? DeletePropertyOrThrow(*O*, *index*).
  - f. Perform ? Set(*O*, "length", *newLen*, true).
  - g. Return *element*.

首先来实现一下 push 方法:

```
Array.prototype.push = function(...items) {  
  let O = Object(this);  
  let len = this.length >>> 0;  
  let argCount = items.length >>> 0;  
  // 2 ** 53 - 1 为JS能表示的最大正整数  
  if (len + argCount > 2 ** 53 - 1) {  
    throw new TypeError("The number of array is over the max value restricted!")  
  }  
  for(let i = 0; i < argCount; i++) {  
    O[len + i] = items[i];  
  }  
  let newLength = len + argCount;  
  O.length = newLength;  
  return newLength;  
}
```

然后来实现 pop 方法:

```
Array.prototype.pop = function() {
  let o = object(this);
  let len = this.length >>> 0;
  if (len === 0) {
    o.length = 0;
    return undefined;
  }
  len--;
  let value = o[len];
  delete o[len];
  o.length = len;
  return value;
}
```

### 34. 能不能实现数组filter方法？

代码如下：

```
Array.prototype.filter = function(callbackfn, thisArg) {
  // 处理数组类型异常
  if (this === null || this === undefined) {
    throw new TypeError("Cannot read property 'filter' of null or undefined");
  }
  // 处理回调类型异常
  if (Object.prototype.toString.call(callbackfn) !== "[object Function]") {
    throw new TypeError(callbackfn + ' is not a function');
  }
  let o = object(this);
  let len = o.length >>> 0;
  let resLen = 0;
  let res = [];
  for(let i = 0; i < len; i++) {
    if (i in o) {
      let element = o[i];
      if (callbackfn.call(thisArg, o[i], i, o)) {
        res[resLen++] = element;
      }
    }
  }
  return res;
}
```

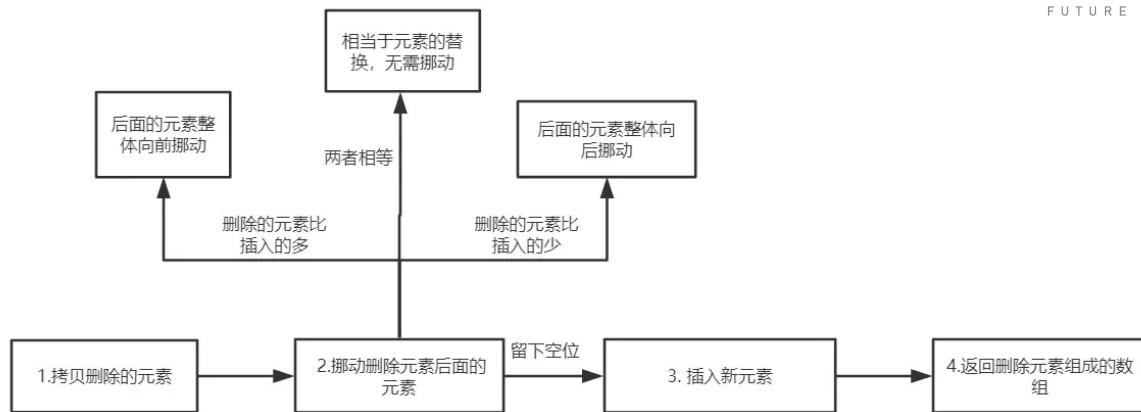
### 35. 能不能实现数组splice方法？

splice 可以说是最受欢迎的数组方法之一，api 灵活，使用方便。现在来梳理一下用法：

- splice(position, count) 表示从 position 索引的位置开始，删除count个元素
- splice(position, 0, ele1, ele2, ...) 表示从 position 索引的元素后面插入一系列的元素
- splice(position, count, ele1, ele2, ...) 表示从 position 索引的位置开始，删除 count 个元素，然后再插入一系列的元素
- 返回值为 被删除元素 组成的 数组。

接下来我们实现这个方法。

首先我们梳理一下实现的思路。



## 初步实现

```
Array.prototype.splice = function(startIndex, deleteCount, ...addElements) {
  let argumentsLen = arguments.length;
  let array = Object(this);
  let len = array.length;
  let deleteArr = new Array(deleteCount);

  // 拷贝删除的元素
  sliceDeleteElements(array, startIndex, deleteCount, deleteArr);
  // 移动删除元素后面的元素
  movePostElements(array, startIndex, len, deleteCount, addElements);
  // 插入新元素
  for (let i = 0; i < addElements.length; i++) {
    array[startIndex + i] = addElements[i];
  }
  array.length = len - deleteCount + addElements.length;
  return deleteArr;
}
```

先拷贝删除的元素，如下所示：

```
const sliceDeleteElements = (array, startIndex, deleteCount, deleteArr) => {
  for (let i = 0; i < deleteCount; i++) {
    let index = startIndex + i;
    if (index in array) {
      let current = array[index];
      deleteArr[i] = current;
    }
  }
};
```

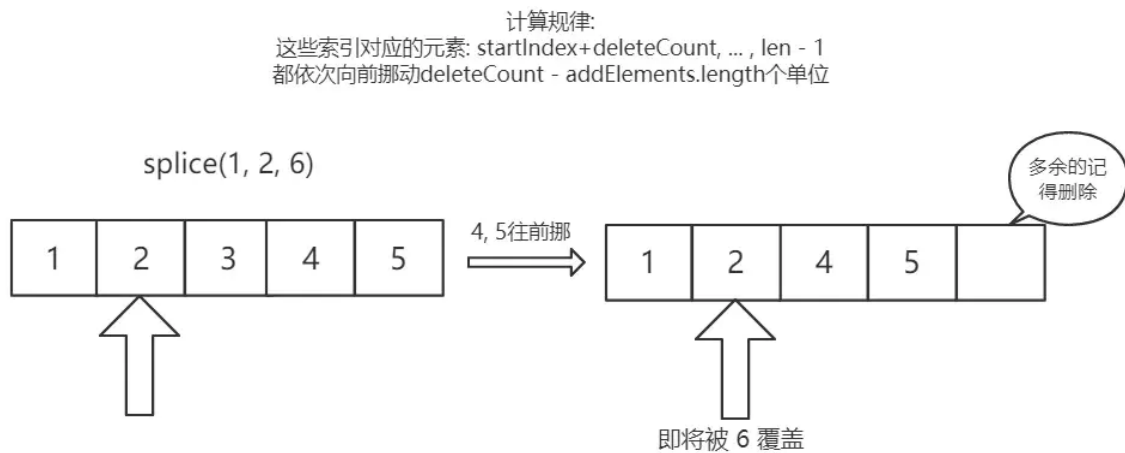
然后对删除元素后面的元素进行挪动，挪动分为三种情况：

- 1) 添加的元素和删除的元素个数相等
- 2) 添加的元素个数小于删除的元素
- 3) 添加的元素个数大于删除的元素

当两者相等时，

```
const movePostElements = (array, startIndex, len, deleteCount, addElements) => {
  if (deleteCount === addElements.length) return;
}
```

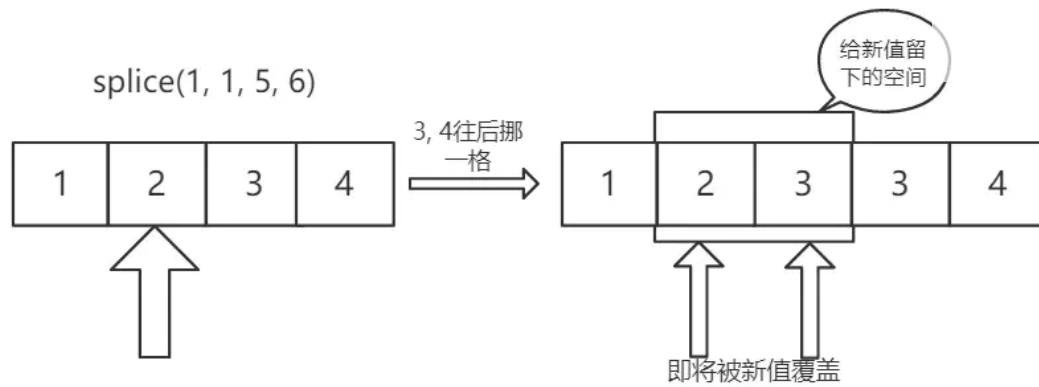
当添加的元素个数小于删除的元素时, 如图所示:



```
const movePostElements = (array, startIndex, len, deleteCount, addElements) => {
  //...
  // 如果添加的元素和删除的元素个数不相等, 则移动后面的元素
  if (deleteCount > addElements.length) {
    // 删除的元素比新增的元素多, 那么后面的元素整体向前挪动
    // 一共需要挪动 len - startIndex - deleteCount 个元素
    for (let i = startIndex + deleteCount; i < len; i++) {
      let fromIndex = i;
      // 将要挪动到的目标位置
      let toIndex = i - (deleteCount - addElements.length);
      if (fromIndex in array) {
        array[toIndex] = array[fromIndex];
      } else {
        delete array[toIndex];
      }
    }
    // 注意注意! 这里我们把后面的元素向前挪, 相当于数组长度减小了, 需要删除冗余元素
    // 目前长度为 len + addElements - deleteCount
    for (let i = len - 1; i >= len + addElements.length - deleteCount; i--) {
      delete array[i];
    }
  }
};
```

当添加的元素个数大于删除的元素时, 如图所示:

计算规律:  
这些索引对应的元素:  $\text{startIndex} + \text{deleteCount}, \dots, \text{len} - 1$   
都依次向后挪动  $\text{addElements.length} - \text{deleteCount}$  个单位



```
const movePostElements = (array, startIndex, len, deleteCount, addElements) => {
  //...
  if(deleteCount < addElements.length) {
    // 删除的元素比新增的元素少，那么后面的元素整体向后挪动
    // 思考一下：这里为什么要从后往前遍历？从前往后会产生什么问题？
    for (let i = len - 1; i >= startIndex + deleteCount; i--) {
      let fromIndex = i;
      // 将要挪动到的目标位置
      let toIndex = i + (addElements.length - deleteCount);
      if (fromIndex in array) {
        array[toIndex] = array[fromIndex];
      } else {
        delete array[toIndex];
      }
    }
  }
};
```

### 优化一: 参数的边界情况

当用户传来非法的 `startIndex` 和 `deleteCount` 或者负索引的时候，需要我们做出特殊的处理。

```
const computeStartIndex = (startIndex, len) => {
  // 处理索引负数的情况
  if (startIndex < 0) {
    return startIndex + len > 0 ? startIndex + len: 0;
  }
  return startIndex >= len ? len: startIndex;
}

const computeDeleteCount = (startIndex, len, deleteCount, argumentsLen) => {
  // 删除数目没有传，默认删除startIndex及后面所有的
  if (argumentsLen === 1)
    return len - startIndex;
  // 删除数目过小
  if (deleteCount < 0)
    return 0;
  // 删除数目过大
  if (deleteCount > len - startIndex)
```

```

    return len - startIndex;
    return deleteCount;
}

Array.prototype.splice = function (startIndex, deleteCount, ...addElements) {
    //,...
    let deleteArr = new Array(deleteCount);

    // 下面参数的清洗工作
    startIndex = computeStartIndex(startIndex, len);
    deleteCount = computeDeleteCount(startIndex, len, deleteCount, argumentsLen);

    // 拷贝删除的元素
    sliceDeleteElements(array, startIndex, deleteCount, deleteArr);
    //...
}

```

## 优化二: 数组为密封对象或冻结对象

什么是密封对象?

密封对象是不可扩展的对象, 而且已有成员的[[Configurable]]属性被设置为false, 这意味着不能添加、删除方法和属性。但是属性值是可以修改的。

什么是冻结对象?

冻结对象是最严格的防篡改级别, 除了包含密封对象的限制外, 还不能修改属性值。

接下来, 我们来把这两种情况——排除。

```

// 判断 sealed 对象和 frozen 对象, 即 密封对象 和 冻结对象
if (Object.isSealed(array) && deleteCount !== addElements.length) {
    throw new TypeError('the object is a sealed object!')
} else if (Object.isFrozen(array) && (deleteCount > 0 || addElements.length > 0)) {
    throw new TypeError('the object is a frozen object!')
}

```

好了, 现在就写了一个比较完整的splice, 如下:

```

const sliceDeleteElements = (array, startIndex, deleteCount, deleteArr) => {
    for (let i = 0; i < deleteCount; i++) {
        let index = startIndex + i;
        if (index in array) {
            let current = array[index];
            deleteArr[i] = current;
        }
    }
};

const movePostElements = (array, startIndex, len, deleteCount, addElements) => {
    // 如果添加的元素和删除的元素个数相等, 相当于元素的替换, 数组长度不变, 被删除元素后面的元素不需要挪动
    if (deleteCount === addElements.length) return;
    // 如果添加的元素和删除的元素个数不相等, 则移动后面的元素
    else if (deleteCount > addElements.length) {

```



```
// 删除的元素比新增的元素多，那么后面的元素整体向前挪动
// 一共需要挪动 len - startIndex - deleteCount 个元素
for (let i = startIndex + deleteCount; i < len; i++) {
  let fromIndex = i;
  // 将要挪动到的目标位置
  let toIndex = i - (deleteCount - addElements.length);
  if (fromIndex in array) {
    array[toIndex] = array[fromIndex];
  } else {
    delete array[toIndex];
  }
}

// 注意注意！这里我们把后面的元素向前挪，相当于数组长度减小了，需要删除冗余元素
// 目前长度为 len + addElements - deleteCount
for (let i = len - 1; i >= len + addElements.length - deleteCount; i--) {
  delete array[i];
}
} else if(deleteCount < addElements.length) {
  // 删除的元素比新增的元素少，那么后面的元素整体向后挪动
  // 思考一下：这里为什么要从后往前遍历？从前往后会产生什么问题？
  for (let i = len - 1; i >= startIndex + deleteCount; i--) {
    let fromIndex = i;
    // 将要挪动到的目标位置
    let toIndex = i + (addElements.length - deleteCount);
    if (fromIndex in array) {
      array[toIndex] = array[fromIndex];
    } else {
      delete array[toIndex];
    }
  }
}
};

const computeStartIndex = (startIndex, len) => {
  // 处理索引负数的情况
  if (startIndex < 0) {
    return startIndex + len > 0 ? startIndex + len: 0;
  }
  return startIndex >= len ? len: startIndex;
}

const computeDeleteCount = (startIndex, len, deleteCount, argumentsLen) => {
  // 删除数目没有传，默认删除startIndex及后面所有的
  if (argumentsLen === 1)
    return len - startIndex;
  // 删除数目过小
  if (deleteCount < 0)
    return 0;
  // 删除数目过大
  if (deleteCount > len - startIndex)
    return len - startIndex;
  return deleteCount;
}

Array.prototype.splice = function(startIndex, deleteCount, ...addElements) {
  let argumentsLen = arguments.length;
  let array = Object(this);
  let len = array.length >>> 0;
```

```
let deleteArr = new Array(deleteCount);

startIndex = computeStartIndex(startIndex, len);
deleteCount = computeDeleteCount(startIndex, len, deleteCount, argumentsLen);

// 判断 sealed 对象和 frozen 对象，即 密封对象 和 冻结对象
if (Object.isSealed(array) && deleteCount !== addElements.length) {
  throw new TypeError('the object is a sealed object!')
} else if (Object.isFrozen(array) && (deleteCount > 0 || addElements.length > 0)) {
  throw new TypeError('the object is a frozen object!')
}

// 拷贝删除的元素
sliceDeleteElements(array, startIndex, deleteCount, deleteArr);
// 移动删除元素后面的元素
movePostElements(array, startIndex, len, deleteCount, addElements);

// 插入新元素
for (let i = 0; i < addElements.length; i++) {
  array[startIndex + i] = addElements[i];
}

array.length = len - deleteCount + addElements.length;

return deleteArr;
}
```

## 36. 能不能实现数组sort方法？

估计大家对 JS 数组的 sort 方法已经不陌生了，之前也对它的用法做了详细的总结。那，它的内部是如何来实现的呢？如果说我们能够进入它的内部去看一看，理解背后的设计，会使我们的思维和素养得到不错的提升。

sort 方法在 V8 内部相对与其他方法而言是一个比较高深的算法，对于很多边界情况做了反复的优化，但是这里我们不会直接拿源码来干讲。我们会来根据源码的思路，实现一个跟引擎性能一样的排序算法，并且一步步拆解其中的奥秘。

### V8 引擎的思路分析

首先大概梳理一下源码中排序的思路：

设要排序的元素个数是  $n$ ：

- 当  $n \leq 10$  时，采用 插入排序
- 当  $n > 10$  时，采用

三路快速排序

- $10 < n \leq 1000$ ，采用中位数作为哨兵元素
- $n > 1000$ ，每隔 200~215 个元素挑出一个元素，放到一个新数组，然后对它排序，找到中间位置的数，以此作为中位数

在动手之前，我觉得我们有必要**为什么**这么做搞清楚。

## 第一、为什么元素个数少的时候要采用插入排序？

虽然插入排序理论上说是 $O(n^2)$ 的算法，快速排序是一个 $O(n\log n)$ 级别的算法。但是别忘了，这只是理论上的估算，在实际情况中两者的算法复杂度前面都会有一个系数的，当  $n$  足够小的时候，快速排序  $n\log n$  的优势会越来越小，倘若插入排序 $O(n^2)$ 前面的系数足够小，那么就会超过快排。而事实上正是如此，插入排序经过优化以后对于小数据集的排序会有非常优越的性能，很多时候甚至会超过快排。

因此，对于很小的数据量，应用插入排序是一个非常不错的选择。

## 第二、为什么要花这么大的力气选择哨兵元素？

因为快速排序的性能瓶颈在于递归的深度，最坏的情况是每次的哨兵都是最小元素或者最大元素，那么进行partition(一边是小于哨兵的元素，另一边是大于哨兵的元素)时，就会有一边是空的，那么这么排下去，递归的层数就达到了 $n$ ，而每一层的复杂度是 $O(n)$ ，因此快排这时候会退化成 $O(n^2)$ 级别。

这种情况是要尽力避免的！如果来避免？

就是让哨兵元素尽可能地处于数组的中间位置，让最大或者最小的情况尽可能少。这时候，你就能理解V8里面所做的种种优化了。

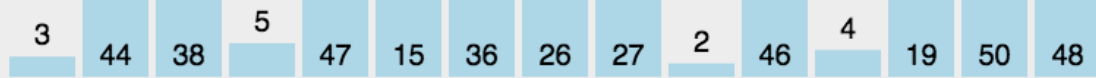
接下来，我们来一步步实现的这样的官方排序算法。

## 插入排序及优化

最初的插入排序可能是这样写的：

```
const insertSort = (arr, start = 0, end) => {
  end = end || arr.length;
  for(let i = start; i < end; i++) {
    let j;
    for(j = i; j > start && arr[j - 1] > arr[j]; j --) {
      let temp = arr[j];
      arr[j] = arr[j - 1];
      arr[j - 1] = temp;
    }
  }
  return;
}
```

看似可以正确的完成排序，但实际上交换元素会有相当大的性能消耗，我们完全可以用变量覆盖的方式来完成，如图所示：



优化后代码如下:

```
const insertSort = (arr, start = 0, end) => {  
  end = end || arr.length;  
  for(let i = start; i < end; i++) {  
    let e = arr[i];  
    let j;  
    for(j = i; j > start && arr[j - 1] > e; j --)  
      arr[j] = arr[j-1];  
    arr[j] = e;  
  }  
  return;  
}
```

接下来正式进入到 sort 方法。

## 寻找哨兵元素

sort的骨架大致如下:

```
Array.prototype.sort = (comparefn) => {  
  let array = Object(this);  
  let length = array.length >>> 0;  
  return InnerArraySort(array, length, comparefn);  
}  
  
const InnerArraySort = (array, length, comparefn) => {  
  // 比较函数未传入  
  if (Object.prototype.toString.call(callbackfn) !== "[object Function]") {  
    comparefn = function (x, y) {  
      if (x === y) return 0;  
      x = x.toString();  
      y = y.toString();  
      if (x == y) return 0;  
    }  
  }  
}
```

```

        else return x < y ? -1 : 1;
    };
}
const insertSort = () => {
    //...
}
const getThirdIndex = (a, from, to) => {
    // 元素个数大于1000时寻找哨兵元素
}
const quickSort = (a, from, to) => {
    //哨兵位置
    let thirdIndex = 0;
    while(true) {
        if(to - from <= 10) {
            insertSort(a, from, to);
            return;
        }
        if(to - from > 1000) {
            thirdIndex = getThirdIndex(a, from, to);
        }else {
            // 小于1000 直接取中点
            thirdIndex = from + ((to - from) >> 2);
        }
    }
    //下面开始快排
}
}

```

先把求取哨兵位置的代码实现一下:

```

const getThirdIndex = (a, from, to) => {
    let tmpArr = [];
    // 递增量, 200~215 之间, 因为任何正数和15做与操作, 不会超过15, 当然是大于0的
    let increment = 200 + ((to - from) & 15);
    let j = 0;
    from += 1;
    to -= 1;
    for (let i = from; i < to; i += increment) {
        tmpArr[j] = [i, a[i]];
        j++;
    }
    // 把临时数组排序, 取中间的值, 确保哨兵的值接近平均位置
    tmpArr.sort(function(a, b) {
        return comparefn(a[1], b[1]);
    });
    let thirdIndex = tmpArr[tmpArr.length >> 1][0];
    return thirdIndex;
}

```

## 完成快排

接下来我们来完成快排的具体代码:

```

const _sort = (a, b, c) => {
    let arr = [a, b, c];

```

```

insertSort(arr, 0, 3);
return arr;
}

const quickSort = (a, from, to) => {
  //...
  // 上面我们拿到了thirdIndex
  // 现在我们拥有三个元素, from, thirdIndex, to
  // 为了再次确保 thirdIndex 不是最值, 把这三个值排序
  [a[from], a[thirdIndex], a[to - 1]] = _sort(a[from], a[thirdIndex], a[to - 1]);
  // 现在正式把 thirdIndex 作为哨兵
  let pivot = a[thirdIndex];
  // 正式进入快排
  let lowEnd = from + 1;
  let highStart = to - 1;
  // 现在正式把 thirdIndex 作为哨兵, 并且lowEnd和thirdIndex交换
  let pivot = a[thirdIndex];
  a[thirdIndex] = a[lowEnd];
  a[lowEnd] = pivot;

  // [lowEnd, i)的元素是和pivot相等的
  // [i, highStart) 的元素是需要处理的
  for(let i = lowEnd + 1; i < highStart; i++) {
    let element = a[i];
    let order = comparefn(element, pivot);
    if (order < 0) {
      a[i] = a[lowEnd];
      a[lowEnd] = element;
      lowEnd++;
    } else if (order > 0) {
      do{
        highStart--;
        if(highStart === i) break;
        order = comparefn(a[highStart], pivot);
      }while(order > 0)
      // 现在 a[highStart] <= pivot
      // a[i] > pivot
      // 两者交换
      a[i] = a[highStart];
      a[highStart] = element;
      if (order < 0) {
        // a[i] 和 a[lowEnd] 交换
        element = a[i];
        a[i] = a[lowEnd];
        a[lowEnd] = element;
        lowEnd++;
      }
    }
  }
  // 永远切分大区间
  if (lowEnd - from > to - highStart) {
    // 继续切分lowEnd ~ from 这个区间
    to = lowEnd;
    // 单独处理小区间
    quickSort(a, highStart, to);
  } else if (lowEnd - from <= to - highStart) {
    from = highStart;
  }
}

```

```
    quickSort(a, from, lowEnd);  
  }  
}
```

## 测试结果

测试结果如下:

一万条数据:

```
10000条数据:  
手写sort: 19.491ms  
手写sort(近乎有序): 12.226ms  
手写sort(完全有序): 1.322ms  
V8 排序: 16.981ms  
V8 排序(近乎有序): 11.108ms  
V8 排序(完全有序): 4.202ms
```

十万条数据:

```
100000条数据:  
手写sort: 71.959ms  
手写sort(近乎有序): 65.896ms  
手写sort(完全有序): 6.359ms  
V8 排序: 80.918ms  
V8 排序(近乎有序): 52.908ms  
V8 排序(完全有序): 27.206ms
```

一百万条数据:

```
1000000条数据:  
手写sort: 239.246ms  
手写sort(近乎有序): 87.710ms  
手写sort(完全有序): 89.522ms  
V8 排序: 502.877ms  
V8 排序(近乎有序): 554.668ms  
V8 排序(完全有序): 326.154ms
```

一千万条数据:

```
10000000条数据:  
手写sort: 2324.992ms  
手写sort(近乎有序): 690.353ms  
手写sort(完全有序): 766.056ms  
V8 排序: 5904.363ms  
V8 排序(近乎有序): 6061.396ms  
V8 排序(完全有序): 3752.349ms
```

结果仅供大家参考, 因为不同的node版本对于部分细节的实现可能不一样, 我现在的版本是v10.15。

从结果可以看到, 目前版本的 node 对于有序程度较高的数据是处理的不够好的, 而我们刚刚实现的排序通过反复确定哨兵的位置就能有效的规避快排在这一场景下的劣势。

最后给大家完整版的sort代码:

```
const sort = (arr, comparefn) => {  
  let array = Object(arr);  
  let length = array.length >>> 0;  
  return InnerArraySort(array, length, comparefn);  
}
```

```

const InnerArraySort = (array, length, comparefn) => {
  // 比较函数未传入
  if (Object.prototype.toString.call(comparefn) !== "[object Function]") {
    comparefn = function (x, y) {
      if (x === y) return 0;
      x = x.toString();
      y = y.toString();
      if (x == y) return 0;
      else return x < y ? -1 : 1;
    };
  }
  const insertSort = (arr, start = 0, end) => {
    end = end || arr.length;
    for (let i = start; i < end; i++) {
      let e = arr[i];
      let j;
      for (j = i; j > start && comparefn(arr[j - 1], e) > 0; j--) {
        arr[j] = arr[j - 1];
      }
      arr[j] = e;
    }
    return;
  }
  const getThirdIndex = (a, from, to) => {
    let tmpArr = [];
    // 递增量, 200~215 之间, 因为任何正数和15做与操作, 不会超过15, 当然是大于0的
    let increment = 200 + ((to - from) & 15);
    let j = 0;
    from += 1;
    to -= 1;
    for (let i = from; i < to; i += increment) {
      tmpArr[j] = [i, a[i]];
      j++;
    }
    // 把临时数组排序, 取中间的值, 确保哨兵的值接近平均位置
    tmpArr.sort(function (a, b) {
      return comparefn(a[1], b[1]);
    });
    let thirdIndex = tmpArr[tmpArr.length >> 1][0];
    return thirdIndex;
  };

  const _sort = (a, b, c) => {
    let arr = [];
    arr.push(a, b, c);
    insertSort(arr, 0, 3);
    return arr;
  }

  const quickSort = (a, from, to) => {
    //哨兵位置
    let thirdIndex = 0;
    while (true) {
      if (to - from <= 10) {
        insertSort(a, from, to);
        return;
      }
      if (to - from > 1000) {

```



```

        thirdIndex = getThirdIndex(a, from, to);
    } else {
        // 小于1000 直接取中点
        thirdIndex = from + ((to - from) >> 2);
    }
    let tmpArr = _sort(a[from], a[thirdIndex], a[to - 1]);
    a[from] = tmpArr[0]; a[thirdIndex] = tmpArr[1]; a[to - 1] = tmpArr[2];
    // 现在正式把 thirdIndex 作为哨兵
    let pivot = a[thirdIndex];
    [a[from], a[thirdIndex]] = [a[thirdIndex], a[from]];
    // 正式进入快排
    let lowEnd = from + 1;
    let highStart = to - 1;
    a[thirdIndex] = a[lowEnd];
    a[lowEnd] = pivot;
    // [lowEnd, i)的元素是和pivot相等的
    // [i, highStart) 的元素是需要处理的
    for (let i = lowEnd + 1; i < highStart; i++) {
        let element = a[i];
        let order = comparefn(element, pivot);
        if (order < 0) {
            a[i] = a[lowEnd];
            a[lowEnd] = element;
            lowEnd++;
        } else if (order > 0) {
            do{
                highStart--;
                if (highStart === i) break;
                order = comparefn(a[highStart], pivot);
            }while (order > 0) ;
            // 现在 a[highStart] <= pivot
            // a[i] > pivot
            // 两者交换
            a[i] = a[highStart];
            a[highStart] = element;
            if (order < 0) {
                // a[i] 和 a[lowEnd] 交换
                element = a[i];
                a[i] = a[lowEnd];
                a[lowEnd] = element;
                lowEnd++;
            }
        }
    }
}
// 永远切分大区间
if (lowEnd - from > to - highStart) {
    // 单独处理小区间
    quickSort(a, highStart, to);
    // 继续切分lowEnd ~ from 这个区间
    to = lowEnd;
} else if (lowEnd - from <= to - highStart) {
    quickSort(a, from, lowEnd);
    from = highStart;
}
}
}
quickSort(array, 0, length);
}

```

### 37. 能不能模拟实现一个new的效果？

new 被调用后做了三件事情:

- 1) 让实例可以访问到私有属性
- 2) 让实例可以访问构造函数原型(creator.prototype)所在原型链上的属性
- 3) 如果构造函数返回的结果不是引用数据类型

```
function newOperator(ctor, ...args) {  
  if(typeof ctor !== 'function'){  
    throw 'newOperator function the first param must be a function';  
  }  
  let obj = Object.create(ctor.prototype);  
  let res = ctor.apply(obj, args);  
  
  let isObject = typeof res === 'object' && res !== null;  
  let isFunction = typeof res === 'function';  
  return isObject || isFunction ? res : obj;  
};
```

### 38. 能不能模拟实现一个 bind 的效果？

实现bind之前，我们首先要知道它做了哪些事情。

- 1) 对于普通函数，绑定this指向
- 2) 对于构造函数，要保证原函数的原型对象上的属性不能丢失

```
Function.prototype.bind = function (context, ...args) {  
  // 异常处理  
  if (typeof this !== "function") {  
    throw new Error("Function.prototype.bind - what is trying to be bound is not callable");  
  }  
  // 保存this的值，它代表调用 bind 的函数  
  var self = this;  
  var fNOP = function () {};  
  
  var fbound = function () {  
    self.apply(this instanceof self ?  
      this :  
      context, args.concat(Array.prototype.slice.call(arguments)));  
  }  
  
  fNOP.prototype = this.prototype;  
  fbound.prototype = new fNOP();  
  
  return fbound;  
}
```

也可以这么用 Object.create 来处理原型:

```
Function.prototype.bind = function (context, ...args) {
  if (typeof this !== "function") {
    throw new Error("Function.prototype.bind - what is trying to be bound is not callable");
  }

  var self = this;

  var fbound = function () {
    self.apply(this instanceof self ?
      this :
      context, args.concat(Array.prototype.slice.call(arguments)));
  }

  fbound.prototype = Object.create(self.prototype);

  return fbound;
}
```

### 39. 能不能实现一个 call/apply 函数？

```
Function.prototype.call = function (context) {
  let context = context || window;
  let fn = Symbol('fn');
  context.fn = this;

  let args = [];
  for(let i = 1, len = arguments.length; i < len; i++) {
    args.push('arguments[' + i + ']');
  }

  let result = eval('context.fn(' + args + ')');

  delete context.fn
  return result;
}
```

不过我认为换成 ES6 的语法会更精炼一些:

```
Function.prototype.call = function (context, ...args) {
  let context = context || window;
  let fn = Symbol('fn');
  context.fn = this;

  let result = eval('context.fn(...args)');

  delete context.fn
  return result;
}
```

类似的，有apply的对应实现:

```
Function.prototype.apply = function (context, args) {  
  let context = context || window;  
  context.fn = this;  
  let result = eval('context.fn(...args)');  
  
  delete context.fn  
  return result;  
}
```

## 40. 谈谈你对JS中this的理解

其实JS中的this是一个非常简单的东西，只需要理解它的执行规则就OK。

在这里不想像其他博客一样展示太多的代码例子弄得天花乱坠，反而不易理解。

call/apply/bind可以显式绑定, 这里就不说了。

主要这些场隐式绑定的场景讨论:

- 1) 全局上下文
- 2) 直接调用函数
- 3) 对象.方法的形式调用
- 4) DOM事件绑定(特殊)
- 5) new构造函数绑定
- 6) 箭头函数

### 全局上下文

全局上下文默认this指向window, 严格模式下指向undefined。

### 直接调用函数

比如:

```
let obj = {  
  a: function() {  
    console.log(this);  
  }  
}  
let func = obj.a;  
func();
```

这种情况是直接调用。this相当于全局上下文的情况。

### 对象.方法的形式调用

还是刚刚的例子，我如果这样写:

```
obj.a();
```

这就是 `对象.方法` 的情况，`this`指向这个对象

## DOM事件绑定

`onclick`和`addEventListener`中 `this` 默认指向绑定事件的元素。

IE比较奇异，使用`attachEvent`，里面的`this`默认指向`window`。

## new+构造函数

此时构造函数中的`this`指向实例对象。

## 箭头函数?

箭头函数没有`this`，因此也不能绑定。里面的`this`会指向当前最近的非箭头函数的`this`，找不到就是`window`(严格模式是`undefined`)。比如：

```
let obj = {
  a: function() {
    let do = () => {
      console.log(this);
    }
    do();
  }
}
obj.a(); // 找到最近的非箭头函数a，a现在绑定着obj，因此箭头函数中的this是obj
```

优先级: `new` > `call`、`apply`、`bind` > `对象.方法` > 直接调用。

## 41. JS中浅拷贝的手段有哪些?

### 重要: 什么是拷贝?

首先来直观的感受一下什么是拷贝。

```
let arr = [1, 2, 3];
let newArr = arr;
newArr[0] = 100;

console.log(arr); //[100, 2, 3]
```

这是直接赋值的情况，不涉及任何拷贝。当改变`newArr`的时候，由于是同一个引用，`arr`指向的值也跟着改变。

现在进行浅拷贝：

```
let arr = [1, 2, 3];
let newArr = arr.slice();
newArr[0] = 100;

console.log(arr);//[1, 2, 3]
```

当修改newArr的时候，arr的值并不改变。什么原因?因为这里newArr是arr浅拷贝后的结果，newArr和arr现在引用的已经不是同一块空间啦!

这就是浅拷贝!

但是这又会带来一个潜在的问题:

```
let arr = [1, 2, {val: 4}];
let newArr = arr.slice();
newArr[2].val = 1000;

console.log(arr);//[ 1, 2, { val: 1000 } ]
```

不是已经不是同一块空间的引用了吗? 为什么改变了newArr改变了第二个元素的val值，arr也跟着变了。

这就是浅拷贝的限制所在了。它只能拷贝一层对象。如果有对象的嵌套，那么浅拷贝将无能为力。但幸运的是，深拷贝就是为了解决这个问题而生的，它能解决无限极的对象嵌套问题，实现彻底的拷贝。当然，这是我们下一篇的重点。现在先让大家有一个基本的概念。

接下来，来研究一下JS中实现浅拷贝到底有多少种方式?

## 手动实现

```
const shallowClone = (target) => {
  if (typeof target === 'object' && target !== null) {
    const cloneTarget = Array.isArray(target) ? [] : {};
    for (let prop in target) {
      if (target.hasOwnProperty(prop)) {
        cloneTarget[prop] = target[prop];
      }
    }
    return cloneTarget;
  } else {
    return target;
  }
}
```

## Object.assign

但是需要注意的是，Object.assign() 拷贝的是对象的属性的引用，而不是对象本身。

```
let obj = { name: 'sy', age: 18 };
const obj2 = Object.assign({}, obj, {name: 'sss'});
console.log(obj2);//{ name: 'sss', age: 18 }
```

```
let arr = [1, 2, 3];
let newArr = arr.concat();
newArr[1] = 100;
console.log(arr); //[ 1, 2, 3 ]
```

### slice浅拷贝

开头的例子不就说的这个嘛!

### ...展开运算符

```
let arr = [1, 2, 3];
let newArr = [...arr]; //跟arr.slice()是一样的效果
```

## 42. 能不能写一个完整的深拷贝?

### 简易版及问题

```
JSON.parse(JSON.stringify());
```

估计这个api能覆盖大多数的应用场景, 没错, 谈到深拷贝, 我第一个想到的也是它。但是实际上, 对于某些严格的场景来说, 这个方法是有巨大的坑的。问题如下:

无法解决 循环引用 的问题。举个例子:

```
const a = {val:2};
a.target = a;
```

拷贝a会出现系统栈溢出, 因为出现了 无限递归 的情况。

无法拷贝一写 特殊的对象, 诸如 RegExp, Date, Set, Map等。

无法拷贝 函数 (划重点)。

因此这个api先pass掉, 我们重新写一个深拷贝, 简易版如下:

```
const deepClone = (target) => {
  if (typeof target === 'object' && target !== null) {
    const cloneTarget = Array.isArray(target) ? []: {};
    for (let prop in target) {
      if (target.hasOwnProperty(prop)) {
        cloneTarget[prop] = deepClone(target[prop]);
      }
    }
    return cloneTarget;
  } else {
    return target;
  }
}
```

现在，我们以刚刚发现的三个问题为导向，一步步来完善、优化我们的深拷贝代码。

## 解决循环引用

现在问题如下：

```
let obj = {val : 100};
obj.target = obj;

deepClone(obj); //报错: RangeError: Maximum call stack size exceeded
```

这就是循环引用。我们怎么来解决这个问题呢？

创建一个Map。记录下已经拷贝过的对象，如果说已经拷贝过，那直接返回它行了。

```
const isObject = (target) => (typeof target === 'object' || typeof target === 'function') && target !== null;

const deepClone = (target, map = new Map()) => {
  if(map.get(target))
    return target;

  if (isObject(target)) {
    map.set(target, true);
    const cloneTarget = Array.isArray(target) ? []: {};
    for (let prop in target) {
      if (target.hasOwnProperty(prop)) {
        cloneTarget[prop] = deepClone(target[prop], map);
      }
    }
    return cloneTarget;
  } else {
    return target;
  }
}
```

现在来试一试：



```
const a = {val:2};
a.target = a;
let newA = deepClone(a);
console.log(newA)//{ val: 2, target: { val: 2, target: [Circular] } }
```

好像是没有问题了,拷贝也完成了。但还是有一个潜在的坑,就是map 上的 key 和 map 构成了强引用关系,这是相当危险的。我给你解释一下与之相对的弱引用的概念你就明白了:

在计算机程序设计中,弱引用与强引用相对,

是指不能确保其引用的对象不会被垃圾回收器回收的引用。一个对象若只被弱引用所引用,则被认为是不可访问(或弱可访问)的,并因此可能在任何时刻被回收。--百度百科

大白话解释一下,被弱引用的对象可以在任何时候被回收,而对于强引用来说,只要这个强引用还在,那么对象无法被回收。拿上面的例子说,map 和 a 一直是强引用的关系,在程序结束之前,a 所占的内存空间一直不会被释放。

怎么解决这个问题?

很简单,让 map 的 key 和 map 构成弱引用即可。ES6给我们提供了这样的数据结构,它的名字叫 `WeakMap`,它是一种特殊的Map,其中的键是弱引用的。其键必须是对象,而值可以是任意的。

稍微改造一下即可:

```
const deepClone = (target, map = new WeakMap()) => {
  //...
}
```

## 拷贝特殊对象

### (1) 可继续遍历

对于特殊的对象,我们使用以下方式来鉴别:

```
Object.prototype.toString.call(obj);
```

梳理一下对于可遍历对象会有什么结果:

```
["object Map"]
["object Set"]
["object Array"]
["object Object"]
["object Arguments"]
```

好,以这些不同的字符串为依据,我们就可以成功地鉴别这些对象。

```
const getType = Object.prototype.toString.call(obj);

const canTraverse = {
  '[object Map]': true,
  '[object Set]': true,
  '[object Array]': true,
  '[object Object]': true,
  '[object Arguments]': true,
```

```
};

const deepClone = (target, map = new Map()) => {
  if(!isObject(target))
    return target;
  let type = getType(target);
  let cloneTarget;
  if(!canTraverse[type]) {
    // 处理不能遍历的对象
    return;
  }else {
    // 这波操作相当关键，可以保证对象的原型不丢失！
    let ctor = target.prototype;
    cloneTarget = new ctor();
  }

  if(map.get(target))
    return target;
  map.put(target, true);

  if(type === mapTag) {
    //处理Map
    target.forEach((item, key) => {
      cloneTarget.set(deepClone(key), deepClone(item));
    })
  }

  if(type === setTag) {
    //处理Set
    target.forEach(item => {
      target.add(deepClone(item));
    })
  }

  // 处理数组和对象
  for (let prop in target) {
    if (target.hasOwnProperty(prop)) {
      cloneTarget[prop] = deepClone(target[prop]);
    }
  }
  return cloneTarget;
}
```

## (2) 不可遍历的对象

```
const boolTag = '[object Boolean]';
const numberTag = '[object Number]';
const stringTag = '[object String]';
const dateTag = '[object Date]';
const errorTag = '[object Error]';
const regexpTag = '[object RegExp]';
const funcTag = '[object Function]';
```

对于不可遍历的对象，不同的对象有不同的处理。

```
const handleRegExp = (target) => {
```

```
const { source, flags } = target;
return new target.constructor(source, flags);
}

const handleFunc = (target) => {
  // 待会的重点部分
}

const handleNotTraverse = (target, tag) => {
  const Ctor = target.constructor;
  switch(tag) {
    case boolTag:
    case numberTag:
    case stringTag:
    case errorTag:
    case dateTag:
      return new Ctor(target);
    case regexpTag:
      return handleRegExp(target);
    case funcTag:
      return handleFunc(target);
    default:
      return new Ctor(target);
  }
}
```

## 拷贝函数

虽然函数也是对象，但是它过于特殊，我们单独把它拿出来拆解。

提到函数，在JS中有两种函数，一种是普通函数，另一种是箭头函数。每个普通函数都是 Function 的实例，而箭头函数不是任何类的实例，每次调用都是不一样的引用。那我们只需要处理普通函数的情况，箭头函数直接返回它本身就好了。

那么如何来区分两者呢？

答案是：利用原型。箭头函数是不存在原型的。

代码如下：

```
const handleFunc = (func) => {
  // 箭头函数直接返回自身
  if(!func.prototype) return func;
  const bodyReg = /^(?<={})(.|\\n)+(?=})/m;
  const paramReg = /^(?<=\\(\\).+(?=\\)\\s+{)/;
  const funcString = func.toString();
  // 分别匹配 函数参数 和 函数体
  const param = paramReg.exec(funcString);
  const body = bodyReg.exec(funcString);
  if(!body) return null;
  if (param) {
    const paramArr = param[0].split(',');
    return new Function(...paramArr, body[0]);
  } else {
    return new Function(body[0]);
  }
}
```

到现在，我们的深拷贝就实现地比较完善了。不过在测试的过程中，我也发现了一个小小的bug。

## 小小的bug

如下所示:

```
const target = new Boolean(false);
const Ctor = target.constructor;
new Ctor(target); // 结果为 Boolean {true} 而不是 false。
```

对于这样一个bug，我们可以对 Boolean 拷贝做最简单的修改，调用valueOf: new target.constructor(target.valueOf())。

但实际上，这种写法是不推荐的。因为在ES6后不推荐使用【new 基本类型()】这样的语法，所以es6中的新类型 Symbol 是不能直接 new 的，只能通过 new Object(SymbolType)。

因此我们接下来统一一下:

```
const handleNotTraverse = (target, tag) => {
  const Ctor = target.constructor;
  switch(tag) {
    case boolTag:
      return new Object(Boolean.prototype.valueOf.call(target));
    case numberTag:
      return new Object(Number.prototype.valueOf.call(target));
    case stringTag:
      return new Object(String.prototype.valueOf.call(target));
    case errorTag:
    case dateTag:
      return new Ctor(target);
    case regexpTag:
      return handleRegExp(target);
    case funcTag:
      return handleFunc(target);
    default:
      return new Ctor(target);
  }
}
```

## 完整代码展示

完整版的深拷贝:

```
const getType = obj => Object.prototype.toString.call(obj);

const isObject = (target) => (typeof target === 'object' || typeof target === 'function') && target !== null;

const canTraverse = {
  '[object Map]': true,
  '[object Set]': true,
  '[object Array]': true,
  '[object Object]': true,
```

```

    '[object Arguments]': true,
  };
  const mapTag = '[object Map]';
  const setTag = '[object Set]';
  const boolTag = '[object Boolean]';
  const numberTag = '[object Number]';
  const stringTag = '[object String]';
  const symbolTag = '[object Symbol]';
  const dateTag = '[object Date]';
  const errorTag = '[object Error]';
  const regexpTag = '[object RegExp]';
  const funcTag = '[object Function]';

  const handleRegExp = (target) => {
    const { source, flags } = target;
    return new target.constructor(source, flags);
  }

  const handleFunc = (func) => {
    // 箭头函数直接返回自身
    if(!func.prototype) return func;
    const bodyReg = /^(?<={})(.|\n)+(?=})/m;
    const paramReg = /^(?<=\\(\\).+(?=\\)\\s+{)/;
    const funcString = func.toString();
    // 分别匹配 函数参数 和 函数体
    const param = paramReg.exec(funcString);
    const body = bodyReg.exec(funcString);
    if(!body) return null;
    if (param) {
      const paramArr = param[0].split(',');
      return new Function(...paramArr, body[0]);
    } else {
      return new Function(body[0]);
    }
  }

  const handleNotTraverse = (target, tag) => {
    const Ctor = target.constructor;
    switch(tag) {
      case boolTag:
        return new Object(Boolean.prototype.valueOf.call(target));
      case numberTag:
        return new Object(Number.prototype.valueOf.call(target));
      case stringTag:
        return new Object(String.prototype.valueOf.call(target));
      case symbolTag:
        return new Object(Symbol.prototype.valueOf.call(target));
      case errorTag:
      case dateTag:
        return new Ctor(target);
      case regexpTag:
        return handleRegExp(target);
      case funcTag:
        return handleFunc(target);
      default:
        return new Ctor(target);
    }
  }
}

```

```
const deepClone = (target, map = new WeakMap()) => {
  if(!isObject(target))
    return target;
  let type = getType(target);
  let cloneTarget;
  if(!canTraverse[type]) {
    // 处理不能遍历的对象
    return handleNotTraverse(target, type);
  }else {
    // 这波操作相当关键，可以保证对象的原型不丢失！
    let ctor = target.constructor;
    cloneTarget = new ctor();
  }

  if(map.get(target))
    return target;
  map.set(target, true);

  if(type === mapTag) {
    //处理Map
    target.forEach((item, key) => {
      cloneTarget.set(deepClone(key, map), deepClone(item, map));
    })
  }

  if(type === setTag) {
    //处理Set
    target.forEach(item => {
      cloneTarget.add(deepClone(item, map));
    })
  }

  // 处理数组和对象
  for (let prop in target) {
    if (target.hasOwnProperty(prop)) {
      cloneTarget[prop] = deepClone(target[prop], map);
    }
  }
  return cloneTarget;
}
```

## 43. 数据是如何存储的？

基本数据类型用 **栈** 存储，引用数据类型用 **堆** 存储。

看起来没有错误，但实际上是有问题的。可以考虑一下闭包的情况，如果变量存在栈中，那函数调用完 **栈顶空间销毁**，闭包变量不就没了吗？

其实还是需要补充一句：

闭包变量是存在堆内存中的。

具体而言，以下数据类型存储在栈中：

- boolean
- null

- undefined
- number
- string
- symbol
- bigint

而所有的对象数据类型存放在堆中。

值得注意的是，对于赋值操作，原始类型的数据直接完整地复制变量值，对象数据类型的数据则是复制引用地址。

因此会有下面的情况：

```
let obj = { a: 1 };  
let newObj = obj;  
newObj.a = 2;  
console.log(obj.a); //变成了2
```

之所以会这样，是因为 obj 和 newObj 是同一份堆空间的地址，改变newObj，等于改变了共同的堆内存，这时候通过 obj 来获取这块内存的值当然会改变。

为什么不全部用栈来保存呢？

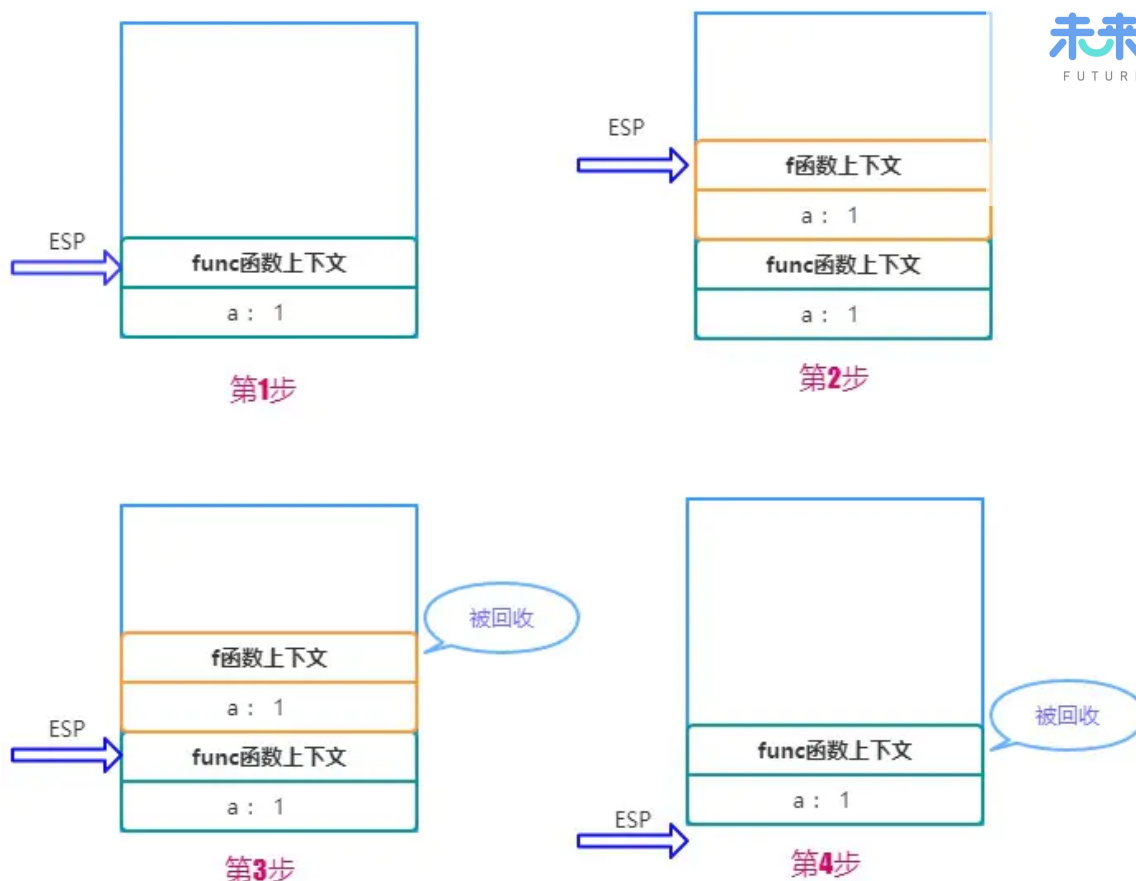
首先，对于系统栈来说，它的功能除了保存变量之外，还有创建并切换函数执行上下文的功能。举个例子：

```
function f(a) {  
  console.log(a);  
}  
  
function func(a) {  
  f(a);  
}  
  
func(1);
```

假设用ESP指针来保存当前的执行状态，在系统栈中会产生如下的过程：

- 1) 调用func, 将 func 函数的上下文压栈，ESP指向栈顶。
- 2) 执行func, 又调用f函数，将 f 函数的上下文压栈，ESP 指针上移。
- 3) 执行完 f 函数，将ESP 下移，f函数对应的栈顶空间被回收。
- 4) 执行完 func, ESP 下移，func对应的空间被回收。

图示如下：



因此你也看到了，如果采用栈来存储相对基本类型更加复杂的对象数据，那么切换上下文的开销将变得巨大！

不过堆内存虽然空间大，能存放大量的数据，但与此同时垃圾内存的回收会带来更大的开销。

## 44. V8 引擎如何进行垃圾内存的回收？

JS 语言不像 C/C++，让程序员自己去开辟或者释放内存，而是类似 Java，采用自己的一套垃圾回收算法进行自动的内存管理。作为一名资深的前端工程师，对于 JS 内存回收的机制是需要非常清楚，以便于在极端的环境下能够分析出系统性能的瓶颈，另一方面，学习这其中的机制，也对我们深入理解 JS 的闭包特性、以及对内存的高效使用，都有很大的帮助。

### V8 内存限制

在其他的后端语言中，如 Java/Go，对于内存的使用没有什么限制，但是 JS 不一样，V8 只能使用系统的一部分内存，具体来说，在 64 位系统下，V8 最多只能分配 1.4G，在 32 位系统中，最多只能分配 0.7G。你想想在前端这样的大内存需求其实并不大，但对于后端而言，nodejs 如果遇到一个 2G 多的文件，那么将无法全部将其读入内存进行各种操作了。

我们知道对于栈内存而言，当 ESP 指针下移，也就是上下文切换之后，栈顶的空间会自动被回收。但对于堆内存而言就比较复杂了，我们下面着重分析堆内存的垃圾回收。

所有的对象类型的数据在 JS 中都是通过堆进行空间分配的。当我们构造一个对象进行赋值操作的时候，其实相应的内存已经分配到了堆上。你可以不断的这样创建对象，让 V8 为它分配空间，直到堆的大小达到上限。

那么问题来了，V8 为什么要给它设置内存上限？明明我的机器大几十 G 的内存，只能让我用这么一点？



究其根本，是由两个因素所共同决定的，一个是JS单线程的执行机制，另一个是JS垃圾回收机制的限制。

首先JS是单线程运行的，这意味着一旦进入到垃圾回收，那么其它的各种运行逻辑都要暂停；另一方面垃圾回收其实是非常耗时间的操作，V8 官方是这样形容的：

以 1.5GB 的垃圾回收堆内存为例，V8 做一次小的垃圾回收需要50ms 以上，做一次非增量式(ps: 后面会解释)的垃圾回收甚至要 1s 以上。

可见其耗时之久，而且在这么长的时间内，我们的JS代码执行会一直没有响应，造成应用卡顿，导致应用性能和响应能力直线下降。因此，V8 做了一个简单粗暴的选择，那就是限制堆内存，也算是一种权衡的手段，因为大部分情况是不会遇到操作几个G内存这样的场景的。

不过，如果你想调整这个内存的限制也不是不行。配置命令如下：

```
// 这是调整老生代这部分的内存，单位是MB。后面会详细介绍新生代和老生代内存
node --max-old-space-size=2048 xxx.js
```

或者

```
// 这是调整新生代这部分的内存，单位是 KB。
node --max-new-space-size=2048 xxx.js
```

## 新生代内存的回收

V8 把堆内存分成了两部分进行处理——新生代内存和老生代内存。顾名思义，新生代就是临时分配的内存，存活时间短，老生代是常驻内存，存活的时间长。V8 的堆内存，也就是两个内存之和。

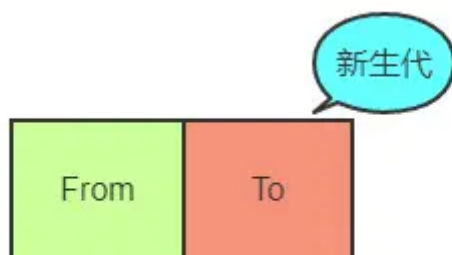


根据这两种不同种类的堆内存，V8 采用了不同的回收策略，来根据不同的场景做针对性的优化。

首先是新生代的内存，刚刚已经介绍了调整新生代内存的方法，那它的内存默认限制是多少？在 64 位和 32 位系统下分别为 32MB 和 16MB。够小吧，不过也很好理解，新生代中的变量存活时间短，来了马上就走，不容易产生太大的内存负担，因此可以将它设的足够小。

那好了，新生代的垃圾回收是怎么做的呢？

首先将新生代内存空间一分为二：



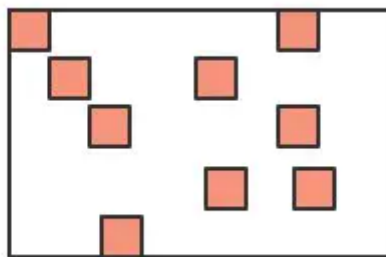
其中From部分表示正在使用的内存，To 是目前闲置的内存。

当进行垃圾回收时，V8 将From部分的对象检查一遍，如果是存活对象那么复制到To内存中(在To内存中按照顺序从头放置的)，如果是非存活对象直接回收即可。

当所有的From中的存活对象按照顺序进入到To内存之后，From 和 To 两者的角色对调，From现在被闲置，To为正在使用，如此循环。

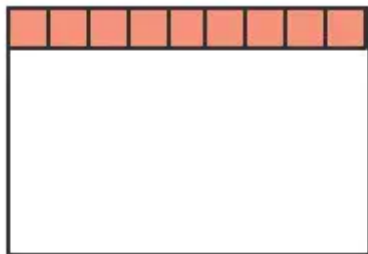
那你很可能会问了，直接将非存活对象回收了不就万事大吉了嘛，为什么还要后面的一系列操作？

注意，我刚刚特别说明了，在To内存中按照顺序从头放置的，这是为了应对这样的场景：



深色的小方块代表存活对象，白色部分表示待分配的内存，由于堆内存是连续分配的，这样零零散散的空间可能会导致稍微大一点的对象没有办法进行空间分配，这种零散的空间也叫做**内存碎片**。刚刚介绍的新生代垃圾回收算法也叫**Scavenge算法**。

Scavenge 算法主要就是解决内存碎片的问题，在进行一顿复制之后，To空间变成了这个样子：



是不是整齐了许多？这样就大大方便了后续连续空间的分配。

不过Scavenge 算法的劣势也非常明显，就是内存只能使用新生代内存的一半，但是它只存放生命周期短的对象，这种对象一般很少，因此时间性能非常优秀。

## 老年代内存的回收

刚刚介绍了新生代的回收方式，那么新生代中的变量如果经过多次回收后依然存在，那么就会被放入到老年代内存中，这种现象就叫**晋升**。

发生晋升其实不只是这一种原因，我们来梳理一下会有那些情况触发晋升：

- 已经经历过一次 Scavenge 回收。
- To（闲置）空间的内存占用超过25%。

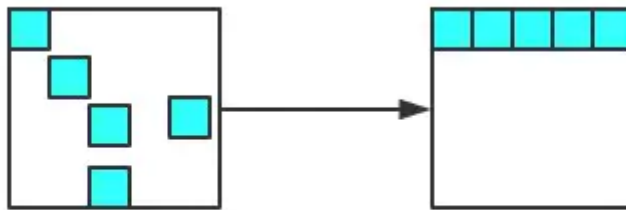
现在进入到老年代的垃圾回收机制当中，老年代中累积的变量空间一般都是很大的，当然不能用 Scavenge 算法啦，浪费一半空间不说，对庞大的内存空间进行复制岂不是劳民伤财？

那么对于老年代而言，究竟是采取怎样的策略进行垃圾回收的呢？

第一步，进行标记-清除。这个过程在《JavaScript高级程序设计(第三版)》中有过详细的介绍，主要分成两个阶段，即标记阶段和清除阶段。首先会遍历堆中的所有对象，对它们做上标记，然后对于代码环境中使用的变量以及被强引用的变量取消标记，剩下的就是要删除的变量了，在随后的清除阶段对其进行空间的回收。

当然这又会引发内存碎片的问题，存活对象的空间不连续对后续的空间分配造成障碍。老年代又是如何处理这个问题的呢？

第二步，整理内存碎片。V8 的解决方式非常简单粗暴，在清除阶段结束后，把存活的对象全部往一端靠拢。



由于是移动对象，它的执行速度不可能很快，事实上也是整个过程中最耗时间的部分。

### 增量标记

由于JS的单线程机制，V8 在进行垃圾回收的时候，不可避免地会阻塞业务逻辑的执行，倘若老生代的垃圾回收任务很重，那么耗时会非常可怕，严重影响应用的性能。那这个时候为了避免这样问题，V8 采取了增量标记的方案，即将一口气完成的标记任务分为很多小的部分完成，每做完一个小的部分就“歇”一下，就js应用逻辑执行一会儿，然后再执行下面的部分，如果循环，直到标记阶段完成才进入内存碎片的整理上面来。其实这个过程跟React Fiber的思路有点像，这里就不展开了。

经过增量标记之后，垃圾回收过程对JS应用的阻塞时间减少到原来了1 / 6, 可以看到，这是一个非常成功的改进。

## 45. 描述一下 V8 执行一段JS代码的过程？

前端相对来说是一个比较新兴的领域，因此各种前端框架和工具层出不穷，让人眼花缭乱，尤其是各大厂商推出 小程序 之后 各自制定标准，让前端开发的工作更加繁琐，在此背景下为了抹平平台之间的差异，诞生的各种 编译工具/框架 也数不胜数。但无论如何，想要赶上这些框架和工具的更新速度是非常难的，即使赶上了也很难产生自己的 技术积淀，一个更好的方式便是学习那些 本质的知识，抓住上层应用中不变的 底层机制，这样我们便能轻松理解上层的框架而不仅仅是被动地使用，甚至能够在适当的场景下自己造出轮子，以满足开发效率的需求。

站在 V8 的角度，理解其中的执行机制，也能够帮助我们理解很多的上层应用，包括Babel、Eslint、前端框架的底层机制。那么，一段 JavaScript 代码放在 V8 当中究竟是如何执行的呢？

首先需要明白的是，机器是读不懂 JS 代码，机器只能理解特定的机器码，那如果要想让 JS 的逻辑在机器上运行起来，就必须将 JS 的代码翻译成机器码，然后让机器识别。JS属于解释型语言，对于解释型的语言说，解释器会对源代码做如下分析：

- 通过词法分析和语法分析生成 AST(抽象语法树)
- 生成字节码

然后解释器根据字节码来执行程序。但 JS 整个执行的过程其实会比这个更加复杂，接下来就来——地拆解。

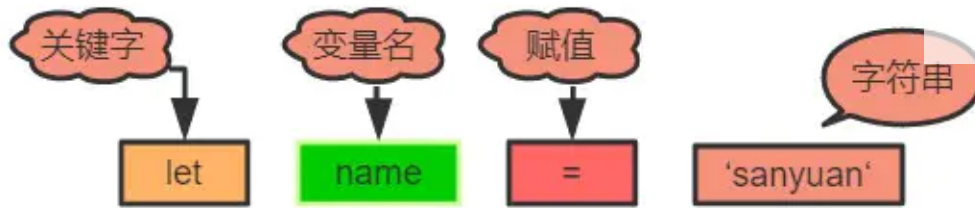
### 生成 AST

生成 AST 分为两步——词法分析和语法分析。

词法分析即分词，它的工作就是将一行行的代码分解成一个个token。比如下面一行代码：

```
let name = 'sanyuan'
```

其中会把句子分解成四个部分：

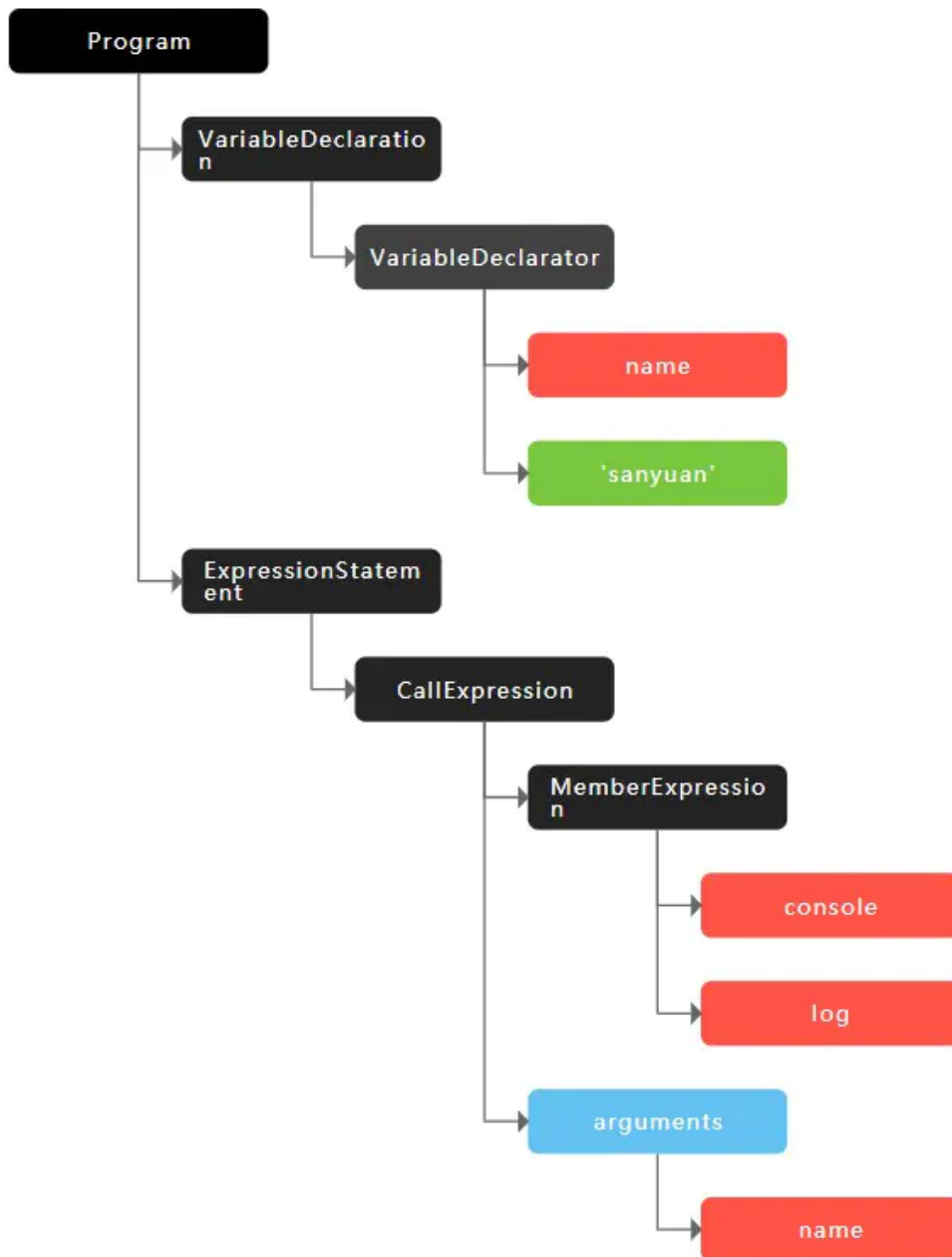


即解析成了四个token，这就是词法分析的作用。

接下来语法分析阶段，将生成的这些 token 数据，根据一定的语法规则转化为AST。举个例子：

```
let name = 'sanyuan'  
console.log(name)
```

最后生成的 AST 是这样的：



当生成了 AST 之后，编译器/解释器后续的工作都要依靠 AST 而不是源代码。顺便补充一句，babel 的工作原理就是将 ES6 的代码解析生成 ES6 的 AST，然后将 ES6 的 AST 转换为 ES5 的 AST，最后才将 ES5 的 AST 转化为具体的 ES5 代码。

生成 AST 后，接下来会生成执行上下文

## 生成字节码

开头就已经提到过了，生成 AST 之后，直接通过 V8 的解释器(也叫 Ignition)来生成字节码。但是字节码并不能让机器直接运行，那你可能就会说了，不能执行还转成字节码干嘛，直接把 AST 转换成机器码不就得了，让机器直接执行。确实，在 V8 的早期是这么做的，但后来因为机器码的体积太大，引发了严重的内存占用问题。

给一张对比图让大家直观地感受以下三者代码量的差异:



很容易得出, 字节码是比机器码轻量得多的代码。那 V8 为什么要使用字节码, 字节码到底是个什么东西?

字节码是介于AST 和 机器码之间的一种代码, 但是与特定类型的机器码无关, 字节码需要通过解释器将其转换为机器码然后执行。

字节码仍然需要转换为机器码, 但和原来不同的是, 现在不用一次性将全部的字节码都转换成机器码, 而是通过解释器来逐行执行字节码, 省去了生成二进制文件的操作, 这样就大大降低了内存的压力。

## 执行代码

在执行字节码的过程中, 如果发现某一部分代码重复出现, 那么 V8 将它记做 **热点代码 (HotSpot)**, 然后将这么代码编译成 **机器码** 保存起来, 这个用来编译的工具就是V8的 **编译器** (也叫做 **TurboFan**), 因此在这样的机制下, 代码执行的时间越久, 那么执行效率会越来越高, 因为有越来越多的字节码被标记为 **热点代码**, 遇到它们时直接执行相应的机器码, 不用再次将转换为机器码。

其实当你听到有人说 JS 就是一门解释器语言的时候, 其实这个说法是有问题的。因为字节码不仅配合了解释器, 而且还和编译器打交道, 所以 JS 并不是完全的解释型语言。而编译器和解释器的 根本区别在于前者会编译生成二进制文件但后者不会。

并且, 这种字节码跟编译器和解释器结合的技术, 我们称之为 **即时编译**, 也就是我们经常听到的 **JIT**。

这就是 V8 中执行一段JS代码的整个过程, 梳理一下:

- 1) 首先通过词法分析和语法分析生成 **AST**
- 2) 将 **AST** 转换为字节码
- 3) 由解释器逐行执行字节码, 遇到热点代码启动编译器进行编译, 生成对应的机器码, 以优化执行效率

## 46. 宏任务(MacroTask)引入

在 JS 中, 大部分的任务都是在主线程上执行, 常见的任务有:

- 1) 渲染事件
- 2) 用户交互事件
- 3) js脚本执行
- 4) 网络请求、文件读写完成事件等等。

为了让这些事件有条不紊地进行, JS引擎需要对之执行的顺序做一定的安排, V8 其实采用的是一种 **队列** 的方式来存储这些任务, 即先进来的先执行。模拟如下:

```
bool keep_running = true;  
void MainTherad(){  
    for(;;){
```

```
//执行队列中的任务
Task task = task_queue.takeTask();
ProcessTask(task);

//执行延迟队列中的任务
ProcessDelayTask()

if(!keep_running) //如果设置了退出标志，那么直接退出线程循环
    break;
}
}
```

这里用到了一个 for 循环，将队列中的任务一一取出，然后执行，这个很好理解。但是其中包含了两种任务队列，除了上述提到的任务队列，还有一个延迟队列，它专门处理诸如setTimeout/setInterval这样的定时器回调任务。

上述提到的，普通任务队列和延迟队列中的任务，都属于**宏任务**。

## 47. 微任务(MicroTask)引入

对于每个宏任务而言，其内部都有一个微任务队列。那为什么要引入微任务？微任务在什么时候执行呢？

其实引入微任务的初衷是为了解决异步回调的问题。想一想，对于异步回调的处理，有多少种方式？总结起来有两点：

- 1) 将异步回调进行宏任务队列的入队操作。
- 2) 将异步回调放到当前宏任务的末尾。

如果采用第一种方式，那么执行回调的时机应该是在前面所有的宏任务完成之后，倘若现在的任务队列非常长，那么回调迟迟得不到执行，造成应用卡顿。

为了规避这样的问题，V8 引入了第二种方式，这就是微任务的解决方式。在每一个宏任务中定义一个**微任务队列**，当该宏任务执行完成，会检查其中的微任务队列，如果为空则直接执行下一个宏任务，如果不为空，则依次执行微任务，执行完成才去执行下一个宏任务。

常见的微任务有MutationObserver、Promise.then(或.reject) 以及以 Promise 为基础开发的其他技术(比如fetch API)，还包括 V8 的垃圾回收过程。

Ok, 这便是宏任务和微任务的概念，接下来正式介绍JS非常重要的运行机制——EventLoop。

## 48. 理解EventLoop：浏览器

例子：

```
console.log('start');
setTimeout(() => {
    console.log('timeout');
});
Promise.resolve().then(() => {
    console.log('resolve');
});
console.log('end');
```



分析一下:

- 1) 刚开始整个脚本作为一个宏任务来执行, 因此先打印start和end
- 2) setTimeout 作为一个宏任务放入宏任务队列
- 3) Promise.then作为一个为微任务放入到微任务队列
- 4) 当本次宏任务执行完, 检查微任务队列, 发现一个Promise.then, 执行
- 5) 接下来进入到下一个宏任务——setTimeout, 执行

因此最后的顺序是:

```
start
end
resolve
timeout
```

这样就带大家直观地感受到了浏览器环境下 EventLoop 的执行流程。不过, 这只是其中的一部分情况, 接下来我们来做一个更完整的总结。

- 1) 一开始整段脚本作为第一个宏任务执行
- 2) 执行过程中同步代码直接执行, 宏任务进入宏任务队列, 微任务进入微任务队列
- 3) 当前宏任务执行完出队, 检查微任务队列, 如果有则依次执行, 直到微任务队列为空
- 4) 执行浏览器 UI 线程的渲染工作
- 5) 检查是否有Web worker任务, 有则执行
- 6) 执行队首新的宏任务, 回到2, 依此循环, 直到宏任务和微任务队列都为空

最后留一道题目练习:

```
Promise.resolve().then(()=>{
  console.log('Promise1')
  setTimeout(()=>{
    console.log('setTimeout2')
  },0)
});
setTimeout(()=>{
  console.log('setTimeout1')
  Promise.resolve().then(()=>{
    console.log('Promise2')
  })
},0);
console.log('start');

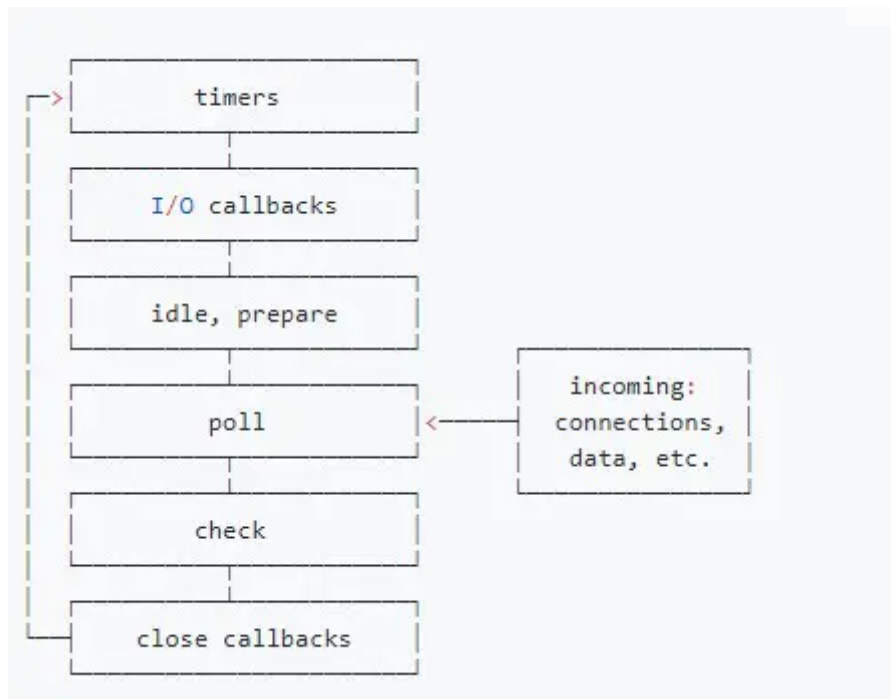
// start
// Promise1
// setTimeout1
// Promise2
// setTimeout2
```



## 49. 理解EventLoop: nodejs

nodejs 和 浏览器的 eventLoop 还是有很大差别的，值得单独拿出来说一说。

不知你是否看过关于 nodejs 中 eventLoop 的一些文章, 是否被这些流程图搞得眼花缭乱、一头雾水:



看到这你不用紧张，这里会抛开这些晦涩的流程图，以最清晰浅显的方式来一步步拆解 nodejs 的事件循环机制。

### 三大关键阶段

首先，梳理一下 nodejs 三个非常重要的执行阶段:

- 1) 执行 **定时器回调** 的阶段。检查定时器，如果到了时间，就执行回调。这些定时器就是 `setTimeout`、`setInterval`。这个阶段暂且叫它 **timer**。
- 2) 轮询(英文叫 **poll**)阶段。因为在node代码中难免会有异步操作，比如文件I/O，网络I/O等等，那么当这些异步操作做完了，就会来通知JS主线程，怎么通知呢？就是通过'data'、'connect'等事件使得事件循环到达 **poll** 阶段。到达了这个阶段后:

如果当前已经存在定时器，而且有定时器到时间了，拿出来执行，eventLoop 将回到timer阶段。

如果没有定时器, 会去看回调函数队列。

- 如果队列不为空，拿出队列中的方法依次执行
- 如果队列为空，检查是否有 `setImmediate` 的回调
  - 有则前往check阶段
  - 没有则继续等待，相当于阻塞了一段时间(阻塞时间是有上限的), 等待 `callback` 函数加入队列，加入后会立刻执行。一段时间后自动进入 `check` 阶段。

- 3) `check` 阶段。这是一个比较简单的阶段，直接执行 `setImmediate` 的回调。

这三个阶段为一个循环过程。不过现在的eventLoop并不完整，我们现在就来——地完善。

### 完善

首先，当第 1 阶段结束后，可能并不会立即等待到异步事件的响应，这时候 nodejs 会进入到 [check 阶段](#)。比如说 TCP 连接遇到 ECONNREFUSED，就会在这个时候执行回调。

并且在 check 阶段结束后还会进入到 [关闭事件的回调阶段](#)。如果一个 socket 或句柄 (handle) 被突然关闭，例如 socket.destroy(), 'close' 事件的回调就会在这个阶段执行。

梳理一下，nodejs 的 eventLoop 分为下面的几个阶段：

1) timer 阶段

2) I/O 异常回调阶段

空闲、预备状态(第2阶段结束，poll 未触发之前)

3) poll 阶段

4) check 阶段

5) 关闭事件的回调阶段

## 实例演示

```
setTimeout(()=>{
  console.log('timer1')
  Promise.resolve().then(function() {
    console.log('promise1')
  })
}, 0)
setTimeout(()=>{
  console.log('timer2')
  Promise.resolve().then(function() {
    console.log('promise2')
  })
}, 0)
```

node版本 >= 11和在 11 以下的会有不同的表现。

首先说 node 版本 >= 11的，它会和浏览器表现一致，一个定时器运行完立即运行相应的微任务。

```
timer1
promise1
timer2
promise2
```

而 node 版本小于 11 的情况下，对于定时器的处理是：

若第一个定时器任务出队并执行完，发现队首的任务仍然是一个定时器，那么就将微任务暂时保存，直接去执行新的定时器任务，当新的定时器任务执行完后，再一一执行中途产生的微任务。

因此会打印出这样的结果：

```
timer1
timer2
promise1
promise2
```

## 50. nodejs 和 浏览器关于eventLoop的主要区别

两者最主要的区别在于浏览器中的微任务是在 每个相应的宏任务 中执行的，而nodejs中的微任务是在不同阶段之间 执行的。

## 51. 关于process.nextTick的一点说明

process.nextTick 是一个独立于 eventLoop 的任务队列。

在每一个 eventLoop 阶段完成后会去检查这个队列，如果里面有任务，会让这部分任务 优先于微任务 执行。

## 52. nodejs中的异步、非阻塞I/O是如何实现的？

在听到 nodejs 相关的特性时，经常会对 异步I/O、非阻塞I/O有所耳闻，听起来好像是差不多的意思，但其实是两码事，下面我们就以原理的角度来剖析一下对 nodejs 来说，这两种技术底层是如何实现的？

### 什么是I/O？

I/O 即Input/Output, 输入和输出的意思。在浏览器端，只有一种 I/O，那就是利用 Ajax 发送网络请求，然后读取返回的内容，这属于 网络I/O。回到 nodejs 中，其实这种的 I/O 的场景就更加广泛了，主要分为两种：

- 文件 I/O。比如用 fs 模块对文件进行读写操作。
- 网络 I/O。比如 http 模块发起网络请求。

### 阻塞和非阻塞I/O

阻塞 和 非阻塞 I/O 其实是针对操作系统内核而言的，而不是 nodejs 本身。阻塞 I/O 的特点就是一定要等到操作系统完成所有操作后才表示调用结束，而非阻塞 I/O 是调用后立马返回，不用等操作系统内核完成操作。

对前者而言，在操作系统进行 I/O 的操作的过程中，我们的应用程序其实是一直处于等待状态的，什么都做不了。那如果换成 非阻塞I/O，调用返回后我们的 nodejs 应用程序可以完成其他的事情，而操作系统同时也在进行 I/O。这样就把等待的时间充分利用了起来，提高了执行效率，但是同时又会产生一个问题，nodejs 应用程序怎么知道操作系统已经完成了 I/O 操作呢？

为了让 nodejs 知道操作系统已经做完 I/O 操作，需要重复地去操作系统那里判断一下是否完成，这种重复判断的方式就是 轮询。对于轮询而言，有以下这么几种方案：

- 1) 一直轮询检查I/O状态，直到 I/O 完成。这是最原始的方式，也是性能最低的，会让 CPU 一直耗用在等待上面。其实跟阻塞 I/O 的效果是一样的。
- 2) 遍历文件描述符(即 文件I/O 时操作系统和 nodejs 之间的文件凭证)的方式来原因 I/O 是否完成，I/O 完成则文件描述符的状态改变。但 CPU 轮询消耗还是很大。
- 3) epoll模式。即在进入轮询的时候如果I/O未完成CPU就休眠，完成之后唤醒CPU。

总之，CPU要么重复检查I/O，要么重复检查文件描述符，要么休眠，都得不到很好的利用，我们希望的是：

这是理想的情况，也是异步 I/O 的效果，那如何实现这样的效果呢？

## 异步 I/O 的本质

Linux 原生存在这样的一种方式，即(AIO), 但两个致命的缺陷:

- 1) 只有 Linux 下存在，在其他系统中没有异步 I/O 支持。
- 2) 无法利用系统缓存。

## nodejs中的异步 I/O 方案

是不是没有办法了呢？在单线程的情况下确实是这样，但是如果把思路放开一点，利用多线程来考虑这个问题，就变得轻松多了。我们可以让一个进程进行计算操作，另外一些进行 I/O 调用，I/O 完成后把信号传给计算的线程，进而执行回调，这不就好了吗？没错，异步 I/O 就是使用这样的线程池来实现的。

只不过在不同的系统下面表现会有所差异，在 Linux 下可以直接使用线程池来完成，在Window系统下则采用 IOCP 这个系统API(其内部还是用线程池完成的)。

有了操作系统的支持，那 nodejs 如何来对接这些操作系统从而实现异步 I/O 呢？

以文件为 I/O 我们以一段代码为例:

```
let fs = require('fs');

fs.readFile('/test.txt', function (err, data) {
  console.log(data);
});
```

## 执行流程

执行代码的过程中大概发生了这些事情:

- 1) 首先，fs.readFile调用Node的核心模块fs.js；
- 2) 接下来，Node的核心模块调用内建模块node\_file.cc，创建对应的文件I/O观察者对象(这个对象后面有大用！)；
- 3) 最后，根据不同平台（Linux或者window），内建模块通过libuv中间层进行系统调用

## libuv调用过程拆解

重点来了！libuv 中是如何来进行进行系统调用的呢？也就是 uv\_fs\_open() 中做了些什么？

### (1) 创建请求对象

以Windows系统为例来说，在这个函数的调用过程中，我们创建了一个文件I/O的请求对象，并往里面注入了回调函数。

```
req_wrap->object->Set(oncomplete_sym, callback);
```

req\_wrap 便是这个请求对象，req\_wrap 中 object\_ 的 oncomplete\_sym 属性对应的值便是我们 nodejs 应用程序代码中传入的回调函数。

## (2) 推入线程池，调用返回

在这个对象包装完成后，QueueUserWorkItem() 方法将这个对象推进线程池中等待执行。

好，至此现在js的调用就直接返回了，我们的js应用程序代码可以继续往下执行，当然，当前的 I/O 操作同时也在线程池中将被执行，这不就完成了异步么：)

等等，别高兴太早，回调都还没执行呢！接下来便是执行回调通知的环节。

## (3) 回调通知

事实上现在线程池中的 I/O 无论是阻塞还是非阻塞都已经无所谓了，因为异步的目的已经达成。重要的是 I/O 完成后会发生什么。

在介绍后续的故事之前，给大家介绍两个重要的方法：GetQueuedCompletionStatus 和 PostQueuedCompletionStatus。

1) 还记得之前讲过的 eventLoop 吗？在每一个Tick当中会调用 GetQueuedCompletionStatus 检查线程池中是否有执行完的请求，如果有则表示时机已经成熟，可以执行回调了。

2) PostQueuedCompletionStatus 方法则是向 IOCP 提交状态，告诉它当前I/O完成了。

把后面的过程串联起来。

当对应线程中的 I/O 完成后，会将获得的结果 存储 起来，保存到 相应的请求对象 中，然后调用 PostQueuedCompletionStatus() 向 IOCP 提交执行完成的状态，并且将线程还给操作系统。一旦 EventLoop 的轮询操作中，调用 GetQueuedCompletionStatus 检测到了完成的状态，就会把 请求对象 塞给I/O观察者(之前埋下伏笔，如今终于闪亮登场)。

I/O 观察者现在的行为就是取出 请求对象 的 存储结果，同时也取出它的 oncomplete\_sym 属性，即回调函数(不懂这个属性的回看第1步的操作)。将前者作为函数参数传入后者，并执行后者。这里，回调函数就成功执行啦！

总结：

1) 阻塞 和 非阻塞 I/O 其实是针对操作系统内核而言的。阻塞 I/O 的特点就是一定要等到操作系统完成所有操作后才表示调用结束，而非阻塞 I/O 是调用后立马返回，不用等操作系统内核完成操作。

2) nodejs中的异步 I/O 采用多线程的方式，由 EventLoop、I/O 观察者，请求对象、线程池 四大要素相互配合，共同实现。

## 53. JS异步编程有哪些方案？为什么会出现这些方案？

关于JS 单线程、EventLoop 以及 异步 I/O 这些底层的特性，我们之前做过了详细的拆解，不在赘述。在探究了底层机制之后，我们还需要对代码的组织方式有所理解，这是离我们最日常开发最接近的部分，异步代码的组织方式直接决定了 开发和 维护 的效率，其重要性也不可小觑。尽管底层机制没变，但异步代码的组织方式却随着 ES 标准的发展，一步步发生了巨大的 变革。接着让我们一起来一探究竟吧！

### 回调函数时代

相信很多 nodejs 的初学者都或多或少踩过这样的坑，node 中很多原生的 api 就是诸如这样的：

```
fs.readFile('xxx', (err, data) => {  
  });
```

典型的高阶函数，将回调函数作为函数参数传给了readFile。但久而久之，就会发现，这种传入回调的方式也存在大坑，比如下面这样：

```
fs.readFile('1.json', (err, data) => {  
  fs.readFile('2.json', (err, data) => {  
    fs.readFile('3.json', (err, data) => {  
      fs.readFile('4.json', (err, data) => {  
  
        });  
      });  
    });  
  });  
});
```

回调当中嵌套回调，也称 **回调地狱**。这种代码的可读性和可维护性都是非常差的，因为嵌套的层级太多。而且还有一个严重的问题，就是每次任务可能会失败，需要在回调里面对每个任务的失败情况进行处理，增加了代码的混乱程度。

## Promise 时代

ES6 中新增的 Promise 就很好解决了 **回调地狱** 的问题，同时合并了错误处理。写出来的代码类似于下面这样：

```
readFilePromise('1.json').then(data => {  
  return readFilePromise('2.json')  
}).then(data => {  
  return readFilePromise('3.json')  
}).then(data => {  
  return readFilePromise('4.json')  
});
```

以链式调用的方式避免了大量的嵌套，也符合人的线性思维方式，大大方便了异步编程。

## co + Generator 方式

利用协程完成 Generator 函数，用 co 库让代码依次执行完，同时以同步的方式书写，也让异步操作按顺序执行。

```
co(function* () {  
  const r1 = yield readFilePromise('1.json');  
  const r2 = yield readFilePromise('2.json');  
  const r3 = yield readFilePromise('3.json');  
  const r4 = yield readFilePromise('4.json');  
})
```

## async + await方式

这是 ES7 中新增的关键字，凡是加上 `async` 的函数都默认返回一个 `Promise` 对象，而更重要的是 `async` + `await` 也能让异步代码以同步的方式来书写，而不需要借助第三方库的支持。

```
const readFileAsync = async function () {
  const f1 = await readFilePromise('1.json')
  const f2 = await readFilePromise('2.json')
  const f3 = await readFilePromise('3.json')
  const f4 = await readFilePromise('4.json')
}
```

## 54. 能不能简单实现一下 node 中回调函数的机制？

回调函数的方式其实内部利用了发布-订阅模式，在这里我们以模拟实现 node 中的 `Event` 模块为例来写实现回调函数的机制。

```
function EventEmitter() {
  this.events = new Map();
}
```

这个 `EventEmitter` 一共需要实现这些方法: `addListener`, `removeListener`, `once`, `removeAllListener`, `emit`。

首先是 `addListener`:

```
// once 参数表示是否只是触发一次
const wrapCallback = (fn, once = false) => ({ callback: fn, once });

EventEmitter.prototype.addListener = function (type, fn, once = false) {
  let handler = this.events.get(type);
  if (!handler) {
    // 为 type 事件绑定回调
    this.events.set(type, wrapCallback(fn, once));
  } else if (handler && typeof handler.callback === 'function') {
    // 目前 type 事件只有一个回调
    this.events.set(type, [handler, wrapCallback(fn, once)]);
  } else {
    // 目前 type 事件回调数 >= 2
    handler.push(wrapCallback(fn, once));
  }
}
```

`removeListener` 的实现如下:

```
EventEmitter.prototype.removeListener = function (type, listener) {
  let handler = this.events.get(type);
  if (!handler) return;
  if (!Array.isArray(handler)) {
    if (handler.callback === listener.callback) this.events.delete(type);
    else return;
  }
  for (let i = 0; i < handler.length; i++) {
    let item = handler[i];
    if (item.callback === listener.callback) {
```



```
// 删除该回调，注意数组塌陷的问题，即后面的元素会往前挪一位。i 要 --
handler.splice(i, 1);
i--;
if (handler.length === 1) {
  // 长度为 1 就不用数组存了
  this.events.set(type, handler[0]);
}
}
}
}
```

once 实现思路很简单，先调用 addListener 添加上了 once 标记的回调对象，然后在 emit 的时候遍历回调列表，将标记了 once: true 的项 remove 掉即可。

```
EventEmitter.prototype.once = function (type, fn) {
  this.addListener(type, fn, true);
}

EventEmitter.prototype.emit = function (type, ...args) {
  let handler = this.events.get(type);
  if (!handler) return;
  if (Array.isArray(handler)) {
    // 遍历列表，执行回调
    handler.map(item => {
      item.callback.apply(this, args);
      // 标记的 once: true 的项直接移除
      if (item.once) this.removeListener(type, item);
    })
  } else {
    // 只有一个回调则直接执行
    handler.callback.apply(this, args);
  }
  return true;
}
```

最后是 removeAllListener:

```
EventEmitter.prototype.removeAllListener = function (type) {
  let handler = this.events.get(type);
  if (!handler) return;
  else this.events.delete(type);
}
```

现在我们测试一下:

```
let e = new EventEmitter();
e.addListener('type', () => {
  console.log("type 事件触发!");
})
e.addListener('type', () => {
  console.log("wow! type 事件又触发了!");
})

function f() {
  console.log("type 事件我只触发一次");
}
```



```
e.once('type', f)
e.emit('type');
e.emit('type');
e.removeAllListener('type');
e.emit('type');
```

```
// type事件触发!
// wow!type事件又触发了!
// type事件我只触发一次
// type事件触发!
// wow!type事件又触发了!
```

一个简易的 Event 就这样实现完成了，为什么说它简易呢？因为还有很多细节的部分没有考虑：

- 1) 在 `参数少` 的情况下，call 的性能优于 apply，反之 apply 的性能更好。因此在执行回调时候可以根据情况调用 call 或者 apply。
- 2) 考虑到内存容量，应该设置 `回调列表的最大值`，当超过最大值的时候，应该选择部分回调进行删除操作。
- 3) 鲁棒性有待提高。对于参数的校验很多地方直接忽略掉了。

## 55. Promise 凭借什么消灭了回调地狱？

### 问题

首先，什么是回调地狱：

- 1) 多层嵌套的问题。
- 2) 每种任务的处理结果存在两种可能性（成功或失败），那么需要在每种任务执行结束后分别处理这两种可能性。

这两种问题在回调函数时代尤为突出。Promise 的诞生就是为了解决这两个问题。

### 解决方法

Promise 利用了三大技术手段来解决回调地狱：

- 回调函数延迟绑定。
- 返回值穿透。
- 错误冒泡。

首先来举个例子：

```
let readFilePromise = (filename) => {
  fs.readFile(filename, (err, data) => {
    if(err) {
      reject(err);
    } else {
      resolve(data);
    }
  })
}
readFilePromise('1.json').then(data => {
  return readFilePromise('2.json')
});
```

看到没有，回调函数不是直接声明的，而是在通过后面的 then 方法传入的，即延迟传入。这就是回调函数延迟绑定。

然后我们做以下微调：

```
let x = readFilePromise('1.json').then(data => {
  return readFilePromise('2.json') //这是返回的Promise
});
x.then(/* 内部逻辑省略 */)
```

我们会根据 then 中回调函数的传入值创建不同类型的 Promise，然后把返回的 Promise 穿透到外层，以供后续的调用。这里的 x 指的就是内部返回的 Promise，然后在 x 后面可以依次完成链式调用。

这便是返回值穿透的效果。

这两种技术一起作用便可以将深层的嵌套回调写成下面的形式：

```
readFilePromise('1.json').then(data => {
  return readFilePromise('2.json');
}).then(data => {
  return readFilePromise('3.json');
}).then(data => {
  return readFilePromise('4.json');
});
```

这样就显得清爽了许多，更重要的是，它更符合人的线性思维模式，开发体验也更好。

两种技术结合产生了 **链式调用** 的效果。

这解决的是多层嵌套的问题，那另一个问题，即每次任务执行结束后 **分别处理成功和失败** 的情况怎么解决的呢？

Promise 采用了 **错误冒泡** 的方式。其实很简单理解，我们来看看效果：

```
readFilePromise('1.json').then(data => {
  return readFilePromise('2.json');
}).then(data => {
  return readFilePromise('3.json');
}).then(data => {
  return readFilePromise('4.json');
}).catch(err => {
  // xxx
});
```

这样前面产生的错误会一直向后传递，被 catch 接收到，就不用频繁地检查错误了。

### 解决效果

- 实现链式调用，解决多层嵌套问题
- 实现错误冒泡后一站式处理，解决每次任务中判断错误、增加代码混乱度的问题

## 56. 为什么Promise要引入微任务？

Promise 中的执行函数是同步进行的，但是里面存在着异步操作，在异步操作结束后会调用 resolve 方法，或者中途遇到错误调用 reject 方法，这两者都是作为微任务进入到 EventLoop 中。但是你有没有想过，Promise 为什么要引入微任务的方式来进行回调操作？

### 解决方式

回到问题本身，其实就是如何处理回调的问题。总结起来有三种方式：

- 1) 使用同步回调，直到异步任务进行完，再进行后面的任务。
- 2) 使用异步回调，将回调函数放在进行 **宏任务队列** 的队尾。
- 3) 使用异步回调，将回调函数放到 **当前宏任务中** 的最后面。

### 优劣对比

第一种方式显然不可取，因为同步的问题非常明显，会让整个脚本阻塞住，当前任务等待，后面的任务都无法得到执行，而这部分等待的时间是可以拿来完成其他事情的，导致 CPU 的利用率非常低，而且还有另外一个致命的问题，就是无法实现延迟绑定的效果。

如果采用第二种方式，那么执行回调(resolve/reject)的时机应该是在前面 **所有的宏任务** 完成之后，倘若现在的任务队列非常长，那么回调迟迟得不到执行，造成 **应用卡顿**。

为了解决上述方案的问题，另外也考虑到延迟绑定的需求，Promise 采取第三种方式，即引入微任务，即把 resolve(reject) 回调的执行放在当前宏任务的末尾。

这样，利用微任务解决了两大痛点：

- 采用**异步回调**替代同步回调解决了浪费 CPU 性能的问题。
- 放到**当前宏任务最后**执行，解决了回调执行的实时性问题。

好，Promise 的基本实现思想已经讲清楚了，相信大家已经知道了它 **为什么这么设计**，接下来就让我们一步步弄清楚它内部到底是 **怎么设计的**。

## 57. Promise 如何实现链式调用？

### 简易版实现

首先写出第一版的代码：

```
//定义三种状态
const PENDING = "pending";
const FULFILLED = "fulfilled";
const REJECTED = "rejected";
```

```
function MyPromise(executor) {
  let self = this; // 缓存当前promise实例
  self.value = null;
  self.error = null;
  self.status = PENDING;
  self.onFulfilled = null; //成功的回调函数
  self.onRejected = null; //失败的回调函数

  const resolve = (value) => {
    if(self.status !== PENDING) return;
    setTimeout(() => {
      self.status = FULFILLED;
      self.value = value;
      self.onFulfilled(self.value); //resolve时执行成功回调
    });
  };

  const reject = (error) => {
    if(self.status !== PENDING) return;
    setTimeout(() => {
      self.status = REJECTED;
      self.error = error;
      self.onRejected(self.error); //resolve时执行成功回调
    });
  };

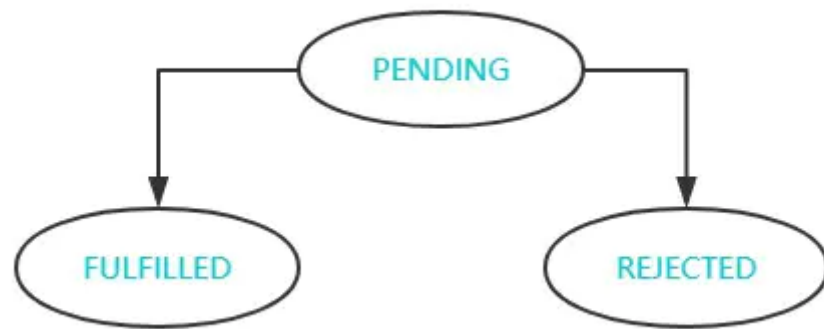
  executor(resolve, reject);
}

MyPromise.prototype.then = function(onFulfilled, onRejected) {
  if (this.status === PENDING) {
    this.onFulfilled = onFulfilled;
    this.onRejected = onRejected;
  } else if (this.status === FULFILLED) {
    //如果状态是fulfilled, 直接执行成功回调, 并将成功值传入
    onFulfilled(this.value)
  } else {
    //如果状态是rejected, 直接执行失败回调, 并将失败原因传入
    onRejected(this.error)
  }
  return this;
}
```

可以看到, Promise 的本质是一个有限状态机, 存在三种状态:

- PENDING(等待)
- FULFILLED(成功)
- REJECTED(失败)

状态改变规则如下图:



对于 Promise 而言, 状态的改变 **不可逆**, 即由等待态变为其他的状态后, 就无法再改变了。

不过, 回到目前这一版的 Promise, 还是存在一些问题的。

### 设置回调数组

首先只能执行一个回调函数, 对于多个回调的绑定就无能为力, 比如下面这样:

```
let promise1 = new MyPromise((resolve, reject) => {
  fs.readFile('./001.txt', (err, data) => {
    if(!err){
      resolve(data);
    }else {
      reject(err);
    }
  })
});

let x1 = promise1.then(data => {
  console.log("第一次展示", data.toString());
});

let x2 = promise1.then(data => {
  console.log("第二次展示", data.toString());
});

let x3 = promise1.then(data => {
  console.log("第三次展示", data.toString());
});
```

这里绑定了三个回调, 想要在 resolve() 之后一起执行, 那怎么办呢?

需要将 `onFulfilled` 和 `onRejected` 改为数组, 调用 resolve 时将其中的方法拿出来——执行即可。

```
self.onFulfilledCallbacks = [];
self.onRejectedCallbacks = [];

MyPromise.prototype.then = function(onFulfilled, onRejected) {
  if (this.status === PENDING) {
    this.onFulfilledCallbacks.push(onFulfilled);
    this.onRejectedCallbacks.push(onRejected);
  } else if (this.status === FULFILLED) {
```

```
onFulfilled(this.value);
} else {
  onRejected(this.error);
}
return this;
}
```

接下来将 resolve 和 reject 方法中执行回调的部分进行修改:

```
// resolve 中
self.onFulfilledCallbacks.forEach((callback) => callback(self.value));
//reject 中
self.onRejectedCallbacks.forEach((callback) => callback(self.error));
```

## 链式调用完成

我们采用目前的代码来进行测试:

```
let fs = require('fs');
let readFilePromise = (filename) => {
  return new MyPromise((resolve, reject) => {
    fs.readFile(filename, (err, data) => {
      if(!err){
        resolve(data);
      }else {
        reject(err);
      }
    })
  })
}
readFilePromise('./001.txt').then(data => {
  console.log(data.toString());
  return readFilePromise('./002.txt');
}).then(data => {
  console.log(data.toString());
})
// 001.txt的内容
// 001.txt的内容
```

咦? 怎么打印了两个 001, 第二次不是读的 002 文件吗?

问题出在这里:

```
MyPromise.prototype.then = function(onFulfilled, onRejected) {
  //...
  return this;
}
```

这么写每次返回的都是第一个 Promise。then 函数当中返回的第二个 Promise 直接被无视了!

说明 then 当中的实现还需要改进, 我们现在需要对 then 中返回值重视起来。

```
MyPromise.prototype.then = function (onFulfilled, onRejected) {
  let bridgePromise;
```

```
let self = this;
if (self.status === PENDING) {
  return bridgePromise = new MyPromise((resolve, reject) => {
    self.onFulfilledCallbacks.push((value) => {
      try {
        // 看到了吗? 要拿到 then 中回调返回的结果。
        let x = onFulfilled(value);
        resolve(x);
      } catch (e) {
        reject(e);
      }
    });
    self.onRejectedCallbacks.push((error) => {
      try {
        let x = onRejected(error);
        resolve(x);
      } catch (e) {
        reject(e);
      }
    });
  });
}
//...
}
```

假若当前状态为 PENDING，将回调数组中添加如上的函数，当 Promise 状态变化后，会遍历相应回调数组并执行回调。

但是这段程度还是存在一些问题:

- 1) 首先 then 中的两个参数不传的情况并没有处理,
- 2) 假如 then 中的回调执行后返回的结果(也就是上面的 x)是一个 Promise, 直接给 resolve 了, 这是我不希望看到的。

怎么来解决这两个问题呢?

先对参数不传的情况做判断:

```
// 成功回调不传给它一个默认函数
onFulfilled = typeof onFulfilled === "function" ? onFulfilled : value => value;
// 对于失败回调直接抛错
onRejected = typeof onRejected === "function" ? onRejected : error => { throw error };
```

然后对返回Promise的情况进行处理:

```
function resolvePromise(bridgePromise, x, resolve, reject) {
  //如果x是一个promise
  if (x instanceof MyPromise) {
    // 拆解这个 promise , 直到返回值不为 promise 为止
    if (x.status === PENDING) {
      x.then(y => {
        resolvePromise(bridgePromise, y, resolve, reject);
      }, error => {
        reject(error);
      });
    } else {

```

```
    x.then(resolve, reject);
  }
} else {
  // 非 Promise 的话直接 resolve 即可
  resolve(x);
}
}
```

然后在 then 的方法实现中作如下修改:

```
resolve(x) -> resolvePromise(bridgePromise, x, resolve, reject);
```

在这里大家好好体会一下拆解 Promise 的过程，其实不难理解，要强调的是其中的递归调用始终传入的 `resolve` 和 `reject` 这两个参数是什么含义，其实他们控制的是最开始传入的 `bridgePromise` 的状态，这一点非常重要。

紧接着，我们实现一下当 Promise 状态不为 PENDING 时的逻辑。

成功状态下调用then:

```
if (self.status === FULFILLED) {
  return bridgePromise = new MyPromise((resolve, reject) => {
    try {
      // 状态变为成功，会有相应的 self.value
      let x = onFulfilled(self.value);
      // 暂时可以理解为 resolve(x)，后面具体实现中有拆解的过程
      resolvePromise(bridgePromise, x, resolve, reject);
    } catch (e) {
      reject(e);
    }
  });
}
```

失败状态下调用then:

```
if (self.status === REJECTED) {
  return bridgePromise = new MyPromise((resolve, reject) => {
    try {
      // 状态变为失败，会有相应的 self.error
      let x = onRejected(self.error);
      resolvePromise(bridgePromise, x, resolve, reject);
    } catch (e) {
      reject(e);
    }
  });
}
```

Promise A+中规定成功和失败的回调都是微任务，由于浏览器中 JS 触碰不到底层微任务的分配，可以直接拿 `setTimeout` (属于宏任务的范畴) 来模拟，用 `setTimeout` 将需要执行的任务包裹，当然，上面的 `resolve` 实现也是同理，大家注意一下即可，其实并不是真正的微任务。



```

if (self.status === FULFILLED) {
  return bridgePromise = new MyPromise((resolve, reject) => {
    setTimeout(() => {
      //...
    })
  })
}

if (self.status === REJECTED) {
  return bridgePromise = new MyPromise((resolve, reject) => {
    setTimeout(() => {
      //...
    })
  })
}

```

好了，到这里，我们基本实现了 then 方法，现在我们拿刚刚的测试代码做一下测试，依次打印如下：

```

001.txt的内容
002.txt的内容

```

可以看到，已经可以顺利地完成链式调用。

## 错误捕获及冒泡机制分析

现在来实现 catch 方法：

```

Promise.prototype.catch = function (onRejected) {
  return this.then(null, onRejected);
}

```

对，就是这么几行，catch 原本就是 then 方法的语法糖。

相比于实现来讲，更重要的是理解其中错误冒泡的机制，即中途一旦发生错误，可以在最后用 catch 捕获错误。

我们回顾一下 Promise 的运作流程也不难理解，贴上一行关键的代码：

```

// then 的实现中
onRejected = typeof onRejected === "function" ? onRejected : error => { throw error };

```

一旦其中有一个PENDING状态的 Promise 出现错误后状态必然会变为失败，然后执行 onRejected函数，而这个 onRejected 执行又会抛错，把新的 Promise 状态变为失败，新的 Promise 状态变为失败后又会执行onRejected.....就这样一直抛下去，直到用catch 捕获到这个错误，才停止往下抛。

这就是 Promise 的错误冒泡机制。

至此，Promise 三大法宝：回调函数延迟绑定、回调返回值穿透 和 错误冒泡。

## 58. 实现Promise的 resolve、reject 和 finally

### 实现 Promise.resolve

实现 resolve 静态方法有三个要点:

- 传参为一个 Promise, 则直接返回它。
- 传参为一个 thenable 对象, 返回的 Promise 会跟随这个对象, 采用它的最终状态 作为 自己的状态。
- 其他情况, 直接返回以该值为成功状态的promise对象。

具体实现如下:

```
Promise.resolve = (param) => {  
  if(param instanceof Promise) return param;  
  return new Promise((resolve, reject) => {  
    if(param && param.then && typeof param.then === 'function') {  
      // param 状态变为成功会调用resolve, 将新 Promise 的状态变为成功, 反之亦然  
      param.then(resolve, reject);  
    }else {  
      resolve(param);  
    }  
  })  
}
```

### 实现 Promise.reject

Promise.reject 中传入的参数会作为一个 reason 原封不动地往下传, 实现如下:

```
Promise.reject = function (reason) {  
  return new Promise((resolve, reject) => {  
    reject(reason);  
  });  
}
```

### 实现 Promise.prototype.finally

无论当前 Promise 是成功还是失败, 调用 finally 之后都会执行 finally 中传入的函数, 并且将值原封不动的往下传。

```
Promise.prototype.finally = function(callback) {  
  this.then(value => {  
    return Promise.resolve(callback()).then(() => {  
      return value;  
    })  
  }, error => {  
    return Promise.resolve(callback()).then(() => {  
      throw error;  
    })  
  })  
}
```

## 59. 实现Promise的 all 和 race

### 实现 Promise.all

对于 all 方法而言，需要完成下面的核心功能：

- 1) 传入参数为一个空的可迭代对象，则直接进行resolve。
- 2) 如果参数中有一个promise失败，那么Promise.all返回的promise对象失败。
- 3) 在任何情况下，Promise.all 返回的 promise 的完成状态的结果都是一个数组

具体实现如下：

```
Promise.all = function(promises) {  
  return new Promise((resolve, reject) => {  
    let result = [];  
    let index = 0;  
    let len = promises.length;  
    if(len === 0) {  
      resolve(result);  
      return;  
    }  
  
    for(let i = 0; i < len; i++) {  
      // 为什么不直接 promise[i].then, 因为promise[i]可能不是一个promise  
      Promise.resolve(promise[i]).then(data => {  
        result[i] = data;  
        index++;  
        if(index === len) resolve(result);  
      }).catch(err => {  
        reject(err);  
      })  
    }  
  })  
}
```

### 实现 Promise.race

race 的实现相比之下就简单一些，只要有一个 promise 执行完，直接 resolve 并停止执行。

```
Promise.race = function(promises) {  
  return new Promise((resolve, reject) => {  
    let len = promises.length;  
    if(len === 0) return;  
    for(let i = 0; i < len; i++) {  
      Promise.resolve(promise[i]).then(data => {  
        resolve(data);  
        return;  
      }).catch(err => {  
        reject(err);  
        return;  
      })  
    }  
  })  
}
```

## 60. 谈谈你对生成器以及协程的理解

生成器(Generator)是 ES6 中的新语法，相对于之前的异步语法，上手的难度还是比较大的。因此这里我们先来好好熟悉一下 Generator 语法。

### 生成器执行流程

上面是生成器函数？

生成器是一个带星号的"函数"(注意：它并不是真正的函数)，可以通过 `yield` 关键字 暂停执行 和 恢复执行的

举个例子：

```
function* gen() {
  console.log("enter");
  let a = yield 1;
  let b = yield (function () {return 2})();
  return 3;
}
var g = gen() // 阻塞住，不会执行任何语句
console.log(typeof g) // object 看到了吗？不是"function"

console.log(g.next())
console.log(g.next())
console.log(g.next())
console.log(g.next())

// enter
// { value: 1, done: false }

// { value: 2, done: false }
// { value: 3, done: true }
// { value: undefined, done: true }
```

由此可以看到，生成器的执行有这样几个关键点：

- 1) 调用 `gen()` 后，程序会阻塞住，不会执行任何语句。
- 2) 调用 `g.next()` 后，程序继续执行，直到遇到 `yield` 程序暂停。
- 3) `next` 方法返回一个对象，有两个属性：`value` 和 `done`。`value` 为当前 `yield` 后面的结果，`done` 表示是否执行完，遇到了 `return` 后，`done` 会由 `false` 变为 `true`。

### yield\*

当一个生成器要调用另一个生成器时，使用 `yield*` 就变得十分方便。比如下面的例子：

```
function* gen1() {  
  yield 1;  
  yield 4;  
}  
function* gen2() {  
  yield 2;  
  yield 3;  
}  
}
```

我们想要按照1234的顺序执行，如何来做呢？

在 gen1 中，修改如下：

```
function* gen1() {  
  yield 1;  
  yield* gen2();  
  yield 4;  
}
```

这样修改之后，之后依次调用 `next` 即可。

## 生成器实现机制——协程

可能你会比较好奇，生成器究竟是如何让函数暂停，又会如何恢复的呢？接下来我们就来对其中的执行机制——`协程`一探究竟。

### 什么是协程？

协程是一种比线程更加轻量级的存在，协程处在线程的环境中，`一个线程可以存在多个协程`，可以将协程理解为线程中的一个任务。不像进程和线程，协程并不受操作系统的管理，而是被具体的应用程序代码所控制。

### 协程的运作过程

那你可能要问了，JS 不是单线程执行的吗，开这么多协程难道可以一起执行吗？

答案是：并不能。一个线程一次只能执行一个协程。比如当前执行 A 协程，另外还有一个 B 协程，如果想要执行 B 的任务，就必须在 A 协程中将 JS 线程的控制权转交给 B 协程，那么现在 B 执行，A 就相当于处于暂停的状态。

举个具体的例子：

```
function* A() {  
  console.log("我是A");  
  yield B(); // A停住，在这里转交线程执行权给B  
  console.log("结束了");  
}  
function B() {  
  console.log("我是B");  
  return 100; // 返回，并且将线程执行权还给A  
}  
let gen = A();  
gen.next();
```

```
gen.next();
```

```
// 我是A  
// 我是B  
// 结束了
```

在这个过程中，A 将执行权交给 B，也就是 A 启动 B，我们也称 A 是 B 的**父协程**。因此 B 当中最后 `return 100` 其实是将 100 传给了父协程。

需要强调的是，对于协程来说，它并不受操作系统的控制，完全由用户自定义切换，因此并没有进程/线程上下文切换的开销，这是**高性能**的重要原因。

OK, 原理就说到这里。可能你还会有疑问: 这个生成器不就暂停-恢复、暂停-恢复这样执行的吗？它和异步有什么关系？而且，每次执行都要调用next，能不能让它一次性执行完毕呢？下一节我们就来仔细拆解这些问题。

## 61. 如何让 Generator 的异步代码按顺序执行完毕？

这里面其实有两个问题:

- 1) `Generator` 如何跟 异步 产生关系？
- 2) 怎么把 `Generator` 按顺序执行完毕？

### thunk 函数

要想知道 `Generator` 跟异步的关系，首先带大家搞清楚一个概念——thunk函数(即 偏函数)，虽然这只是实现两者关系的方式之一。(另一种方式是 `Promise`，后面会讲到)

举个例子，比如我们现在要判断数据类型。可以写如下的判断逻辑:

```
let isString = (obj) => {  
  return Object.prototype.toString.call(obj) === '[object String]';  
};  
let isFunction = (obj) => {  
  return Object.prototype.toString.call(obj) === '[object Function]';  
};  
let isArray = (obj) => {  
  return Object.prototype.toString.call(obj) === '[object Array]';  
};  
let isSet = (obj) => {  
  return Object.prototype.toString.call(obj) === '[object Set]';  
};  
// ...
```

可以看到，出现了非常多重复的逻辑。我们将它们做一下封装:

```
let isType = (type) => {  
  return (obj) => {  
    return Object.prototype.toString.call(obj) === `[object ${type}]`;  
  }  
}
```

现在我们这样做即可:

```
let isString = isType('String');
let isFunction = isType('Function');
//...
```

相应的 `isString` 和 `isFunction` 是由 `isType` 生产出来的函数，但它们依然可以判断出参数是否为 `String` (`Function`)，而且代码简洁了不少。

```
isString("123");
isFunction(val => val);
```

`isType` 这样的函数我们称为 **thunk 函数**。它的核心逻辑是接收一定的参数，生产出定制化的函数，然后使用定制化的函数去完成功能。thunk 函数的实现会比单个的判断函数复杂一点点，但就是这一点点的复杂，大大方便了后续的操作。

## Generator 和 异步

### thunk 版本

以文件操作为例，我们来看看 异步操作 如何应用于 Generator。

```
const readFileThunk = (filename) => {
  return (callback) => {
    fs.readFile(filename, callback);
  }
}
```

`readFileThunk` 就是一个 **thunk 函数**。异步操作核心的一环就是绑定回调函数，而 **thunk 函数** 可以帮助我们做到。首先传入文件名，然后生成一个针对某个文件的定制化函数。这个函数中传入回调，这个回调就会成为异步操作的回调。这样就让 **Generator** 和 **异步** 关联起来了。

紧接着我们做如下的操作：

```
const gen = function* () {
  const data1 = yield readFileThunk('001.txt')
  console.log(data1.toString())
  const data2 = yield readFileThunk('002.txt')
  console.log(data2.toString())
}
```

接着我们让它执行完：

```
let g = gen();
// 第一步：由于进场是暂停的，我们调用next，让它开始执行。
// next返回值中有一个value值，这个value是yield后面的结果，放在这里也就是是thunk函数生成的定制化函数，里面需要传一个回调函数作为参数
g.next().value((err, data1) => {
  // 第二步：拿到上一次得到的结果，调用next，将结果作为参数传入，程序继续执行。
  // 同理，value传入回调
  g.next(data1).value((err, data2) => {
    g.next(data2);
  })
})
})
```

打印结果如下:

001.txt的内容  
002.txt的内容

上面嵌套的情况还算简单, 如果任务多起来, 就会产生很多层的嵌套, 可操作性不强, 有必要把执行的代码封装一下:

```
function run(gen){
  const next = (err, data) => {
    let res = gen.next(data);
    if(res.done) return;
    res.value(next);
  }
  next();
}
run(g);
```

再次执行, 依然打印正确的结果。代码虽然就这么几行, 但包含了递归的过程, 好好体会一下。

这是通过 `thunk` 完成异步操作的情况。

## Promise 版本

还是拿上面的例子, 用 `Promise` 来实现就轻松一些:

```
const readFilePromise = (filename) => {
  return new Promise((resolve, reject) => {
    fs.readFile(filename, (err, data) => {
      if(err) {
        reject(err);
      } else {
        resolve(data);
      }
    })
  })
}.then(res => res);

const gen = function* () {
  const data1 = yield readFilePromise('001.txt')
  console.log(data1.toString())
  const data2 = yield readFilePromise('002.txt')
  console.log(data2.toString())
}
```

执行的代码如下:



```
let g = gen();
function getGenPromise(gen, data) {
  return gen.next(data).value;
}
getGenPromise(g).then(data1 => {
  return getGenPromise(g, data1);
}).then(data2 => {
  return getGenPromise(g, data2)
})
```

打印结果如下:

```
001.txt的内容
002.txt的内容
```

同样，我们可以对执行 Generator 的代码加以封装:

```
function run(g) {
  const next = (data) => {
    let res = g.next();
    if(res.done) return;
    res.value.then(data => {
      next(data);
    })
  }
  next();
}
```

同样能输出正确的结果。代码非常精炼，希望能参照刚刚链式调用的例子，仔细体会一下递归调用的过程。

## 采用 co 库

以上我们针对 `thunk` 函数和 `Promise` 两种 Generator 异步操作的一次性执行完毕做了封装，但实际场景中已经存在成熟的工具包了，如果大名鼎鼎的 `co` 库，其实核心原理就是我们已经手写过了（就是刚刚封装的 `Promise` 情况下的执行代码），只不过源码会各种边界情况做了处理。使用起来非常简单:

```
const co = require('co');
let g = gen();
co(g).then(res =>{
  console.log(res);
})
```

打印结果如下:

```
001.txt的内容
002.txt的内容
100
```

简单几行代码就完成了 Generator 所有的操作，真不愧 `co` 和 `Generator` 天生一对啊！

## 62. 解释一下async/await的运行机制。

async/await被称为 JS 中异步终极解决方案。它既能够像 co + Generator 一样用同步的方式来书写异步代码，又得到底层的语法支持，无需借助任何第三方库。接下来，我们从原理的角度来重新审视这个语法糖背后究竟做了些什么。

### async

什么是 async ?

MDN 的定义: async 是一个通过异步执行并隐式返回 Promise 作为结果的函数。

注意重点: **返回结果为Promise。**

举个例子:

```
async function func() {  
  return 100;  
}  
console.log(func());  
// Promise {<resolved>: 100}
```

这就是隐式返回 Promise 的效果。

### await

我们来看看 await 做了些什么事情。

以一段代码为例:

```
async function test() {  
  console.log(100)  
  let x = await 200  
  console.log(x)  
  console.log(200)  
}  
console.log(0)  
test()  
console.log(300)
```

我们来分析一下这段程序。首先代码同步执行，打印出 0，然后将 test 压入执行栈，打印出 100，下面注意了，遇到了关键角色 **await**。

放个慢镜头:

```
await 100;
```

被 JS 引擎转换成一个 Promise :

```
let promise = new Promise((resolve, reject) => {  
  resolve(100);  
})
```

这里调用了 resolve，resolve 的任务进入微任务队列。

然后, JS 引擎将暂停当前协程的运行, 把线程的执行权交给 父协程 (父协程不懂是什么的, 上上篇才讲回去补课)。

回到父协程中, 父协程的第一件事情就是对 `await` 返回的 `Promise` 调用 `then`, 来监听这个 `Promise` 的状态改变。

```
promise.then(value => {  
  // 相关逻辑, 在resolve 执行之后来调用  
})
```

然后往下执行, 打印出 `300`。

根据 `EventLoop` 机制, 当前主线程的宏任务完成, 现在检查 微任务队列, 发现还有一个 `Promise` 的 `resolve`, 执行, 现在父协程在 `then` 中传入的回调执行。我们来看看这个回调具体做的是是什么。

```
promise.then(value => {  
  // 1. 将线程的执行权交给test协程  
  // 2. 把 value 值传递给 test 协程  
})
```

Ok, 现在执行权到了 `test` 协程 手上, `test` 接收到 父协程 传来的 `200`, 赋值给 `a`, 然后依次执行后面的语句, 打印 `200`、`200`。

最后的输出为:

```
0  
100  
300  
200  
200
```

总结一下, `async/await` 利用协程和 `Promise` 实现了同步方式编写异步代码的效果, 其中 `Generator` 是对协程的一种实现, 虽然语法简单, 但引擎在背后做了大量的工作, 我们也对这些工作做了一一的拆解。用 `async/await` 写出的代码也更加优雅、美观, 相比于之前的 `Promise` 不断调用 `then` 的方式, 语义化更加明显, 相比于 `co + Generator` 性能更高, 上手成本也更低, 不愧是 `JS` 异步终极解决方案!

## 63. `forEach` 中用 `await` 会产生什么问题?怎么解决这个问题?

### 问题

问题:对于异步代码, `forEach` 并不能保证按顺序执行。

举个例子:

```
async function test() {  
  let arr = [4, 2, 1]  
  arr.forEach(async item => {  
    const res = await handle(item)  
    console.log(res)  
  })  
  console.log('结束')  
}  
  
function handle(x) {
```

```
return new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve(x)
  }, 1000 * x)
})
}

test()
```

我们期望的结果是:

```
4
2
1
结束
```

但是实际上会输出:

```
结束
1
2
4
```

## 问题原因

这是为什么呢? 我想我们有必要看看 `forEach` 底层怎么实现的。

```
// 核心逻辑
for (var i = 0; i < length; i++) {
  if (i in array) {
    var element = array[i];
    callback(element, i, array);
  }
}
```

可以看到, `forEach` 拿过来直接执行了, 这就导致它无法保证异步任务的执行顺序。比如后面的任务用时短, 那么就又可能抢在前面的任务之前执行。

## 解决方案

如何解决这个问题呢?

其实也很简单, 我们利用 `for...of` 就能轻松解决。

```
async function test() {
  let arr = [4, 2, 1]
  for(const item of arr) {
    const res = await handle(item)
    console.log(res)
  }
  console.log('结束')
}
```

## 解决原理——Iterator

for...of并不像forEach那么简单粗暴的方式去遍历执行，而是采用一种特别的手段——迭代器去遍历。

首先，对于数组来讲，它是一种可迭代数据类型。那什么是可迭代数据类型呢？

原生具有[Symbol.iterator]属性数据类型为可迭代数据类型。如数组、类数组（如arguments、NodeList）、Set和Map。

可迭代对象可以通过迭代器进行遍历。

```
let arr = [4, 2, 1];  
// 这就是迭代器  
let iterator = arr[Symbol.iterator]();  
console.log(iterator.next());  
console.log(iterator.next());  
console.log(iterator.next());  
console.log(iterator.next());  
  
// {value: 4, done: false}  
// {value: 2, done: false}  
// {value: 1, done: false}  
// {value: undefined, done: true}
```

因此，我们的代码可以这样来组织：

```
async function test() {  
  let arr = [4, 2, 1]  
  let iterator = arr[Symbol.iterator]();  
  let res = iterator.next();  
  while(!res.done) {  
    let value = res.value;  
    console.log(value);  
    await handle(value);  
    res = iterator.next();  
  }  
  console.log('结束')  
}  
// 4  
// 2  
// 1  
// 结束
```

多个任务成功地按顺序执行！

## 重新认识生成器

回头再看看用iterator遍历[4,2,1]这个数组的代码。

```
let arr = [4, 2, 1];
// 迭代器
let iterator = arr[Symbol.iterator]();
console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());

// {value: 4, done: false}
// {value: 2, done: false}
// {value: 1, done: false}
// {value: undefined, done: true}
```

咦？返回值有value和done属性，生成器也可以调用 next,返回的也是这样的数据结构，这么巧？！

没错，生成器本身就是一个迭代器。

既然属于迭代器，那它就可以用for...of遍历了吧？

当然没错，不信来写一个简单的斐波那契数列(50以内)：

```
function* fibonacci(){
  let [prev, cur] = [0, 1];
  console.log(cur);
  while(true) {
    [prev, cur] = [cur, prev + cur];
    yield cur;
  }
}

for(let item of fibonacci()) {
  if(item > 50) break;
  console.log(item);
}

// 1
// 1
// 2
// 3
// 5
// 8
// 13
// 21
// 34
```

## 64. 关于JS中一些重要的api实现

### 用ES5实现数组的map方法

核心要点：

- 1) 回调函数的参数有哪些，返回值如何处理。
- 2) 不修改原来的数组。

```
Array.prototype.MyMap = function(fn, context){
    var arr = Array.prototype.slice.call(this); //由于是ES5所以就不用...展开符号了
    var mappedArr = [];
    for (var i = 0; i < arr.length; i++){
        mappedArr.push(fn.call(context, arr[i], i, this));
    }
    return mappedArr;
}
```

## 用ES5实现数组的reduce方法

核心要点:

- 1) 初始值不传怎么处理
- 2) 回调函数的参数有哪些，返回值如何处理。

```
Array.prototype.myReduce = function(fn, initialValue) {
    var arr = Array.prototype.slice.call(this);
    var res, startIndex;
    res = initialValue ? initialValue : arr[0];
    startIndex = initialValue ? 0 : 1;
    for(var i = startIndex; i < arr.length; i++) {
        res = fn.call(null, res, arr[i], i, this);
    }
    return res;
}
```

## 实现call/apply

思路: 利用this的上下文特性。

```
//实现apply只要把下一行中的...args换成args即可
Function.prototype.myCall = function(context = window, ...args) {
    let func = this;
    let fn = Symbol("fn");
    context[fn] = func;

    let res = context[fn](...args); //重点代码，利用this指向，相当于
    context.caller(...args)

    delete context[fn];
    return res;
}
```

## 实现Object.create方法(常用)

```
function create(proto) {
  function F() {};
  F.prototype = proto;
  F.prototype.constructor = F;

  return new F();
}
```

## 实现bind方法

核心要点:

- 1) 对于普通函数，绑定this指向
- 2) 对于构造函数，要保证原函数的原型对象上的属性不能丢失

```
Function.prototype.bind = function(context, ...args) {
  let self = this; // 谨记this表示调用bind的函数
  let fBound = function() {
    // this instanceof fBound为true表示构造函数的情况。如new func.bind(obj)
    return self.apply(this instanceof fBound ? this : context || window,
      args.concat(Array.prototype.slice.call(arguments)));
  }
  fBound.prototype = Object.create(this.prototype); // 保证原函数的原型对象上的属性不丢失
  return fBound;
}
```

## 实现new关键字

核心要点:

- 1) 创建一个全新的对象，这个对象的proto要指向构造函数的原型对象
- 2) 执行构造函数
- 3) 返回值为object类型则作为new方法的返回值返回，否则返回上述全新对象

```
function myNew(fn, ...args) {
  let instance = Object.create(fn.prototype);
  let res = fn.apply(instance, args);
  return typeof res === 'object' ? res : instance;
}
```

## 实现instanceof的作用

核心要点：原型链的向上查找。



```
function myInstanceOf(left, right) {
  let proto = Object.getPrototypeOf(left);
  while(true) {
    if(proto == null) return false;
    if(proto == right.prototype) return true;
    proto = Object.getPrototypeOf(proto);
  }
}
```

## 实现单例模式

核心要点: 用闭包和Proxy属性拦截

```
function proxy(func) {
  let instance;
  let handler = {
    construct(target, args) {
      if(!instance) {
        instance = Reflect.construct(func, args);
      }
      return instance;
    }
  }
  return new Proxy(func, handler);
}
```

## 实现防抖功能

核心要点:

如果在定时器的时间范围内再次触发，则重新计时。

```
const debounce = (fn, delay) => {
  let timer = null;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
};
```

## 实现节流功能

核心要点:

如果在定时器的时间范围内再次触发，则不予理睬，等当前定时器完成，才能启动下一个定时器。

```
const throttle = (fn, delay = 500) => {
  let flag = true;
  return (...args) => {
    if (!flag) return;
    flag = false;
    setTimeout(() => {
      fn.apply(this, args);
      flag = true;
    }, delay);
  };
};
```

## 实现深拷贝

以下为简易版深拷贝，没有考虑循环引用的情况和Buffer、Promise、Set、Map的处理，如果——实现，过于复杂，面试短时间写出来不太现实

```
const clone = parent => {
  // 判断类型
  const isType = (target, type) => `[object ${type}]` ===
    Object.prototype.toString.call(target)

  // 处理正则
  const getRegExp = re => {
    let flags = "";
    if (re.global) flags += "g";
    if (re.ignoreCase) flags += "i";
    if (re.multiline) flags += "m";
    return flags;
  };

  const _clone = parent => {
    if (parent === null) return null;
    if (typeof parent !== "object") return parent;

    let child, proto;

    if (isType(parent, "Array")) {
      // 对数组做特殊处理
      child = [];
    } else if (isType(parent, "RegExp")) {
      // 对正则对象做特殊处理
      child = new RegExp(parent.source, getRegExp(parent));
      if (parent.lastIndex) child.lastIndex = parent.lastIndex;
    } else if (isType(parent, "Date")) {
      // 对Date对象做特殊处理
      child = new Date(parent.getTime());
    } else {
      // 处理对象原型
      proto = Object.getPrototypeOf(parent);
      // 利用Object.create切断原型链
      child = Object.create(proto);
    }

    for (let i in parent) {
      // 递归

```

```

    child[i] = _clone(parent[i]);
  }
  return child;
};
return _clone(parent);
};

```

## 实现Promise

重点难点，比较复杂

### 用法

```

new Promise((resolve, reject) => {
  //异步成功执行resolve，否则执行reject
}).then((res) => {
  //resolve触发第一个回调函数执行
}, (err) => {
  //reject触发第二个回调函数执行
}).then(res => {
  //需要保证then方法返回的依然是promise
  //这样才能实现链式调用
}).catch(reason => {

});
//等待所有的promise都成功执行then，
//反之只要有一个失败就会执行catch
Promise.all([promise1, ...]).then();

```

js

### (1) 初步实现Promise:

- 1) 实现三种状态: 'pending', 'fulfilled', 'rejected'
- 2) 能够实现then方法两种回调函数的处理

```

//promise.js
class Promise{
  //传一个异步函数进来
  constructor(excutorCallback){
    this.status = 'pending';
    this.value = undefined;
    this.fulfillAry = [];
    this.rejectedAry = [];
    //=>执行Excutor
    let resolveFn = result => {
      if(this.status !== 'pending') return;
      let timer = setTimeout(() => {
        this.status = 'fulfilled';
        this.value = result;
        this.fulfillAry.forEach(item => item(this.value));
      }, 0);
    };
    let rejectFn = reason => {
      if(this.status !== 'pending')return;

```

```

    let timer = setTimeout(() => {
      this.status = 'rejected';
      this.value = reason;
      this.rejectedAry.forEach(item => item(this.value))
    })
  };
  try{
    //执行这个异步函数
    excutorCallback(resolveFn, rejectFn);
  } catch(err) {
    //=>有异常信息按照rejected状态处理
    rejectFn(err);
  }
}
then(fulfilledCallback, rejectedCallback) {
  //resolve和reject函数其实一个作为微任务
  //因此他们不是立即执行，而是等then调用完成后执行
  this.fulfilledAry.push(fulfilledCallback);
  this.rejectedAry.push(rejectedCallback);
  //一顿push过后他们被执行
}
}
}

module.exports = Promise;

```

测试如下：

```

let Promise = require('./promise');

let p1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    Math.random() < 0.5 ? resolve(100) : reject(-100);
  }, 1000)
}).then(res => {
  console.log(res);
}, err => {
  console.log(err);
})

```

## (2) 完成链式效果

最大的难点在于链式调用的实现，具体来说就是then方法的实现。

```

//then传进两个函数
then(fulfilledCallback, rejectedCallback) {
  //保证两者为函数
  typeof fulfilledCallback !== 'function' ? fulfilledCallback = result =>
  result:null;
  typeof rejectedCallback !== 'function' ? rejectedCallback = reason => {
    throw new Error(reason instanceof Error ? reason.message : reason);
  } : null
  //返回新的Promise对象，后面称它为“新Promise”
  return new Promise((resolve, reject) => {
    //注意这个this指向目前的Promise对象，而不是新的Promise
    //再强调一遍，很重要：

```

```
//目前的Promise(不是这里return的新Promise)的resolve和reject函数其实一个作为微任务
//因此他们不是立即执行，而是等then调用完成后执行
this.fulfillAry.push(() => {
  try {
    //把then里面的方法拿过来执行
    //执行的目的已经达到
    let x = fulfilledCallback(this.value);
    //下面执行之后的下一步，也就是记录执行的状态，决定新Promise如何表现
    //如果返回值x是一个Promise对象，就执行then操作
    //如果不是Promise，直接调用新Promise的resolve函数，
    //新Promise的fulfillAry现在为空，在新Promise的then操作后，新Promise的resolve执行

    x instanceof Promise ? x.then(resolve, reject):resolve(x);
  }catch(err){
    reject(err)
  }
});
//以下同理
this.rejectedAry.push(() => {
  try {
    let x = this.rejectedCallback(this.value);
    x instanceof Promise ? x.then(resolve, reject):resolve(x);
  }catch(err){
    reject(err)
  }
});
});
}
```

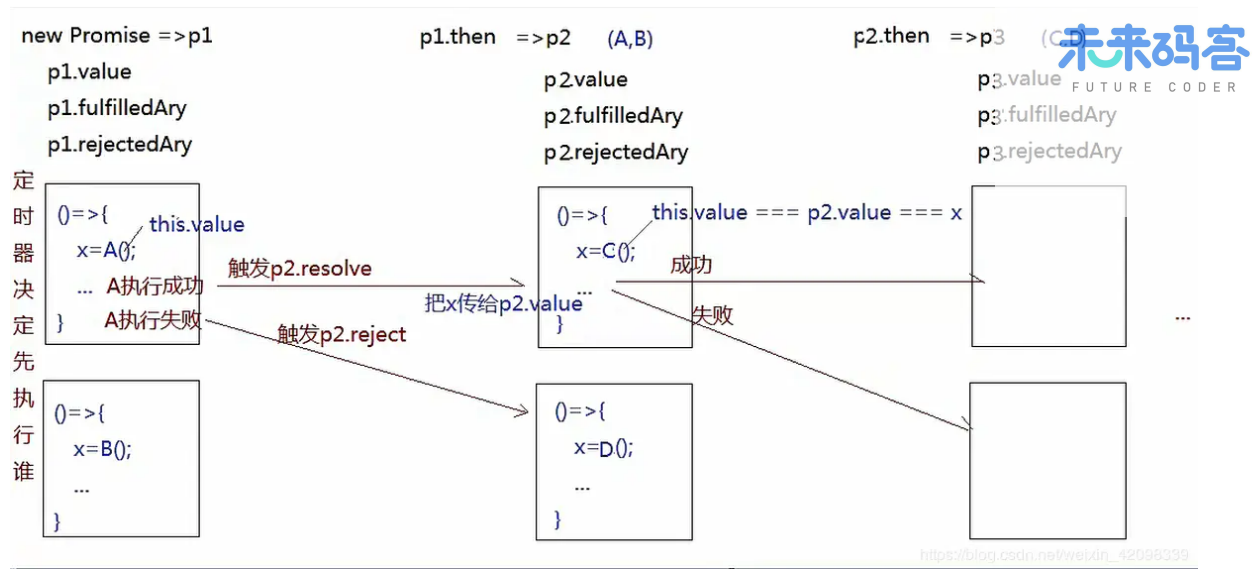
测试用例：

```
let p1 = new Promise((resolve, reject) => {
  setTimeout(() => {
    Math.random()<0.5?resolve(100):reject(-100);
  }, 1000)
})

let p2 = p1.then(result => {
  //执行then返回的是一个新的Promise
  return result + 100;
})

let p3 = p2.then(result => {
  console.log(result);
}, reason => {
  console.log(reason)
})
```

简单画图来模拟一下链式调用的内部流程：



有了then方法，catch自然而然调用即可：

```

catch(rejectedCallback) {
  return this.then(null, rejectedCallback);
}
  
```

### (3) Promise.all()

接下来实现Promise.all()

```

//为类的静态方法，而不是在原型上
static all(promiseAry = []) {
  let index = 0,
      result = [];
  return new Promise((resolve, reject) => {
    for(let i = 0; i < promiseAry.length; i++){
      promiseAry[i].then(val => {
        index++;
        result[i] = val;
        if( index === promiseAry.length){
          resolve(result)
        }
      }, reject);
    }
  })
}
  
```

### (4) Promise.race()

接下来是race方法

```

static race(promises) {
  return new Promise((resolve, reject) => {
    if (promises.length === 0) {
      return;
    } else {
      for(let i = 0; i < promises.length; i++){
        promises[i].then(val => {
          resolve(val);
        }, reject);
      }
    }
  })
}
  
```

```

        promises[i].then(val => {
            resolve(result);
            return;
        }, reject);
    }
}
});
}

```

## (5) Promise.resolve()

```

static resolve (value) {
    if (value instanceof Promise) return value
    return new Promise(resolve => resolve(value))
}

```

## (6) Promise.reject()

```

static reject (value) {
    return new Promise((resolve, reject) => reject(value))
}

```

## 完整代码

现在手写一个简陋但是功能较为完备的Promise就大功告成了。

```

class Promise{
    constructor(excutorCallback){
        this.status = 'pending';
        this.value = undefined;
        this.fulfillAry = [];
        this.rejectedAry = [];
        //=>执行Excutor
        let resolveFn = result => {
            if(this.status !== 'pending') return;
            let timer = setTimeout(() => {
                this.status = 'fulfilled';
                this.value = result;
                this.fulfillAry.forEach(item => item(this.value));
            }, 0);
        };
        let rejectFn = reason => {
            if(this.status !== 'pending')return;
            let timer = setTimeout(() => {
                this.status = 'rejected';
                this.value = reason;
                this.rejectedAry.forEach(item => item(this.value))
            })
        };
        try{

```

```

        excutorCallback(resolveFn, rejectFn);
    } catch(err) {
        //=>有异常信息按照rejected状态处理
        rejectFn(err);
    }
}

then(fulfilledCallback, rejectedCallback) {
    typeof fulfilledCallback !== 'function' ? fulfilledCallback = result =>
result:null;
    typeof rejectedCallback !== 'function' ? rejectedCallback = reason => {
        throw new Error(reason instanceof Error? reason.message:reason);
    } : null

    return new Promise((resolve, reject) => {
        this.fulfillAry.push(() => {
            try {
                let x = fulfilledCallback(this.value);
                x instanceof Promise ? x.then(resolve, reject):resolve(x);
            } catch(err){
                reject(err)
            }
        });
        this.rejectedAry.push(() => {
            try {
                let x = this.rejectedCallback(this.value);
                x instanceof Promise ? x.then(resolve, reject):resolve(x);
            } catch(err){
                reject(err)
            }
        })
    });
}

catch(rejectedCallback) {
    return this.then(null, rejectedCallback);
}

static all(promiseAry = []) {
    let index = 0,
        result = [];
    return new Promise((resolve, reject) => {
        for(let i = 0; i < promiseAry.length; i++){
            promiseAry[i].then(val => {
                index++;
                result[i] = val;
                if( index === promiseAry.length){
                    resolve(result)
                }
            }, reject);
        }
    })
}

static race(promiseAry) {
    return new Promise((resolve, reject) => {
        if (promiseAry.length === 0) {
            return;
        }
        for (let i = 0; i < promiseAry.length; i++) {
            promiseAry[i].then(val => {
                resolve(val);
            });
        }
    });
}

```



```
        return;  
      }, reject);  
    }  
  })  
}  
  
static resolve (value) {  
  if (value instanceof Promise) return value  
  return new Promise(resolve => resolve(value))  
}  
  
static reject (value) {  
  return new Promise((resolve, reject) => reject(value))  
}  
}  
  
module.exports = Promise;
```

## 65. 事件流向

- (1) 冒泡：子节点一层层冒泡到根节点
- (2) 捕获顺序与冒泡相反
- (3) addEventListener 最后个参数 true 代表捕获反之代表冒泡
- (4) 阻止冒泡不停止父节点捕获

## 66. 事件委托

参考定义：事件委托就是利用事件冒泡，只指定一个事件处理程序，就可以管理某一类型的所有事件 好处：给重复的节点添加相同操作，减少 dom 交互，提高性能 实现思路：给父组件添加事件，通过事件冒泡，排查元素是否为指定元素，并进行系列操作

# HTTP

## 1. HTTP 报文结构是怎样的？

对于 TCP 而言，在传输的时候分为两个部分：**TCP头**和**数据部分**。

而 HTTP 类似，也是 **header + body** 的结构，具体而言：

起始行 + 头部 + 空行 + 实体

由于 http **请求报文** 和 **响应报文** 是有一定区别，因此我们分开介绍。

### 起始行

对于请求报文来说，起始行类似下面这样：

GET /home HTTP/1.1

也就是**方法 + 路径 + http版本**。

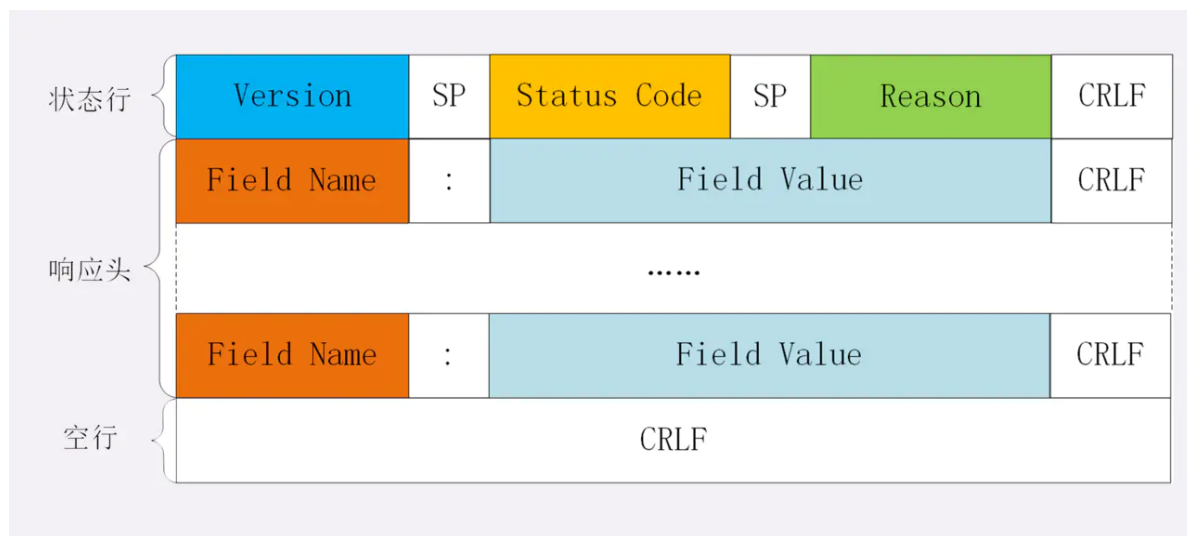
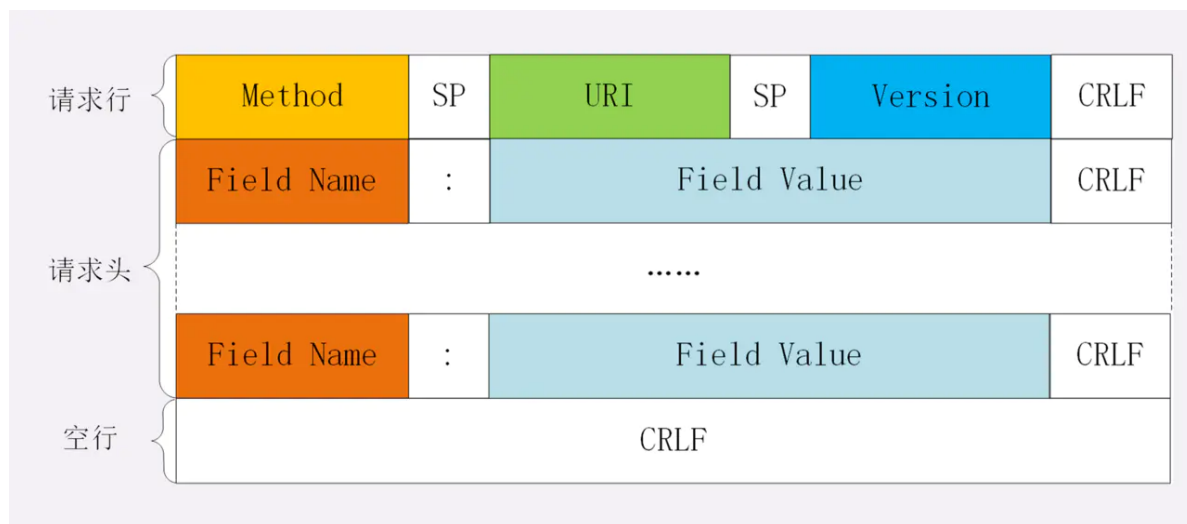
对于响应报文来说，起始行一般长这个样：

响应报文的起始行也叫做 **状态行**。由 **http版本**、**状态码**和**原因**三部分组成。

值得注意的是，在起始行中，每两个部分之间用**空格**隔开，最后一个部分后面应该接一个**换行**，严格遵循 **ABNF** 语法规则。

## 头部

展示一下请求头和响应头在报文中的位置:



不管是请求头还是响应头，其中的字段是相当多的，而且牵扯到 **http** 非常多的特性，这里就不一一列举的，重点看看这些头部字段的格式：

- 字段名不区分大小写
- 字段名不允许出现空格，不可以出现下划线 `_`
- 字段名后面必须紧接着 `:`

## 空行

很重要，用来区分开 **头部** 和 **实体**。

问: 如果说在头部中间故意加一个空行会怎么样?

那么空行后的内容全部被视为实体。

## 实体

就是具体的数据了，也就是 body 部分。请求报文对应 请求体，响应报文对应 响应体。

## 2. HTTP有哪些请求方法？

http/1.1 规定了以下请求方法(注意，都是大写):

- GET: 通常用来获取资源
- HEAD: 获取资源的元信息
- POST: 提交数据，即上传数据
- PUT: 修改数据
- DELETE: 删除资源(几乎用不到)
- CONNECT: 建立连接隧道，用于代理服务器
- OPTIONS: 列出可对资源实行的请求方法，用来跨域请求
- TRACE: 追踪请求-响应的传输路径

## 3. GET 和 POST 有什么区别？

首先最直观的是语义上的区别。

而后又有这样一些具体的差别:

- 从缓存的角度，GET 请求会被浏览器主动缓存下来，留下历史记录，而 POST 默认不会。
- 从编码的角度，GET 只能进行 URL 编码，只能接收 ASCII 字符，而 POST 没有限制。
- 从参数的角度，GET 一般放在 URL 中，因此不安全，POST 放在请求体中，更适合传输敏感信息。
- 从幂等性的角度，GET 是幂等的，而 POST 不是。(幂等表示执行相同的操作，结果也是相同的)
- 从TCP的角度，GET 请求会把请求报文一次性发出去，而 POST 会分为两个 TCP 数据包，首先发 header 部分，如果服务器响应 100(continue)，然后发 body 部分。(火狐浏览器除外，它的 POST 请求只发一个 TCP 包)

## 4. 如何理解 URI？

URI, 全称为(Uniform Resource Identifier), 也就是**统一资源标识符**，它的作用很简单，就是区分互联网上不同的资源。

但是，它并不是我们常说的 网址，网址指的是 URL，实际上 URI 包含了 URN 和 URL 两个部分，由于 URL 过于普及，就默认将 URI 视为 URL 了。

### URI 的结构

URI 真正最完整的结构是这样的。



可能你会有疑问，好像跟平时见到的不太一样啊！先别急，我们来——拆解。

**scheme** 表示协议名，比如 `http`，`https`，`file` 等等。后面必须和 `://` 连在一起。

**user:passwd@** 表示登录主机时的用户信息，不过很不安全，不推荐使用，也不常用。

**host:port**表示主机名和端口。

**path**表示请求路径，标记资源所在位置。

**query**表示查询参数，为 `key=val` 这种形式，多个键值对之间用 `&` 隔开。

**fragment**表示 URI 所定位的资源内的一个锚点，浏览器可以根据这个锚点跳转到对应的位置。

举个例子：

```
https://www.baidu.com/s?wd=HTTP&rsv_spt=1
```

这个 URI 中，`https` 即 **scheme** 部分，`www.baidu.com` 为 **host:port** 部分（注意，`http` 和 `https` 的默认端口分别为 80、443），`/s` 为 **path** 部分，而 `wd=HTTP&rsv_spt=1` 就是 **query** 部分。

## URI 编码

URI 只能使用 **ASCII**，ASCII 之外的字符是不支持显示的，而且还有一部分符号是界定符，如果不加以处理就会导致解析出错。

因此，URI 引入了 **编码** 机制，将所有**非 ASCII 码字符**和**界定符**转为十六进制字节值，然后在前面加个 `%`。

如，空格被转义成了 `%20`，**三元**被转义成了 `%E4%B8%89%E5%85%83`。

## 5. 如何理解 HTTP 状态码？

RFC 规定 HTTP 的状态码为**三位数**，被分为五类：

- **1xx**: 表示目前是协议处理的中间状态，还需要后续操作。
- **2xx**: 表示成功状态。
- **3xx**: 重定向状态，资源位置发生变动，需要重新请求。
- **4xx**: 请求报文有误。
- **5xx**: 服务器端发生错误。

接下来就——分析这里具体的状态码。

### 1xx

**101 Switching Protocols**。在 `HTTP` 升级为 `WebSocket` 的时候，如果服务器同意变更，就会发送状态码 101。

### 2xx

**200 OK**是见得最多的成功状态码。通常在响应体中放有数据。

**204 No Content**含义与 200 相同，但响应头后没有 `body` 数据。

**206 Partial Content**顾名思义，表示部分内容，它的使用场景为 `HTTP` 分块下载和断点续传，当然也会带上相应的响应头字段 `Content-Range`。

### 3xx

**301 Moved Permanently**即永久重定向，对应着**302 Found**，即临时重定向。

比如你的网站从 HTTP 升级到了 HTTPS 了，以前的站点再也不用了，应当返回 **301**，这个时候浏览器默认会做缓存优化，在第二次访问的时候自动访问重定向的那个地址。

而如果只是暂时不可用，那么直接返回 302 即可，和 301 不同的是，浏览器并不会做缓存优化。

**304 Not Modified**: 当协商缓存命中时会返回这个状态码。

### 4xx

**400 Bad Request**: 开发者经常看到一头雾水，只是笼统地提示了一下错误，并不知道哪里出错了。

**403 Forbidden**: 这实际上并不是请求报文出错，而是服务器禁止访问，原因有很多，比如法律禁止、信息敏感。

**404 Not Found**: 资源未找到，表示没在服务器上找到相应的资源。

**405 Method Not Allowed**: 请求方法不被服务器端允许。

**406 Not Acceptable**: 资源无法满足客户端的条件。

**408 Request Timeout**: 服务器等待了太长时间。

**409 Conflict**: 多个请求发生了冲突。

**413 Request Entity Too Large**: 请求体的数据过大。

**414 Request-URI Too Long**: 请求行里的 URI 太大。

**429 Too Many Request**: 客户端发送的请求过多。

**431 Request Header Fields Too Large**请求头的字段内容太大。

### 5xx

**500 Internal Server Error**: 仅仅告诉你服务器出错了，出了啥错咱也不知道。

**501 Not Implemented**: 表示客户端请求的功能还不支持。

**502 Bad Gateway**: 服务器自身是正常的，但访问的时候出错了，啥错误咱也不知道。

**503 Service Unavailable**: 表示服务器当前很忙，暂时无法响应服务。

## 6. 简要概括一下 HTTP 的特点？HTTP 有哪些缺点？

### HTTP 特点

HTTP 的特点概括如下：

1. 灵活可扩展，主要体现在两个方面。一个是语义上的自由，只规定了基本格式，比如空格分隔单词，换行分隔字段，其他的各个部分都没有严格的语法限制。另一个是传输形式的多样性，不仅仅可以传输文本，还能传输图片、视频等任意数据，非常方便。
2. 可靠传输。HTTP 基于 TCP/IP，因此把这一特性继承了下来。这属于 TCP 的特性，不具体介绍了。

3. 请求-应答。也就是**一发一收**、**有来有回**，当然这个请求方和应答方不单单指客户端和服务端之间，如果某台服务器作为代理来连接后端的服务端，那么这台服务器也会扮演**请求方**的角色。
4. 无状态。这里的状态是指**通信过程的上下文信息**，而每次 http 请求都是独立、无关的，默认不需要保留状态信息。

## HTTP 缺点

### 无状态

所谓的优点和缺点还是要分场景来看的，对于 HTTP 而言，最具争议的地方在于它的无状态。

在需要长连接的场景中，需要保存大量的上下文信息，以免传输大量重复的信息，那么这时候无状态就是 http 的缺点了。

但与此同时，另外一些应用仅仅只是为了获取一些数据，不需要保存连接上下文信息，无状态反而减少了网络开销，成为了 http 的优点。

### 明文传输

即协议里的报文(主要指的是头部)不使用二进制数据，而是文本形式。

这当然对于调试提供了便利，但同时也让 HTTP 的报文信息暴露给了外界，给攻击者也提供了便利。

**WIFI陷阱** 就是利用 HTTP 明文传输的缺点，诱导你连上热点，然后疯狂抓你所有的流量，从而拿到你的敏感信息。

### 队头阻塞问题

当 http 开启长连接时，共用一个 TCP 连接，同一时刻只能处理一个请求，那么当前请求耗时过长的情况下，其它的请求只能处于阻塞状态，也就是著名的**队头阻塞**问题。接下来会有一小节讨论这个问题。

## 7. 对 Accept 系列字段了解多少？

对于 Accept 系列字段的介绍分为四个部分：**数据格式**、**压缩方式**、**支持语言**和**字符集**。

### 数据格式

HTTP 灵活的特性，它支持非常多的数据格式，那么这么多格式的数据一起到达客户端，客户端怎么知道它的格式呢？

当然，最低效的方式是直接猜，有没有更好的方式呢？直接指定可以吗？

答案是肯定的。不过首先需要介绍一个标准——**MIME**(Multipurpose Internet Mail Extensions, **多用途互联网邮件扩展**)。它首先用在电子邮件系统中，让邮件可以发任意类型的数据，这对于 HTTP 来说也是通用的。

因此，HTTP 从**MIME type**取了一部分来标记报文 body 部分的数据类型，这些类型体现在 **Content-Type** 这个字段，当然这是针对于发送端而言，接收端想要收到特定类型的数据，也可以用 Accept 字段。

具体而言，这两个字段的取值可以分为下面几类：

- text: text/html, text/plain, text/css 等
- image: image/gif, image/jpeg, image/png 等

- audio/video: audio/mpeg, video/mp4 等
- application: application/json, application/javascript, application/pdf, application/octet-stream 等

## 压缩方式

当然一般这些数据都是会进行编码压缩的，采取什么样的压缩方式就体现在了发送方的 `Content-Encoding` 字段上，同样的，接收什么样的压缩方式体现在了接受方的 `Accept-Encoding` 字段上。这个字段的取值有下面几种：

- gzip: 当今最流行的压缩格式
- deflate: 另外一种著名的压缩格式
- br: 一种专门为 HTTP 发明的压缩算法

```
// 发送端
Content-Encoding: gzip
// 接收端
Accept-Encoding: gzip
```

## 支持语言

对于发送方而言，还有一个 `Content-Language` 字段，在需要实现国际化的方案当中，可以用来指定支持的语言，在接受方对应的字段为 `Accept-Language`。如：

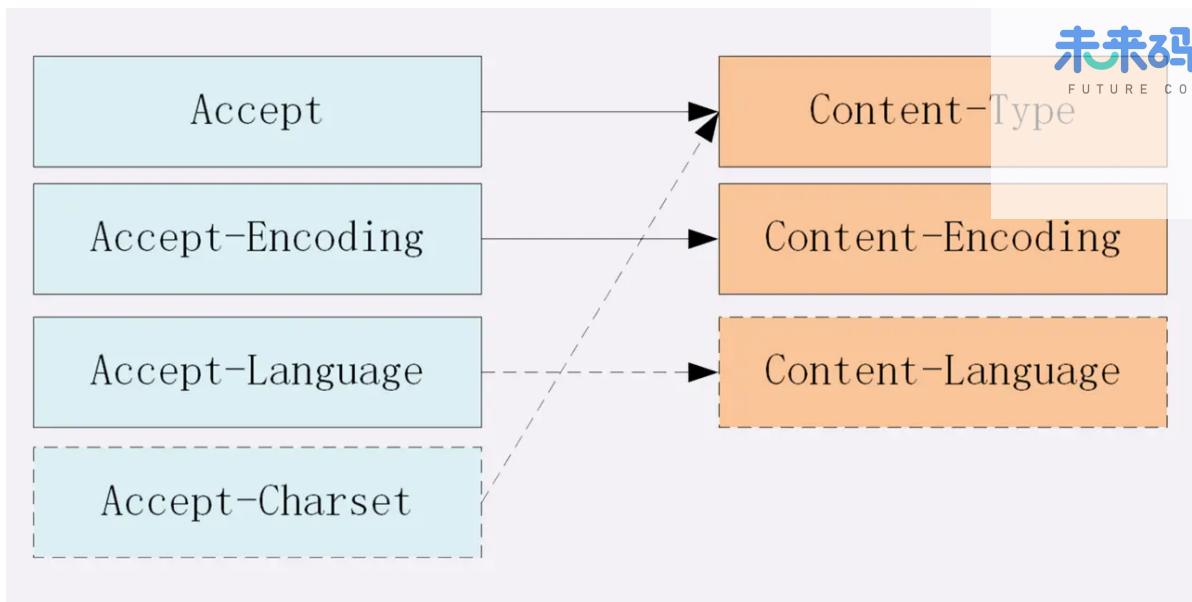
```
// 发送端
Content-Language: zh-CN, zh, en
// 接收端
Accept-Language: zh-CN, zh, en
```

## 字符集

最后是一个比较特殊的字段，在接收端对应为 `Accept-Charset`，指定可以接受的字符集，而在发送端并没有对应的 `Content-Charset`，而是直接放在了 `Content-Type` 中，以 `charset` 属性指定。如：

```
// 发送端
Content-Type: text/html; charset=utf-8
// 接收端
Accept-Charset: charset=utf-8
```

最后以一张图来总结一下吧：



## 8. 对于定长和不定长的数据，HTTP 是怎么传输的？

### 定长包体

对于定长包体而言，发送端在传输的时候一般会带上 `Content-Length`，来指明包体的长度。

我们用一个 `nodejs` 服务器来模拟一下：

```
const http = require('http');

const server = http.createServer();

server.on('request', (req, res) => {
  if(req.url === '/') {
    res.setHeader('Content-Type', 'text/plain');
    res.setHeader('Content-Length', 10);
    res.write("helloworld");
  }
})

server.listen(8081, () => {
  console.log("成功启动");
})
```

启动后访问: `localhost:8081`。

浏览器中显示如下：

```
helloworld
```

这是长度正确的情况，那不正确的情况是如何处理的呢？

我们试着把这个长度设置的小一些：

```
res.setHeader('Content-Length', 8);
```

重启服务，再次访问，现在浏览器中内容如下：



那后面的 `1d` 哪里去了呢？实际上在 http 的响应体中直接被截去了。

然后我们试着将这个长度设置得大一些：

```
res.setHeader('Content-Length', 12);
```

此时浏览器显示如下：



该网页无法正常运行

localhost 意外终止了连接。

直接无法显示了。可以看到 `Content-Length` 对于 http 传输过程起到了十分关键的作用，如果设置不当可以直接导致传输失败。

## 不定长包体

上述是针对 `定长包体`，那么对于 `不定长包体` 而言是如何传输的呢？

这里就必须介绍另外一个 http 头部字段了：

```
Transfer-Encoding: chunked
```

表示分块传输数据，设置这个字段后会自动产生两个效果：

- `Content-Length` 字段会被忽略
- 基于长连接持续推送动态内容

我们依然以一个实际的例子来模拟分块传输，nodejs 程序如下：

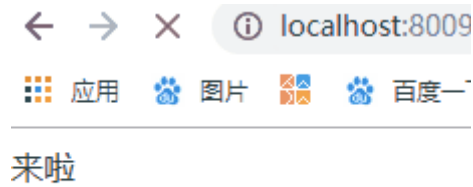
```
const http = require('http');

const server = http.createServer();

server.on('request', (req, res) => {
  if(req.url === '/') {
    res.setHeader('Content-Type', 'text/html; charset=utf8');
    res.setHeader('Content-Length', 10);
    res.setHeader('Transfer-Encoding', 'chunked');
    res.write("<p>来啦</p>");
    setTimeout(() => {
      res.write("第一次传输<br/>");
    }, 1000);
    setTimeout(() => {
      res.write("第二次传输");
      res.end()
    }, 2000);
  }
})
```

```
}  
  
server.listen(8009, () => {  
  console.log("成功启动");  
})
```

访问效果入下:



用 telnet 抓到的响应如下:

```
HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf8  
Content-Length: 10  
Transfer-Encoding: chunked  
Date: Sun, 23 Feb 2020 10:04:43 GMT  
Connection: keep-alive  
  
d  
<p>来啦</p>  
14  
第一次传输<br/>  
f  
第二次传输  
0
```

注意, `Connection: keep-alive` 及之前的为响应行和响应头, 后面的内容为响应体, 这两部分用换行符隔开。

响应体的结构比较有意思, 如下所示:

```
chunk长度(16进制的数)
第一个chunk的内容
chunk长度(16进制的数)
第二个chunk的内容
.....
0
```

最后是留有有一个空行的，这一点请大家注意。

以上便是 http 对于定长数据和不定长数据的传输方式。

## 9. HTTP 如何处理大文件的传输？

对于几百 M 甚至上 G 的大文件来说，如果要一口气全部传输过来显然是不现实的，会有大量的等待时间，严重影响用户体验。因此，HTTP 针对这一场景，采取了范围请求的解决方案，允许客户端仅仅请求一个资源的一部分。

### 如何支持

当然，前提是服务器要支持范围请求，要支持这个功能，就必须加上这样一个响应头：

```
Accept-Ranges: none
```

用来告知客户端这边是支持范围请求的。

### Range 字段拆解

而对于客户端而言，它需要指定请求哪一部分，通过 Range 这个请求头字段确定，格式为 bytes=x-y。接下来就来讨论一下这个 Range 的书写格式：

- 0-499表示从开始到第 499 个字节。
- 500- 表示从第 500 字节到文件终点。
- -100表示文件的最后100个字节。

服务器收到请求之后，首先验证范围是否合法，如果越界了那么返回 416 错误码，否则读取相应片段，返回 206 状态码。

同时，服务器需要添加 Content-Range 字段，这个字段的格式根据请求头中 Range 字段的的不同而有所差异。

具体来说，请求单段数据和请求多段数据，响应头是不一样的。

举个例子：

```
// 单段数据
Range: bytes=0-9
// 多段数据
Range: bytes=0-9, 30-39
```

接下来我们就分别来讨论着两种情况。

对于单段数据的请求，返回的响应如下：

```
HTTP/1.1 206 Partial Content
Content-Length: 10
Accept-Ranges: bytes
Content-Range: bytes 0-9/100

i am xxxxx
```

值得注意的是 Content-Range 字段，0-9 表示请求的返回，100 表示资源的总大小，很好理解。

## 多段数据

接下来我们看看多段请求的情况。得到的响应会是下面这个形式：

```
HTTP/1.1 206 Partial Content
Content-Type: multipart/byteranges; boundary=00000010101
Content-Length: 189
Connection: keep-alive
Accept-Ranges: bytes

--00000010101
Content-Type: text/plain
Content-Range: bytes 0-9/96

i am xxxxx
--00000010101
Content-Type: text/plain
Content-Range: bytes 20-29/96

eex jspy e
--00000010101--
```

这个时候出现了一个非常关键的字段 Content-Type：

multipart/byteranges;boundary=00000010101，它代表了信息量是这样的：

- 请求一定是多段数据请求
- 响应体中的分隔符是 00000010101

因此，在响应体中各段数据之间会由这里指定的分隔符分开，而且在最后的分隔末尾添上 -- 表示结束。

以上就是 http 针对大文件传输所采用的手段。

## 10. HTTP 中如何处理表单数据的提交？

在 http 中，有两种主要的表单提交的方式，体现在两种不同的 Content-Type 取值：

- application/x-www-form-urlencoded
- multipart/form-data

由于表单提交一般是 POST 请求，很少考虑 GET，因此这里我们将默认提交的数据放在请求体中。

## application/x-www-form-urlencoded

对于 `application/x-www-form-urlencoded` 格式的表单内容，有以下特点：

- 其中的数据会被编码成以 `&` 分隔的键值对
- 字符以 **URL 编码方式** 编码。

如：

```
// 转换过程：{a: 1, b: 2} -> a=1&b=2 -> 如下(最终形式)
"a%3D1%26b%3D2"
```

## multipart/form-data

对于 `multipart/form-data` 而言：

- 请求头中的 `Content-Type` 字段会包含 `boundary`，且 `boundary` 的值有浏览器默认指定。例：  
`Content-Type: multipart/form-data;boundary=----WebKitFormBoundaryRRJKewfHPGrS4LKe。`
- 数据会分为多个部分，每两个部分之间通过分隔符来分隔，每部分表述均有 HTTP 头部描述子包体，如 `Content-Type`，在最后的分隔符会加上 `--` 表示结束。

相应的 请求体 是下面这样：

```
Content-Disposition: form-data;name="data1";
Content-Type: text/plain
data1
----WebKitFormBoundaryRRJKewfHPGrS4LKe
Content-Disposition: form-data;name="data2";
Content-Type: text/plain
data2
----WebKitFormBoundaryRRJKewfHPGrS4LKe--
```

### 小结

值得一提的是，`multipart/form-data` 格式最大的特点在于：**每一个表单元素都是独立的资源表述**。另外，你可能在写业务的过程中，并没有注意到其中还有 `boundary` 的存在，如果你打开抓包工具，确实可以看到不同的表单元素被拆分开了，之所以在平时感觉不到，是以为浏览器和 HTTP 给你封装了这一系列操作。

而且，在实际的场景中，对于图片等文件的上传，基本采用 `multipart/form-data` 而不用 `application/x-www-form-urlencoded`，因为没有必要做 URL 编码，带来巨大耗时的同时也占用了更多的空间。

## 11. HTTP1.1 如何解决 HTTP 的队头阻塞问题？

### 什么是 HTTP 队头阻塞？

HTTP 传输是基于 **请求-应答** 的模式进行的，报文必须是一发一收，但值得注意的是，里面的任务被放在一个任务队列中串行执行，一旦队首的请求处理太慢，就会阻塞后面请求的处理。这就是著名的 **HTTP 队头阻塞** 问题。

## 并发连接

对于一个域名允许分配多个长连接，那么相当于增加了任务队列，不至于一个队伍的任务阻塞其它所有任务。在RFC2616规定过客户端最多并发 2 个连接，不过事实上在现在的浏览器标准中，这个上限要多很多，Chrome 中是 6 个。

但其实，即使是提高了并发连接，还是不能满足人们对性能的需求。

## 域名分片

一个域名不是可以并发 6 个长连接吗？那我就多分几个域名。

比如 content1.sanyuan.com、content2.sanyuan.com。

这样一个 sanyuan.com 域名下可以分出非常多的二级域名，而它们都指向同样的一台服务器，能够并发的长连接数更多了，事实上也更好地解决了队头阻塞的问题。

## 12. 对 Cookie 了解多少？

### Cookie 简介

HTTP 是一个无状态的协议，每次 http 请求都是独立、无关的，默认不需要保留状态信息。但有时候需要保存一些状态，怎么办呢？

HTTP 为此引入了 Cookie。Cookie 本质上就是浏览器里面存储的一个很小的文本文件，内部以键值对的方式来存储(在chrome开发者面板的Application这一栏可以看到)。向同一个域名下发送请求，都会携带相同的 Cookie，服务器拿到 Cookie 进行解析，便能拿到客户端的状态。而服务端可以通过响应头中的 Set-Cookie 字段来对客户端写入 Cookie。举例如下：

```
// 请求头
Cookie: a=xxx;b=xxx
// 响应头
Set-Cookie: a=xxx
set-cookie: b=xxx
```

### Cookie 属性

#### 1) 生存周期

Cookie 的有效期可以通过Expires和Max-Age两个属性来设置。

- Expires即 过期时间
- Max-Age用的是一段间隔，单位是秒，从浏览器收到报文开始计算。

若 Cookie 过期，则这个 Cookie 会被删除，并不会发送给服务端。

#### 2) 作用域

关于作用域也有两个属性: Domain和path, 给 Cookie 绑定了域名和路径，在发送请求之前，发现域名或者路径和这两个属性不匹配，那么就不会带上 Cookie。值得注意的是，对于路径来说，/ 表示域名下的任意路径都允许使用 Cookie。

#### 3) 安全相关

如果带上 `Secure`，说明只能通过 HTTPS 传输 cookie。

如果 cookie 字段带上 `HttpOnly`，那么说明只能通过 HTTP 协议传输，不能通过 JS 访问，这也是预防 XSS 攻击的重要手段。

相应的，对于 CSRF 攻击的预防，也有 `SameSite` 属性。

`SameSite` 可以设置为三个值，`Strict`、`Lax` 和 `None`。

- a. 在 `Strict` 模式下，浏览器完全禁止第三方请求携带 Cookie。比如请求 `sanyuan.com` 网站只能在 `sanyuan.com` 域名当中请求才能携带 Cookie，在其他网站请求都不能。
- b. 在 `Lax` 模式，就宽松一点了，但是只能在 `get` 方法提交表单 况或者 `a` 标签发送 `get` 请求 的情况下可以携带 Cookie，其他情况均不能。
- c. 在 `None` 模式下，也就是默认模式，请求会自动携带上 Cookie。

### Cookie 的缺点

- 1) 容量缺陷。Cookie 的体积上限只有 4KB，只能用来存储少量的信息。
- 2) 性能缺陷。Cookie 紧跟域名，不管域名下面的某一个地址需不需要这个 Cookie，请求都会携带上完整的 Cookie，这样随着请求数的增多，其实会造成巨大的性能浪费的，因为请求携带了很多不必要的内容。但可以通过 `Domain` 和 `Path` 指定作用域来解决。
- 3) 安全缺陷。由于 Cookie 以纯文本的形式在浏览器和服务器中传递，很容易被非法用户截获，然后进行一系列的篡改，在 Cookie 的有效期内重新发送给服务器，这是相当危险的。另外，在 `HttpOnly` 为 `false` 的情况下，Cookie 信息能直接通过 JS 脚本来读取。

## 13. 如何理解 HTTP 代理？

我们知道在 HTTP 是基于 请求-响应 模型的协议，一般由客户端发请求，服务器来进行响应。

当然，也有特殊情况，就是代理服务器的情况。引入代理之后，作为代理的服务器相当于一个中间人的角色，对于客户端而言，表现为服务器进行响应；而对于源服务器，表现为客户端发起请求，具有**双重身份**。

那代理服务器到底是用来做什么的呢？

### 功能

- 1) **负载均衡**。客户端的请求只会先到达代理服务器，后面到底有多少源服务器，IP 都是多少，客户端是不知道的。因此，这个代理服务器可以拿到这个请求之后，可以通过特定的算法分发给不同的源服务器，让各台源服务器的负载尽量平均。当然，这样的算法有很多，包括**随机算法**、**轮询**、**一致性hash**、**LRU**（最近最少使用）等等，不过这些算法并不是本文的重点，大家有兴趣自己可以研究一下。
- 2) **保障安全**。利用**心跳**机制监控后台的服务器，一旦发现故障机就将其踢出集群。并且对于上下行的数据进行过滤，对非法 IP 限流，这些都是代理服务器的工作。
- 3) **缓存代理**。将内容缓存到代理服务器，使得客户端可以直接从代理服务器获得而不用到源服务器那里。下一节详细拆解。

### 相关头部字段

## Via

代理服务器需要标明自己的身份，在 HTTP 传输中留下自己的痕迹，怎么办呢？

通过 `via` 字段来记录。举个例子，现在中间有两台代理服务器，在客户端发送请求后会经历这样一个过程：

```
客户端 -> 代理1 -> 代理2 -> 源服务器
```

在源服务器收到请求后，会在 `请求头` 拿到这个字段：

```
via: proxy_server1, proxy_server2
```

而源服务器响应时，最终在客户端会拿到这样的 `响应头`：

```
via: proxy_server2, proxy_server1
```

可以看到，`via` 中代理的顺序即为在 HTTP 传输中报文传达的顺序。

## X-Forwarded-For

字面意思就是 `为谁转发`，它记录的是 **请求方** 的 IP 地址(注意，和 `via` 区分开，`X-Forwarded-For` 记录的是请求方这一个 IP)。

## X-Real-IP

是一种获取用户真实 IP 的字段，不管中间经过多少代理，这个字段始终记录最初的客户端的 IP。

相应的，还有 `X-Forwarded-Host` 和 `X-Forwarded-Proto`，分别记录 **客户端**(注意哦，不包括代理)的 `域名` 和 `协议名`。

## X-Forwarded-For产生的问题

前面可以看到，`X-Forwarded-For` 这个字段记录的是请求方的 IP，这意味着每经过一个不同的代理，这个字段的 `名字` 都要变，从 `客户端` 到 `代理1`，这个字段是客户端的 IP，从 `代理1` 到 `代理2`，这个字段就变为了代理1的 IP。

但是这会产生两个问题：

- 1) 意味着代理必须解析 HTTP 请求头，然后修改，比直接转发数据性能下降。
- 2) 在 HTTPS 通信加密的过程中，原始报文是不允许修改的。

由此产生了 `代理协议`，一般使用明文版本，只需要在 HTTP 请求行上面加上这样格式的文本即可：

```
// PROXY + TCP4/TCP6 + 请求方地址 + 接收方地址 + 请求端口 + 接收端口
PROXY TCP4 0.0.0.1 0.0.0.2 1111 2222
GET / HTTP/1.1
...
```

这样就可以解决 `X-Forwarded-For` 带来的问题了。



## 14. 如何理解 HTTP 缓存及缓存代理?

首先通过 `Cache-Control` 验证强缓存是否可用

- 如果强缓存可用, 直接使用
- 否则进入协商缓存, 即发送 HTTP 请求, 服务器通过请求头中的 `If-Modified-Since` 或者 `If-None-Match` 这些条件请求字段检查资源是否更新
  - 若资源更新, 返回资源和 200 状态码
- 否则, 返回 304, 告诉浏览器直接从缓存获取资源

## 15. 为什么产生代理缓存?

对于源服务器来说, 它也是有缓存的, 比如 **Redis, Memcache**, 但对于 HTTP 缓存来说, 如果每次客户端缓存失效都要到源服务器获取, 那给源服务器的压力是很大的。

由此引入了**缓存代理**的机制。让 **代理服务器** 接管一部分的服务端 HTTP 缓存, 客户端缓存过期后**就近**到代理缓存中获取, 代理缓存过期了才请求源服务器, 这样流量巨大的时候能明显降低源服务器的压力。

那缓存代理究竟是如何做到的呢?

总的来说, 缓存代理的控制分为两部分, 一部分是**源服务器端**的控制, 一部分是**客户端**的控制。

## 16. 源服务器的缓存控制

### private 和 public

在源服务器的响应头中, 会加上 `Cache-Control` 这个字段进行缓存控制字段, 那么它的值当中可以加入 `private` 或者 `public` 表示是否允许代理服务器缓存, 前者禁止, 后者为允许。

比如对于一些非常私密的数据, 如果缓存到代理服务器, 别人直接访问代理就可以拿到这些数据, 是非常危险的, 因此对于这些数据一般是不会允许代理服务器进行缓存的, 将响应头部的 `Cache-Control` 设为 `private`, 而不是 `public`。

### proxy-revalidate

`must-revalidate` 的意思是**客户端**缓存过期就去源服务器获取, 而 `proxy-revalidate` 则表示**代理服务器**的缓存过期后到源服务器获取。

### s-maxage

`s` 是 `share` 的意思, 限定了缓存在代理服务器中可以存放多久, 和限制客户端缓存时间的 `max-age` 并不冲突。

讲了这几个字段, 我们不妨来举个小例子, 源服务器在响应头中加入这样一个字段:

```
Cache-Control: public, max-age=1000, s-maxage=2000
```

相当于源服务器说: 我这个响应是允许代理服务器缓存的, 客户端缓存过期了到代理中拿, 并且在客户端的缓存时间为 1000 秒, 在代理服务器中的缓存时间为 2000 s。

## 17. 客户端的缓存控制

### max-stale 和 min-fresh

在客户端的请求头中，可以加入这两个字段，来对代理服务器上的缓存进行**宽容**和**限制**操作。比如：

```
max-stale: 5
```

表示客户端到代理服务器上拿缓存的时候，即使代理缓存过期了也不要紧，只要过期时间在5秒之内，还是可以从代理中获取的。

又比如：

```
min-fresh: 5
```

表示代理缓存需要一定的新鲜度，不要等到缓存刚好到期再拿，一定要在**到期前 5 秒**之前的时间拿，否则拿不到。

### only-if-cached

这个字段加上后表示客户端只会接受代理缓存，而不会接受源服务器的响应。如果代理缓存无效，则直接返回 504 (Gateway Timeout)。

以上便是缓存代理的内容，涉及的字段比较多，希望能好好回顾一下，加深理解。

## 18. 什么是跨域？浏览器如何拦截响应？如何解决？

在前后端分离的开发模式中，经常会遇到跨域问题，即 Ajax 请求发出去了，服务器也成功响应了，前端就是拿不到这个响应。接下来我们就来好好讨论一下这个问题。

### 1) 什么是跨域

回顾一下 URI 的组成：



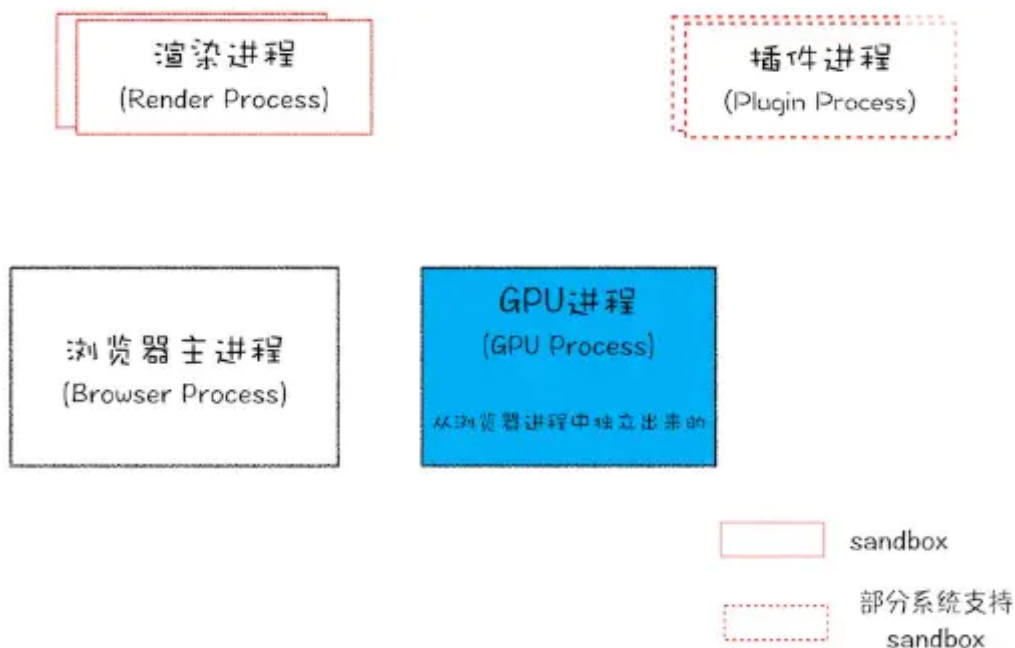
浏览器遵循**同源政策**（**scheme**（协议）、**host**（主机）和 **port**（端口）都相同则为**同源**）。非同源站点有这样一些限制：

- 不能读取和修改对方的 DOM
- 不读访问对方的 Cookie、IndexedDB 和 LocalStorage
- 限制 XMLHttpRequest 请求。（后面的话题着重围绕这个）

当浏览器向目标 URI 发 Ajax 请求时，只要当前 URL 和目标 URL 不同源，则产生跨域，被称为**跨域请求**。

跨域请求的响应一般会被浏览器所拦截，注意，是被浏览器拦截，响应其实是成功到达客户端了。那这个拦截是如何发生呢？

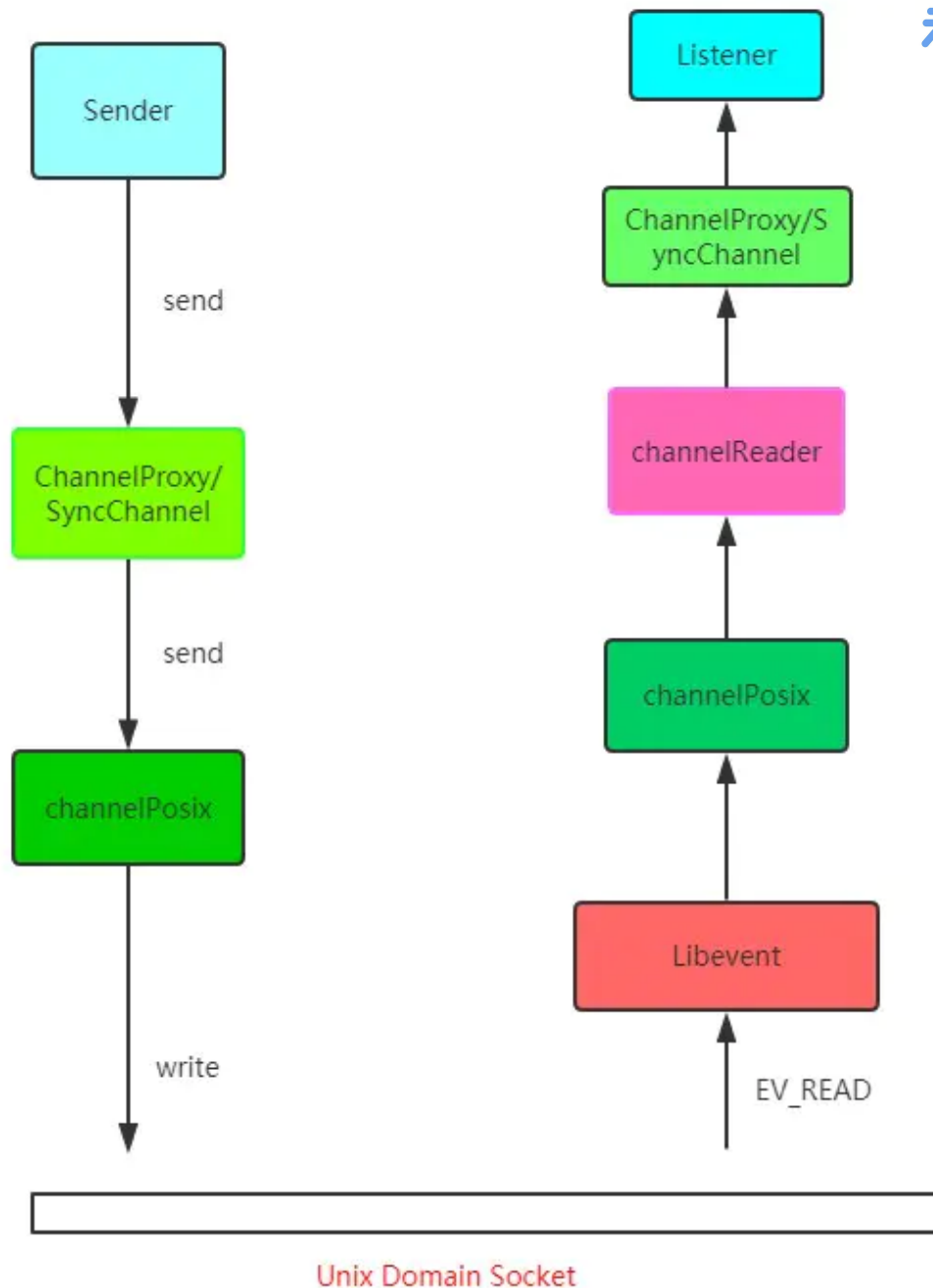
首先要知道的是，浏览器是多进程的，以 Chrome 为例，进程组成如下：



**WebKit 渲染引擎**和**V8 引擎**都在渲染进程当中。

当 `xhr.send` 被调用，即 Ajax 请求准备发送的时候，其实还只是在渲染进程的处理。为了防止黑客通过脚本触碰到系统资源，浏览器将每一个渲染进程装进了沙箱，并且为了防止 CPU 芯片一直存在的 **Spectre** 和 **Meltdown** 漏洞，采取了 **站点隔离** 的手段，给每一个不同的站点(一级域名不同)分配了沙箱，互不干扰。

在沙箱当中的渲染进程是没有办法发送网络请求的，那怎么办？只能通过网络进程来发送。那这样就涉及到进程间通信(IPC, Inter Process Communication)了。接下来我们看看 chromium 当中进程间通信是如何完成的，在 chromium 源码中调用顺序如下：



总的来说就是利用 Unix Domain Socket 套接字，配合事件驱动的高性能网络并发库 libevent 完成进程的 IPC 过程。

好，现在数据传递给了浏览器主进程，主进程接收到后，才真正地发出相应的网络请求。

在服务端处理完数据后，将响应返回，主进程检查到跨域，且没有cors(后面会详细说)响应头，将响应体全部丢掉，并不会发送给渲染进程。这就达到了拦截数据的目的。

接下来我们来说一说解决跨域问题的几种方案。

## CORS

CORS 其实是 W3C 的一个标准，全称是 跨域资源共享。它需要浏览器和服务器的共同支持，具体来说，非 IE 和 IE10 以上支持CORS，服务器需要附加特定的响应头，后面具体拆解。不过在弄清楚 CORS 的原理之前，我们需要清楚两个概念: **简单请求**和**非简单请求**。

浏览器根据请求方法和请求头的特定字段，将请求做了一下分类，具体来说规则是这样，凡是满足下面条件的属于**简单请求**:

- 请求方法为 GET、POST 或者 HEAD
- 请求头的取值范围: Accept、Accept-Language、Content-Language、Content-Type(只限于三个值 application/x-www-form-urlencoded、multipart/form-data、text/plain)

浏览器画了这样一个圈, 在这个圈里面的就是**简单请求**, 圈外面的就是**非简单请求**, 然后针对这两种不同的请求进行不同的处理。

## 简单请求

请求发出去之前, 浏览器做了什么?

它会自动在请求头当中, 添加一个 `Origin` 字段, 用来说明请求来自哪个源。服务器拿到请求之后, 在回应时对应地添加 `Access-Control-Allow-Origin` 字段, 如果 `Origin` 不在这个字段的范围中, 那么浏览器就会将响应拦截。

因此, `Access-Control-Allow-Origin` 字段是服务器用来决定浏览器是否拦截这个响应, 这是必需的字段。与此同时, 其它一些可选的功能性的字段, 用来描述如果不会拦截, 这些字段将会发挥各自的作用。

**Access-Control-Allow-Credentials.** 这个字段是一个布尔值, 表示是否允许发送 Cookie, 对于跨域请求, 浏览器对这个字段默认值设为 `false`, 而如果需要拿到浏览器的 Cookie, 需要添加这个响应头并设为 `true`, 并且在前端也需要设置 `withCredentials` 属性:

```
let xhr = new XMLHttpRequest();
xhr.withCredentials = true;
```

**Access-Control-Expose-Headers.** 这个字段是给 `XMLHttpRequest` 对象赋能, 让它不仅可以拿到基本的 6 个响应头字段 (包括 `Cache-Control`、`Content-Language`、`Content-Type`、`Expires`、`Last-Modified` 和 `Pragma`), 还能拿到这个字段声明的**响应头字段**。比如这样设置:

```
Access-Control-Expose-Headers: aaa
```

那么在前端可以通过 `XMLHttpRequest.getResponseHeader('aaa')` 拿到 `aaa` 这个字段的值。

## 非简单请求

非简单请求相对而言会有些不同, 体现在两个方面: **预检请求**和**响应字段**。

我们以 PUT 方法为例。

```
var url = 'http://xxx.com';
var xhr = new XMLHttpRequest();
xhr.open('PUT', url, true);
xhr.setRequestHeader('X-Custom-Header', 'xxx');
xhr.send();
复制代码
```

当这段代码执行后, 首先会发送**预检请求**。这个预检请求的请求行和请求体是下面这个格式:

```
OPTIONS / HTTP/1.1
Origin: 当前地址
Host: xxx.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: X-Custom-Header
复制代码
```

预检请求的方法是 `OPTIONS`，同时会加上 `Origin` 源地址和 `Host` 目标地址，这很简单。同时也会加上两个关键的字段：

- `Access-Control-Request-Method`，列出 CORS 请求用到哪个 HTTP 方法
- `Access-Control-Request-Headers`，指定 CORS 请求将要加上什么请求头

这是 `预检请求`。接下来是 **响应字段**，响应字段也分为两部分，一部分是对于 **预检请求** 的响应，一部分是对于 **CORS 请求** 的响应。

**预检请求的响应**。如下面的格式：

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: X-Custom-Header
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 1728000
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
Content-Length: 0
```

其中有这样几个关键的 **响应头字段**：

- `Access-Control-Allow-Origin`：表示可以允许请求的源，可以填具体的源名，也可以填 `*` 表示允许任意源请求。
- `Access-Control-Allow-Methods`：表示允许的请求方法列表。
- `Access-Control-Allow-Credentials`：简单请求中已经介绍。
- `Access-Control-Allow-Headers`：表示允许发送的请求头字段
- `Access-Control-Max-Age`：预检请求的有效期，在此期间，不用发出另外一条预检请求。

在预检请求的响应返回后，如果请求不满足响应头的条件，则触发 `XMLHttpRequest` 的 `onerror` 方法，当然后面真正的 **CORS 请求** 也不会发出去了。

**CORS 请求的响应**。绕了这么一大转，到了真正的 CORS 请求就容易多了，现在它和 **简单请求** 的情况是一样的。浏览器自动加上 `Origin` 字段，服务端响应头返回 **`Access-Control-Allow-Origin`**。可以参考以上简单请求部分的内容。

## JSONP

虽然 `XMLHttpRequest` 对象遵循同源政策，但是 `script` 标签不一样，它可以通过 `src` 填上目标地址从而发出 GET 请求，实现跨域请求并拿到响应。这也就是 JSONP 的原理，接下来我们就来封装一个 JSONP：

```
const jsonp = ({ url, params, callbackName }) => {
  const generateURL = () => {
    let dataStr = '';
    for (let key in params) {
      dataStr += `${key}=${params[key]}&`;
    }
  }
}
```

```

    }
    dataStr += `callback=${callbackName}`;
    return `${url}?${dataStr}`;
  };
  return new Promise((resolve, reject) => {
    // 初始化回调函数名称
    callbackName = callbackName || Math.random().toString.replace('.', '');
    // 创建 script 元素并加入到当前文档中
    let scriptEle = document.createElement('script');
    scriptEle.src = generateURL();
    document.body.appendChild(scriptEle);
    // 绑定到 window 上, 为了后面调用
    window[callbackName] = (data) => {
      resolve(data);
      // script 执行完了, 成为无用元素, 需要清除
      document.body.removeChild(scriptEle);
    }
  });
}

```

当然在服务端也会有响应的操作, 以 express 为例:

```

let express = require('express')
let app = express()
app.get('/', function(req, res) {
  let { a, b, callback } = req.query
  console.log(a); // 1
  console.log(b); // 2
  // 注意哦, 返回给script标签, 浏览器直接把这部分字符串执行
  res.end(`${callback}('数据包')`);
})
app.listen(3000)

```

前端这样简单地调用一下就好了:

```

jsonp({
  url: 'http://localhost:3000',
  params: {
    a: 1,
    b: 2
  }
}).then(data => {
  // 拿到数据进行处理
  console.log(data); // 数据包
})

```

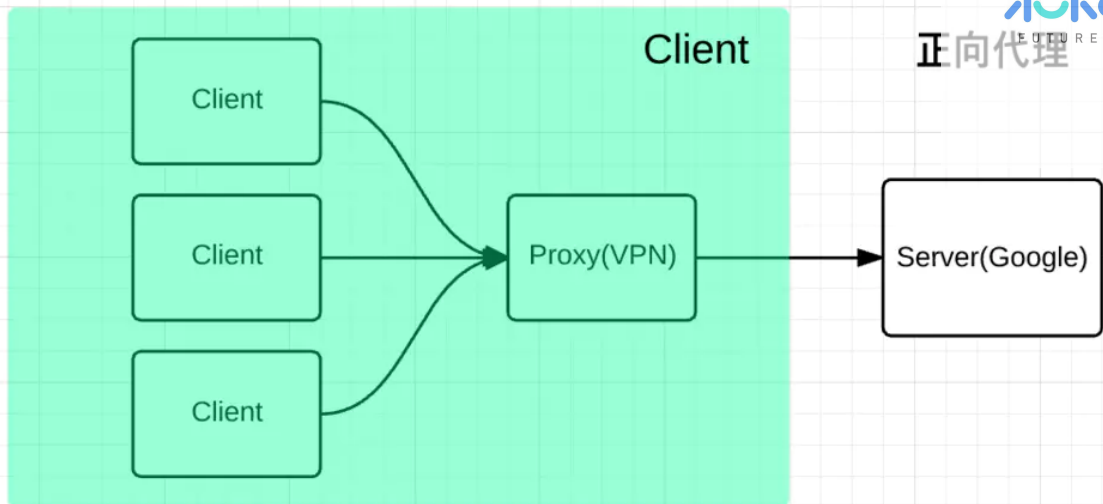
和 CORS 相比, JSONP 最大的优势在于兼容性好, IE 低版本不能使用 CORS 但可以使用 JSONP, 缺点也很明显, 请求方法单一, 只支持 GET 请求。

## Nginx

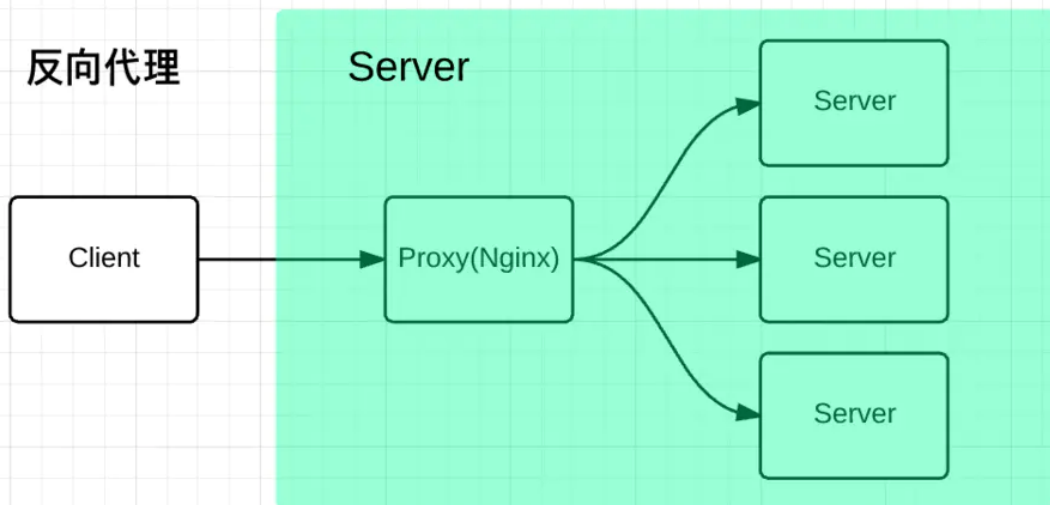
Nginx 是一种高性能的 反向代理 服务器, 可以用来轻松解决跨域问题。

what? 反向代理? 我给你看一张图你就懂了。





## 反向代理



正向代理帮助客户端**访问**客户端自己访问不到的服务器，然后将结果返回给客户端。

反向代理拿到客户端的请求，将请求转发给其他的服务器，主要的场景是维持服务器集群的**负载均衡**，换句话说，反向代理帮**其它的服务器**拿到请求，然后选择一个合适的服务器，将请求转交给它。

因此，两者的区别就很明显了，正向代理服务器是帮**客户端**做事情，而反向代理服务器是帮其它的**服务器**做事情。

好了，那 Nginx 是如何来解决跨域的呢？

比如说现在客户端的域名为**client.com**，服务器的域名为**server.com**，客户端向服务器发送 Ajax 请求，当然会跨域了，那这个时候让 Nginx 登场了，通过下面这个配置：

```
server {
    listen 80;
    server_name client.com;
    location /api {
        proxy_pass server.com;
    }
}
```

Nginx 相当于起了一个跳板机，这个跳板机的域名也是 **client.com**，让客户端首先访问 **client.com/api**，这当然没有跨域，然后 Nginx 服务器作为反向代理，将请求转发给 **server.com**，当响应返回时又将响应给到客户端，这就完成整个跨域请求的过程。



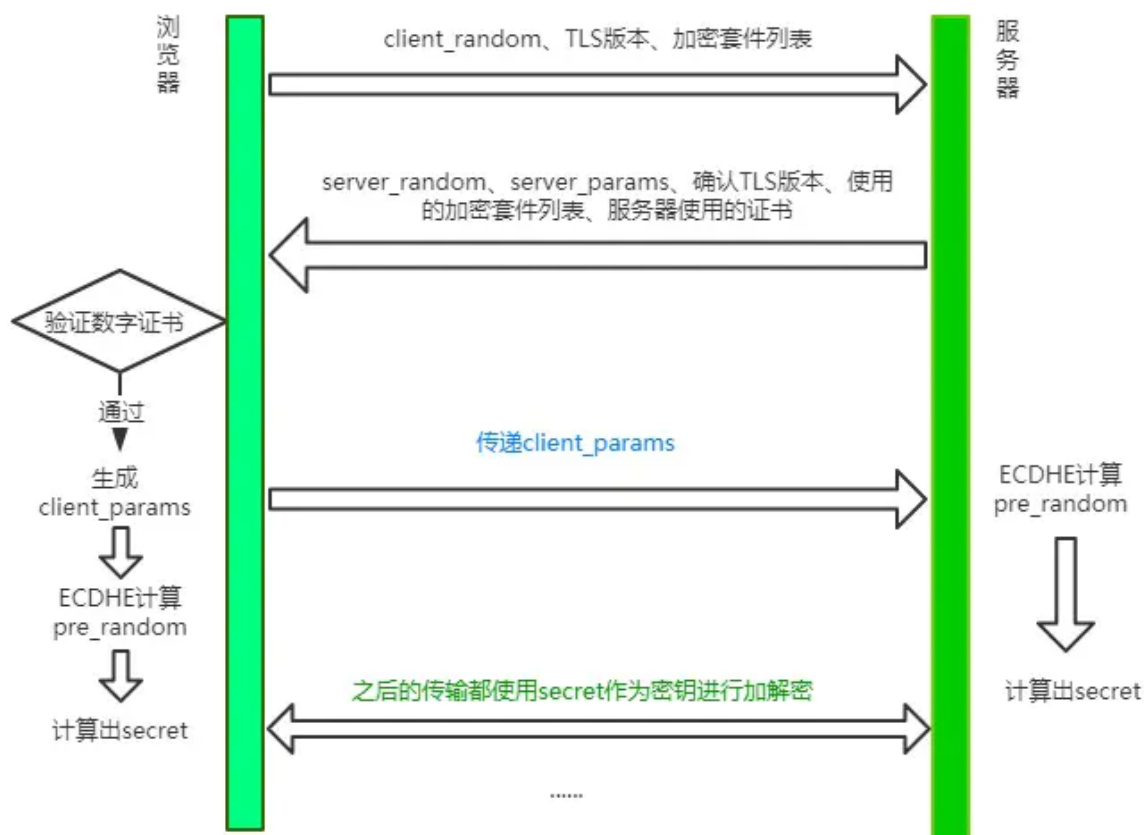
其实还有一些不太常用的方式，大家了解即可，比如 `postMessage`，当然 `WebSocket` 也是一种方式，但是已经不属于 HTTP 的范畴，另外一些奇技淫巧就不建议大家去死记硬背了，一方面都难得记住，另一方面临时背下来，面试官也不会对你印象加分，因为看得出来是背的。当然没有背并不代表减分，把跨域原理和前面三种主要的跨域方式理解清楚，经得起更深一步的推敲，反而会让别人觉得你是一个靠谱的人。

## 19. 传统 RSA 握手

先来说说传统的 TLS 握手，也是大家在网上经常看到的。

### TLS 1.2 握手过程

现在我们来讲讲主流的 TLS 1.2 版本所采用的方式。



刚开始你可能会比较懵，先别着急，过一遍下面的流程再来看会豁然开朗。

#### step 1: Client Hello

首先，浏览器发送 `client_random`、TLS版本、加密套件列表。

`client_random` 是什么？用来最终 `secret` 的一个参数。

加密套件列表是什么？我举个例子，加密套件列表一般长这样：

```
TLS_ECDHE_WITH_AES_128_GCM_SHA256
```

意思是 TLS 握手过程中, 使用 ECDHE 算法生成 `pre_random` (这个数后面会介绍), 128位的 AES 算法进行对称加密, 在对称加密的过程中使用主流的 GCM 分组模式, 因为对称加密中很重要的一个问题就是如何分组。最后一个为哈希摘要算法, 采用 SHA256 算法。

其中值得解释一下的是这个哈希摘要算法, 试想一个这样的场景, 服务端现在给客户端发消息来了, 客户端并不知道此时的消息到底是服务端发的, 还是中间人伪造的消息呢? 现在引入这个哈希摘要算法, 将服务端的证书信息通过**这个算法**生成一个摘要(可以理解为 比较短的字符串), 用来**标识**这个服务端的身份, 用私钥加密后把**加密后的标识和自己的公钥**传给客户端。客户端拿到**这个公钥**来解密, 生成另外一份摘要。两个摘要进行对比, 如果相同则能确认服务端的身份。这也就是所谓**数字签名**的原理。其中除了哈希算法, 最重要的过程是**私钥加密, 公钥解密**。

## step 2: Server Hello

可以看到服务器一口气给客户端回复了非常多的内容。

`server_random` 也是最后生成 `secret` 的一个参数, 同时确认 TLS 版本、需要使用的加密套件和自己的证书, 这都不难理解。那剩下的 `server_params` 是干嘛的呢?

我们先埋个伏笔, 现在你只需要知道, `server_random` 到达了客户端。

## step 3: Client 验证证书, 生成secret

客户端验证服务端传来的 证书 和 签名 是否通过, 如果验证通过, 则传递 `client_params` 这个参数给服务器。

接着客户端通过 ECDHE 算法计算出 `pre_random`, 其中传入两个参数:`server_params`和 `client_params`。现在你应该清楚这两个参数的作用了吧, 由于 ECDHE 基于 椭圆曲线离散对数, 这两个参数也称作 椭圆曲线的公钥。

客户端现在拥有了 `client_random`、`server_random` 和 `pre_random`, 接下来将这三个数通过一个伪随机数函数来计算出最终的 `secret`。

## step4: Server 生成 secret

刚刚客户端不是传了 `client_params` 过来了吗?

现在服务端开始用 ECDHE 算法生成 `pre_random`, 接着用和客户端同样的伪随机数函数生成最后的 `secret`。

## 注意事项

TLS的过程基本上讲完了, 但还有两点需要注意。

**第一**、实际上 TLS 握手是一个**双向认证**的过程, 从 step1 中可以看到, 客户端有能力验证服务器的身份, 那服务器能不能验证客户端的身份呢?

当然是可以的。具体来说, 在 step3 中, 客户端传送 `client_params`, 实际上给服务器传一个验证消息, 让服务器将相同的验证流程(哈希摘要 + 私钥加密 + 公钥解密)走一遍, 确认客户端的身份。

**第二**、当客户端生成 `secret` 后, 会给服务端发送一个收尾的消息, 告诉服务器之后的都用对称加密, 对称加密的算法就用第一次约定的。服务器生成完 `secret` 也会向客户端发送一个收尾的消息, 告诉客户端以后就直接用对称加密来通信。

这个收尾的消息包括两部分，一部分是 `Change Cipher Spec`，意味着后面加密传输了，另一个是 `Finished` 消息，这个消息是对之前所有发送的数据做的摘要，对摘要进行加密，让对方验证。

当双方都验证通过之后，握手才正式结束。后面的 HTTP 正式开始传输加密报文。

## 20. RSA 和 ECDHE 握手过程的区别

- 1) ECDHE 握手，也就是主流的 TLS1.2 握手中，使用 ECDHE 实现 `pre_random` 的加密解密，没有用到 RSA。
- 2) 使用 ECDHE 还有一个特点，就是客户端发送完收尾消息后可以提前 `抢跑`，直接发送 HTTP 报文，节省了一个 RTT，不必等到收尾消息到达服务器，然后等服务器返回收尾消息给自己，直接开始发请求。这也叫 `TLS False Start`。

## 21. TLS 1.3 做了哪些改进？

TLS 1.2 虽然存在了 10 多年，经历了无数的考验，但历史的车轮总是不断向前的，为了获得更强的安全、更优秀的性能，在 2018 年就推出了 TLS1.3，对于 TLS1.2 做了一系列的改进，主要分为这几个部分：**强化安全、提高性能**。

### 强化安全

在 TLS1.3 中废除了非常多的加密算法，最后只保留五个加密套件：

- TLS\_AES\_128\_GCM\_SHA256
- TLS\_AES\_256\_GCM\_SHA384
- TLS\_CHACHA20\_POLY1305\_SHA256
- TLS\_AES\_128\_GCM\_SHA256
- TLS\_AES\_128\_GCM\_8\_SHA256

可以看到，最后剩下的对称加密算法只有 **AES** 和 **CHACHA20**，之前主流的也会这两种。分组模式也只剩下 **GCM** 和 **POLY1305**，哈希摘要算法只剩下了 **SHA256** 和 **SHA384** 了。

那你可能会问了，之前 **RSA** 这么重要的非对称加密算法怎么不在了？

我觉得有两方面的原因：

**第一**、2015 年发现了 **FREAK** 攻击，即已经有人发现了 RSA 的漏洞，能够进行破解了。

**第二**、一旦私钥泄露，那么中间人可以通过私钥计算出之前所有报文的 `secret`，破解之前所有的密文。

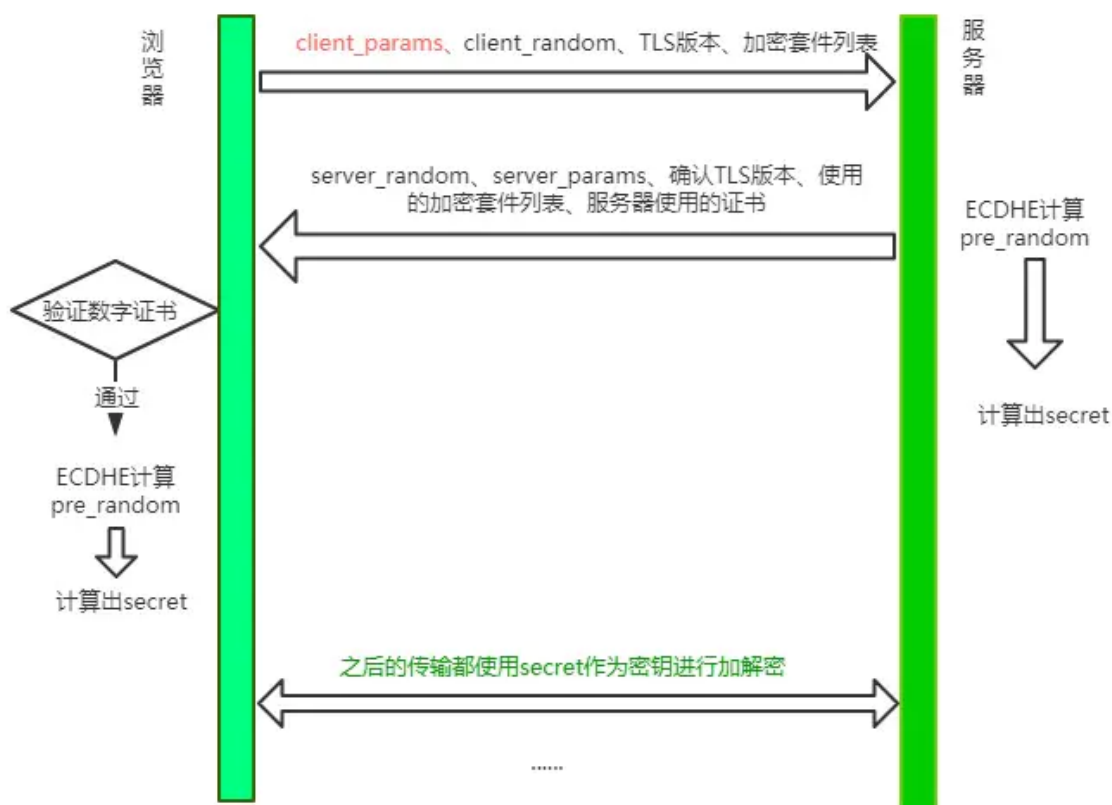
为什么？回到 RSA 握手的过程中，客户端拿到服务器的证书后，提取出服务器的公钥，然后生成 `pre_random` 并用公钥加密传给服务器，服务器通过私钥解密，从而拿到真实的 `pre_random`。当中间人拿到了服务器私钥，并且截获之前所有报文的时候，那么就能拿到 `pre_random`、`server_random` 和 `client_random` 并根据对应的随机数函数生成 `secret`，也就是拿到了 TLS 最终的会话密钥，每一个历史报文都能通过这样的方式进行破解。

但 ECDHE 在每次握手时都会生成临时的密钥对，即使私钥被破解，之前的历史消息并不会收到影响。这种一次破解并不影响历史信息性质也叫**前向安全性**。

**RSA** 算法不具备前向安全性，而 **ECDHE** 具备，因此在 TLS1.3 中彻底取代了 **RSA**。

## 握手改进

流程如下:



大体的方式和 TLS1.2 差不多, 不过和 TLS 1.2 相比少了一个 RTT, 服务端不必等待对方验证证书之后才拿到 `client_params`, 而是直接在第一次握手的时候就能够拿到, 拿到之后立即计算 `secret`, 节省了之前不必要的等待时间。同时, 这也意味着在第一次握手的时候客户端需要传送更多的信息, 一口气给传完。

这种 TLS 1.3 握手方式也被叫做**1-RTT握手**。但其实这种 1-RTT 的握手方式还是有一些优化的空间的, 接下来我们来一一介绍这些优化方式。

## 会话复用

会话复用有两种方式: **Session ID**和**Session Ticket**。

先说说最早出现的**Session ID**, 具体做法是客户端和服务端首次连接后各自保存会话的 ID, 并存储会话密钥, 当再次连接时, 客户端发送 ID 过来, 服务器查找这个 ID 是否存在, 如果找到了就直接复用之前的会话状态, 会话密钥不用重新生成, 直接用原来的那份。

但这种方式也存在一个弊端, 就是当客户端数量庞大的时候, 对服务端的存储压力非常大。

因而出现了第二种方式——**Session Ticket**。它的思路就是: 服务端的压力大, 那就把压力分摊给客户端呗。具体来说, 双方连接成功后, 服务器加密会话信息, 用**Session Ticket**消息发给客户端, 让客户端保存下来。下次重连的时候, 就把这个 Ticket 进行解密, 验证它过没过期, 如果没过期那就直接恢复之前的会话状态。

这种方式虽然减小了服务端的存储压力，但也带来了安全问题，即每次用一个固定的密钥来解密 Ticket 数据，一旦黑客拿到这个密钥，之前所有的历史记录也被破解了。因此为了尽量避免这样的问题，密钥需要定期进行更换。

总的来说，这些会话复用的技术在保证 1-RTT 的同时，也节省了生成会话密钥这些算法所消耗的时间，是一笔可观的性能提升。

## PSK

刚刚说的都是 1-RTT 情况下的优化，那能不能优化到 0-RTT 呢？

答案是可以的。做法其实也很简单，在发送 Session Ticket 的同时带上应用数据，不用等到服务端确认，这种方式被称为 Pre-Shared Key，即 PSK。

这种方式虽然方便，但也带来了安全问题。中间人截获 PSK 的数据，不断向服务器重复发，类似于 TCP 第一次握手携带数据，增加了服务器被攻击的风险。

## 总结

TLS1.3 在 TLS1.2 的基础上废除了大量的算法，提升了安全性。同时利用会话复用节省了重新生成密钥的时间，利用 PSK 做到了 0-RTT 连接。

## 22. HTTP/2 有哪些改进？

由于 HTTPS 在安全方面已经做的非常好了，HTTP 改进的关注点放在了性能方面。对于 HTTP/2 而言，它对于性能的提升主要在于两点：

- 头部压缩
- 多路复用

当然还有一些颠覆性的功能实现：

- 设置请求优先级
- 服务器推送

这些重大的提升本质上也是为了解决 HTTP 本身的问题而产生的。接下来我们来看看 HTTP/2 解决了哪些问题，以及解决方式具体是如何的。

### 头部压缩

在 HTTP/1.1 及之前的时代，**请求体**一般会有响应的压缩编码过程，通过 Content-Encoding 头部字段来指定，但你有没有想过头部字段本身的压缩呢？当请求字段非常复杂的时候，尤其对于 GET 请求，请求报文几乎全是请求头，这个时候还是存在非常大的优化空间的。HTTP/2 针对头部字段，也采用了对应的压缩算法——HPACK，对请求头进行压缩。

HPACK 算法是专门为 HTTP/2 服务的，它主要的亮点有两个：

- 首先是在服务器和客户端之间建立哈希表，将用到的字段存放在这张表中，那么在传输的时候对于之前出现过的值，只需要把**索引**(比如 0, 1, 2, ...)传给对方即可，对方拿到索引查表就行了。这种**传索引**的方式，可以说让请求头字段得到极大程度的精简和复用。

| Index | Header Name      | Header Value |
|-------|------------------|--------------|
| 1     | :authority       |              |
| 2     | :method          | GET          |
| 3     | :method          | POST         |
| 4     | :path            | /            |
| 5     | :path            | /index.html  |
| 6     | :scheme          | http         |
| 7     | :scheme          | https        |
| 8     | :status          | 200          |
| ...   | ....             | ...          |
| 59    | vary             |              |
| 60    | via              |              |
| 61    | www-authenticate |              |

HTTP/2 当中废除了起始行的概念，将起始行中的请求方法、URI、状态码转换成了头字段，不过这些字段都有一个 ":" 前缀，用来和其它请求头区分开。

- 其次是对整数和字符串进行**哈夫曼编码**，哈夫曼编码的原理就是先将所有出现的字符建立一张索引表，然后让出现次数多的字符对应的索引尽可能短，传输的时候也是传输这样的**索引序列**，可以达到非常高的压缩率。

## 多路复用

### HTTP 队头阻塞

我们之前讨论了 HTTP 队头阻塞的问题，其根本原因在于 HTTP 基于 **请求-响应** 的模型，在同一个 TCP 长连接中，前面的请求没有得到响应，后面的请求就会被阻塞。

后面我们又讨论到用**并发连接**和**域名分片**的方式来解决这个问题，但这并没有真正从 HTTP 本身的层面解决问题，只是增加了 TCP 连接，分摊风险而已。而且这么做也有弊端，多条 TCP 连接会竞争**有限的带宽**，让真正优先级高的请求不能优先处理。

而 HTTP/2 便从 HTTP 协议本身解决了 **队头阻塞** 问题。注意，这里并不是指的 **TCP 队头阻塞**，而是 **HTTP 队头阻塞**，两者并不是一回事。TCP 的队头阻塞是在 **数据包** 层面，单位是 **数据包**，前一个报文没有收到便不会将后面收到的报文上传给 HTTP，而 HTTP 的队头阻塞是在 **HTTP 请求-响应** 层面，前一个请求没处理完，后面的请求就要阻塞住。两者所在的层次不一样。

那么 HTTP/2 如何来解决所谓的队头阻塞呢？

### 二进制分帧

首先，HTTP/2 认为明文传输对机器而言太麻烦了，不方便计算机的解析，因为对于文本而言会有多义性的字符，比如回车换行到底是内容还是分隔符，在内部需要用到状态机去识别，效率比较低。于是 HTTP/2 干脆把报文全部换成二进制格式，全部传输 **01** 串，方便了机器的解析。



原来 Headers + Body 的报文格式如今被拆分成了一个二进制的帧，用**Headers 帧**存放头部字段，**Data 帧**存放请求体数据。分帧之后，服务器看到的不再是一个个完整的 HTTP 请求报文，而是一堆乱序的二进制帧。这些二进制帧不存在先后关系，因此也就不会排队等待，也就没有了 HTTP 的队头阻塞问题。

通信双方都可以给对方发送二进制帧，这种二进制帧的**双向传输的序列**，也叫做**流 (Stream)**。HTTP/2 用**流**来在一个 TCP 连接上来进行多个数据帧的通信，这就是**多路复用**的概念。

可能你会有一个疑问，既然是乱序首发，那最后如何来处理这些乱序的数据帧呢？

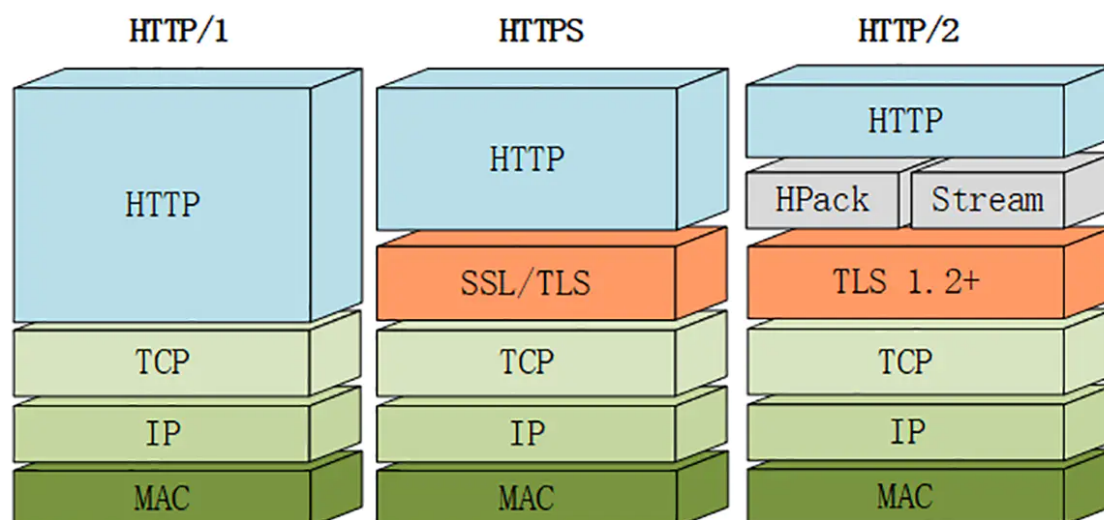
首先要声明的是，所谓的乱序，指的是不同 ID 的 Stream 是乱序的，但同一个 Stream ID 的帧一定是按顺序传输的。二进制帧到达后对方会将 Stream ID 相同的二进制帧组装成完整的**请求报文**和**响应报文**。当然，在二进制帧当中还有其他的一些字段，实现了**优先级**和**流量控制**等功能，我们放到下一节再来介绍。

## 服务器推送

另外值得一说的是 HTTP/2 的服务器推送(Server Push)。在 HTTP/2 当中，服务器已经不再是完全被动地接收请求，响应请求，它也能新建 stream 来给客户端发送消息，当 TCP 连接建立之后，比如浏览器请求一个 HTML 文件，服务器就可以在返回 HTML 的基础上，将 HTML 中引用到的其他资源文件一起返回给客户端，减少客户端的等待。

## 总结

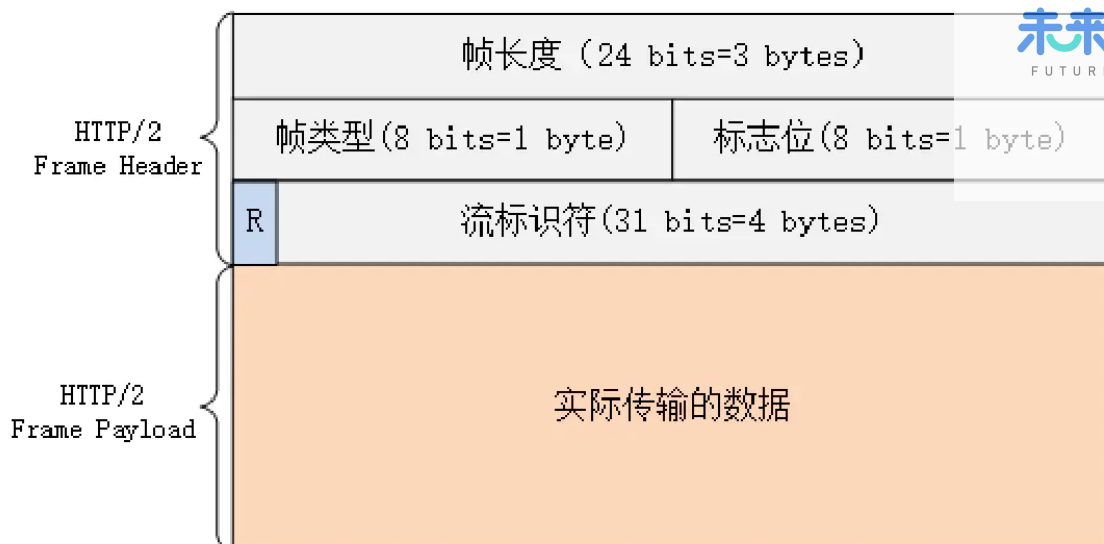
当然，HTTP/2 新增那么多的特性，是不是 HTTP 的语法要重新学呢？不需要，HTTP/2 完全兼容之前 HTTP 的语法和语义，如**请求头**、**URI**、**状态码**、**头部字段**都没有改变，完全不用担心。同时，在安全方面，HTTP 也支持 TLS，并且现在主流的浏览器都公开只支持加密的 HTTP/2，因此你现在能看到的 HTTP/2 也基本上都是跑在 TLS 上面的了。最后放一张分层图给大家参考：



## 23. HTTP/2 中的二进制帧是如何设计的？

### 帧结构

HTTP/2 中传输的帧结构如下图所示：



每个帧分为 帧头 和 帧体。先是三个字节的帧长度，这个长度表示的是 帧体 的长度。

然后是帧类型，大概可以分为 数据帧 和 控制帧 两种。数据帧用来存放 HTTP 报文，控制帧用来管理 流 的传输。

接下来的一个字节是 帧标志，里面一共有 8 个标志位，常用的有 **END\_HEADERS** 表示头数据结束，**END\_STREAM** 表示单方向数据发送结束。

后 4 个字节是 Stream ID，也就是 流标识符，有了它，接收方就能从乱序的二进制帧中选择出 ID 相同的帧，按顺序组装成请求/响应报文。

### 流的状态变化

从前面可以知道，在 HTTP/2 中，所谓的 流，其实就是二进制帧的 双向传输的序列。那么在 HTTP/2 请求和响应的过程中，流的状态是如何变化的呢？

HTTP/2 其实也是借鉴了 TCP 状态变化的思想，根据帧的标志位来实现具体的状态改变。这里我们以一个普通的 请求-响应 过程为例来说明：





最开始两者都是空闲状态，当客户端发送 Headers 帧后，开始分配 Stream ID，此时客户端的流打开，服务端接收之后服务端的流也打开，两端的流都打开之后，就可以互相传递数据帧和控制帧了。

当客户端要关闭时，向服务端发送 END\_STREAM 帧，进入半关闭状态，这个时候客户端只能接收数据，而不能发送数据。

服务端收到这个 END\_STREAM 帧后也进入半关闭状态，不过此时服务端的情况是只能发送数据，而不能接收数据。随后服务端也向客户端发送 END\_STREAM 帧，表示数据发送完毕，双方进入关闭状态。

如果下次要开启新的流，流 ID 需要自增，直到上限为止，到达上限后开一个新的 TCP 连接重头开始计数。由于流 ID 字段长度为 4 个字节，最高位又被保留，因此范围是 0 ~ 2 的 31 次方，大约 21 亿个。

## 流的特性

刚刚谈到了流的状态变化过程，这里顺便就来总结一下流传输的特性：

- 并发性。一个 HTTP/2 连接上可以同时发多个帧，这一点和 HTTP/1 不同。这也是实现多路复用的基础。
- 自增性。流 ID 是不可重用的，而是会按顺序递增，达到上限之后又新开 TCP 连接从头开始。
- 双向性。客户端和服务端都可以创建流，互不干扰，双方都可以作为发送方或者接收方。
- 可设置优先级。可以设置数据帧的优先级，让服务端先处理重要资源，优化用户体验。

## TCP协议

## 1. 能不能说一说 TCP 和 UDP 的区别？

首先概括一下基本的区别：

**TCP是一个面向连接的、可靠的、基于字节流的传输层协议。**

而**UDP是一个面向无连接的传输层协议。**(就这么简单，其它TCP的特性也就没有了)。

具体分析，和 UDP 相比，TCP 有三大核心特性：

**1) 面向连接。**所谓的连接，指的是客户端和服务器的连接，在双方互相通信之前，TCP 需要三次握手建立连接，而 UDP 没有相应建立连接的过程。

**2) 可靠性。**TCP 花了非常多的功夫保证连接的可靠，这个可靠性体现在哪些方面呢？一个是有状态，另一个是可控制。

TCP 会精准记录哪些数据发送了，哪些数据被对方接收了，哪些没有被接收到，而且保证数据包按序到达，不允许半点差错。这是**有状态**。

当意识到丢包了或者网络环境不佳，TCP 会根据具体情况调整自己的行为，控制自己的发送速度或者重发。这是**可控制**。

相应的，UDP 就是**无状态**，**不可控**的。

**3) 面向字节流。**UDP 的数据传输是基于数据报的，这是因为仅仅只是继承了 IP 层的特性，而 TCP 为了维护状态，将一个个 IP 包变成了字节流。

## 2. 说说 TCP 三次握手的过程？

### 恋爱模拟

以谈恋爱为例，两个人能够在一起最重要的事情是首先确认各自**爱**和**被爱**的能力。接下来我们以此来模拟三次握手的过程。

第一次：

男：**我爱你。**

女方收到。

由此证明男方拥有**爱**的能力。

第二次：

女：**我收到了你的爱，我也爱你。**

男方收到。

OK，现在的情况说明，女方拥有**爱**和**被爱**的能力。

第三次：

男：**我收到了你的爱。**

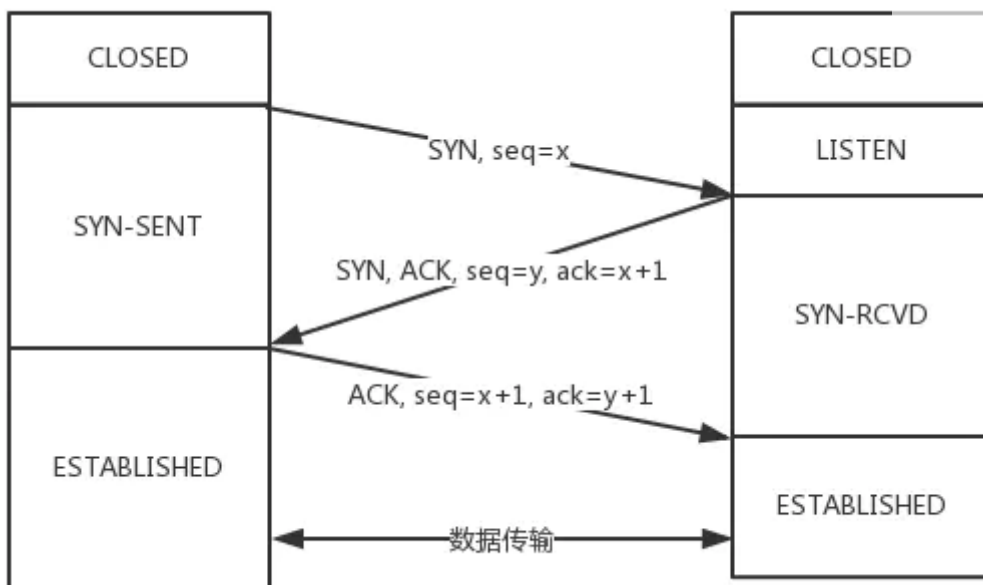
女方收到。

现在能够保证男方具备**被爱**的能力。

由此完整地确认了双方**爱**和**被爱**的能力，两人开始一段甜蜜的爱情。

### 真实握手

当然刚刚那段属于扯淡，不代表本人价值观，目的是让大家理解整个握手过程的意义，因为两个过程非常相似。对应到 TCP 的三次握手，也是需要确认双方的两样能力：发送的能力和接收的能力。于是便会有下面的三次握手的过程：



从最开始双方都处于 `CLOSED` 状态。然后服务端开始监听某个端口，进入了 `LISTEN` 状态。

然后客户端主动发起连接，发送 `SYN`，自己变成了 `SYN-SENT` 状态。

服务端接收到，返回 `SYN` 和 `ACK` (对应客户端发来的 `SYN`)，自己变成了 `SYN-RCVD`。

之后客户端再发送 `ACK` 给服务端，自己变成了 `ESTABLISHED` 状态；服务端收到 `ACK` 之后，也变成了 `ESTABLISHED` 状态。

另外需要提醒你注意的是，从图中可以看出，`SYN` 是需要消耗一个序列号的，下次发送对应的 `ACK` 序列号要加1，为什么呢？只需要记住一个规则：

凡是需要对端确认的，一定消耗TCP报文的序列号。

`SYN` 需要对端的确认，而 `ACK` 并不需要，因此 `SYN` 消耗一个序列号而 `ACK` 不需要。

### 3. 为什么是三次而不是两次、四次？

#### 1) 为什么不是两次？

根本原因: 无法确认客户端的接收能力。

分析如下:

如果是两次，你现在发了 `SYN` 报文想握手，但是这个包`滞留`在了当前的网络中迟迟没有到达，TCP 以为这是丢了包，于是重传，两次握手建立好了连接。

看似没有问题，但是连接关闭后，如果这个`滞留`在网路中的包到达了服务端呢？这时候由于是两次握手，服务端只要接收到然后发送相应的数据包，就默认`建立连接`，但是现在客户端已经断开了。

看到问题的吧，这就带来了连接资源的浪费。

#### 2) 为什么不是四次？

三次握手的目的是确认双方 `发送` 和 `接收` 的能力，那四次握手可以嘛？

当然可以，100 次都可以。但为了解决问题，三次就足够了，再多用处就不大了。

#### 4. 三次握手过程中可以携带数据么？

第三次握手的时候，可以携带。前两次握手不能携带数据。

如果前两次握手能够携带数据，那么一旦有人想攻击服务器，那么他只需要在第一次握手时的 SYN 报文中放大量数据，那么服务器势必会消耗更多的时间和内存空间去处理这些数据，增大了服务器被攻击的风险。

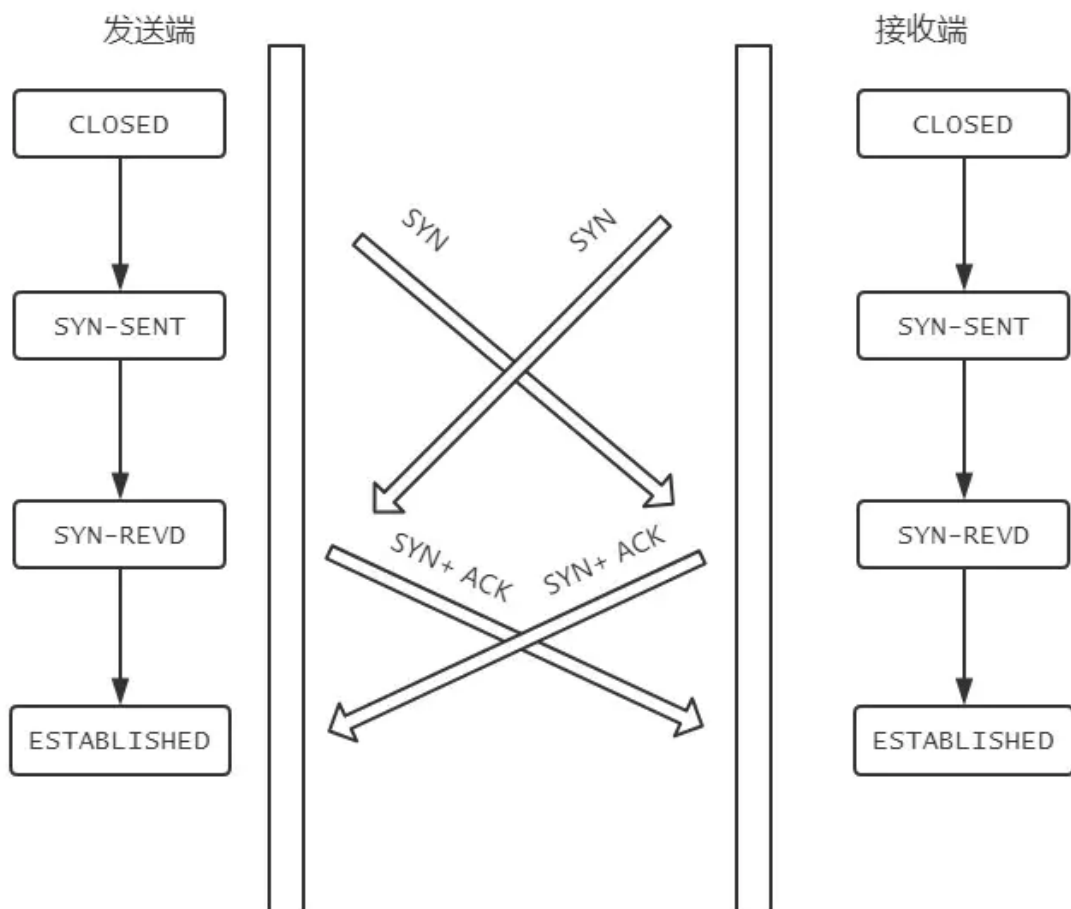
第三次握手的时候，客户端已经处于 ESTABLISHED 状态，并且已经能够确认服务器的接收、发送能力正常，这个时候相对安全了，可以携带数据。

#### 5. 同时打开会怎样？

如果双方同时发 SYN 报文，状态变化会是怎样的呢？

这是一个可能会发生的情况。

状态变迁如下：



在发送方给接收方发 SYN 报文的同时，接收方也给发送方发 SYN 报文，两个人刚上了！

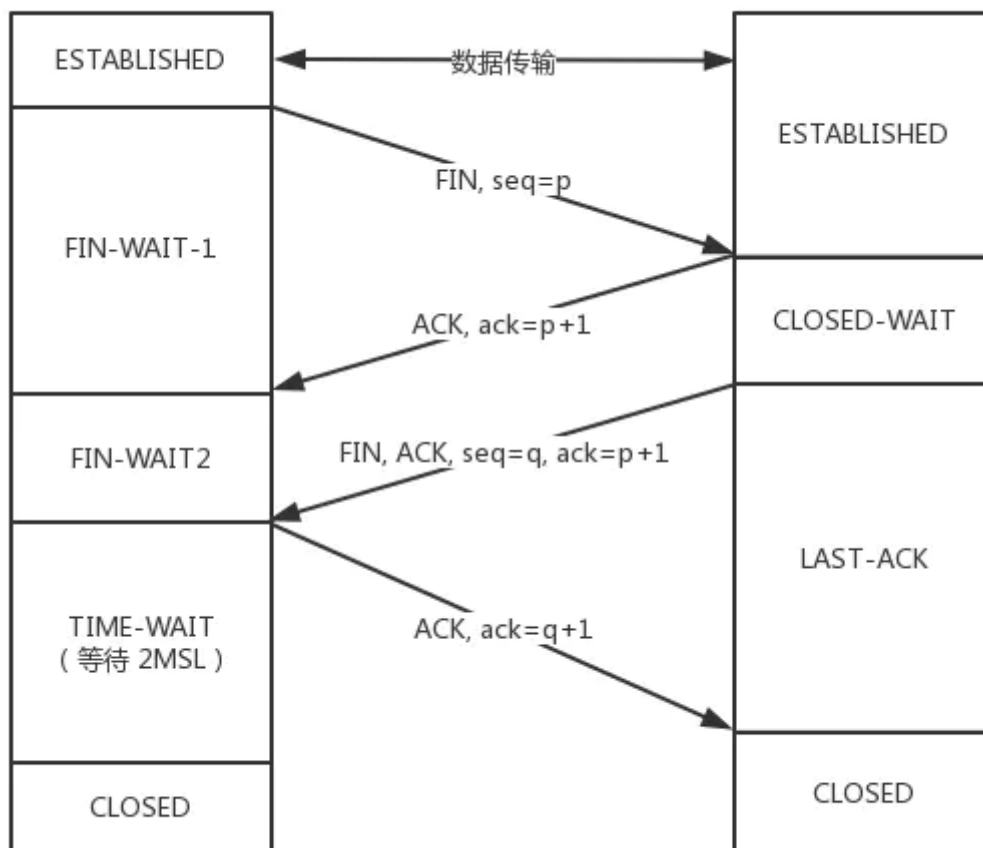
发完 SYN，两者的状态都变为 SYN-SENT。

在各自收到对方的 SYN 后，两者状态都变为 SYN-REVD。

接着会回复对应的 `ACK + SYN`，这个报文在对方接收之后，两者状态一起变为 `ESTABLISHED`。  
这就是同时打开情况下的状态变迁。

## 6. 说说 TCP 四次挥手的过程

### 过程拆解



刚开始双方处于 `ESTABLISHED` 状态。

客户端要断开了，向服务器发送 `FIN` 报文，在 TCP 报文中的位置如下图：

|                            |    |        |             |                        |             |                      |             |             |             |             |                   |
|----------------------------|----|--------|-------------|------------------------|-------------|----------------------|-------------|-------------|-------------|-------------|-------------------|
| 0                          |    | 1      |             | 2                      |             | 3                    |             |             |             |             |                   |
| 源端口（Source port）           |    |        |             | 目标端口（Destination port） |             |                      |             |             |             |             |                   |
| 序列号（Sequence number）       |    |        |             |                        |             |                      |             |             |             |             |                   |
| 确认号（Acknowledgment number） |    |        |             |                        |             |                      |             |             |             |             |                   |
| 头部长度                       | 保留 | N<br>S | C<br>W<br>R | E<br>C<br>E            | U<br>R<br>G | A<br>C<br>K<br>H     | P<br>S<br>T | R<br>S<br>T | S<br>Y<br>N | F<br>I<br>N | 窗口大小（Window Size） |
| 校验和（Checksum）              |    |        |             |                        |             | 紧急指针（Urgent pointer） |             |             |             |             |                   |
| 选项（Options）、填充（Padding）    |    |        |             |                        |             |                      |             |             |             |             |                   |

发送后客户端变成了 `FIN-WAIT-1` 状态。注意，这时候客户端同时也变成了 `half-close`(半关闭) 状态，即无法向服务端发送报文，只能接收。

服务端接收后向客户端确认，变成了 `CLOSED-WAIT` 状态。

客户端接收到了服务端的确认，变成了 `FIN-WAIT2` 状态。

随后，服务端向客户端发送 `FIN`，自己进入 `LAST-ACK` 状态，

客户端收到服务端发来的 `FIN` 后，自己变成了 `TIME-WAIT` 状态，然后发送 `ACK` 给服务端。

注意了，这个时候，客户端需要等待足够长的时间，具体来说，是 2 个 MSL (Maximum Segment Lifetime, 报文最大生存时间)，在这段时间内如果客户端没有收到服务端的重发请求，那么表示 ACK 成功到达，挥手结束，否则客户端重发 ACK。

## 等待2MSL的意义

如果不等待会怎样？

如果不等待，客户端直接跑路，当服务端还有很多数据包要给客户端发，且还在路上的时候，若客户端的端口此时刚好被新的应用占用，那么就接收到了无用数据包，造成数据包混乱。所以，最保险的做法是等服务器发来的数据包都死翘翘再启动新的应用。

那，照这样说一个 MSL 不就够了吗，为什么要等待 2 MSL？

- 1 个 MSL 确保四次挥手中主动关闭方最后的 ACK 报文最终能达到对端
- 1 个 MSL 确保对端没有收到 ACK 重传的 FIN 报文可以到达

这就是等待 2MSL 的意义。

## 7. 为什么是四次挥手而不是三次？

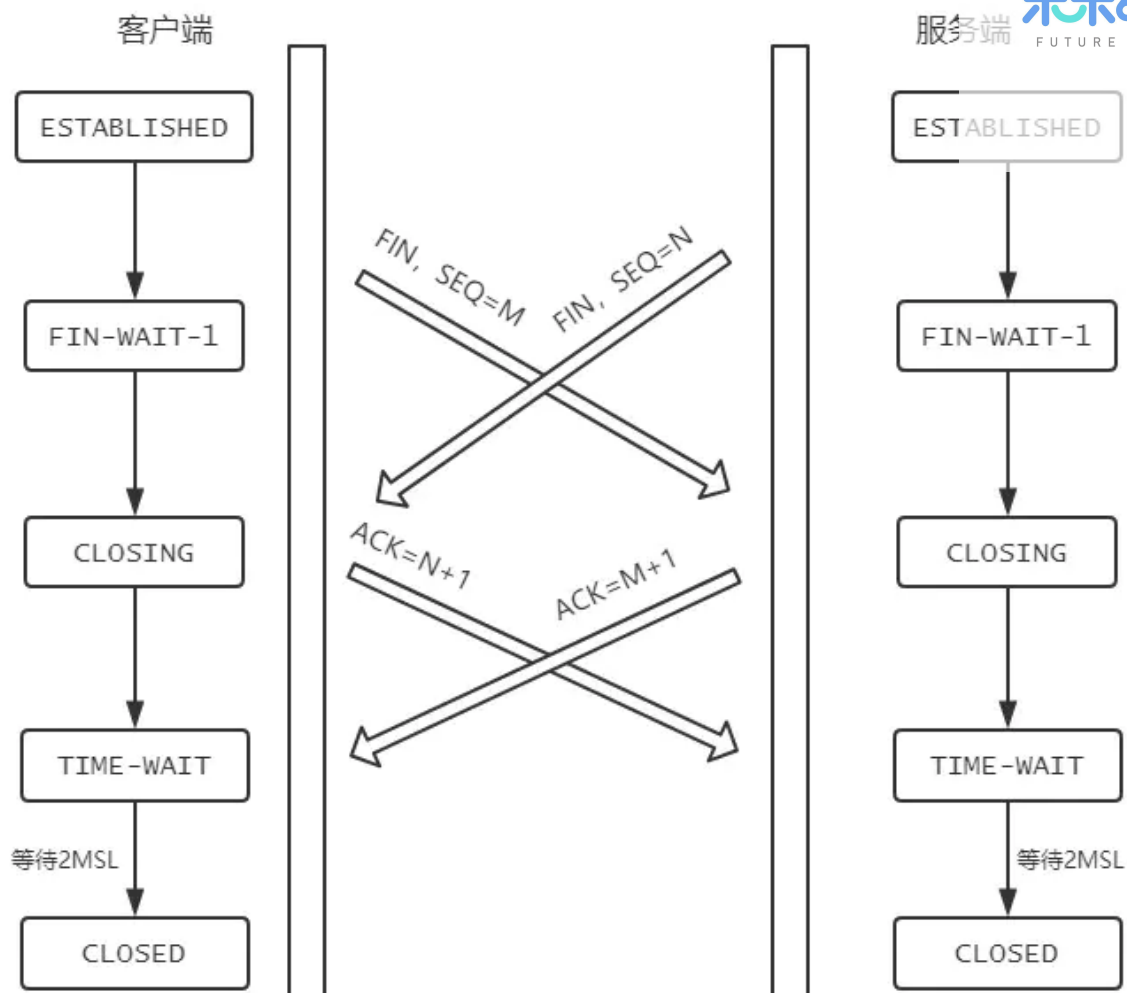
因为服务端在接收到 FIN，往往不会立即返回 FIN，必须等到服务端所有的报文都发送完毕了，才能发 FIN。因此先发一个 ACK 表示已经收到客户端的 FIN，延迟一段时间才发 FIN。这就造成了四次挥手。

如果是三次挥手会有什么问题？

等于说服务端将 ACK 和 FIN 的发送合并为一次挥手，这个时候长时间的延迟可能会导致客户端误以为 FIN 没有到达客户端，从而让客户端不断的重发 FIN。

## 8. 同时关闭会怎样？

如果客户端和服务端同时发送 FIN，状态会如何变化？如图所示：



## 9. 说说半连接队列和 SYN Flood 攻击的关系

三次握手前，服务端的状态从 `CLOSED` 变为 `LISTEN`，同时在内部创建了两个队列：**半连接队列**和**全连接队列**，即**SYN队列**和**ACCEPT队列**。

### 半连接队列

当客户端发送 `SYN` 到服务端，服务端收到以后回复 `ACK` 和 `SYN`，状态由 `LISTEN` 变为 `SYN_RCVD`，此时这个连接就被推入了**SYN队列**，也就是**半连接队列**。

### 全连接队列

当客户端返回 `ACK`，服务端接收后，三次握手完成。这个时候连接等待被具体的应用取走，在被取走之前，它会被推入另外一个 `TCP` 维护的队列，也就是**全连接队列(Accept Queue)**。

### SYN Flood 攻击原理

SYN Flood 属于典型的 DoS/DDoS 攻击。其攻击的原理很简单，就是用客户端在短时间内伪造大量不存在的 IP 地址，并向服务端疯狂发送 `SYN`。对于服务端而言，会产生两个危险的后果：

1) 处理大量的 SYN 包并返回对应 ACK, 势必有大量连接处于 SYN\_RCVD 状态, 从而占满整个半连接队列, 无法处理正常的请求。

2) 由于是不存在的 IP, 服务端长时间收不到客户端的 ACK, 会导致服务端不断重发数据, 直到耗尽服务端的资源。

## 10. 如何应对 SYN Flood 攻击?

1. 增加 SYN 连接, 也就是增加半连接队列的容量。
2. 减少 SYN + ACK 重试次数, 避免大量的超时重发。
3. 利用 SYN Cookie 技术, 在服务端接收到 SYN 后不立即分配连接资源, 而是根据这个 SYN 计算出一个 Cookie, 连同第二次握手回复给客户端, 在客户端回复 ACK 的时候带上这个 Cookie 值, 服务端验证 Cookie 合法之后才分配连接资源。

## 11. 介绍一下 TCP 报文头部的字段

报文头部结构如下(单位为字节):

|                            |    |        |             |                        |                            |                                                                         |                   |
|----------------------------|----|--------|-------------|------------------------|----------------------------|-------------------------------------------------------------------------|-------------------|
| 0                          |    | 1      |             | 2                      |                            | 3                                                                       |                   |
| 源端口（Source port）           |    |        |             | 目标端口（Destination port） |                            |                                                                         |                   |
| 序列号（Sequence number）       |    |        |             |                        |                            |                                                                         |                   |
| 确认号（Acknowledgment number） |    |        |             |                        |                            |                                                                         |                   |
| 头部长度                       | 保留 | N<br>S | C<br>W<br>R | E<br>C<br>R<br>E       | U<br>R<br>G<br>E<br>N<br>T | A<br>C<br>K<br>N<br>O<br>W<br>L<br>E<br>D<br>G<br>E<br>M<br>E<br>N<br>T | 窗口大小（Window Size） |
| 校验和（Checksum）              |    |        |             | 紧急指针（Urgent pointer）   |                            |                                                                         |                   |
| 选项（Options）、填充（Padding）    |    |        |             |                        |                            |                                                                         |                   |

### 源端口、目标端口

如何标识唯一标识一个连接? 答案是 TCP 连接的 四元组——源 IP、源端口、目标 IP 和目标端口。

那 TCP 报文怎么没有源 IP 和目标 IP 呢? 这是因为在 IP 层就已经处理了 IP。TCP 只需要记录两者的端口即可。

### 序列号

即 Sequence number, 指的是本报文段第一个字节的序列号。

从图中可以看出, 序列号是一个长为 4 个字节, 也就是 32 位的无符号整数, 表示范围为  $0 \sim 2^{32} - 1$ 。如果到达最大值了后就循环到 0。

序列号在 TCP 通信的过程中有两个作用:

1. 在 SYN 报文中交换彼此的初始序列号。
2. 保证数据包按正确的顺序组装。

### ISN

即 Initial Sequence Number (初始序列号), 在三次握手的过程当中, 双方会用过 SYN 报文来交换彼此的 ISN。



ISN 并不是一个固定的值，而是每 4 ms 加一，溢出则回到 0，这个算法使得猜测 ISN 变得很困难，那为什么要这么做？

如果 ISN 被攻击者预测到，要知道源 IP 和源端口号都是很容易伪造的，当攻击者猜测 ISN 之后，直接伪造一个 RST 后，就可以强制连接关闭的，这是非常危险的。

而动态增长的 ISN 大大提高了猜测 ISN 的难度。

## 确认号

即 ACK(Acknowledgment number)。用来告知对方下一个期望接收的序列号，**小于ACK**的所有字节已经全部收到。

## 标记位

常见的标记位有 SYN, ACK, FIN, RST, PSH。

SYN 和 ACK 已经在上文说过，后三个解释如下：FIN：即 Finish，表示发送方准备断开连接。

RST：即 Reset，用来强制断开连接。

PSH：即 Push，告知对方这些数据包收到后应该马上交给上层的应用，不能缓存。

## 窗口大小

占用两个字节，也就是 16 位，但实际上是不够用的。因此 TCP 引入了窗口缩放的选项，作为窗口缩放的比例因子，这个比例因子的范围在 0 ~ 14，比例因子可以将窗口的值扩大为原来的  $2^n$  次方。

## 校验和

占用两个字节，防止传输过程中数据包有损坏，如果遇到校验和有差错的报文，TCP 直接丢弃之，等待重传。

## 可选项

可选项的格式如下：

| 种类 (Kind) 1 byte | 长度 (Length) 1 byte | 值 (value) |
|------------------|--------------------|-----------|
|------------------|--------------------|-----------|

常用的可选项有以下几个：

- Timestamp: TCP 时间戳，后面详细介绍。
- MSS: 指的是 TCP 允许的从对方接收的最大报文段。
- SACK: 选择确认选项。
- Window Scale: 窗口缩放选项。

## 12. 说说 TCP 快速打开的原理(TFO)

第一节讲了 TCP 三次握手，可能有人会说，每次都三次握手好麻烦呀！能不能优化一点？

可以啊。今天来说说这个优化后的 TCP 握手流程，也就是 TCP 快速打开(TCP Fast Open, 即TFO)的原理。

优化的过程是这样的，还记得我们说 SYN Flood 攻击时提到的 SYN Cookie 吗？这个 Cookie 可不是浏览器的 Cookie，用它同样可以实现 TFO。

### 1) TFO 流程

#### 首轮三次握手

首先客户端发送 SYN 给服务端，服务端接收到。

注意哦！现在服务端不是立刻回复 SYN + ACK，而是通过计算得到一个 SYN Cookie，将这个 Cookie 放到 TCP 报文的 Fast Open 选项中，然后才给客户端返回。

客户端拿到这个 Cookie 的值缓存下来。后面正常完成三次握手。

首轮三次握手就是这样的流程。而后面的三次握手就不一样啦！

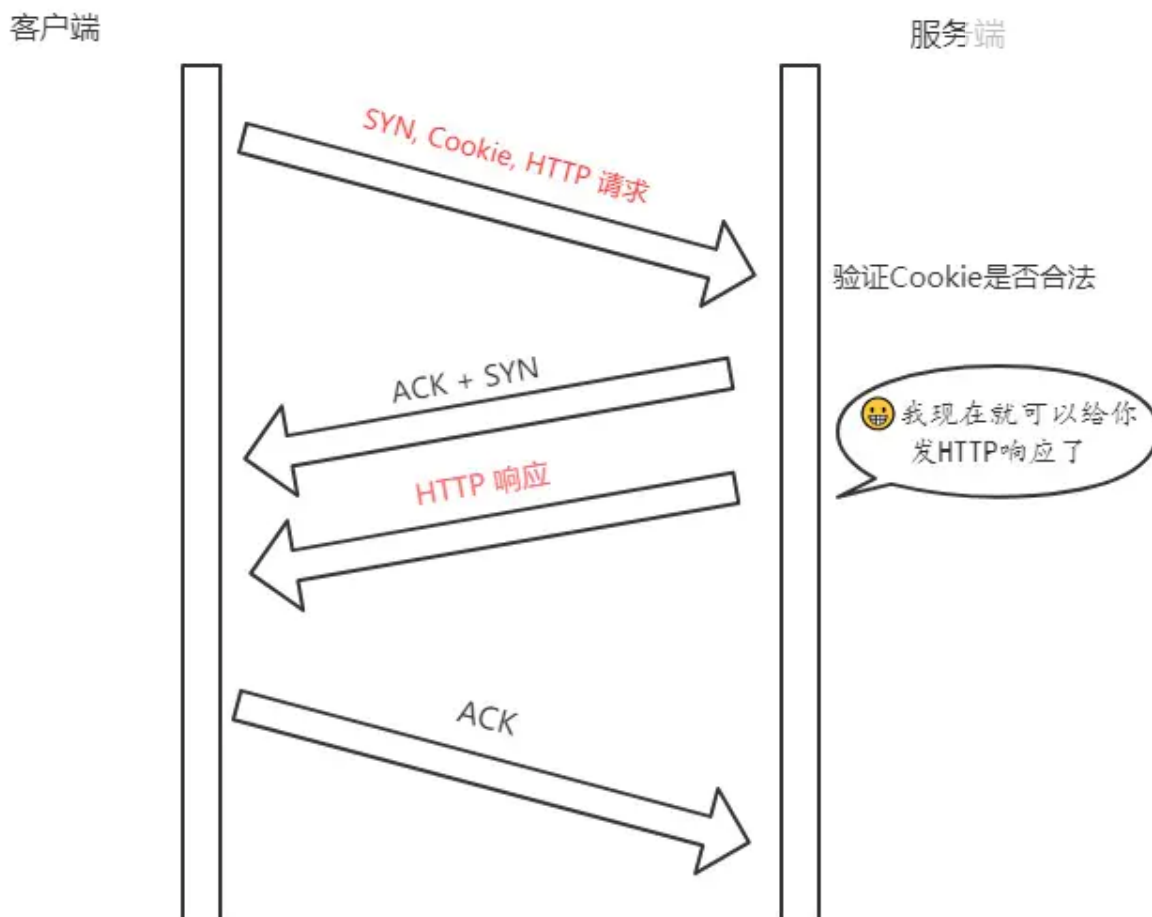
#### 后面的三次握手

在后面的三次握手中，客户端会将之前缓存的 Cookie、SYN 和 HTTP 请求 (是的，你没看错)发送给服务端，服务端验证了 Cookie 的合法性，如果不合法直接丢弃；如果是合法的，那么就正常返回 SYN + ACK。

重点来了，现在服务端能向客户端发 HTTP 响应了！这是最显著的改变，三次握手还没建立，仅仅验证了 Cookie 的合法性，就可以返回 HTTP 响应了。

当然，客户端的 ACK 还得正常传过来，不然怎么叫三次握手嘛。

流程如下:



注意: 客户端最后握手的 ACK 不一定要等到服务端的 HTTP 响应到达才发送, 两个过程没有任何关系。

## 2) TFO 的优势

TFO 的优势并不在与首轮三次握手, 而在于后面的握手, 在拿到客户端的 Cookie 并验证通过以后, 可以直接返回 HTTP 响应, 充分利用了 **1 个 RTT** (Round-Trip Time, 往返时延) 的时间 **提前进行数据传输**, 积累起来还是一个比较大的优势。

## 13. 能不能说说 TCP 报文中时间戳的作用?

timestamp 是 TCP 报文首部的一个可选项, 一共占 10 个字节, 格式如下:

```
kind(1 字节) + length(1 字节) + info(8 个字节)
```

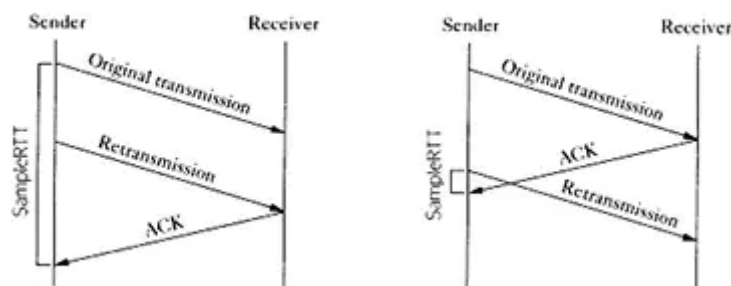
其中 kind = 8, length = 10, info 有两部分构成: **timestamp** 和 **timestamp echo**, 各占 4 个字节。

那么这些字段都是干嘛的呢? 它们用来解决那些问题?

接下来我们就来——梳理, TCP 的时间戳主要解决两大问题:

- 计算往返时延 RTT(Round-Trip Time)
- 防止序列号的回绕问题

在没有时间戳的时候，计算 RTT 会遇到的问题如下图所示：



如果以第一次发包为开始时间的话，就会出现左图的问题，RTT 明显偏大，开始时间应该采用第二次的；

如果以第二次发包为开始时间的话，就会导致右图的问题，RTT 明显偏小，开始时间应该采用第一次发包的。

实际上无论开始时间以第一次发包还是第二次发包为准，都是不准确的。

那这个时候引入时间戳就很好的解决了这个问题。

比如现在 a 向 b 发送一个报文 s1，b 向 a 回复一个含 ACK 的报文 s2 那么：

- **step 1:** a 向 b 发送的时候，`timestamp` 中存放的内容就是 a 主机发送时的内核时刻 `ta1`。
- **step 2:** b 向 a 回复 s2 报文的时候，`timestamp` 中存放的是 b 主机的时刻 `tb`，`timestamp echo` 字段为从 s1 报文中解析出来的 `ta1`。
- **step 3:** a 收到 b 的 s2 报文之后，此时 a 主机的内核时刻是 `ta2`，而在 s2 报文中的 `timestamp echo` 选项中可以得到 `ta1`，也就是 s2 对应的报文最初的发送时刻。然后直接采用 `ta2 - ta1` 就得到了 RTT 的值。

## 防止序列号回绕问题

现在我们来模拟一下这个问题。

序列号的范围其实是在  $0 \sim 2^{32} - 1$ ，为了方便演示，我们缩小一下这个区间，假设范围是  $0 \sim 4$ ，那么到达 4 的时候会回到 0。

| 第几次发包 | 发送字节  | 对应序列号 | 状态           |
|-------|-------|-------|--------------|
| 1     | 0 ~ 1 | 0 ~ 1 | 成功接收         |
| 2     | 1 ~ 2 | 1 ~ 2 | 滞留在网络中       |
| 3     | 2 ~ 3 | 2 ~ 3 | 成功接收         |
| 4     | 3 ~ 4 | 3 ~ 4 | 成功接收         |
| 5     | 4 ~ 5 | 0 ~ 1 | 成功接收，序列号从0开始 |
| 6     | 5 ~ 6 | 1 ~ 2 | ???          |

假设在第 6 次的时候，之前还滞留在网路中的包回来了，那么就有两个序列号为 `1 ~ 2` 的数据包了，怎么区分谁是谁呢？这个时候就产生了序列号回绕的问题。

那么用 timestamp 就能很好地解决这个问题，因为每次发包的时候都是将发包机器当时的内核时间记录在报文中，那么两次发包序列号即使相同，时间戳也不可能相同，这样就能够区分开两个数据包了。

## 14. TCP 的超时重传时间是如何计算的？

TCP 具有超时重传机制，即间隔一段时间没有等到数据包的回复时，重传这个数据包。

那么这个重传间隔是如何来计算的呢？

今天我们就来讨论一下这个问题。

这个重传间隔也叫做**超时重传时间**(Retransmission TimeOut, 简称RTO)，它的计算跟上一节提到的 RTT 密切相关。这里我们将介绍两种主要的方法，一个是经典方法，一个是标准方法。

### 经典方法

经典方法引入了一个新的概念——SRTT(Smoothed round trip time，即平滑往返时间)，没产生一次新的 RTT 就根据一定的算法对 SRTT 进行更新，具体而言，计算方式如下(SRTT 初始值为0)：

$$SRTT = (\alpha * SRTT) + ((1 - \alpha) * RTT)$$

其中， $\alpha$  是**平滑因子**，建议值是 0.8，范围是 0.8 ~ 0.9。

拿到 SRTT，我们就可以计算 RTO 的值了：

$$RTO = \min(\text{ubound}, \max(\text{lbound}, \beta * SRTT))$$

$\beta$  是加权因子，一般为 1.3 ~ 2.0，**lbound** 是下界，**ubound** 是上界。

其实这个算法过程还是很简单的，但是也存在一定的局限，就是在 RTT 稳定的地方表现还可以，而在 RTT 变化较大的地方就不行了，因为平滑因子  $\alpha$  的范围是 0.8 ~ 0.9，RTT 对于 RTO 的影响太小。

### 标准方法

为了解决经典方法对于 RTT 变化不敏感的问题，后面又引出了标准方法，也叫 Jacobson / Karels 算法。

一共有三步。

**第一步：**计算 SRTT，公式如下：

$$SRTT = (1 - \alpha) * SRTT + \alpha * RTT$$

注意这个时候的  $\alpha$  跟经典方法中的  $\alpha$  取值不一样了，建议值是 1/8，也就是 0.125。

**第二步：**计算 RTTVAR (round-trip time variation)这个中间变量。

$$RTTVAR = (1 - \beta) * RTTVAR + \beta * (|RTT - SRTT|)$$

$\beta$  建议值为 0.25。这个值是这个算法中出彩的地方，也就是说，它记录了最新的 RTT 与当前 SRTT 之间的差值，给我们在后续感知到 RTT 的变化提供了抓手。

**第三步：**计算最终的 RTO：

$$RTO = \mu * SRTT + \theta * RTTVAR$$

$\mu$  建议值取 1,  $\theta$  建议值取 4。

这个公式在 SRTT 的基础上加上了最新 RTT 与它的偏移, 从而很好的感知了 RTT 的变化, 这种算法下, RTO 与 RTT 变化的差值关系更加密切。

## 15. 能不能说一说 TCP 的流量控制?

对于发送端和接收端而言, TCP 需要把发送的数据放到**发送缓存区**, 将接收的数据放到**接收缓存区**。

而流量控制索要做的事情, 就是在通过接收缓存区的大小, 控制发送端的发送。如果对方的接收缓存区满了, 就不能再继续发送了。

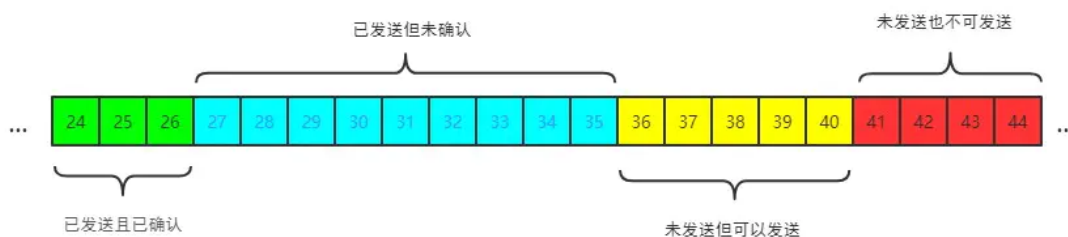
要具体理解流量控制, 首先需要了解 **滑动窗口** 的概念。

### TCP 滑动窗口

TCP 滑动窗口分为两种: **发送窗口**和**接收窗口**。

#### (1) 发送窗口

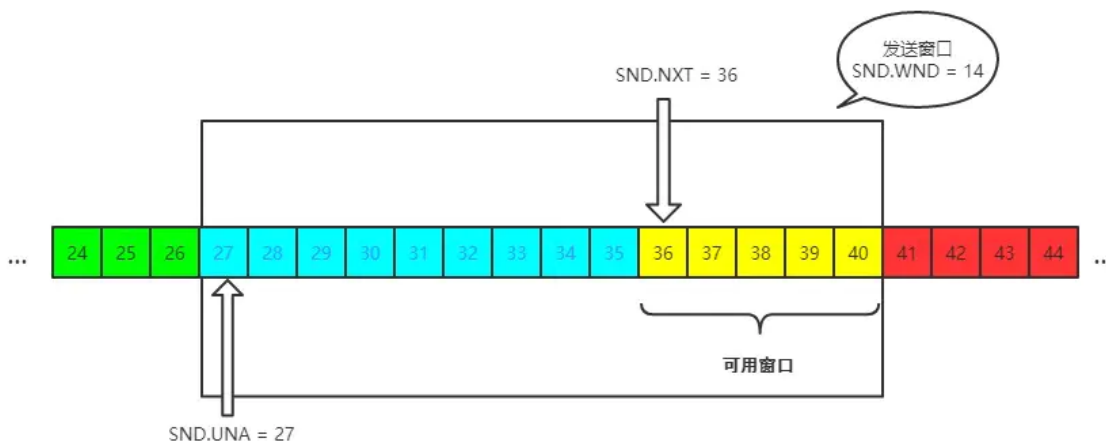
发送端的滑动窗口结构如下:



其中包含四大部分:

- 已发送且已确认
- 已发送但未确认
- 未发送但可以发送
- 未发送也不可以发送

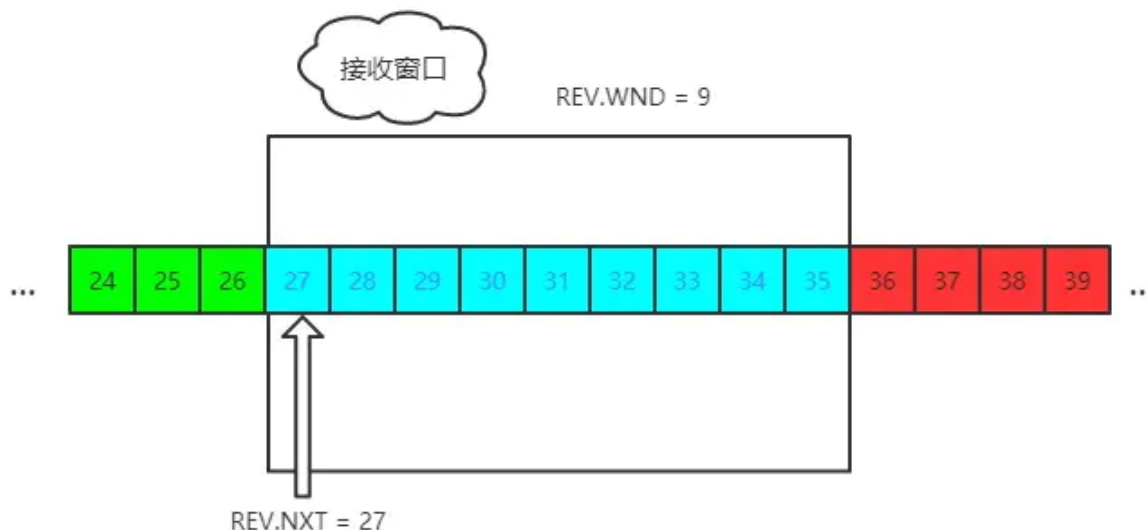
其中有一些重要的概念, 我标注在图中:



发送窗口就是图中被框住的范围。SND 即 send, WND 即 window, UNA 即 unacknowledged, 表示未被确认, NXT 即 next, 表示下一个发送的位置。

## (2) 接收窗口

接收端的窗口结构如下:



REV 即 receive, NXT 表示下一个接收的位置, WND 表示接收窗口大小。

## 流量控制过程

这里我们不用太复杂的例子, 以一个最简单的来回来模拟一下流量控制的过程, 方便大家理解。

首先双方三次握手, 初始化各自的窗口大小, 均为 200 个字节。

假如当前发送端给接收端发送 100 个字节, 那么此时对于发送端而言, SND.NXT 当然要右移 100 个字节, 也就是说当前的可用窗口减少了 100 个字节, 这很好理解。

现在这 100 个到达了接收端, 被放到接收端的缓冲队列中。不过此时由于大量负载的原因, 接收端处理不了这么多字节, 只能处理 40 个字节, 剩下的 60 个字节被留在了缓冲队列中。

注意了, 此时接收端的情况是处理能力不够用啦, 你发送端给我少发点, 所以此时接收端的接收窗口应该缩小, 具体来说, 缩小 60 个字节, 由 200 个字节变成了 140 字节, 因为缓冲队列还有 60 个字节没被应用拿走。

因此, 接收端会在 ACK 的报文首部带上缩小后的滑动窗口 140 字节, 发送端对应地调整发送窗口的大小为 140 个字节。

此时对于发送端而言, 已经发送且确认的部分增加 40 字节, 也就是 SND.UNA 右移 40 个字节, 同时发送窗口缩小为 140 个字节。

这也就是流量控制的过程。尽管回合再多, 整个控制的过程和原理是一样的。

## 16. 能不能说说 TCP 的拥塞控制?

上一节所说的流量控制发生在发送端跟接收端之间, 并没有考虑到整个网络环境的影响, 如果说当前网络特别差, 特别容易丢包, 那么发送端就应该注意一些了。而这, 也正是拥塞控制需要处理的问题。

对于拥塞控制来说, TCP 每条连接都需要维护两个核心状态:

- 拥塞窗口 (Congestion Window, cwnd)
- 慢启动阈值 (Slow Start Threshold, ssthresh)

涉及到的算法有这几个:

- 慢启动
- 拥塞避免
- 快速重传和快速恢复

接下来, 我们就来——拆解这些状态和算法。首先, 从拥塞窗口说起。

## 拥塞窗口

拥塞窗口 (Congestion Window, cwnd) 是指目前自己还能传输的数据量大小。

那么之前介绍了接收窗口的概念, 两者有什么区别呢?

- 接收窗口(rwnd)是 接收端 给的限制
- 拥塞窗口(cwnd)是 发送端 的限制

限制谁呢?

限制的是 发送窗口 的大小。

有了这两个窗口, 如何来计算 发送窗口?

```
发送窗口大小 = min(rwnd, cwnd)
```

取两者的较小值。而拥塞控制, 就是来控制 cwnd 的变化。

## 慢启动

刚开始进入传输数据的时候, 你是不知道现在的网路到底是稳定还是拥堵的, 如果做的太激进, 发包太急, 那么疯狂丢包, 造成雪崩式的网络灾难。

因此, 拥塞控制首先就是要采用一种保守的算法来慢慢地适应整个网路, 这种算法叫 慢启动。运作过程如下:

- 首先, 三次握手, 双方宣告自己的接收窗口大小
- 双方初始化自己的拥塞窗口(cwnd)大小
- 在开始传输的一段时间, 发送端每收到一个 ACK, 拥塞窗口大小加 1, 也就是说, 每经过一个 RTT, cwnd 翻倍。如果说初始窗口为 10, 那么第一轮 10 个报文传完且发送端收到 ACK 后, cwnd 变为 20, 第二轮变为 40, 第三轮变为 80, 依次类推。

难道就这么无止境地翻倍下去? 当然不可能。它的阈值叫做慢启动阈值, 当 cwnd 到达这个阈值之后, 好比踩了下刹车, 别涨了那么快了, 老铁, 先 hold 住!

在到达阈值后, 如何来控制 cwnd 的大小呢?

这就是拥塞避免做的事情了。

## 拥塞避免

原来每收到一个 ACK, cwnd 加1, 现在到达阈值了, cwnd 只能加这么一点:  $1 / cwnd$ 。那你仔细算算, 一轮 RTT 下来, 收到 cwnd 个 ACK, 那最后拥塞窗口的大小 cwnd 总共才增加 1。



也就是说，以前一个 RTT 下来，`cwnd` 翻倍，现在 `cwnd` 只是增加 1 而已。

当然，**慢启动**和**拥塞避免**是一起作用的，是一体的。

## 快速重传和快速恢复

### 快速重传

在 TCP 传输的过程中，如果发生了丢包，即接收端发现数据段不是按序到达的时候，接收端的处理是重复发送之前的 ACK。

比如第 5 个包丢了，即使第 6、7 个包到达的接收端，接收端也一律返回第 4 个包的 ACK。当发送端收到 3 个重复的 ACK 时，意识到丢包了，于是马上进行重传，不用等到一个 RTO 的时间到了才重传。

这就是**快速重传**，它解决的是**是否需要重传**的问题。

### 选择性重传

那你可能会问了，既然要重传，那么只重传第 5 个包还是第 5、6、7 个包都重传呢？

当然第 6、7 个都已经到达了，TCP 的设计者也不傻，已经传过去干嘛还要传？干脆记录一下哪些包到了，哪些没到，针对性地重传。

在收到发送端的报文后，接收端回复一个 ACK 报文，那么在这个报文首部的可选项中，就可以加上 `SACK` 这个属性，通过 `left edge` 和 `right edge` 告知发送端已经收到了哪些区间的数据报。因此，即使第 5 个包丢包了，当收到第 6、7 个包之后，接收端依然会告诉发送端，这两个包到了。剩下第 5 个包没到，就重传这个包。这个过程也叫做**选择性重传(SACK, Selective Acknowledgment)**，它解决的是**如何重传**的问题。

### 快速恢复

当然，发送端收到三次重复 ACK 之后，发现丢包，觉得现在的网络已经有些拥塞了，自己会进入**快速恢复阶段**。

在这个阶段，发送端如下改变：

- 拥塞阈值降低为 `cwnd` 的一半
- `cwnd` 的大小变为拥塞阈值
- `cwnd` 线性增加

以上就是 TCP 拥塞控制的经典算法：**慢启动**、**拥塞避免**、**快速重传**和**快速恢复**。

## 17. 能不能说说 Nagle 算法和延迟确认？

### Nagle 算法

试想一个场景，发送端不停地给接收端发很小的包，一次只发 1 个字节，那么发 1 千个字节需要发 1000 次。这种频繁的发送是存在问题的，不光是传输的时延消耗，发送和确认本身也是需要耗时的，频繁的发送接收带来了巨大的时延。

而避免小包的频繁发送，这就是 Nagle 算法要做的事情。

具体来说，Nagle 算法的规则如下：

- 当第一次发送数据时不用等待，就算是 1byte 的小包也立即发送
- 后面发送满足下面条件之一就可以发了：
  - 数据包大小达到最大段大小(Max Segment Size, 即 MSS)
  - 之前所有包的 ACK 都已接收到

### 延迟确认

试想这样一个场景，当我收到了发送端的一个包，然后在极短的时间内又接收到了第二个包，那我是一个个地回复，还是稍微等一下，把两个包的 ACK 合并后一起回复呢？

**延迟确认**(delayed ack)所做的事情，就是后者，稍稍延迟，然后合并 ACK，最后才回复给发送端。TCP 要求这个延迟的时延必须小于 500ms，一般操作系统实现都不会超过 200ms。

不过需要主要的是，有一些场景是不能延迟确认的，收到了就要马上回复：

- 接收到了大于一个 frame 的报文，且需要调整窗口大小
- TCP 处于 quickack 模式（通过 `tcp_in_quickack_mode` 设置）
- 发现了乱序包

### 两者一起使用会怎样？

前者意味着延迟发，后者意味着延迟接收，会造成更大的延迟，产生性能问题。

## 18. 如何理解 TCP 的 keep-alive？

大家都听说过 http 的 `keep-alive`，不过 TCP 层面也是有 `keep-alive` 机制，而且跟应用层不太一样。

试想一个场景，当有一方因为网络故障或者宕机导致连接失效，由于 TCP 并不是一个轮询的协议，在下一个数据包到达之前，对端对连接失效的情况是一无所知的。

这个时候就出现了 `keep-alive`，它的作用就是探测对端的连接有没有失效。

在 Linux 下，可以这样查看相关的配置：

```
sudo sysctl -a | grep keepalive

// 每隔 7200 s 检测一次
net.ipv4.tcp_keepalive_time = 7200
// 一次最多重传 9 个包
net.ipv4.tcp_keepalive_probes = 9
// 每个包的间隔重传间隔 75 s
net.ipv4.tcp_keepalive_intvl = 75
```

不过，现状是大部分的应用并没有默认开启 TCP 的 `keep-alive` 选项，为什么？

站在应用的角度：

- 7200s 也就是两个小时检测一次，时间太长
- 时间再短一些，也难以体现其设计的初衷，即检测长时间的死连接

## 浏览器

## 1. 能不能说一说浏览器缓存?

缓存是性能优化中非常重要的一环，浏览器的缓存机制对开发也是非常重要的知识点。接下来以三个部分来把浏览器的缓存机制说清楚：

- 强缓存
- 协商缓存
- 缓存位置

### 1) 强缓存

浏览器中的缓存作用分为两种情况，一种是需要发送 HTTP 请求，一种是不需要发送。

首先是检查强缓存，这个阶段 **不需要** 发送HTTP请求。

如何来检查呢？通过相应的字段来进行，但是说起这个字段就有点门道了。

在HTTP/1.0和HTTP/1.1当中，这个字段是不一样的。在早期，也就是 HTTP/1.0 时期，使用的是 **Expires**，而HTTP/1.1使用的是**Cache-Control**。让我们首先来看看Expires。

#### (1) Expires

Expires即过期时间，存在于服务端返回的响应头中，告诉浏览器在这个过期时间之前可以直接从缓存里面获取数据，无需再次请求。比如下面这样：

```
Expires: Wed, 22 Nov 2019 08:41:00 GMT
```

表示资源在2019年11月22号8点41分过期，过期了就得向服务端发请求。

这个方式看上去没什么问题，合情合理，但其实潜藏了一个坑，那就是服务器的时间和浏览器的时间可能并不一致，那服务器返回的这个过期时间可能就是不准确的。因此这种方式很快在后来的HTTP1.1版本中被抛弃了。

#### (2) Cache-Control

在HTTP1.1中，采用了一个非常关键的字段：**Cache-Control**。这个字段也是存在于

它和 **Expires** 本质的不同在于它并没有采用 **具体的过期时间点** 这个方式，而是采用过期时长来控制缓存，对应的字段是max-age。比如这个例子：

```
Cache-Control:max-age=3600
```

代表这个响应返回后在 3600 秒，也就是一个小时之内可以直接使用缓存。

如果你觉得它只有 **max-age** 一个属性的话，那就大错特错了。

它其实可以组合非常多的指令，完成更多场景的缓存判断，将一些关键的属性列举如下：**public**：客户端和代理服务器都可以缓存。因为一个请求可能要经过不同的 **代理服务器** 最后才到达目标服务器，那么结果就是不仅仅浏览器可以缓存数据，中间的任何代理节点都可以进行缓存。

**private**：这种情况就是只有浏览器能缓存了，中间的代理服务器不能缓存。

**no-cache**：跳过当前的强缓存，发送HTTP请求，即直接进入 **协商缓存阶段**。

**no-store**：非常粗暴，不进行任何形式的缓存。

**s-maxage**: 这和 **max-age** 长得比较像, 但是区别在于 **s-maxage** 是针对代理服务器的缓存时间。

值得注意的是, 当 **Expires** 和 **Cache-Control** 同时存在的时候, **Cache-Control** 会优先考虑。

当然, 还存在一种情况, 当资源缓存时间超时了, 也就是**强缓存失效**了, 接下来怎么办? 没错, 这样就进入到第二级屏障——**协商缓存**了。

## 2) 协商缓存

强缓存失效之后, 浏览器在请求头中携带相应的 **缓存tag** 来向服务器发请求, 由服务器根据这个tag, 来决定是否使用缓存, 这就是协商缓存。

具体来说, 这样的缓存tag分为两种: **Last-Modified** 和 **ETag**。这两者各有优劣, 并不存在谁对谁有绝对的优势, 跟上面强缓存的两个 tag 不一样。

### (1) Last-Modified

即最后修改时间。在浏览器第一次给服务器发送请求后, 服务器会在响应头中加上这个字段。

浏览器接收到后, 如果再次请求, 会在请求头中携带 **If-Modified-Since** 字段, 这个字段的值也就是服务器传来的最后修改时间。

服务器拿到请求头中的 **If-Modified-Since** 的字段后, 其实会和这个服务器中该资源的最后修改时间对比:

- 如果请求头中的这个值小于最后修改时间, 说明是时候更新了。返回新的资源, 跟常规的HTTP请求响应的流程一样。
- 否则返回304, 告诉浏览器直接用缓存。

### (2) ETag

**ETag** 是服务器根据当前文件的内容, 给文件生成的唯一标识, 只要里面的内容有改动, 这个值就会变。服务器通过 **响应头** 把这个值给浏览器。

浏览器接收到ETag的值, 会在下次请求时, 将这个值作为**If-None-Match**这个字段的内容, 并放到请求头中, 然后发给服务器。

服务器接收到**If-None-Match**后, 会跟服务器上该资源的ETag进行比对:

- 如果两者不一样, 说明要更新了。返回新的资源, 跟常规的HTTP请求响应的流程一样。
- 否则返回304, 告诉浏览器直接用缓存。

### (3) 两者对比

1) 在 **精准度** 上, **ETag** 优于 **Last-Modified**。优于 **ETag** 是按照内容给资源上标识, 因此能准确感知资源的变化。而 **Last-Modified** 就不一样了, 它在一些特殊的情况并不能准确感知资源变化, 主要有两种情况:

- 编辑了资源文件, 但是文件内容并没有更改, 这样也会造成缓存失效。
- **Last-Modified** 能够感知的单位时间是秒, 如果文件在 1 秒内改变了多次, 那么这时候的 **Last-Modified** 并没有体现出修改了。

2) 在性能上, **Last-Modified** 优于 **ETag**, 也很简单理解, **Last-Modified** 仅仅只是记录一个时间点, 而 **ETag** 需要根据文件的具体内容生成哈希值。

另外，如果两种方式都支持的话，服务器会优先考虑 ETag。

### 3) 缓存位置

前面我们已经提到，当强缓存命中或者协商缓存中服务器返回304的时候，我们直接从缓存中获取资源。那这些资源究竟缓存在什么位置呢？

浏览器中的缓存位置一共有四种，按优先级从高到低排列分别是：

- Service Worker
- Memory Cache
- Disk Cache
- Push Cache

### 4) Service Worker

Service Worker 借鉴了 Web Worker 的思路，即让 JS 运行在主线程之外，由于它脱离了浏览器的窗体，因此无法直接访问 DOM。虽然如此，但它仍然能帮助我们完成很多有用的功能，比如 离线缓存、消息推送 和 网络代理 等功能。其中的 离线缓存 就是 **Service Worker Cache**。

Service Worker 同时也是 PWA 的重要实现机制，关于它的细节和特性，我们将会在后面的 PWA 的分享中详细介绍。

### 5) Memory Cache 和 Disk Cache

**Memory Cache**指的是内存缓存，从效率上讲它是最快的。但是从存活时间来讲又是最短的，当渲染进程结束后，内存缓存也就不存在了。

**Disk Cache**就是存储在磁盘中的缓存，从存取效率上讲是比内存缓存慢的，但是他的优势在于存储容量和存储时长。稍微有些计算机基础的应该很好理解，就不展开了。

好，现在问题来了，既然两者各有优劣，那浏览器如何决定将资源放进内存还是硬盘呢？主要策略如下：

- 比较大的JS、CSS文件会直接被丢进磁盘，反之丢进内存
- 内存使用率比较高的时候，文件优先进入磁盘

### 6) Push Cache

即推送缓存，这是浏览器缓存的最后一道防线。它是 HTTP/2 中的内容，虽然现在应用的并不广泛，但随着 HTTP/2 的推广，它的应用越来越广泛。

## 总结

对浏览器的缓存机制来做个简要的总结：

首先通过 Cache-Control 验证强缓存是否可用

- 如果强缓存可用，直接使用
- 否则进入协商缓存，即发送 HTTP 请求，服务器通过请求头中的 If-Modified-Since 或者 If-None-Match 字段检查资源是否更新
  - 若资源更新，返回资源和200状态码

- 否则，返回304，告诉浏览器直接从缓存获取资源

## 2. 能不能说一说浏览器的本地存储？各自优劣如何？

浏览器的本地存储主要分为 `Cookie`、`webStorage` 和 `IndexedDB`，其中 `webStorage` 又可以分为 `localStorage` 和 `sessionStorage`。接下来我们就来——分析这些本地存储方案。

### 1) Cookie

`Cookie` 最开始被设计出来其实并不是来做本地存储的，而是为了弥补HTTP在状态管理上的不足。

HTTP协议是一个无状态协议，客户端向服务器发请求，服务器返回响应，故事就这样结束了，但是下次发请求如何让服务端知道客户端是谁呢？

这种背景下，就产生了 `Cookie`。

`Cookie` 本质上就是浏览器里面存储的一个很小的文本文件，内部以键值对的方式来存储(在chrome开发者面板的 `Application` 这一栏可以看到)。向同一个域名下发送请求，都会携带相同的 `Cookie`，服务器拿到 `Cookie` 进行解析，便能拿到客户端的状态。

`Cookie` 的作用很好理解，就是用来做状态存储的，但它也是有诸多致命的缺陷的：

- 容量缺陷。`Cookie` 的体积上限只有 `4KB`，只能用来存储少量的信息。
- 性能缺陷。`Cookie` 紧跟域名，不管域名下面的某一个地址需不需要这个 `Cookie`，请求都会携带上完整的 `Cookie`，这样随着请求数的增多，其实会造成巨大的性能浪费的，因为请求携带了很多不必要的内容。
- 安全缺陷。由于 `Cookie` 以纯文本的形式在浏览器和服务器中传递，很容易被非法用户截获，然后进行一系列的篡改，在 `Cookie` 的有效期内重新发送给服务器，这是相当危险的。另外，在 `httpOnly` 为 `false` 的情况下，`Cookie` 信息能直接通过 `JS` 脚本来读取。

### 2) localStorage和Cookie异同

`localStorage` 有一点跟 `Cookie` 一样，就是针对一个域名，即在同一个域名下，会存储相同的一段 `localStorage`。

不过它相对 `Cookie` 还是有相当多的区别的：

- 容量。`localStorage` 的容量上限为 `5M`，相比于 `Cookie` 的 `4K` 大大增加。当然这个 `5M` 是针对一个域名的，因此对于一个域名是持久存储的。
- 只存在客户端，默认不参与与服务端的通信。这样就很好地避免了 `Cookie` 带来的性能问题和安全问题。
- 接口封装。通过 `localStorage` 暴露在全局，并通过它的 `setItem` 和 `getItem` 等方法进行操作，非常方便。

### 操作方式

接下来我们来具体看看如何来操作 `localStorage`。

```
let obj = { name: "sanyuan", age: 18 };
localStorage.setItem("name", "sanyuan");
localStorage.setItem("info", JSON.stringify(obj));
```



接着进入相同的域名时就能拿到相应的值:

```
let name = localStorage.getItem("name");  
let info = JSON.parse(localStorage.getItem("info"));
```

从这里可以看出, `localStorage` 其实存储的都是字符串, 如果是存储对象需要调用 `JSON` 的 `stringify` 方法, 并且用 `JSON.parse` 来解析成对象。

## 应用场景

利用 `localStorage` 的较大容量和持久特性, 可以利用 `localStorage` 存储一些内容稳定的资源, 比如官网的 `logo`, 存储 `Base64` 格式的图片资源, 因此利用 `localStorage`

## 3) sessionStorage

### 特点

`sessionStorage` 以下方面和 `localStorage` 一致:

- 容量。容量上限也为 5M。
- 只存在客户端, 默认不参与与服务端的通信。
- 接口封装。除了 `sessionStorage` 名字有所变化, 存储方式、操作方式均和 `localStorage` 一样。

但 `sessionStorage` 和 `localStorage` 有一个本质的区别, 那就是前者只是会话级别的存储, 并不是持久化存储。会话结束, 也就是页面关闭, 这部分 `sessionStorage` 就不复存在了。

## 应用场景

- 1) 可以用它对表单信息进行维护, 将表单信息存储在里面, 可以保证页面即使刷新也不会让之前的表单信息丢失。
- 2) 可以用它存储本次浏览记录。如果关闭页面后不需要这些记录, 用 `sessionStorage` 就再合适不过了。事实上微博就采取了这样的存储方式。

## 4) IndexedDB

`IndexedDB` 是运行在浏览器中的 `非关系型数据库`, 本质上是数据库, 绝不是和刚才 `WebStorage` 的 5M 一个量级, 理论上这个容量是没有上限的。

接着我们来分析一下 `IndexedDB` 的一些重要特性, 除了拥有数据库本身的特性, 比如 `支持事务`, `存储二进制数据`, 还有这样一些特性需要格外注意:

- 键值对存储。内部采用 `对象仓库` 存放数据, 在这个对象仓库中数据采用 `键值对` 的方式来存储。
- 异步操作。数据库的读写属于 `I/O` 操作, 浏览器中对异步 `I/O` 提供了支持。
- 受同源策略限制, 即无法访问跨域的数据库。

## 总结

浏览器中各种本地存储和缓存技术的发展, 给前端应用带来了大量的机会, `PWA` 也正是依托了这些优秀的存储方案才得以发展起来。重新梳理一下这些本地存储方案:

- (1) `cookie` 并不适合存储, 而且存在非常多的缺陷。

(2) `web storage` 包括 `localStorage` 和 `sessionStorage`, 默认不会参与和服务器的通信

(3) `IndexedDB` 为运行在浏览器上的非关系型数据库, 为大型数据的存储提供了接口。

### 3. 说一说从输入URL到页面呈现发生了什么? (网络)

这是一个可以无限难的问题。出这个题目的目的就是为了考察你的 web 基础深入到什么程度。由于水平和篇幅有限, 在这里我将把其中一些重要的过程给大家梳理一遍, 相信能在绝大部分的情况下给出一个比较惊艳的答案。

这里我提前声明, 由于是一个综合性非常强的问题, 可能会在某一个点上深挖出非常多的细节, 我个人觉得学习是一个循序渐进的过程, 在明白了整体过程后再去自己研究这些细节, 会对整个知识体系有更深入的理解。同时, 关于延申出来的细节点我都有参考资料, 看完这篇之后不妨再去深入学习一下, 扩展知识面。

好, 正题开始。

此时此刻, 你在浏览器地址栏输入了百度的网址:

```
https://www.baidu.com/
```

#### 网络请求

##### (1) 构建请求

浏览器会构建请求行:

```
// 请求方法是GET, 路径为根路径, HTTP协议版本为1.1  
GET / HTTP/1.1
```

##### (2) 查找强缓存

先检查强缓存, 如果命中直接使用, 否则进入下一步。

##### (3) DNS解析

由于我们输入的是域名, 而数据包是通过 IP地址 传给对方的。因此我们需要得到域名对应的IP地址。这个过程需要依赖一个服务系统, 这个系统将域名和 IP 一一映射, 我们将这个系统就叫做DNS (域名系统)。得到具体 IP 的过程就是 DNS 解析。

当然, 值得注意的是, 浏览器提供了DNS数据缓存功能。即如果一个域名已经解析过, 那会把解析的结果缓存下来, 下次处理直接走缓存, 不需要经过 DNS解析。

另外, 如果不指定端口的话, 默认采用对应的 IP 的 80 端口。

##### (4) 建立 TCP 连接

这里要提醒一点, Chrome 在同一个域名下要求同时最多只能有 6 个 TCP 连接, 超过 6 个的话剩下的请求就得等待。

假设现在不需要等待, 我们进入了 TCP 连接的建立阶段。首先解释一下什么是 TCP:



TCP (Transmission Control Protocol, 传输控制协议) 是一种面向连接的、可靠的、基于字节流的传输层通信协议。

建立 TCP 连接 经历了下面三个阶段:

- 1) 通过三次握手(即总共发送3个数据包确认已经建立连接)建立客户端和服务端之间的连接。
- 2) 进行数据传输。这里有一个重要的机制, 就是接收方接收到数据包后必须要向发送方 确认, 如果发送方没有接到这个 确认 的消息, 就判定为数据包丢失, 并重新发送该数据包。当然, 发送的过程中还有一个优化策略, 就是把 大的数据包拆成一个个小包, 依次传输到接收方, 接收方按照这个小包的顺序把它们 组装 成完整数据包。
- 3) 断开连接的阶段。数据传输完成, 现在要断开连接了, 通过四次挥手来断开连接。

读到这里, 你应该明白 TCP 连接通过什么手段来保证数据传输的可靠性, 一是三次握手确认连接, 二是数据包校验保证数据到达接收方, 三是通过四次挥手断开连接。

## (5) 发送 HTTP 请求

现在 TCP 连接 建立完毕, 浏览器可以和服务器开始通信, 即开始发送 HTTP 请求。浏览器发 HTTP 请求要携带三样东西: 请求行、请求头和请求体。

首先, 浏览器会向服务器发送请求行, 关于请求行, 我们在这一部分的第一步就构建完了, 贴一下内容:

```
// 请求方法是GET, 路径为根路径, HTTP协议版本为1.1  
GET / HTTP/1.1
```

结构很简单, 由请求方法、请求URI和HTTP版本协议组成。

同时也要带上请求头, 比如我们之前说的Cache-Control、If-Modified-Since、If-None-Match都由可能被放入请求头中作为缓存的标识信息。当然了还有一些其他的属性, 列举如下:

```
Accept:  
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;  
q=0.8,application/signed-exchange;v=b3  
Accept-Encoding: gzip, deflate, br  
Accept-Language: zh-CN,zh;q=0.9  
Cache-Control: no-cache  
Connection: keep-alive  
Cookie: /* 省略cookie信息 */  
Host: www.baidu.com  
Pragma: no-cache  
Upgrade-Insecure-Requests: 1  
User-Agent: Mozilla/5.0 (iPhone; CPU iPhone OS 11_0 like Mac OS X)  
AppleWebKit/604.1.38 (KHTML, like Gecko) Version/11.0 Mobile/15A372 Safari/604.1
```

最后是请求体, 请求体只有在 POST 方法下存在, 常见的场景是表单提交。

## 网络响应

HTTP 请求到达服务器, 服务器进行对应的处理。最后要把数据传给浏览器, 也就是返回网络响应。

跟请求部分类似, 网络响应具有三个部分: 响应行、响应头和响应体。

响应行类似下面这样:

由HTTP协议版本、状态码和状态描述组成。

响应头包含了服务器及其返回数据的一些信息, 服务器生成数据的时间、返回的数据类型以及对即将写入的Cookie信息。

举例如下:

```
Cache-Control: no-cache
Connection: keep-alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Wed, 04 Dec 2019 12:29:13 GMT
Server: apache
Set-Cookie:
rsv_i=f9a0SIItKqzv7kqgAAgphbGyRts3RwTg%2FLyU3Y5Eh5Lwyf00rAsvdezbay0QqkDqFZ0DfQxb
y4wXKT8Au807ZT9UuMSBq2k; path=/; domain=.baidu.com
```

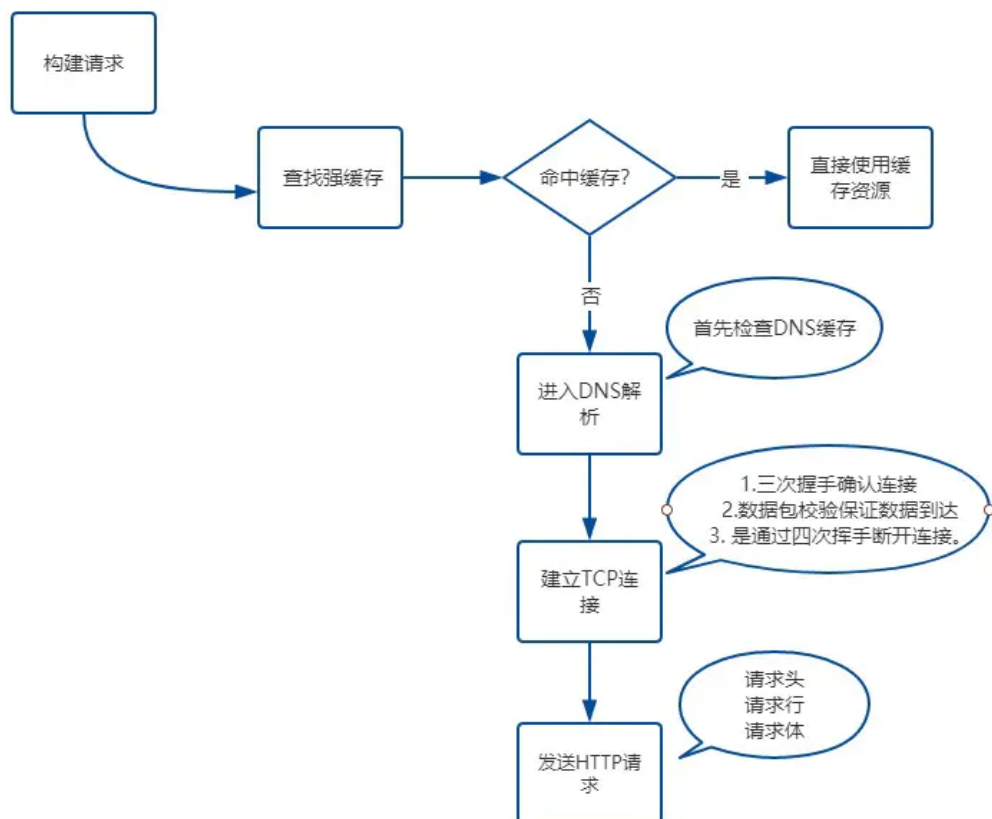
响应完成之后怎么办? TCP 连接就断开了吗?

不一定。这时候要判断 `Connection` 字段, 如果请求头或响应头中包含 `Connection: Keep-Alive`, 表示建立了持久连接, 这样 TCP 连接会一直保持, 之后请求统一站点的资源会复用这个连接。

否则断开 TCP 连接, 请求-响应流程结束。

## 总结

到此, 我们来总结一下主要内容, 也就是浏览器端的网络请求过程:



## 4. 说一说从输入URL到页面呈现发生了什么？（解析算法）

完成了网络请求和响应，如果响应头中 `Content-Type` 的值是 `text/html`，那么接下来就是浏览器的解析和渲染工作了。

首先来介绍解析部分，主要分为以下几个步骤：

- 构建 DOM 树
- 样式 计算
- 生成 布局树 (Layout Tree)

### 构建 DOM 树

由于浏览器无法直接理解 HTML 字符串，因此将这一系列的字节流转换为一种有意义并且方便操作的数据结构，这种数据结构就是 DOM 树。DOM 树本质上是一个以 `document` 为根节点的多叉树。

那通过什么样的方式来进行解析呢？

#### (1) HTML 文法的本质

首先，我们应该清楚把握一点：HTML 的文法并不是 上下文无关文法。

这里，有必要讨论一下什么是 上下文无关文法。

在计算机科学的编译原理学科中，有非常明确的定义：

若一个形式文法  $G = (N, \Sigma, P, S)$  的产生式规则都取如下的形式： $V \rightarrow w$ ，则叫上下文无关语法。其中  $V \in N$ ， $w \in (N \cup \Sigma)^*$ 。

其中把  $G = (N, \Sigma, P, S)$  中各个参量的意义解释一下：

- $N$  是非终结符(顾名思义，就是说最后一个符号不是它，下面同理)集合。
- $\Sigma$  是终结符集合。
- $P$  是开始符，它必须属于  $N$ ，也就是非终结符。
- $S$  就是不同的产生式的集合。如  $S \rightarrow aSb$  等等。

通俗一点讲，上下文无关的文法就是说这个文法中所有产生式的左边都是一个非终结符。

看到这里，如果还有一点懵圈，我举个例子你就明白了。

比如：

```
A -> B
```

这个文法中，每个产生式左边都会有一个非终结符，这就是上下文无关的文法。在这种情况下， $xBy$  一定是可以规约出  $xAy$  的。

我们下面看看一个反例：

```
aA -> B
Aa -> B
```

这种情况就是不是上下文无关的文法，当遇到  $B$  的时候，我们不知道到底能不能规约出  $A$ ，取决于左边或者右边是否有  $a$  存在，也就是说和上下文有关。

关于它为什么是非上下文无关文法，首先需要让大家注意的是，规范的 HTML 语法，是符合上下文无关文法的，能够体现它非上下文无关的是不标准的语法。在此我仅举一个反例即可证明。

比如解析器扫描到 form 标签的时候，上下文无关文法的处理方式是直接创建对应 form 的 DOM 对象，而真实的 HTML5 场景中却不是这样，解析器会查看 form 的上下文，如果这个 form 标签的父标签也是 form，那么直接跳过当前的 form 标签，否则才创建 DOM 对象。

常规的编程语言都是上下文无关的，而 HTML 却相反，也正是它非上下文无关的特性，决定了 HTML Parser 并不能使用常规编程语言的解析器来完成，需要另辟蹊径。

## (2) 解析算法

HTML5 规范详细地介绍了解析算法。这个算法分为两个阶段：

- 标记化。
- 建树。

对应的两个过程就是**词法分析**和**语法分析**。

### 标记化算法

这个算法输入为 HTML 文本，输出为 HTML 标记，也成为**标记生成器**。其中运用**有限自动状态机**来完成。即在当当前状态下，接收一个或多个字符，就会更新到下一个状态。

```
<html>
  <body>
    Hello sanyuan
  </body>
</html>
```

通过一个简单的例子来演示一下**标记化**的过程。

遇到 <，状态为**标记打开**。

接收 [a-z] 的字符，会进入**标记名称状态**。

这个状态一直保持，直到遇到 >，表示标记名称记录完成，这时候变为**数据状态**。

接下来遇到 body 标签做同样的处理。

这个时候 html 和 body 的标记都记录好了。

现在来到 <body> 中的 >，进入**数据状态**，之后保持这样状态接收后面的字符 hello sanyuan。

接着接收 </body> 中的 <，回到**标记打开**，接收下一个 / 后，这时候会创建一个 end tag 的 token。

随后进入**标记名称状态**，遇到 > 回到**数据状态**。

接着以同样的样式处理 </body>。

### 建树算法

之前提到过，DOM 树是一个以 document 为根节点的多叉树。因此解析器首先会创建一个 document 对象。标记生成器会把每个标记的信息发送给**建树器**。建树器接收到相应的标记时，会**创建对应的 DOM 对象**。创建这个 DOM 对象 后会做两件事情：

1. 将 DOM 对象 加入 DOM 树中。
2. 将对应标记压入存放开放(与 闭合标签 意思对应)元素的栈中。

还是拿下面这个例子说:

```
<html>
  <body>
    Hello sanyuan
  </body>
</html>
```

首先, 状态为初始化状态。

接收到标记生成器传来的 `html` 标签, 这时候状态变为**before html状态**。同时创建一个 `HTMLHtmlElement` 的 DOM 元素, 将其加到 `document` 根对象上, 并进行压栈操作。

接着状态自动变为**before head**, 此时从标记生成器那边传来 `body`, 表示并没有 `head`, 这时候**建树器**会自动创建一个 `HTMLHeadElement` 并将其加入到 DOM 树 中。

现在进入到**in head**状态, 然后直接跳到**after head**。

现在**标记生成器**传来了 `body` 标记, 创建 `HTMLBodyElement`, 插入到 DOM 树中, 同时压入开放标记栈。

接着状态变为**in body**, 然后接收后面一系列的字符: `Hello sanyuan`。接收到第一个字符的时候, 会创建一个 `Text` 节点并把字符插入其中, 然后把 `Text` 节点插入到 DOM 树中 `body` 元素的下面。随着不断接收后面的字符, 这些字符会附在 `Text` 节点上。

现在, **标记生成器**传过来一个 `body` 的结束标记, 进入到**after body**状态。

**标记生成器**最后传过来一个 `html` 的结束标记, 进入到**after after body**的状态, 表示解析过程到此结束。

## 容错机制

讲到 HTML5 规范, 就不得不说它强大的**宽容策略**, 容错能力非常强, 虽然大家褒贬不一, 不过我想作为一名资深的前端工程师, 有必要知道 `HTML Parser` 在容错方面做了哪些事情。

接下来是 WebKit 中一些经典的容错示例, 发现有其他的也欢迎来补充。

1. 使用 `</br>` 而不是 `<br>`

```
if (t->isCloseTag(brTag) && m_document->inCompatMode()) {
  reportError(MalformedBRError);
  t->beginTag = true;
}
```

全部换为 `<br>` 的形式。

- 表格离散

```
<table>
  <table>
    <tr><td>inner table</td></tr>
  </table>
<tr><td>outer table</td></tr>
</table>
```

WebKit 会自动转换为:

```
<table>
  <tr><td>outer table</td></tr>
</table>
<table>
  <tr><td>inner table</td></tr>
</table>
```

- 表单元素嵌套

这时候直接忽略里面的 `form`。

## 样式计算

关于CSS样式，它的来源一般是三种：

- link标签引用
- style标签中的样式
- 元素的内嵌style属性

### (1) 格式化样式表

首先，浏览器是无法直接识别 CSS 样式文本的，因此渲染引擎接收到 CSS 文本之后第一件事情就是将其转化为一个结构化的对象，即styleSheets。

这个格式化的过程过于复杂，而且对于不同的浏览器会有不同的优化策略，这里就不展开了。

在浏览器控制台能够通过 `document.styleSheets` 来查看这个最终的结构。当然，这个结构包含了以上三种CSS来源，为后面的样式操作提供了基础。

### (2) 标准化样式属性

有一些 CSS 样式的数值并不容易被渲染引擎所理解，因此需要在计算样式之前将它们标准化，如 `em` -> `px`, `red` -> `#ff0000`, `bold` -> `700` 等等。

### (3) 计算每个节点的具体样式

样式已经被 格式化 和 标准化,接下来就可以计算每个节点的具体样式信息了。

其实计算的方式也并不复杂，主要就是两个规则: **继承**和**层叠**。

每个子节点都会默认继承父节点的样式属性，如果父节点中没有找到，就会采用浏览器默认样式，也叫 `UserAgent` 样式。这就是继承规则，非常容易理解。

然后是层叠规则，CSS 最大的特点在于它的层叠性，也就是最终的样式取决于各个属性共同作用的效果，甚至有很多诡异的层叠现象，看过《CSS世界》的同学应该对此深有体会，具体的层叠规则属于深入 CSS 语言的范畴，这里就不过多介绍了。

不过值得注意的是，在计算完样式之后，所有的样式值会被挂在到 `window.getComputedStyle` 当中，也就是可以通过JS来获取计算后的样式，非常方便。

## 生成布局树

现在已经生成了 DOM 树和 DOM 样式，接下来要做的就是通过浏览器的布局系统确定元素的位置，也就是要生成一棵布局树 (Layout Tree)。

布局树生成的大致工作如下：

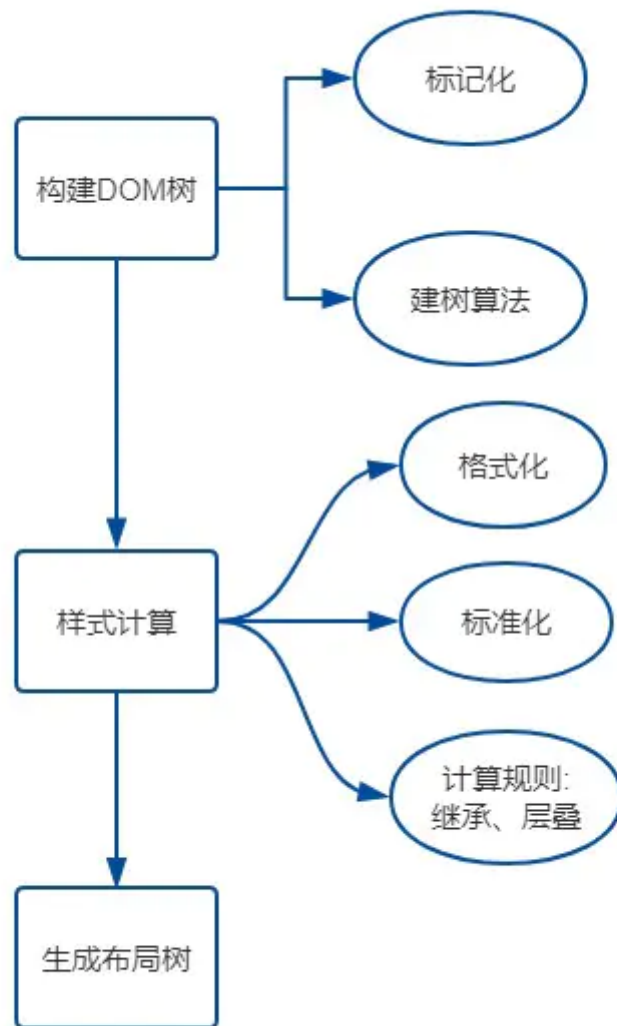
- 1) 遍历生成的 DOM 树节点，并把他们添加到布局树中。
- 2) 计算布局树节点的坐标位置。

值得注意的是，这棵布局树值包含可见元素，对于 `head` 标签和设置了 `display: none` 的元素，将不会被放入其中。

有人说首先会生成 `Render Tree`，也就是渲染树，其实这还是 16 年之前的事情，现在 Chrome 团队已经做了大量的重构，已经没有生成 `Render Tree` 的过程了。而布局树的信息已经非常完善，完全拥有 `Render Tree` 的功能。

## (3) 总结

梳理一下这一节的主要脉络：



## 5. 说一说从输入URL到页面呈现发生了什么？（渲染过程）

上一节介绍了浏览器解析的过程,其中包含构建DOM、样式计算和构建布局树。

接下来就来拆解下一个过程——渲染。分为以下几个步骤:

- 建立 图层树 (Layer Tree)
- 生成 绘制列表
- 生成 图块 并 栅格化
- 显示器显示内容

### 建图层树

如果你觉得现在 DOM节点 也有了, 样式和位置信息也有了, 可以开始绘制页面了, 那你就错了。

因为你考虑掉了另外一些复杂的场景, 比如3D动画如何呈现出变换效果, 当元素含有层叠上下文时如何控制显示和隐藏等等。

为了解决如上所述的问题, 浏览器在构建完 布局树 之后, 还会对特定的节点进行分层, 构建一棵 图层树 (Layer Tree)。

那这棵图层树是根据什么来构建的呢?



一般情况下，节点的图层会默认属于父亲节点的图层(这些图层也称为**合成层**)。那什么时候会提升为一个单独的合成层呢？

有两种情况需要分别讨论，一种是**显式合成**，一种是**隐式合成**。

## (1) 显式合成

下面是**显式合成**的情况：

### 一、拥有**层叠上下文**的节点。

层叠上下文也基本上是有一些特定的CSS属性创建的，一般有以下情况：

- HTML根元素本身就具有层叠上下文。
- 普通元素设置**position不为static**并且**设置了z-index属性**，会产生层叠上下文。
- 元素的 **opacity** 值不是 1
- 元素的 **transform** 值不是 none
- 元素的 **filter** 值不是 none
- 元素的 **isolation** 值是isolate
- **will-change**指定的属性值为上面任意一个。(will-change的作用后面会详细介绍)

### 二、需要**剪裁**的地方。

比如一个div，你只给他设置 100 \* 100 像素的大小，而你在里面放了非常多的文字，那么超出的文字部分就需要被剪裁。当然如果出现了滚动条，那么滚动条会被单独提升为一个图层。

## (2) 隐式合成

接下来是**隐式合成**，简单来说就是**层叠等级低**的节点被提升为单独的图层之后，那么**所有层叠等级比它高的节点都会**成为一个单独的图层。

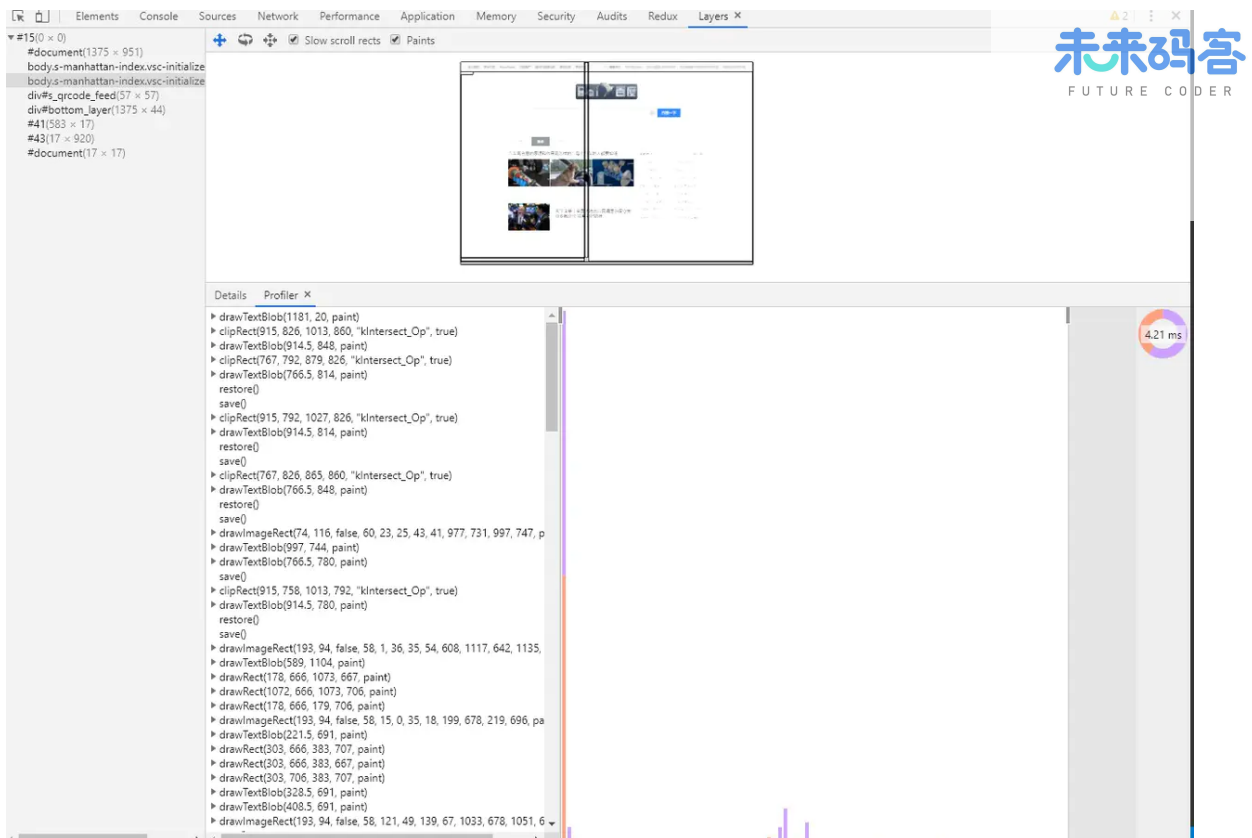
这个隐式合成其实隐藏着巨大的风险，如果在一个大型应用中，当一个**z-index**比较低的元素被提升为单独图层之后，层叠在它上面的元素统统都会被提升为单独的图层，可能会增加上千个图层，大大增加内存的压力，甚至直接让页面崩溃。这就是**层爆炸**的原理。

值得注意的是，当需要 **repaint** 时，只需要 **repaint** 本身，而不会影响到其他的层。

## 生成绘制列表

接下来渲染引擎会将图层的绘制拆分成一个个绘制指令，比如先画背景、再描绘边框.....然后将这些指令按顺序组合成一个待绘制列表，相当于给后面的绘制操作做了一波计划。

这里我以百度首页为例，大家可以在 Chrome 开发者工具中在设置栏中展开 **more tools**，然后选择 **Layers** 面板，就能看到下面的绘制列表：



## 生成图块和生成位图

现在开始绘制操作，实际上在渲染进程中绘制操作是由专门的线程来完成的，这个线程叫**合成线程**。

绘制列表准备好了之后，渲染进程的主线程会给**合成线程**发送 `commit` 消息，把绘制列表提交给合成线程。接下来就是合成线程一展宏图的时候啦。

首先，考虑到视口就这么大，当页面非常大的时候，要滑很长时间才能滑到底，如果要一口气全部绘制出来是相当浪费性能的。因此，合成线程要做的第一件事情就是将图层**分块**。这些块的大小一般不会特别大，通常是  $256 * 256$  或者  $512 * 512$  这个规格。这样可以大大加速页面的首屏展示。

因为后面图块数据要进入 GPU 内存，考虑到浏览器内存上传到 GPU 内存的操作比较慢，即使是绘制一部分图块，也可能会耗费大量时间。针对这个问题，Chrome 采用了一个策略：在首次合成图块时只采用一个**低分辨率**的图片，这样首屏展示的时候只是展示出低分辨率的图片，这个时候继续进行合成操作，当正常的图块内容绘制完毕后，会将当前低分辨率的图块内容替换。这也是 Chrome 底层优化首屏加载速度的一个手段。

顺便提醒一点，渲染进程中专门维护了一个**栅格化线程池**，专门负责把**图块**转换为**位图数据**。

然后合成线程会选择视口附近的**图块**，把它交给**栅格化线程池**生成位图。

生成位图的过程实际上都会使用 GPU 进行加速，生成的位图最后发送给**合成线程**。

## 显示器显示内容

栅格化操作完成后，**合成线程**会生成一个绘制命令，即"DrawQuad"，并发送给浏览器进程。

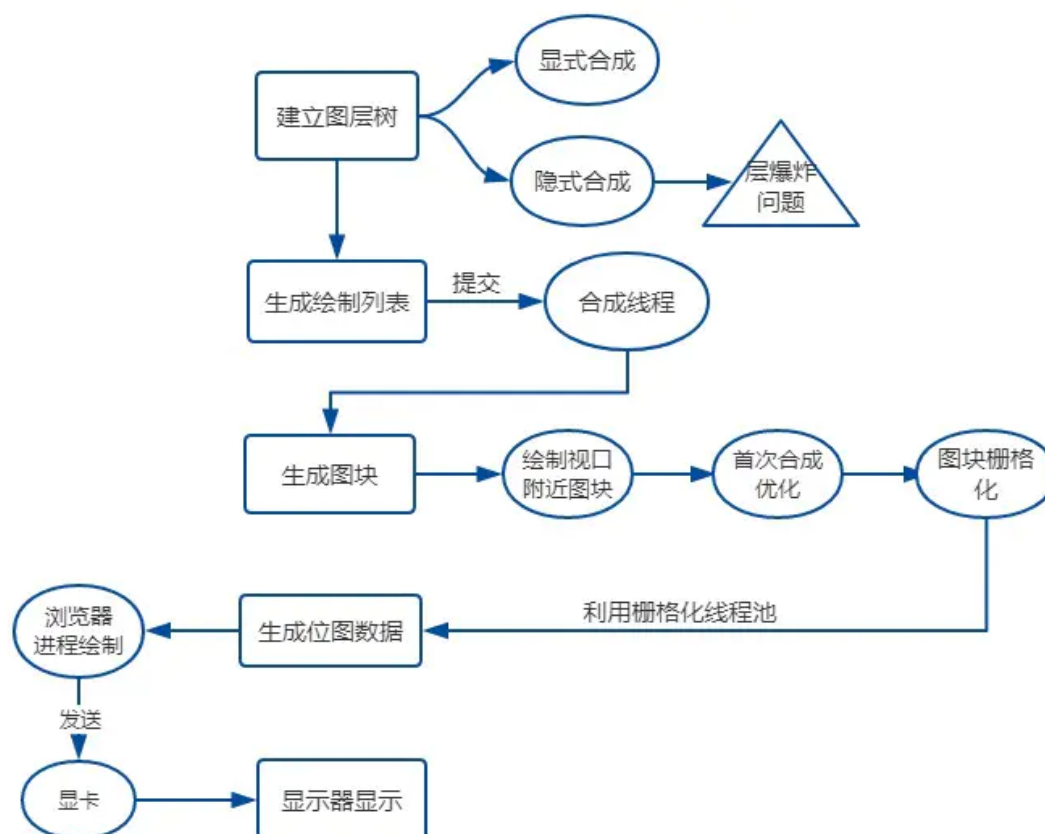
浏览器进程中的 `viz` 组件 接收到这个命令，根据这个命令，把页面内容绘制到内存，也就是生成了页面，然后把这部分内存发送给显卡。为什么发给显卡呢？我想有必要先聊一聊显示器显示图像的原理。

无论是 PC 显示器还是手机屏幕，都有一个固定的刷新频率，一般是 60 HZ，即 60 帧，也就是一秒更新 60 张图片，一张图片停留的时间约为 16.7 ms。而每次更新的图片都来自显卡的**前缓冲区**。而显卡接收来自浏览器进程传来的页面后，会合成相应的图像，并将图像保存到**后缓冲区**，然后系统自动将**前缓冲区**和**后缓冲区**对换位置，如此循环更新。

看到这里你也就是明白，当某个动画大量占用内存的时候，浏览器生成图像的时候会变慢，图像传送给显卡就会不及时，而显示器还是以不变的频率刷新，因此会出现卡顿，也就是明显的掉帧现象。

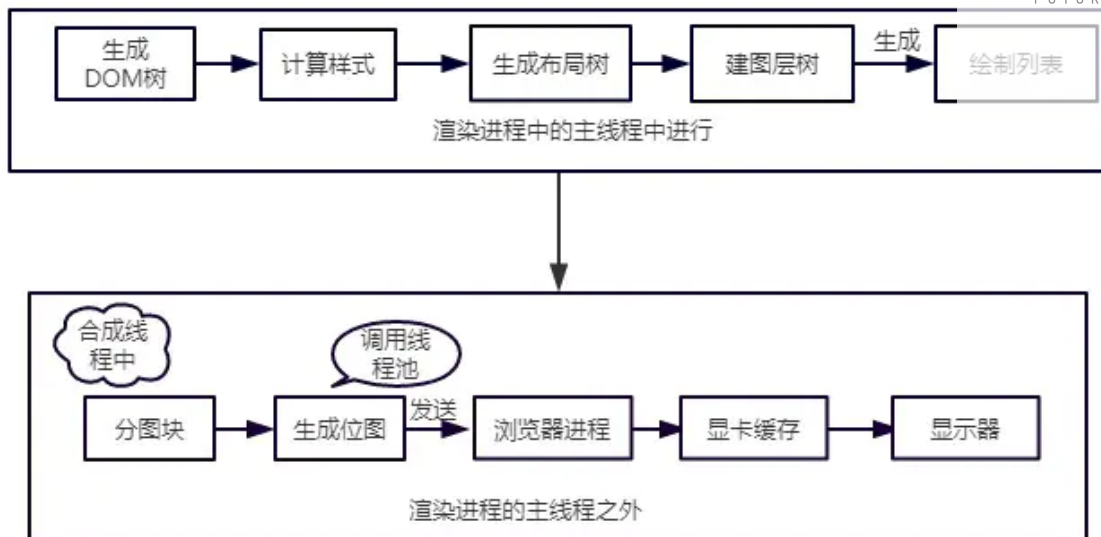
## 总结

到这里，我们算是把整个过程给走通了，现在重新来梳理一下页面渲染的流程。



## 6. 谈谈你对重绘和回流的理解

我们首先来回顾一下渲染流水线的流程:



接下来, 我们将来以此为依据来介绍重绘和回流, 以及让更新视图的另外一种方式——合成。

## 回流

首先介绍回流。回流也叫重排。

### (1) 触发条件

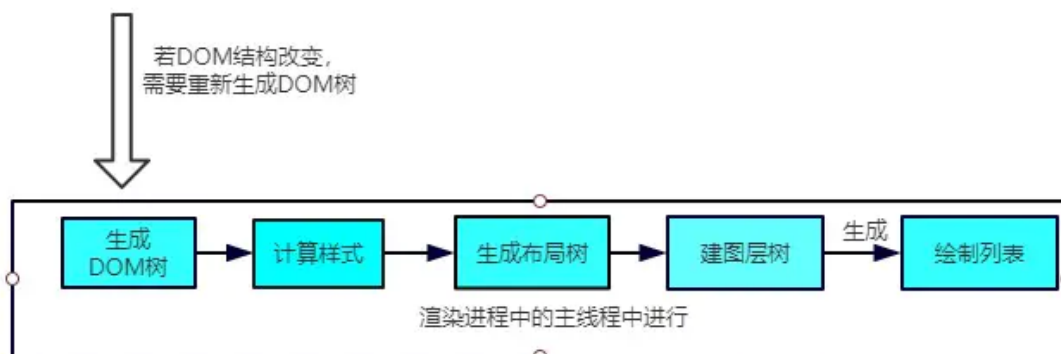
简单来说, 就是当我们对 DOM 结构的修改引发 DOM 几何尺寸变化的时候, 会发生回流的过程。

具体一点, 有以下的操作会触发回流:

1. 一个 DOM 元素的几何属性变化, 常见的几何属性有 `width`、`height`、`padding`、`margin`、`left`、`top`、`border` 等等, 这个很好理解。
2. 使 DOM 节点发生 增减 或者 移动。
3. 读写 `offset` 族、`scroll` 族和 `client` 族属性的时候, 浏览器为了获取这些值, 需要进行回流操作。
4. 调用 `window.getComputedStyle` 方法。

### (2) 回流过程

依照上面的渲染流水线, 触发回流的时候, 如果 DOM 结构发生改变, 则重新渲染 DOM 树, 然后将后面的流程(包括主线程之外的任务)全部走一遍。



相当于将解析和合成的过程重新又走了一遍, 开销是非常大的。

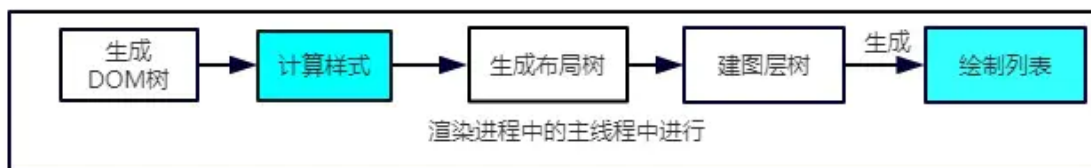
## 重绘

## (1) 触发条件

当 DOM 的修改导致了样式的变化，并且没有影响几何属性的时候，会导致重绘 (repaint)。

## (2) 重绘过程

由于没有导致 DOM 几何属性的变化，因此元素的位置信息不需要更新，从而省去布局的过程。流程如下：



跳过了生成布局树和建图层树的阶段，直接生成绘制列表，然后继续进行分块、生成位图等后面一系列操作。

可以看到，重绘不一定导致回流，但回流一定发生了重绘。

## 合成

还有一种情况，是直接合成。比如利用 CSS3 的 transform、opacity、filter 这些属性就可以实现合成的效果，也就是大家常说的 GPU 加速。

### (1) GPU 加速的原因

在合成的情况下，会直接跳过布局和绘制流程，直接进入非主线程处理的部分，即直接交给合成线程处理。交给它处理有两大好处：

- 能够充分发挥 GPU 的优势。合成线程生成位图的过程中会调用线程池，并在其中使用 GPU 进行加速生成，而 GPU 是擅长处理位图数据的。
- 没有占用主线程的资源，即使主线程卡住了，效果依然能够流畅地展示。

## 实践意义

- 知道上面的原理之后，对于开发过程有什么指导意义呢？
- 避免频繁使用 style，而是采用修改 class 的方式。
- 使用 createDocumentFragment 进行批量的 DOM 操作。
- 对于 resize、scroll 等进行防抖/节流处理。

添加 will-change: transform，让渲染引擎为其单独实现一个图层，当这些变换发生时，仅仅只是利用合成线程去处理这些变换，而不牵扯到主线程，大大提高渲染效率。当然这个变化不限于 transform，任何可以实现合成效果的 CSS 属性都能用 will-change 来声明。这里有一个实际的例子，一行 will-change: transform 拯救一个项目。

## 7. 能不能说一说 XSS 攻击？

### 1) 什么是 XSS 攻击？

XSS 全称是 Cross Site Scripting (即 跨站脚本)，为了和 CSS 区分，故叫它 XSS。XSS 攻击是指浏览器中执行恶意脚本 (无论是跨域还是同域)，从而拿到用户的信息并进行操作。

这些操作一般可以完成下面这些事情：

- 窃取 Cookie。
- 监听用户行为，比如输入账号密码后直接发送到黑客服务器。
- 修改 DOM 伪造登录表单。

- 在页面中生成浮窗广告。

通常情况，XSS 攻击的实现有三种方式——**存储型**、**反射型**和**文档型**。原理都比较简单，先来介绍一下。

### (1) 存储型

**存储型**，顾名思义就是将恶意脚本存储了起来，确实，存储型的 XSS 将脚本存储到了服务端的数据库，然后在客户端执行这些脚本，从而达到攻击的效果。

常见的场景是留言评论区提交一段脚本代码，如果前后端没有做好转义的工作，那评论内容存到了数据库，在页面渲染过程中**直接执行**，相当于执行一段未知逻辑的 JS 代码，是非常恐怖的。这就是存储型的 XSS 攻击。

### (2) 反射型

**反射型**XSS 指的是恶意脚本作为**网络请求的一部分**。

比如我输入：

```
http://sanyuan.com?q=<script>alert("你完蛋了")</script>
```

这杨，在服务器端会拿到 q 参数,然后将内容返回给浏览器端，浏览器将这些内容作为HTML的一部分解析，发现是一个脚本，直接执行，这样就被攻击了。

之所以叫它**反射型**，是因为恶意脚本是通过作为网络请求的参数，经过服务器，然后再反射到HTML文档中，执行解析。和**存储型**不一样的是，服务器并不会存储这些恶意脚本。

### (3) 文档型

文档型的 XSS 攻击并不会经过服务端，而是作为中间人的角色，在数据传输过程劫持到网络数据包，然后**修改里面的 html 文档**！

这样的劫持方式包括 **WIFI路由器劫持** 或者 **本地恶意软件** 等。

## 2) 防范措施

明白了三种XSS攻击的原理，我们能发现一个共同点: 都是让恶意脚本直接能在浏览器中执行。

那么要防范它，就是要避免这些脚本代码的执行。

为了完成这一点，必须做到一个信念，两个利用。

### (1) 一个信念

千万不要相信任何用户的输入！

无论是在前端和服务端，都要对用户的输入进行**转码**或者**过滤**。

如：

```
<script>alert('你完蛋了')</script>
```

转码后变为:

```
<script>alert('你完蛋了');</script>
```

这样的代码在 html 解析的过程中是无法执行的。

## (2) 利用 CSP

CSP, 即浏览器中的内容安全策略, 它的核心思想就是服务器决定浏览器加载哪些资源, 具体来说可以完成以下功能:

1. 限制其他域下的资源加载。
2. 禁止向其它域提交数据。
3. 提供上报机制, 能帮助我们及时发现 XSS 攻击。

## (3) 利用 HttpOnly

很多 XSS 攻击脚本都是用来窃取 Cookie, 而设置 Cookie 的 HttpOnly 属性后, JavaScript 便无法读取 Cookie 的值。这样也能很好的防范 XSS 攻击。

## 总结

XSS 攻击是指浏览器中执行恶意脚本, 然后拿到用户的信息进行操作。主要分为 存储型、反射型和 文档型。防范的措施包括:

- 一个信念: 不要相信用户的输入, 对输入内容转码或者过滤, 让其不可执行。
- 两个利用: 利用 CSP, 利用 Cookie 的 HttpOnly 属性。

## 8. 能不能说一说 CSRF 攻击?

### 什么是 CSRF 攻击?

CSRF(Cross-site request forgery), 即跨站请求伪造, 指的是黑客诱导用户点击链接, 打开黑客的网站, 然后黑客利用用户 **目前的登录状态** 发起跨站请求。

举个例子, 你在某个论坛点击了黑客精心挑选的小姐姐图片, 你点击后, 进入了一个新的页面。

那么恭喜你, 被攻击了:)

你可能会比较好奇, 怎么突然就被攻击了呢? 接下来我们就来拆解一下当你点击了链接之后, 黑客在背后做了哪些事情。

可能会做三样事情。列举如下:

### (1) 自动发 GET 请求

黑客网页里面可能有一段这样的代码:

```

```

进入页面后自动发送 get 请求, 值得注意的是, 这个请求会自动带上关于 xxx.com 的 cookie 信息(这里是假定你已经在 xxx.com 中登录过)。



假如服务器端没有相应的验证机制，它可能认为发请求的是一个正常的用户，因为携带了相应的 cookie，然后进行相应的各种操作，可以是转账汇款以及其他的恶意操作。

## (2) 自动发 POST 请求

黑客可能自己填了一个表单，写了一段自动提交的脚本。

```
<form id='hacker-form' action="https://xxx.com/info" method="POST">
  <input type="hidden" name="user" value="hhh" />
  <input type="hidden" name="count" value="100" />
</form>
<script>document.getElementById('hacker-form').submit();</script>
```

同样也会携带相应的用户 cookie 信息，让服务器误以为是一个正常的用户在操作，让各种恶意的操作变为可能。

## (3) 诱导点击发送 GET 请求

在黑客的网站上，可能会放上一个链接，驱使你点击：

```
<a href="https://xxx/info?user=hhh&count=100" target="_blank">点击进入修仙世界</a>
```

点击后，自动发送 get 请求，接下来和 自动发 GET 请求 部分同理。

这就是 CSRF 攻击的原理。和 XSS 攻击对比，CSRF 攻击并不需要将恶意代码注入用户当前页面的 html 文档中，而是跳转到新的页面，利用服务器的验证漏洞和用户之前的登录状态来模拟用户进行操作。

## 防范措施

### (1) 利用Cookie的SameSite属性

CSRF 攻击 中重要的一环就是自动发送目标站点下的 Cookie，然后就是这一份 Cookie 模拟了用户的身份。因此在 Cookie 上面下文章是防范的不二之选。

恰好，在 Cookie 当中有一个关键的字段，可以对请求中 Cookie 的携带作一些限制，这个字段就是 SameSite。

SameSite 可以设置为三个值，Strict、Lax 和 None。

- a. 在 Strict 模式下，浏览器完全禁止第三方请求携带 Cookie。比如请求 sanyuan.com 网站只能在 sanyuan.com 域名当中请求才能携带 Cookie，在其他网站请求都不能。
- b. 在 Lax 模式，就宽松一点了，但是只能在 get 方法提交表单 况或者 a 标签发送 get 请求 的情况下可以携带 Cookie，其他情况均不能。
- c. 在 None 模式下，也就是默认模式，请求会自动携带上 Cookie。

### (2) 验证来源站点

这就需要用到请求头中的两个字段：Origin 和 Referer。

其中，Origin 只包含域名信息，而 Referer 包含了 具体的 URL 路径。

当然，这两者都是可以伪造的，通过 Ajax 中自定义请求头即可，安全性略差。



### (3) CSRF Token

Django 作为 Python 的一门后端框架，如果是用它开发过的同学就知道，在它的模板(template)中，开发表单时，经常会附上这样一行代码：

```
{% csrf_token %}
```

复制代码

这就是 CSRF Token 的典型应用。那它的原理是怎样的呢？

首先，浏览器向服务器发送请求时，服务器生成一个字符串，将其植入到返回的页面中。

然后浏览器如果要发送请求，就必须带上这个字符串，然后服务器来验证是否合法，如果不合法则不予响应。这个字符串也就是 CSRF Token，通常第三方站点无法拿到这个 token，因此也就是被服务器给拒绝。

### 总结

CSRF(Cross-site request forgery)，即跨站请求伪造，指的是黑客诱导用户点击链接，打开黑客的网站，然后黑客利用用户目前的登录状态发起跨站请求。

CSRF 攻击一般会有三种方式：

- 自动 GET 请求
- 自动 POST 请求
- 诱导点击发送 GET 请求。

防范措施：利用 Cookie 的 SameSite 属性、验证来源站点和 CSRF Token。

## 9. HTTPS为什么让数据传输更安全？

谈到 HTTPS，就不得不谈到与之相对的 HTTP。HTTP 的特性是明文传输，因此在传输的每一个环节，数据都有可能被第三方窃取或者篡改，具体来说，HTTP 数据经过 TCP 层，然后经过 WIFI 路由器、运营商和目标服务器，这些环节中都可能被中间人拿到数据并进行篡改，也就是我们常说的中间人攻击。

为了防范这样一类攻击，我们不得已要引入新的加密方案，即 HTTPS。

HTTPS 并不是一个新的协议，而是一个加强版的 HTTP。其原理是在 HTTP 和 TCP 之间建立了一个中间层，当 HTTP 和 TCP 通信时并不是像以前那样直接通信，直接经过了一个中间层进行加密，将加密后的数据包传给 TCP，响应的，TCP 必须将数据包解密，才能传给上面的 HTTP。这个中间层也叫安全层。安全层的核心就是对数据加解密。

接下来我们就来剖析一下 HTTPS 的加解密是如何实现的。

### 对称加密和非对称加密

#### (1) 概念

首先需要理解 对称加密 和 非对称加密 的概念，然后讨论两者应用后的效果如何。

对称加密 是最简单的方式，指的是 加密 和 解密 用的是同样的密钥。

而对于 非对称加密，如果有 A、B 两把密钥，如果用 A 加密过的数据包只能用 B 解密，反之，如果用 B 加密过的数据包只能用 A 解密。

## (2) 加解密过程

接着我们来谈谈 浏览器 和 服务器 进行协商加解密的过程。

首先，浏览器会给服务器发送一个随机数 `client_random` 和一个加密的方法列表。

服务器接收后给浏览器返回另一个随机数 `server_random` 和加密方法。

现在，两者拥有三样相同的凭证: `client_random`、`server_random` 和加密方法。

接着用这个加密方法将两个随机数混合起来生成密钥，这个密钥就是浏览器和服务端通信的 暗号。

## (3) 各自应用的效果

如果用 对称加密 的方式，那么第三方可以在中间获取到 `client_random`、`server_random` 和加密方法，由于这个加密方法同时可以解密，所以中间人可以成功对暗号进行解密，拿到数据，很容易就将这种加密方式破解了。

既然 对称加密 这么不堪一击，我们就来试一试 非对称 加密。在这种加密方式中，服务器手里有两把钥匙，一把是 公钥，也就是说每个人都能拿到，是公开的，另一把是 私钥，这把私钥只有服务器自己知道。

好，现在开始传输。

浏览器把 `client_random` 和加密方法列表传过来，服务器接收到，把 `server_random`、加密方法和 公钥 传给浏览器。

现在两者拥有相同的 `client_random`、`server_random` 和加密方法。然后浏览器用公钥将 `client_random` 和 `server_random` 加密，生成与服务器通信的 暗号。

这时候由于是非对称加密，公钥加密过的数据只能用 私钥 解密，因此中间人就算拿到浏览器传来的数据，由于他没有私钥，照样无法解密，保证了数据的安全性。

这难道一定就安全吗？聪明的小伙伴早就发现了端倪。回到 非对称加密 的定义，公钥加密的数据可以用私钥解密，那私钥加密的数据也可以用公钥解密呀！

服务器的数据只能用私钥进行加密(因为如果它用公钥那么浏览器也没法解密啦)，中间人一旦拿到公钥，那么就可以对服务端传来的数据进行解密了，就这样又被破解了。而且，只是采用非对称加密，对于服务器性能消耗也是相当巨大的，因此我们暂且不采用这种方案。

## 对称加密和非对称加密的结合

可以发现，对称加密和非对称加密，单独应用任何一个，都会存在安全隐患。那我们能不能把两者结合，进一步保证安全呢？

其实是可以的，演示一下整个流程：

- 1) 浏览器向服务器发送 `client_random` 和加密方法列表。
- 2) 服务器接收到，返回 `server_random`、加密方法以及公钥。
- 3) 浏览器接收，接着生成另一个随机数 `pre_random`，并且用公钥加密，传给服务器。
- 4) 服务器用私钥解密这个被加密后的 `pre_random`。

现在浏览器和服务端有三样相同的凭证: `client_random`、`server_random` 和 `pre_random`。然后两者用相同的加密方法混合这三个随机数，生成最终的 密钥。

然后浏览器和服务器尽管用一样的密钥进行通信，即使用 对称加密。

这个最终的密钥是很难被中间人拿到的，为什么呢？因为中间人没有私钥，从而拿不到 `pre_random`，也就无法生成最终的密钥了。

回头比较一下和单纯的使用非对称加密，这种方式做了什么改进呢？本质上是防止了私钥加密的数据外传。单独使用非对称加密，最大的漏洞在于服务器传数据给浏览器只能用私钥加密，这是危险产生的根源。利用 对称和非对称 加密结合的方式，就防止了这一点，从而保证了安全。

## 证书

尽管通过两者加密方式的结合，能够很好地实现加密传输，但实际上还是存在一些问题。黑客如果采用 DNS 劫持，将目标地址替换成黑客服务器的地址，然后黑客自己造一份公钥和私钥，照样能进行数据传输。而对于浏览器用户而言，他是不知道自己正在访问一个危险的服务器的。

事实上 HTTPS 在上述 结合对称和非对称加密 的基础上，又添加了 数字证书认证 的步骤。其目的就是让服务器证明自己的身份。

### (1) 传输过程

为了获取这个证书，服务器运营者需要向第三方认证机构获取授权，这个第三方机构也叫 CA (Certificate Authority)，认证通过后 CA 会给服务器颁发数字证书。

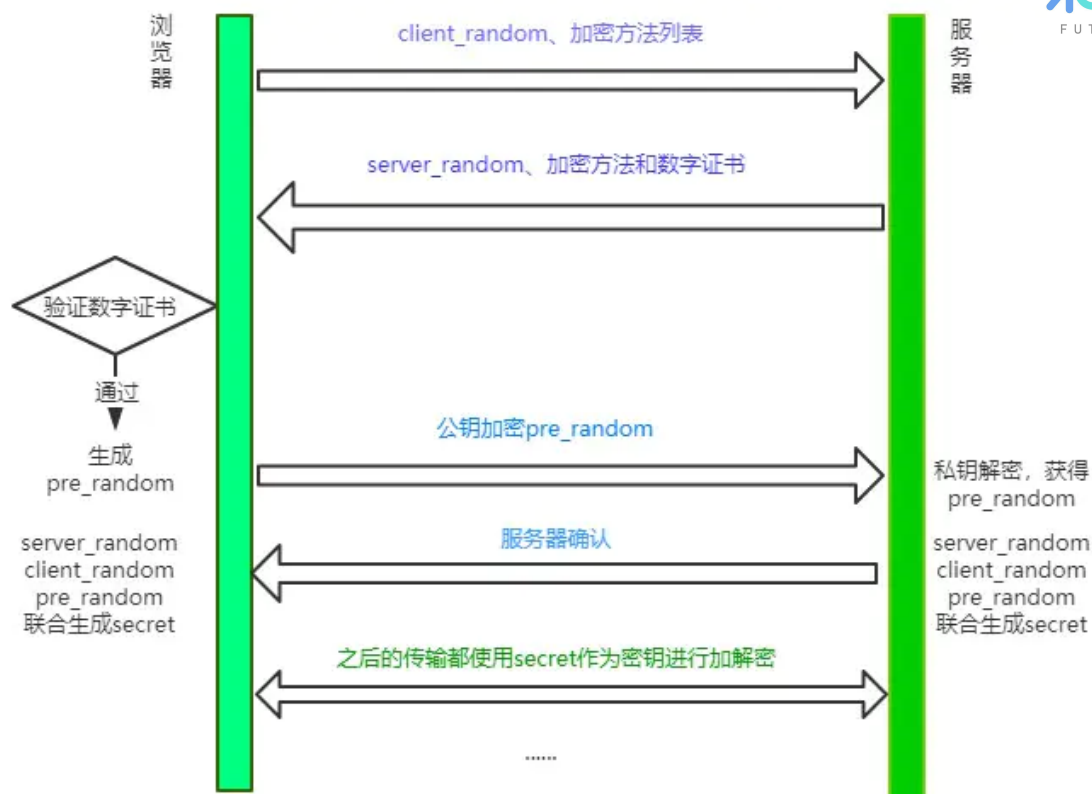
这个数字证书有两个作用：

1. 服务器向浏览器证明自己的身份。
2. 把公钥传给浏览器。

这个验证的过程发生在什么时候呢？

当服务器传送 `server_random`、加密方法的时候，顺便会带上 数字证书 (包含了 公钥)，接着浏览器接收之后就会开始验证数字证书。如果验证通过，那么后面的过程照常进行，否则拒绝执行。

现在我们来梳理一下 HTTPS 最终的加解密过程:



## (2) 认证过程

浏览器拿到数字证书后，如何来对证书进行认证呢？

首先，会读取证书中的明文内容。CA 进行数字证书的签名时会保存一个 Hash 函数，来这个函数来计算明文内容得到 信息A，然后用公钥解密明文内容得到 信息B，两份信息做比对，一致则表示认证合法。

当然有时候对于浏览器而言，它不知道哪些 CA 是值得信任的，因此会继续查找 CA 的上级 CA，以同样的信息比对方式验证上级 CA 的合法性。一般根级的 CA 会内置在操作系统当中，当然如果向上找没有找到根级的 CA，那么将被视为不合法。

## 总结

HTTPS并不是一个新的协议，它在 HTTP 和 TCP 的传输中建立了一个安全层，利用 对称加密 和 非对称加密 结合数字证书认证的方式，让传输过程的安全性大大提高。

## 10. 能不能实现事件的防抖和节流？

### 节流

节流的核心思想: 如果在定时器的时间范围内再次触发，则不予理睬，等当前定时器 完成，才能启动下一个定时器任务。这好比公交车，10 分钟一趟，10 分钟内有多少人在公交站等我不管，10 分钟一到我就要发车走人！

代码如下:

```
function throttle(fn, interval) {
  let flag = true;
  return function(...args) {
    let context = this;
    if (!flag) return;
    flag = false;
    setTimeout(() => {
      fn.apply(context, args);
      flag = true;
    }, interval);
  };
};
```

写成下面的方式也是表达一样的意思:

```
const throttle = function(fn, interval) {
  let last = 0;
  return function (...args) {
    let context = this;
    let now = +new Date();
    // 还没到时间
    if(now - last < interval) return;
    last = now;
    fn.apply(this, args)
  }
}
```

## 防抖

核心思想: 每次事件触发则删除原来的定时器, 建立新的定时器。跟王者荣耀的回城功能类似, 你反复触发回城功能, 那么只认最后一次, 从最后一次触发开始计时。

```
function debounce(fn, delay) {
  let timer = null;
  return function (...args) {
    let context = this;
    if(timer) clearTimeout(timer);
    timer = setTimeout(function() {
      fn.apply(context, args);
    }, delay);
  }
}
```

## 加强版节流

现在我们可以把 防抖 和 节流 放到一起, 为什么呢? 因为防抖有时候触发的太频繁会导致一次响应都没有, 我们希望到了固定的时间必须给用户一个响应, 事实上很多前端库就是采取了这样的思路。

```
function throttle(fn, delay) {
  let last = 0, timer = null;
  return function (...args) {
    let context = this;
```

```
let now = new Date();
if(now - last < delay){
  clearTimeout(timer);
  setTimeout(function() {
    last = now;
    fn.apply(context, args);
  }, delay);
} else {
  // 这个时候表示时间到了，必须给响应
  last = now;
  fn.apply(context, args);
}
}
```

## 11. 能不能实现图片懒加载?

### 方案一:clientHeight、scrollTop 和 offsetTop

首先给图片一个占位资源:

```

```

接着，通过监听 scroll 事件来判断图片是否到达视口:

```
let img = document.getElementsByTagName("img");
let num = img.length;
let count = 0; //计数器，从第一张图片开始计

lazyload(); //首次加载别忘了显示图片

window.addEventListener('scroll', lazyload);

function lazyload() {
  let viewHeight = document.documentElement.clientHeight; //视口高度
  let scrollTop = document.documentElement.scrollTop ||
document.body.scrollTop; //滚动条卷去的高度
  for(let i = count; i < num; i++) {
    // 元素现在已经出现在视口中
    if(img[i].offsetTop < scrollHeight + viewHeight) {
      if(img[i].getAttribute("src") !== "default.jpg") continue;
      img[i].src = img[i].getAttribute("data-src");
      count ++;
    }
  }
}
```

当然，最好对 scroll 事件做节流处理，以免频繁触发:

```
// throttle函数我们上节已经实现
window.addEventListener('scroll', throttle(lazyload, 200));
```

## 方案二: getBoundingClientRect

现在我们用另外一种方式来判断图片是否出现在了当前视口, 即 DOM 元素的 `getBoundingClientRect` API。

上述的 lazyload 函数改成下面这样:

```
function lazyload() {
  for(let i = count; i < num; i++) {
    // 元素现在已经出现在视口中
    if(img[i].getBoundingClientRect().top <
document.documentElement.clientHeight) {
      if(img[i].getAttribute("src") !== "default.jpg") continue;
      img[i].src = img[i].getAttribute("data-src");
      count ++;
    }
  }
}
```

## 方案三: IntersectionObserver

这是浏览器内置的一个 API, 实现了 监听window的scroll事件、判断是否在视口中 以及 节流 三大功能。

我们来具体试一把:

```
let img = document.getElementsByTagName("img");

const observer = new IntersectionObserver(changes => {
  //changes 是被观察的元素集合
  for(let i = 0, len = changes.length; i < len; i++) {
    let change = changes[i];
    // 通过这个属性判断是否在视口中
    if(change.isIntersecting) {
      const imgElement = change.target;
      imgElement.src = imgElement.getAttribute("data-src");
      observer.unobserve(imgElement);
    }
  }
})

Array.from(img).forEach(item => observer.observe(item));
```

这样就很方便地实现了图片懒加载, 当然这个 `IntersectionObserver` 也可以用作其他资源的预加载, 功能非常强大。

# Vue

## 1.什么是MVVM?

MVVM是Model-View-ViewModel的缩写。MVVM是一种设计思想。Model 层代表数据模型, 也可以在Model中定义数据修改和操作的业务逻辑; View 代表UI 组件, 它负责将数据模型转化成UI 展现出来, ViewModel 是一个同步View 和 Model的对象。

在MVVM架构下，View 和 Model 之间并没有直接的联系，而是通过ViewModel进行交互，Model和ViewModel之间的交互是双向的，因此View数据的变化会同步到Model中，而Model数据的变化也会立即反应到View上。

ViewModel 通过双向数据绑定把 View 层和 Model 层连接了起来，而View 和 Model 之间的同步工作完全是自动的，无需人为干涉，因此开发者只需关注业务逻辑，不需要手动操作DOM, 不需要关注数据状态的同步问题，复杂的数据状态维护完全由 MVVM 来统一管理。

## 2. mvvm和mvc区别？它和其它框架（jquery）的区别是什么？哪些场景适合？

mvc和mvvm其实区别并不大。都是一种设计思想。主要就是mvc中Controller演变成mvvm中的viewModel。mvvm主要解决了mvc中大量的DOM 操作使页面渲染性能降低，加载速度变慢，影响用户体验。

区别：vue数据驱动，通过数据来显示视图层而不是节点操作。

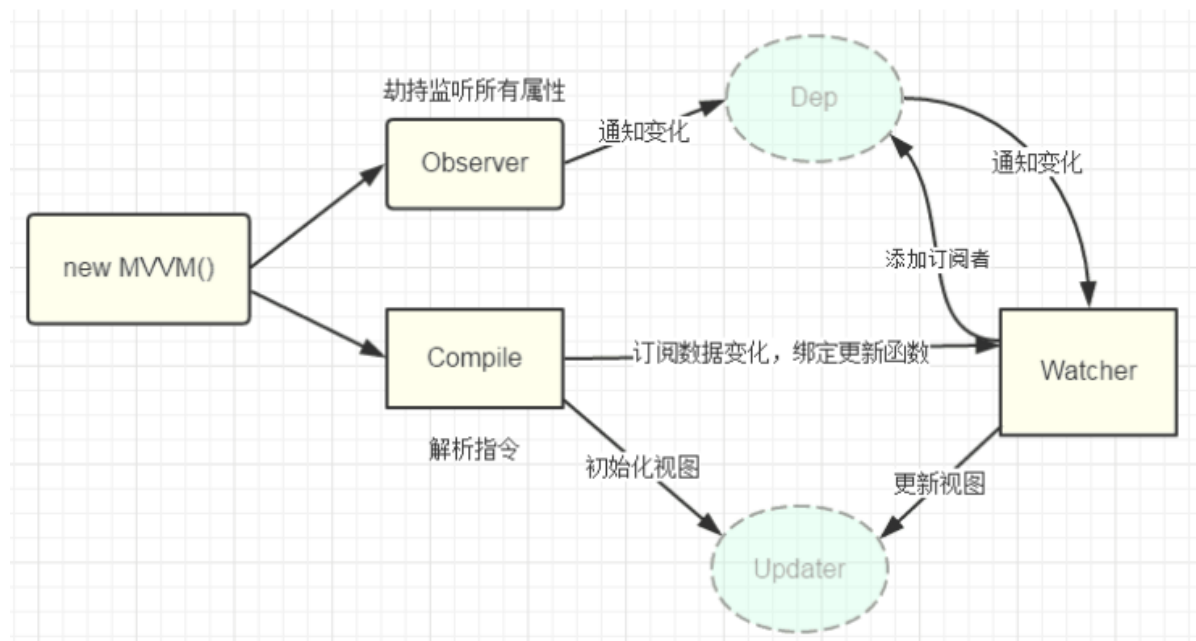
场景：数据操作比较多的场景，更加便捷

## 3. 组件之间的传值？

- 父组件与子组件传值  
父组件通过标签上面定义传值  
子组件通过props方法接受数据
- 子组件向父组件传递数据  
子组件通过\$emit方法传递参数

## 4. Vue 双向绑定原理

mvvm 双向绑定，采用数据劫持结合发布者-订阅者模式的方式，通过 Object.defineProperty() 来劫持各个属性的 setter、getter，在数据变动时发布消息给订阅者，触发相应的监听回调。



### 几个要点：

- 1) 实现一个数据监听器 Observer，能够对数据对象的所有属性进行监听，如有变动可拿到最新值并通知订阅者



2) 实现一个指令解析器 Compile，对每个元素节点的指令进行扫描和解析，根据指令模板替换数据以及绑定相应的更新函数

3) 实现一个 Watcher，作为连接 Observer 和 Compile 的桥梁，能够订阅并收到每个属性变动的通知，执行指令绑定的相应回调函数，从而更新视图

4) mvvm 入口函数，整合以上三者

#### 具体步骤：

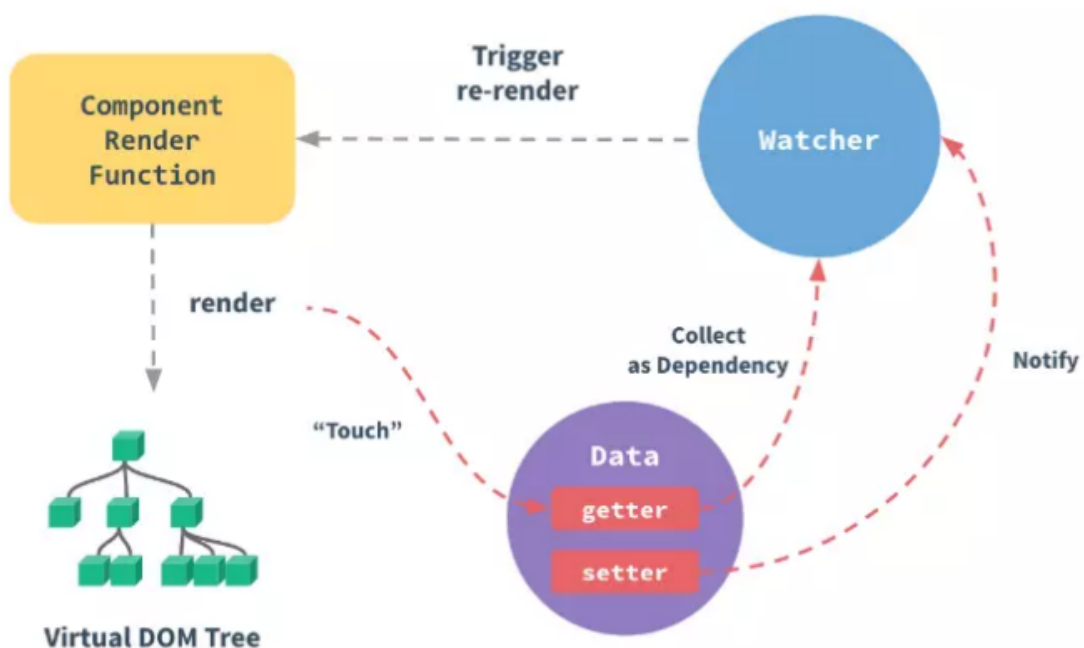
- 需要 observe 的数据对象进行递归遍历，包括子属性对象的属性，都加上 setter 和 getter 这样的话，给这个对象的某个值赋值，就会触发 setter，那么就能监听到了数据变化
- compile 解析模板指令，将模板中的变量替换成数据，然后初始化渲染页面视图，并将每个指令对应的节点绑定更新函数，添加监听数据的订阅者，一旦数据有变动，收到通知，更新视图
- Watcher 订阅者是 Observer 和 Compile 之间通信的桥梁，主要做的事情是：在自身实例化时往属性订阅器(dep)里面添加自己自身必须有一个 update() 方法待属性变动 dep.notice() 通知时，能调用自身的 update() 方法，并触发 Compile 中绑定的回调，则功成身退。
- MVVM 作为数据绑定的入口，整合 Observer、Compile 和 Watcher 三者，通过 Observer 来监听自己的 model 数据变化，通过 Compile 来解析编译模板指令，最终利用 Watcher 搭起 Observer 和 Compile 之间的通信桥梁，达到数据变化 -> 视图更新；视图交互变化(input) -> 数据 model 变更的双向绑定效果。

## 5. 描述下 vue 从初始化页面--修改数据--刷新页面 UI 的过程？

当 Vue 进入初始化阶段时，一方面 Vue 会遍历 data 中的属性，并用 Object.defineProperty 将它转化成 getter/setter 的形式，实现数据劫持(暂不谈 Vue3.0 的 Proxy)；另一方面，Vue 的指令编译器 Compiler 对元素节点的各个指令进行解析，初始化视图，并订阅 Watcher 来更新视图，此时 Watcher 会将自己添加到消息订阅器 Dep 中，此时初始化完毕。

当数据发生变化时，触发 Observer 中 setter 方法，立即调用 Dep.notify(), Dep 这个数组开始遍历所有的订阅者，并调用其 update 方法，Vue 内部再通过 diff 算法，patch 相应的更新完成对订阅者视图的改变。

## 6. 你是如何理解 Vue 的响应式系统的？



- 任何一个 Vue Component 都有一个与之对应的 Watcher 实例
- Vue 的 data 上的属性会被添加 getter 和 setter 属性
- 当 Vue Component render 函数被执行的时候, data 上会被 触碰(touch), 即被读, getter 方法会被调用, 此时 Vue 会去记录此 Vue component 所依赖的所有 data。(这一过程被称为依赖收集)
- data 被改动时 (主要是用户操作), 即被写, setter 方法会被调用, 此时 Vue 会去通知所有依赖于此 data 的组件去调用他们的 render 函数进行更新

## 7. 虚拟 DOM 实现原理

- 虚拟DOM本质上是JavaScript对象,是对真实DOM的抽象
- 状态变更时, 记录新树和旧树的差异
- 最后把差异更新到真正的dom

## 8. Vue 中 key 值的作用?

当 Vue.js 用 v-for 正在更新已渲染过的元素列表时, 它默认用“就地复用”策略。如果数据项的顺序被改变, Vue 将不会移动 DOM 元素来匹配数据项的顺序, 而是简单复用此处每个元素, 并且确保它在特定索引下显示已被渲染过的每个元素。key 的作用主要是为了高效的更新虚拟DOM。

## 9. Vue 的生命周期

总共分为8个阶段创建前/后, 载入前/后, 更新前/后, 销毁前/后。

- 创建前/后: 在beforeCreate阶段, vue实例的挂载元素el和数据对象data都为undefined, 还未初始化。在created阶段, vue实例的数据对象data有了, el还没有。
- 载入前/后: 在beforeMount阶段, vue实例的\$el和data都初始化了, 但还是挂载之前为虚拟的 dom 节点, data.message还未替换。在mounted阶段, vue实例挂载完成, data.message成功渲染。
- 更新前/后: 当data变化时, 会触发beforeUpdate和updated方法。
- 销毁前/后: 在执行destroy方法后, 对data的改变不会再触发周期函数, 说明此时vue实例已经解除了事件监听以及和dom的绑定, 但是dom结构依然存在

### (1) 什么是vue生命周期

Vue 实例从创建到销毁的过程, 就是生命周期。也就是从开始创建、初始化数据、编译模板、挂载 Dom→渲染、更新→渲染、卸载等一系列过程, 我们称这是 Vue 的生命周期。

### (2) vue生命周期的作用是什么

它的生命周期中有多个事件钩子, 让我们在控制整个Vue实例的过程时更容易形成好的逻辑。

### (3) 第一次页面加载会触发哪几个钩子

第一次页面加载时会触发 beforeCreate, created, beforeMount, mounted 这几个钩子

### (4) DOM 渲染在 哪个周期中就已经完成

DOM 渲染在 mounted 中就已经完成了

### (5) 简单描述每个周期具体适合哪些场景

生命周期钩子的一些使用方法：

- beforecreate : 可以在这加个loading事件，在加载实例时触发
- created : 初始化完成时的事件写在这里，如在这结束loading事件，异步请求也适宜在这里调用
- mounted : 挂载元素，获取到DOM节点
- updated : 如果对数据统一处理，在这里写上相应函数
- beforeDestroy : 可以做一个确认停止事件的确认框
- nextTick : 更新数据后立即操作dom

## 10. Vue 组件间通信有哪些方式？

- props/\$emit
- \$emit/\$on
- vuex
- \$attrs/\$listeners
- provide/inject
- \$parent/\$children 与 ref

## 11. vue 中怎么重置 data？

使用Object.assign(), vm.\$data可以获取当前状态下的data, vm.\$options.data(this)可以获取到组件初始化状态下的data。

```
Object.assign(this.$data, this.$options.data(this)) // 注意加this，不然取不到 data()
{ a: this.methodA } 中的this.methodA。
```

## 12. 组件中写 name 选项有什么作用？

- 项目使用 keep-alive 时，可搭配组件 name 进行缓存过滤
- DOM 做递归组件时需要调用自身 name
- vue-devtools 调试工具里显示的组见名称是由vue中组件name决定的

## 13. vue-router 有哪些钩子函数？

- 全局前置守卫 router.beforeEach
- 全局解析守卫 router.beforeResolve
- 全局后置钩子 router.afterEach
- 路由独享的守卫 beforeEnter
- 组件内的守卫 beforeRouteEnter、beforeRouteUpdate、beforeRouteLeave

## 14. route 和 router 的区别是什么？

route 是“路由信息对象”，包括 path , params , hash , query , fullPath , matched , name 等路由信息参数。

router 是“路由实例对象”，包括了路由的跳转方法( push 、 replace )，钩子函数等。

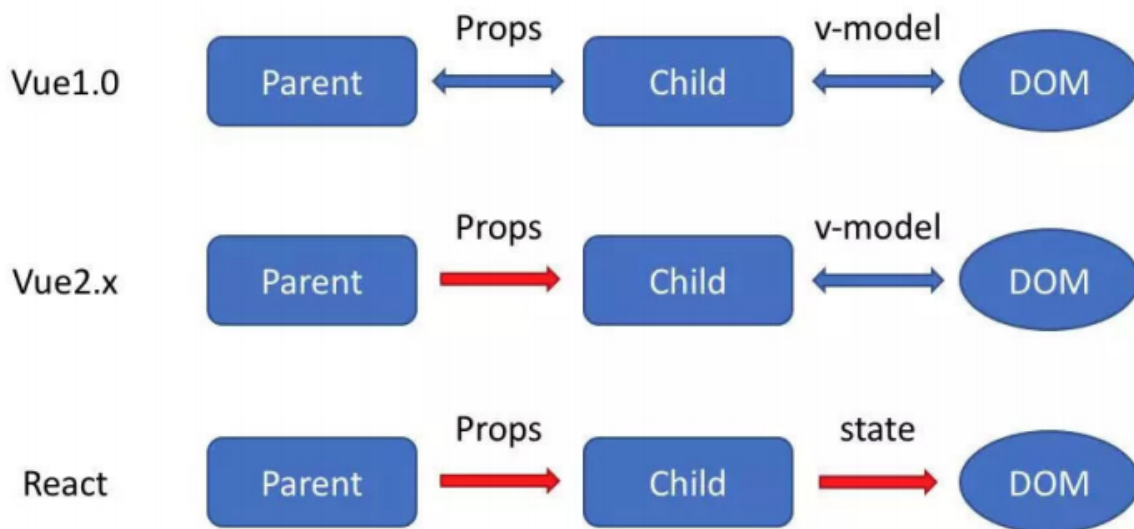
## 15. 说一下 Vue 和 React 的认识，做一个简单的对比

### 1) 监听数据变化的实现原理不同

- Vue 通过 getter/setter 以及一些函数的劫持，能精确快速的计算出 Virtual DOM 的差异。这是由于它在渲染过程中，会跟踪每一个组件的依赖关系，不需要重新渲染整个组件树。
- React 默认是通过比较引用的方式进行，如果不优化，每当应用的状态被改变时，全部子组件都会重新渲染，可能导致大量不必要的 VDOM 的重新渲染。
- Vue 不需要特别的优化就能达到很好的性能，而对于 React 而言，需要通过 PureComponent/shouldComponentUpdate 这个生命周期方法来进行控制。如果你的应用中，交互复杂，需要处理大量的 UI 变化，那么使用 Virtual DOM 是一个好主意。如果你更新元素并不频繁，那么 Virtual DOM 并不一定适用，性能很可能还不如直接操控 DOM。

为什么 React 不精确监听数据变化呢？这是因为 Vue 和 React 设计理念上的区别，Vue 使用的是可变数据，而 React 更强调数据的不可变。

### 2) 数据流的不同



- Vue 中默认支持双向绑定，组件与 DOM 之间可以通过 v-model 双向绑定。但是，父子组件之间，props 在 2.x 版本是单向数据流
- React 一直提倡的是单向数据流，他称之为 onChange/setState() 模式。不过由于我们一般都会用 Vuex 以及 Redux 等单向数据流的状态管理框架，因此很多时候我们感受不到这一点的区别了。

### 3) 模板渲染方式的不同

在表层上，模板的语法不同

- React 是通过 JSX 渲染模板
- 而 Vue 是通过一种拓展的 HTML 语法进行渲染

在深层上，模板的原理不同，这才是他们的本质区别：

- React 是在组件 JS 代码中，通过原生 JS 实现模板中的常见语法，比如插值，条件，循环等，都是通过 JS 语法实现的
  - Vue 是在和组件 JS 代码分离的单独的模板中，通过指令来实现的，比如条件语句就需要 v-if 来实现
- 对这一点，我个人比较喜欢 React 的做法，因为他更加纯粹更加原生，而 Vue 的做法显得有些独特，会把 HTML 弄得很乱。举个例子，说明 React 的好处：react 中 render 函数是支持闭包特性的，所以我们 import 的组件在 render 中可以直接调用。但是在 Vue 中，由于模板中使用的数据都必须挂在 this 上进行一次中转，所以我们 import 一个组件完了之后，还需要在 components 中再声明下，这样显然是很奇怪但又不得不这样的做法。

## 16. Vue 的 nextTick 的原理是什么？

### 1) 为什么需要 nextTick

Vue 是异步修改 DOM 的并且不鼓励开发者直接接触 DOM，但有时候业务需要必须对数据更改--刷新后的 DOM 做相应的处理，这时候就可以使用 Vue.nextTick(callback)这个 api 了。

### 2) 理解原理前的准备

首先需要知道事件循环中宏任务和微任务这两个概念(这其实也是面试常考点)。

常见的宏任务有 script, setTimeout, setInterval, setImmediate, I/O, UI rendering

常见的微任务有 process.nextTick(Nodejs), Promise.then(), MutationObserver;

### 3) 理解 nextTick

而 nextTick 的原理正是 vue 通过异步队列控制 DOM 更新和 nextTick 回调函数先后执行的方式。如果大家看过这部分的源码，会发现其中做了很多 isNative()的判断，因为这里还存在兼容性优雅降级的问题。可见 Vue 开发团队的深思熟虑，对性能的良苦用心。

## 17. Vuex 有哪几种属性？

有五种，分别是 State、Getter、Mutation、Action、Module

## 18. vue 首屏加载优化

- 把不常改变的库放到 index.html 中，通过 cdn 引入，然后找到 build/webpack.base.conf.js 文件，在 module.exports = {} 中添加以下代码

```
externals: {  
  'vue': 'Vue',  
  'vue-router': 'VueRouter',  
  'element-ui': 'ELEMENT',  
},
```

这样 webpack 就不会把 vue.js, vue-router, element-ui 库打包了。

- vue 路由的懒加载  
import 或者 require 懒加载。
- 不生成 map 文件  
找到 config/index.js，修改为 productionSourceMap: false
- vue 组件尽量不要全局引入
- 使用更轻量级的工具库

## 19. vuex

### (1) vuex是什么？怎么使用？哪种功能场景使用它？

vue框架中状态管理。在main.js引入store，注入。新建一个目录store，..... export。

场景有：单页应用中，组件之间的状态。音乐播放、登录状态、加入购物车

### (2) vuex有哪几种属性？

有五种，分别是 State、Getter、Mutation、Action、Module

- vuex的State特性

A, Vuex就是一个仓库，仓库里面放了很多对象。其中state就是数据源存放地，对应于一般Vue对象里面的data

B, state里面存放的数据是响应式的，Vue组件从store中读取数据，若是store中的数据发生改变，依赖这个数据的组件也会发生更新

C、它通过mapState把全局的 state 和 getters 映射到当前组件的 computed 计算属性中

- vuex的Getter特性

A、getters 可以对State进行计算操作，它就是Store的计算属性

B、虽然在组件内也可以做计算属性，但是getters 可以在多组件之间复用

C、如果一个状态只在一个组件内使用，是可以不用getters

- vuex的Mutation特性

Action 类似于 mutation，不同在于：Action 提交的是 mutation，而不是直接变更状态；Action可以包含任意异步操作。

### (3) 不用Vuex会带来什么问题？

- 可维护性会下降，想修改数据要维护三个地方；
- 可读性会下降，因为一个组件里的数据，根本就看不出来是从哪来的；
- 增加耦合，大量的上传派发，会让耦合性大大增加，本来Vue用Component就是为了减少耦合，现在这么用，和组件化的初衷相背

## 20. v-show和v-if指令的共同点和不同点

- v-show指令是通过修改元素的display的CSS属性让其显示或者隐藏
- v-if指令是直接销毁和重建DOM达到让元素显示和隐藏的效果

## 数据结构和算法

### 链表

## 1. 简单的反转链表

反转一个单链表。

示例:

```
输入: 1->2->3->4->5->NULL
输出: 5->4->3->2->1->NULL
```

### &循环解决方案

这道题是链表中的经典题目，充分体现链表这种数据结构 操作思路简单，但是 实现上 并没有那么简单的特点。

那在实现上应该注意一些什么问题呢？

保存后续节点。作为新手来说，很容易将当前节点的 `next` 指针直接指向前一个节点，但其实当前节点 下一个节点 的指针也就丢失了。因此，需要在遍历的过程当中，先将下一个节点保存，然后再操作 `next` 指向。

链表结构声定义如下:

```
function ListNode(val) {
  this.val = val;
  this.next = null;
}
```

实现如下:

```
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
let reverseList = (head) => {
  if (!head)
    return null;
  let pre = null, cur = head;
  while (cur) {
    // 关键: 保存下一个节点的值
    let next = cur.next;
    cur.next = pre;
    pre = cur;
    cur = next;
  }
  return pre;
};
```

由于逻辑比较简单，代码直接一气呵成。不过仅仅写完还不够，对于链表问题，边界检查的习惯能帮助我们进一步保证代码的质量。

但作为系统性的训练而言，单单让程序通过未免太草率了，我们后续会尽可能地用不同的方式去解决相同的问题，达到融会贯通的效果，也是对自己思路的开拓，有时候或许能达到更优解。

### 递归解决方案

由于之前的思路已经介绍得非常清楚了，因此在这我们贴上代码，大家好好体会：



```
let reverseList = (head) => {  
  let reverse = (pre, cur) => {  
    if(!cur) return pre;  
    // 保存 next 节点  
    let next = cur.next;  
    cur.next = pre;  
    return reverse(cur, next);  
  }  
  return reverse(null, head);  
}
```

## 2. 区间反转

反转从位置  $m$  到  $n$  的链表。请使用一趟扫描完成反转。

说明:  $1 \leq m \leq n \leq$  链表长度。

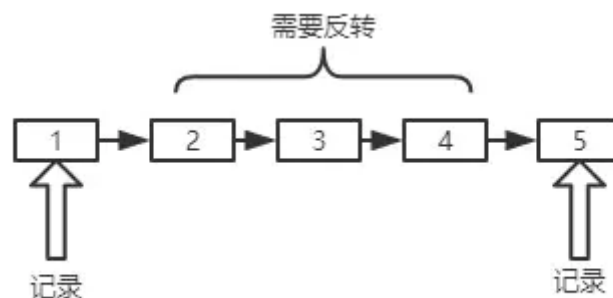
示例:

输入: 1->2->3->4->5->NULL,  $m = 2$ ,  $n = 4$   
输出: 1->4->3->2->5->NULL

### 思路

这一题相比上一个整个链表反转的题，其实是换汤不换药。我们依然有两种类型的解法：**循环解法**和**递归解法**。

需要注意的问题就是 **前后节点** 的保存(或者记录)，什么意思呢？看这张图你就明白了。



关于前节点和后节点的定义，大家在图上应该能看的比较清楚了，后面会经常用到。

反转操作上一题已经拆解过，这里不再赘述。值得注意的是反转后的工作，那么对于整个区间反转后的工作，其实就是一个移花接木的过程，首先将**前节点**的 next 指向区间终点，然后将区间起点的 next 指向**后节点**。因此这一题中有四个需要重视的节点：**前节点**、**后节点**、**区间起点**和**区间终点**。接下来我们开始实际的编码操作。

### 循环解法

```
/**  
 * @param {ListNode} head  
 * @param {number} m  
 * @param {number} n
```



```

* @return {ListNode}
*/
var reverseBetween = function(head, m, n) {
    let count = n - m;
    let p = dummyHead = new ListNode();
    let pre, cur, start, tail;
    p.next = head;
    for(let i = 0; i < m - 1; i++) {
        p = p.next;
    }
    // 保存前节点
    front = p;
    // 同时保存区间首节点
    pre = tail = p.next;
    cur = pre.next;
    // 区间反转
    for(let i = 0; i < count; i++) {
        let next = cur.next;
        cur.next = pre;
        pre = cur;
        cur = next;
    }
    // 前节点的 next 指向区间末尾
    front.next = pre;
    // 区间首节点的 next 指向后节点(循环完后的cur就是区间后面第一个节点, 即后节点)
    tail.next = cur;
    return dummyHead.next;
};

```

## 递归解法

对于递归解法, 唯一的不同就在于对于区间的处理, 采用递归程序进行处理, 大家也可以趁着复习一下递归反转的实现。

```

var reverseBetween = function(head, m, n) {
    // 递归反转函数
    let reverse = (pre, cur) => {
        if(!cur) return pre;
        // 保存 next 节点
        let next = cur.next;
        cur.next = pre;
        return reverse(cur, next);
    }
    let p = dummyHead = new ListNode();
    dummyHead.next = head;
    let start, end; // 区间首尾节点
    let front, tail; // 前节点和后节点
    for(let i = 0; i < m - 1; i++) {
        p = p.next;
    }
    front = p; // 保存前节点
    start = front.next;
    for(let i = m - 1; i < n; i++) {
        p = p.next;
    }
}

```

```
end = p;
tail = end.next; //保存后节点
end.next = null;
// 开始穿针引线啦，前节点指向区间首，区间首指向后节点
front.next = reverse(null, start);
start.next = tail;
return dummyHead.next;
}
```

### 3. 两个一组翻转链表

给定一个链表，两两交换其中相邻的节点，并返回交换后的链表。

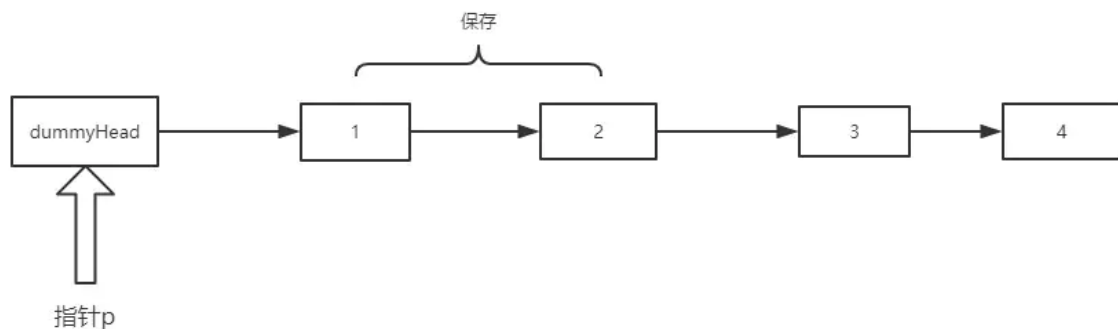
你不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

示例:

给定 1->2->3->4，你应该返回 2->1->4->3。

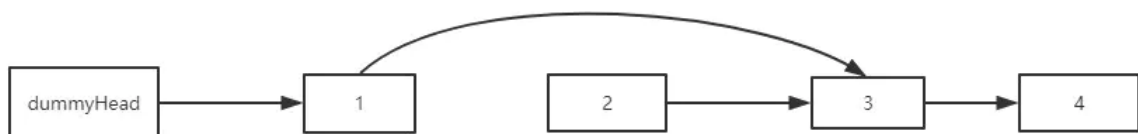
#### 思路

如图所示，我们首先建立一个虚拟头节点(dummyHead)，辅助我们分析。

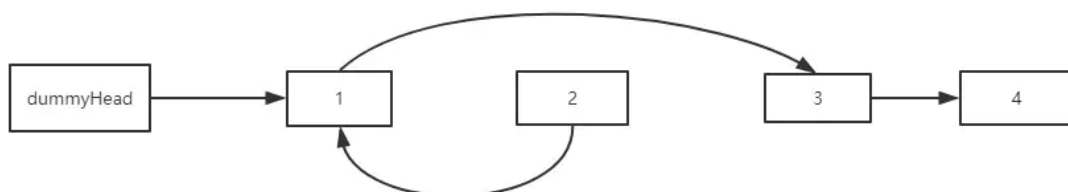


首先让 p 处在 dummyHead 的位置，记录下 p.next 和 p.next.next 的节点，也就是 node1 和 node2。

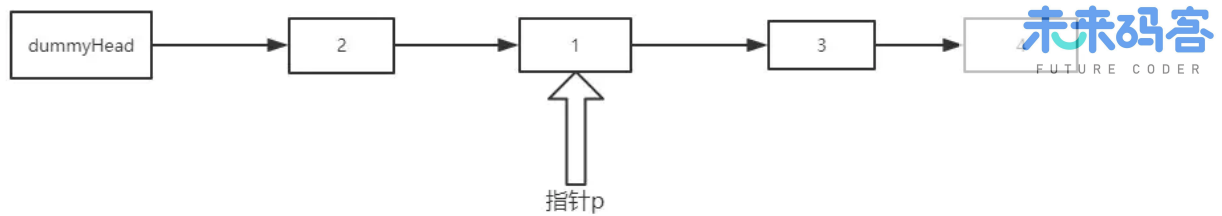
随后让 **node1.next = node2.next**, 效果:



然后让 **node2.next = node1**, 效果:



最后，**dummyHead.next = node2**，本次翻转完成。同时 p 指针指向node1，效果如下：



依此循环，如果 `p.next` 或者 `p.next.next` 为空，也就是找不到新的一组节点了，循环结束。

### 循环解决

思路清楚了，其实实现还是比较容易的，代码如下：

```
var swapPairs = function(head) {
  if(head == null || head.next == null)
    return head;
  let dummyHead = p = new ListNode();
  let node1, node2;
  dummyHead.next = head;
  while((node1 = p.next) && (node2 = p.next.next)) {
    node1.next = node2.next;
    node2.next = node1;
    p.next = node2;
    p = node1;
  }
  return dummyHead.next;
};
```

### 递归方式

```
var swapPairs = function(head) {
  if(head == null || head.next == null)
    return head;
  let node1 = head, node2 = head.next;
  node1.next = swapPairs(node2.next);
  node2.next = node1;
  return node2;
};
```

## 4. K个一组翻转链表

给你一个链表，每  $k$  个节点一组进行翻转，请你返回翻转后的链表。

$k$  是一个正整数，它的值小于或等于链表的长度。

如果节点总数不是  $k$  的整数倍，那么请将最后剩余的节点保持原有顺序。

示例：

给定这个链表: 1->2->3->4->5

当 k = 2 时, 应当返回: 2->1->4->3->5

当 k = 3 时, 应当返回: 3->2->1->4->5

说明:

- 你的算法只能使用常数的额外空间。
- 你不能只是单纯的改变节点内部的值, 而是需要实际的进行节点交换。

## 思路

思路类似No.3中的两个一组翻转。唯一的不同在于两个一组的情况下每一组只需要反转两个节点, 而在K个一组的情况下对应的操作是将K个元素的链表进行反转。

## 递归解法

以下代码的注释中首节点、尾结点等概念都是针对反转前的链表而言的。

```
/**
 * @param {ListNode} head
 * @param {number} k
 * @return {ListNode}
 */
var reverseKGroup = function(head, k) {
  let pre = null, cur = head;
  let p = head;
  // 下面的循环用来检查后面的元素是否能组成一组
  for(let i = 0; i < k; i++) {
    if(p == null) return head;
    p = p.next;
  }
  for(let i = 0; i < k; i++){
    let next = cur.next;
    cur.next = pre;
    pre = cur;
    cur = next;
  }
  // pre为本组最后一个节点, cur为下一组的起点
  head.next = reverseKGroup(cur, k);
  return pre;
};
```

## 循环解法

重点都放在注释里面了。

```
var reverseKGroup = function(head, k) {
  let count = 0;
  // 看是否能构成一组, 同时统计链表元素个数
  for(let p = head; p != null; p = p.next) {
    if(p == null && i < k) return head;
    count++;
  }
```

```

    }
    let loopCount = Math.floor(count / k);
    let p = dummyHead = new ListNode();
    dummyHead.next = head;
    // 分成了 loopCount 组，对每一个组进行反转
    for(let i = 0; i < loopCount; i++) {
        let pre = null, cur = p.next;
        for(let j = 0; j < k; j++) {
            let next = cur.next;
            cur.next = pre;
            pre = cur;
            cur = next;
        }
        // 当前 pre 为该组的尾结点，cur 为下一组首节点
        let start = p.next; // start 是该组首节点
        // 开始穿针引线！思路和2个一组的情况一模一样
        p.next = pre;
        start.next = cur;
        p = start;
    }
    return dummyHead.next;
};

```

## 5. 如何检测链表形成环？

给定一个链表，判断链表中是否形成环。

### 思路

思路一：循环一遍，用 Set 数据结构保存节点，利用节点的内存地址来进行判重，如果同样的节点走过两次，则表明已经形成了环。

思路二：利用快慢指针，快指针一次走两步，慢指针一次走一步，如果 **两者相遇**，则表明已经形成了环。

可能你会纳闷，为什么思路二用两个指针在环中一定会相遇呢？

其实很简单，如果有环，两者一定同时走到环中，那么在环中，**选慢指针为参考系**，快指针每次 **相对参考系** 向前走一步，终究会绕回原点，也就是回到慢指针的位置，从而让两者相遇。如果没有环，则两者的相对距离越来越远，永远不会相遇。

接下来我们来编程实现。

### 方法一：Set 判重

```

/**
 * @param {ListNode} head
 * @return {boolean}
 */
var hasCycle = (head) => {
    let set = new Set();
    let p = head;
    while(p) {
        // 同一个节点再次碰到，表示有环
        if(set.has(p)) return true;
        set.add(p);
    }
}

```

```
    p = p.next;
  }
  return false;
}
```

## 方法二: 快慢指针

```
var hasCycle = function(head) {
  let dummyHead = new ListNode();
  dummyHead.next = head;
  let fast = slow = dummyHead;
  // 零个结点或者一个结点，肯定无环
  if(fast.next == null || fast.next.next == null)
    return false;
  while(fast && fast.next) {
    fast = fast.next.next;
    slow = slow.next;
    // 两者相遇了
    if(fast == slow) {
      return true;
    }
  }
  return false;
};
```

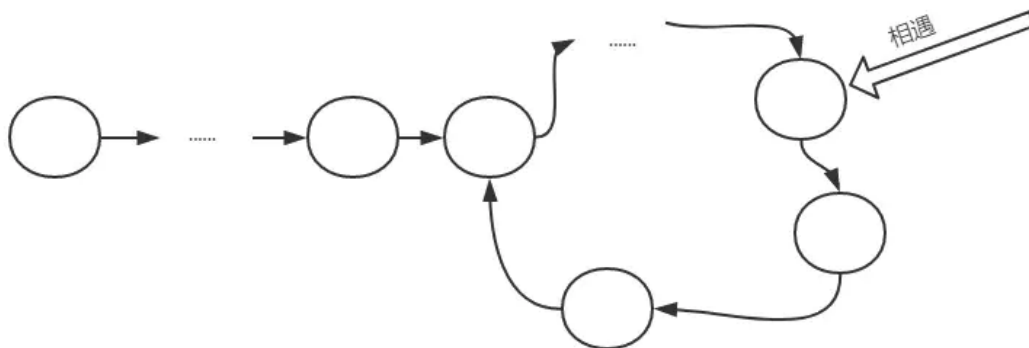
## 6. 如何找到环的起点

给定一个链表，返回链表开始入环的第一个节点。如果链表无环，则返回 null。

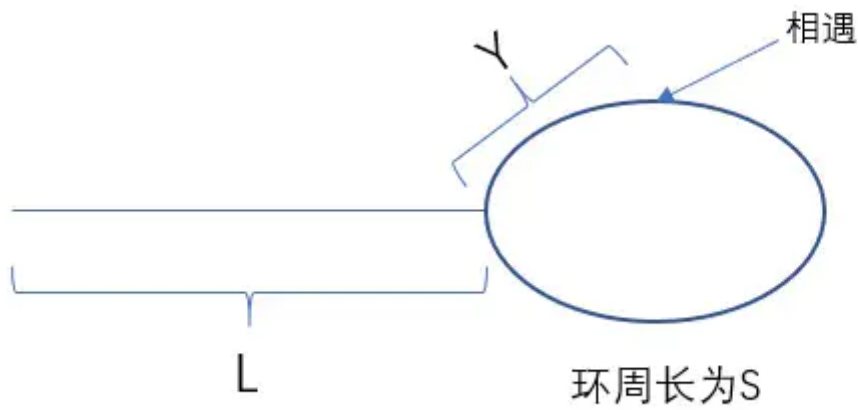
**说明：**不允许修改给定的链表。

### 思路分析

刚刚已经判断了如何判断出现环，那如何找到环的节点呢？我们来分析一波。



看上去比较繁琐，我们把它做进一步的抽象：



设快慢指针走了  $x$  秒，慢指针一秒走一次。

对快指针，有： $2x - L = m * S + Y$  -----①

对慢指针，有： $x - L = n * S + Y$  -----②

其中， $m$ 、 $n$  均为自然数。

① - ② \* 2 得：

$$L = (m - n) * S - Y \text{-----③}$$

好，这是一个非常重要的等式。我们现在假设有一个新的指针在  $L$  段的最左端，慢指针现在还在相遇处。

让 **新指针** 和 **慢指针** 都每次走一步，那么，当 **新指针** 走了  $L$  步之后**到达环起点**，而与此同时，我们看看 **慢指针** 情况如何。

由③式，慢指针走了  $(m - n) * S - Y$  个单位，以环起点为参照物，相遇时的位置为  $Y$ ，而现在由  $Y + (m - n) * S - Y$  即  $(m - n) * S$ ，得知慢指针实际上参照环起点，走了整整  $(m - n)$  圈。也就是说，**慢指针此时也到达了环起点**。:::tip 结论 现在的解法就很清晰了，当快慢指针相遇之后，让新指针从头出发，和慢指针同时前进，且每次前进一步，两者相遇的地方，就是**环起点**。

## 编程实现

懂得原理之后，实现起来就容易很多了。

```
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var detectCycle = function(head) {
    let dummyHead = new ListNode();
    dummyHead.next = head;
    let fast = slow = dummyHead;
    // 零个结点或者一个结点，肯定无环
    if(fast.next == null || fast.next.next == null)
        return null;
    while(fast && fast.next) {
        fast = fast.next.next;
        slow = slow.next;
        // 两者相遇了
    }
```

```

        if(fast == slow) {
            let p = dummyHead;
            while(p != slow) {
                p = p.next;
                slow = slow.next;
            }
            return p;
        }
    }
    return null;
};

```

## 7. 合并两个有序链表

将两个有序链表合并为一个新的有序链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

**示例:**

输入: 1->2->4, 1->3->4  
输出: 1->1->2->3->4->4

### 递归解法

递归解法更容易理解，我们先用递归来做一下：

```

/**
 * @param {ListNode} l1
 * @param {ListNode} l2
 * @return {ListNode}
 */
var mergeTwoLists = function(l1, l2) {
    const merge = (l1, l2) => {
        if(l1 == null) return l2;
        if(l2 == null) return l1;
        if(l1.val > l2.val) {
            l2.next = merge(l1, l2.next);
            return l2;
        } else {
            l1.next = merge(l1.next, l2);
            return l1;
        }
    }
    return merge(l1, l2);
};

```

### 循环解法

```

var mergeTwoLists = function(l1, l2) {
    if(l1 == null) return l2;
    if(l2 == null) return l1;

```



```

let p = dummyHead = new ListNode();
let p1 = l1, p2 = l2;
while(p1 && p2) {
    if(p1.val > p2.val) {
        p.next = p2;
        p = p.next;
        p2 = p2.next;
    }else {
        p.next = p1;
        p = p.next;
        p1 = p1.next;
    }
}
// 循环完成后务必检查剩下的部分
if(p1) p.next = p1;
else p.next = p2;
return dummyHead.next;
};

```

## 8. 合并 K 个有序链表

合并 k 个排序链表，返回合并后的排序链表。请分析和描述算法的复杂度。

示例:

```

输入:
[
  1->4->5,
  1->3->4,
  2->6
]
输出: 1->1->2->3->4->4->5->6

```

自上而下(递归)实现

```

/**
 * @param {ListNode[]} lists
 * @return {ListNode}
 */
var mergeKLists = function(lists) {
    // 上面已经实现
    var mergeTwoLists = function(l1, l2) { /*上面已经实现*/ };
    const _mergeLists = (lists, start, end) => {
        if(end - start < 0) return null;
        if(end - start == 0) return lists[end];
        let mid = Math.floor(start + (end - start) / 2);
        return mergeTwoList(_mergeLists(lists, start, mid), _mergeLists(lists,
mid + 1, end));
    }
    return _mergeLists(lists, 0, lists.length - 1);
};

```

在自下而上的实现方式中，为每一个链表绑定了一个虚拟头指针(dummyHead)，为什么这么做？

这是为了方便链表的合并，比如 l1 和 l2 合并之后，合并后链表的头指针就直接是 l1 的 dummyHead.next 值，等于说两个链表都合并到了 l1 当中，方便了后续的合并操作。

```
var mergeKLists = function(lists) {  
  var mergeTwoLists = function(l1, l2) { /*上面已经实现*/ };  
  // 边界情况  
  if(!lists || !lists.length) return null;  
  // 虚拟头指针集合  
  let dummyHeads = [];  
  // 初始化虚拟头指针  
  for(let i = 0; i < lists.length; i++) {  
    let node = new ListNode();  
    node.next = lists[i];  
    dummyHeads[i] = node;  
  }  
  // 自底向上进行merge  
  for(let size = 1; size < lists.length; size += size){  
    for(let i = 0; i + size < lists.length; i += 2 * size) {  
      dummyHeads[i].next = mergeTwoLists(dummyHeads[i].next, dummyHeads[i  
+ size].next);  
    }  
  }  
  return dummyHeads[0].next;  
};
```

多个链表的合并到这里就实现完成了，这种归并的方式同时也是对链表进行归并排序的核心代码。

## 9. 判断回文链表

请判断一个单链表是否为回文链表。

示例1:

输入: 1->2  
输出: false

示例2:

输入: 1->2->2->1  
输出: true

你能否用  $O(n)$  时间复杂度和  $O(1)$  空间复杂度解决此题？

### 思路分析

这一题如果不考虑性能的限制，其实是非常简单的。但考虑到  $O(n)$  时间复杂度和  $O(1)$  空间复杂度，恐怕就值得停下来好好想想了。

题目的要求是单链表，没有办法访问前面的节点，那我们只得另辟蹊径：

找到链表中点，然后将后半部分反转，就可以依次比较得出结论了。下面我们来实现一波。

## 代码实现

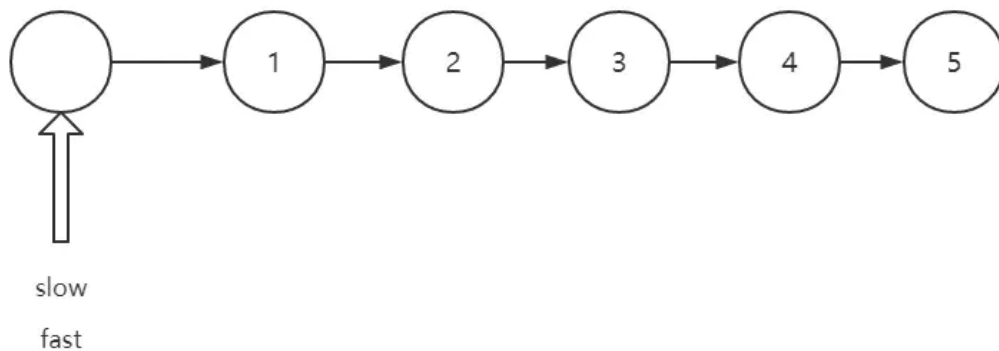
其实关键部分的代码就是找中点了。先亮剑：

```
let dummyHead = slow = fast = new ListNode();  
dummyHead.next = head;  
// 注意注意，来找中点了  
while(fast && fast.next) {  
    slow = slow.next;  
    fast = fast.next.next;  
}
```

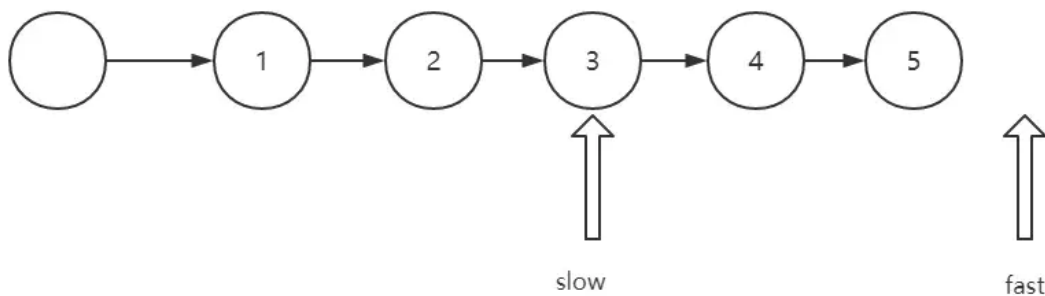
为什么边界要设成这样？

不妨来分析一下，分链表节点个数为 **奇数** 和 **偶数** 的时候分别讨论。

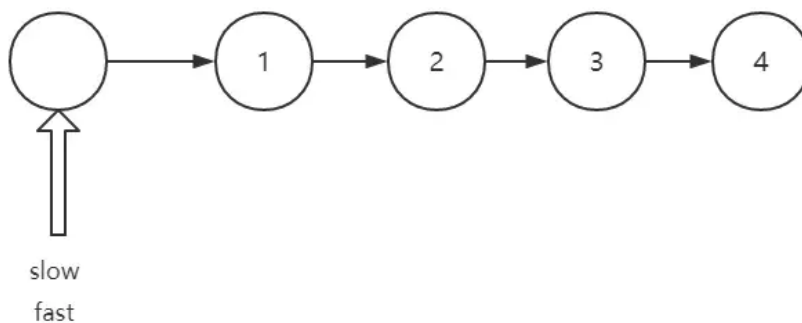
- 当链表节点个数为奇数



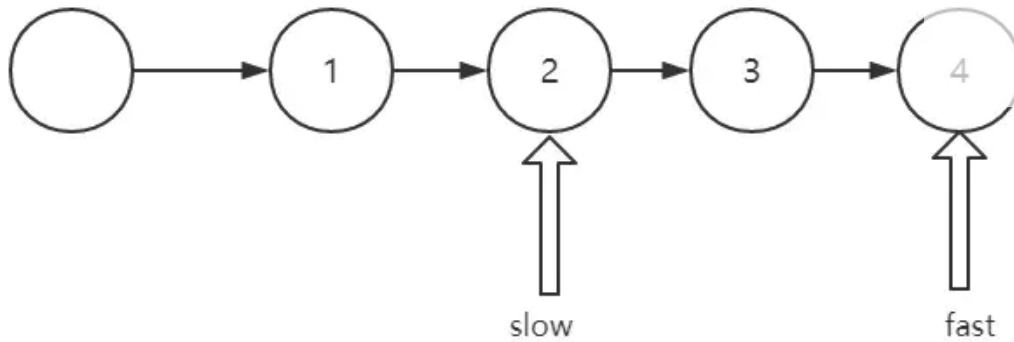
试着模拟一下，fast 为空的时候，停止循环，状态如下：



- 当链表节点个数为偶数



模拟走一遍，当 fast.next 为空的时候，停止循环，状态如下：



对于 fast 为空 和 fast.next 为空 两个条件，在奇数的情况下，总是 fast 为空 先出现，偶数的情况下，总是 fast.next 先出现。

也就是说：一旦 fast 为空，链表节点个数一定为奇数，否则为偶数。因此两种情况可以合并来讨论，当 fast 为空或者 fast.next 为空，循环就可以终止了。

完整实现如下：

```
/**
 * @param {ListNode} head
 * @return {boolean}
 */
var isPalindrome = function(head) {
    let reverse = (pre, cur) => {
        if(!cur) return pre;
        let next = cur.next;
        cur.next = pre;
        return reverse(cur, next);
    }
    let dummyHead = slow = fast = new ListNode();
    dummyHead.next = head;
    // 注意注意，来找中点了，黄金模板
    while(fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let next = slow.next;
    slow.next = null;
    let newStart = reverse(null, next);
    for(let p = head, newP = newStart; newP != null; p = p.next, newP = newP.next) {
        if(p.val !== newP.val) return false;
    }
    return true;
};
```

## 1. 有效括号

给定一个只包括 '(' , ')' , '{' , '}' , '[' , ']' 的字符串，判断字符串是否有效。

有效字符串需满足：

左括号必须用相同类型的右括号闭合。左括号必须以正确的顺序闭合。注意空字符串可被认为是有效字符串。

示例:

```
输入: "()"
输出: true
```

### 代码实现

```
/**
 * @param {string} s
 * @return {boolean}
 */
var isValid = function(s) {
    let stack = [];
    for(let i = 0; i < s.length; i++) {
        let ch = s.charAt(i);
        if(ch == '(' || ch == '[' || ch == '{')
            stack.push(ch);
        if(!stack.length) return false;
        if(ch == ')' && stack.pop() !== '(') return false;
        if(ch == ']' && stack.pop() !== '[') return false;
        if(ch == '}' && stack.pop() !== '{') return false;
    }
    return stack.length === 0;
};
```

## 2. 多维数组 flatten

将多维数组转化为一维数组。

示例:

```
[1, [2, [3, [4, 5]]], 6] -> [1, 2, 3, 4, 5, 6]
```

### 代码实现

```
/**
 * @constructor
 * @param {NestedInteger[]} nestedList
 * @return {Integer[]}
 */
```

```
let flatten = (nestedList) => {
  let result = [];
  let fn = function (target, ary) {
    for (let i = 0; i < ary.length; i++) {
      let item = ary[i];
      if (Array.isArray(ary[i])) {
        fn(target, item);
      } else {
        target.push(item);
      }
    }
  }
  fn(result, nestedList)
  return result;
}
```

同时可采用 reduce 的方式, 一行就可以解决, 非常简洁。

```
let flatten = (nestedList) => nestedList.reduce((pre, cur) =>
pre.concat(Array.isArray(cur) ? flatten(cur): cur), [])
```

### 3. 普通的层次遍历

给定一个二叉树, 返回其按层次遍历的节点值。 (即逐层地, 从左到右访问所有节点)。

示例:

```

  3
 / \
9  20
 / \
15  7
```

结果应输出:

```
[
  [3],
  [9,20],
  [15,7]
]
```

实现

```
/**
 * @param {TreeNode} root
 * @return {number[][]}
 */
var levelOrder = function(root) {
  if(!root) return [];
  let queue = [];
  let res = [];
  let level = 0;
  queue.push(root);
```

```
let temp;
while(queue.length) {
  res.push([]);
  let size = queue.length;
  // 注意一下: size -- 在层次遍历中是一个非常重要的技巧
  while(size --) {
    // 出队
    let front = queue.shift();
    res[level].push(front.val);
    // 入队
    if(front.left) queue.push(front.left);
    if(front.right) queue.push(front.right);
  }
  level++;
}
return res;
};
```

#### 4. 二叉树的锯齿形层次遍历

给定一个二叉树，返回其节点值的锯齿形层次遍历。（即先从左往右，再从右往左进行下一层遍历，以此类推，层与层之间交替进行）。

例如：

给定二叉树 [3,9,20,null,null,15,7], 输出应如下：

```

  3
 / \
9   20
 / \
15  7
```

返回锯齿形层次遍历如下：

```
[
  [3],
  [20,9],
  [15,7]
]
```

#### 思路

这一题思路稍微不同，但如果把握住层次遍历的思路，就会非常简单。

#### 代码实现

```
var zigzagLevelOrder = function(root) {
  if(!root) return [];
  let queue = [];
  let res = [];
  let level = 0;
```

```

queue.push(root);
let temp;
while(queue.length) {
    res.push([]);
    let size = queue.length;
    while(size --) {
        // 出队
        let front = queue.shift();
        res[level].push(front.val);
        if(front.left) queue.push(front.left);
        if(front.right) queue.push(front.right);
    }
    // 仅仅增加下面一行代码即可
    if(level % 2) res[level].reverse();
    level++;
}
return res;
};

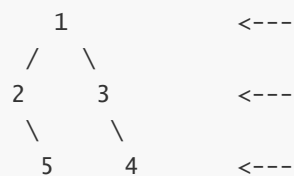
```

## 5. 二叉树的右视图

给定一棵二叉树，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例:

输入: [1,2,3,null,5,null,4]  
 输出: [1, 3, 4]  
 解释:



### 思路

右视图？如果你以 DFS 即深度优先搜索的思路来想，你会感觉异常的痛苦。

但如果用广度优先搜索的思想，即用层序遍历的方式，求解这道题目也变得轻而易举。

### 代码实现

```

/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var rightSideView = function(root) {
    if(!root) return [];
    let queue = [];
    let res = [];
    queue.push(root);

```



```
while(queue.length) {  
    res.push(queue[0].val);  
    let size = queue.length;  
    while(size --) {  
        // 一个size的循环就是一层的遍历，在这一层只拿最右边的结点  
        let front = queue.shift();  
        if(front.right) queue.push(front.right);  
        if(front.left) queue.push(front.left);  
    }  
}  
return res;  
};
```

## 6. 完全平方数

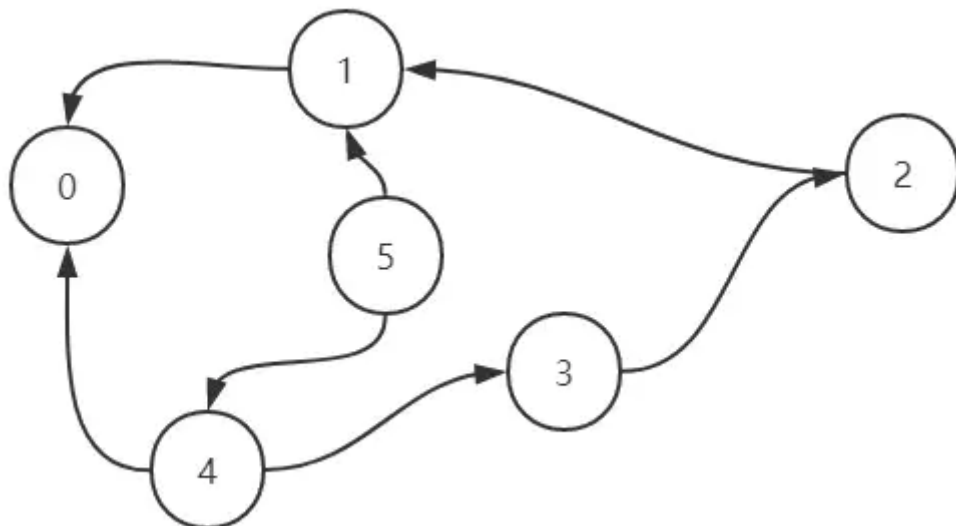
给定正整数  $n$ ，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于  $n$ 。你需要让组成和的完全平方数的个数最少。

示例:

输入:  $n = 12$   
输出: 3  
解释:  $12 = 4 + 4 + 4$ .

### 思路

这一题其实最容易想到的思路是动态规划，我们放到后面专门来拆解。实际上用队列进行图的建模，也是可以顺利地用广度优先遍历的方式解决的。



看到这个图，你可能会有点懵，我稍微解释一下你就明白了。

在这个无权图中，每一个点指向的都是它可能经过的上一个节点。举例来说，对 5 而言，可能是 4 加上了 1 的平方 转换而来，也可能是 1 加上了 2 的平方 转换而来，因此跟 1 和 2 都有联系，依次类推。

那么我们现在要做了就是寻找到从  $n$  转换到 0 最短的连线数。

举个例子， $n = 8$  时，我们需要找到它的邻居节点 4 和 7，此时到达 4 和到达 7 的步数都为 1，然后分别从 4 和 7 出发，4 找到邻居节点 3 和 0，达到 3 和 0 的步数都为 2，考虑到此时已经到达 0，遍历终止，返回到达 0 的步数 2 即可。

Talk is cheap, show me your code. 我们接下来一步步实现这个寻找的过程。

## 实现

接下来我们来实现第一版的代码。

```
/**
 * @param {number} n
 * @return {number}
 */
var numSquares = function(n) {
  let queue = [];
  queue.push([n, 0]);
  while(queue.length) {
    let [num, step] = queue.shift();
    for(let i = 1; ; i++) {
      let nextNum = num - i * i;
      if(nextNum < 0) break;
      // 还差最后一步就到了，直接返回 step + 1
      if(nextNum == 0) return step + 1;
      queue.push([nextNum, step + 1]);
    }
  }
  // 最后是不需要返回另外的值的，因为 1 也是完全平方数，所有的数都能用 1 来组合
};
```

这个解法从功能上来讲是没有问题的，但是其中隐藏了巨大的性能问题

那为什么会出现这样的问题？

出就在这样一行代码：

```
queue.push([nextNum, step + 1]);
```

只要是大于 0 的数，统统塞进队列。要知道  $2 - 1 = 1$ ， $5 - 4 = 1$ ， $9 - 8 = 1$  .....这样会重复非常多的 1，依次类推，也会重复非常多的 2, 3 等等等等。

这样大量的重复数字不仅仅消耗了更多的循环次数，同时也造成更加巨大的内存空间压力。

因此，我们需要对已经推入队列的数字进行标记，避免重复推入。改善代码如下：

```
var numSquares = function(n) {
  let map = new Map();
  let queue = [];
  queue.push([n, 0]);
  map.set(n, true);
  while(queue.length) {
    let [num, step] = queue.shift();
    for(let i = 1; ; i++) {
      let nextNum = num - i * i;
      if(nextNum < 0) break;
      if(nextNum == 0) return step + 1;
    }
  }
};
```

```
// nextNum 未被访问过
if(!map.get(nextNum)){
    queue.push([nextNum, step + 1]);
    // 标记已经访问过
    map.set(nextNum, true);
}
}
};
```

## 7. 单词接龙

给定两个单词 (beginWord 和 endWord) 和一个字典，找到从 beginWord 到 endWord 的最短转换序列的长度。转换需遵循如下规则：

- 每次转换只能改变一个字母。
- 转换过程中的中间单词必须是字典中的单词。

说明:

- 1) 如果不存在这样的转换序列，返回 0。
- 2) 所有单词具有相同的长度。
- 3) 所有单词只由小写字母组成。
- 4) 字典中不存在重复的单词。
- 5) 你可以假设 beginWord 和 endWord 是非空的，且二者不相同。

示例:

输入：  
beginword = "hit",  
endword = "cog",  
wordList = ["hot","dot","dog","lot","log","cog"]

输出：5

解释：一个最短转换序列是 "hit" -> "hot" -> "dot" -> "dog" -> "cog",  
返回它的长度 5。

### 思路

这一题是一个更加典型的用图建模的问题。如果每一个单词都是一个节点，那么只要和这个单词仅有一个字母不同，那么就是它的相邻节点。

这里我们可以通过 BFS 的方式来进行遍历。实现如下:

### 代码实现

```
/**
 * @param {string} beginword
 * @param {string} endword
```

```
* @param {string[]} wordList
* @return {number}
*/
var ladderLength = function(beginWord, endWord, wordList) {
    // 两个单词在图中是否相邻
    const issimilar = (a, b) => {
        let diff = 0
        for(let i = 0; i < a.length; i++) {
            if(a.charAt(i) !== b.charAt(i)) diff++;
            if(diff > 1) return false;
        }
        return true;
    }
    let queue = [beginWord];
    let index = wordList.indexOf(beginWord);
    if(index !== -1) wordList.splice(index, 1);
    let res = 2;
    while(queue.length) {
        let size = queue.length;
        while(size --) {
            let front = queue.shift();
            for(let i = 0; i < wordList.length; i++) {
                if(!issimilar(front, wordList[i]))continue;
                // 找到了
                if(wordList[i] === endWord) {
                    return res;
                }
                else {
                    queue.push(wordList[i]);
                }
                // wordList[i]已经成功推入，现在不需要了，删除即可
                // 这一步性能优化，相当关键，不然100%超时
                wordList.splice(i, 1);
                i --;
            }
        }
        // 步数 +1
        res += 1;
    }
    return 0;
};
```

## 8. 优先队列

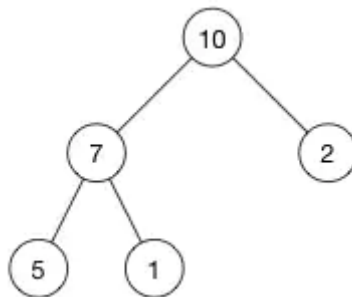
所谓优先队列，就是一种特殊的**队列**，其底层使用**堆**的结构，使得每次添加或者删除，让队首元素始终是优先级最高的。关于优先级通过什么字段、按照什么样的比较方式来设定，可以由我们自己来决定。

要实现优先队列，首先来实现一个堆的结构。

## 9. 关于堆的说明

可能你以前没有接触过堆这种数据结构，但是其实是很简单的一种结构，其本质就是一棵二叉树。但是这棵二叉树比较特殊，除了用数组来依次存储各个节点(节点对应的数组下标和层序遍历的序号一致)之外，它需要保证任何一个父节点的优先级大于子节点，这也是它最关键的性质，因为保证了根元素一定是优先级最高的。

举一个例子：



现在这个堆的数组就是: [10, 7, 2, 5, 1]

因此也会产生两个非常关键的操作——siftUp 和 siftDown。

对于siftUp操作，我们试想一下现在有一个正常的堆，满足任何父元素优先级大于子元素，这时候向这个堆的数组末尾又添加了一个元素，那现在可能就不符合堆的结构特点了。那么现在我将新增的节点和其父节点进行比较，如果父节点优先级小于它，则两者交换，不断向上比较直到根节点为止，这样就保证了堆的正确结构。而这样的操作就是siftUp。

siftDown是与其相反方向的操作，从上到下比较，原理相同，也是为了保证堆的正确结构。

## 10. 实现一个最大堆

最大堆，即堆顶元素为优先级最高的元素。

```
// 以最大堆为例来实现一波
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {number[]}
 */
class MaxHeap {
  constructor(arr = [], compare = null) {
    this.data = arr;
    this.size = arr.length;
    this.compare = compare;
  }
  getSize() {
    return this.size;
  }
  isEmpty() {
    return this.size === 0;
  }
  // 增加元素
  add(value) {
    this.data.push(value);
    this.size++;
    // 增加的时候把添加的元素进行 siftup
  }
}
```

```

        this._siftUp(this.getSize() - 1);
    }
    // 找到优先级最高的元素
    findMax() {
        if (this.getSize() === 0)
            return;
        return this.data[0];
    }
    // 让优先级最高的元素(即队首元素)出队
    extractMax() {
        // 1.保存队首元素
        let ret = this.findMax();
        // 2.让队首和队尾元素交换位置
        this._swap(0, this.getSize() - 1);
        // 3. 把队尾踢出去, size--
        this.data.pop();
        this.size--;
        // 4. 新的队首 siftDown
        this._siftDown(0);
        return ret;
    }

    toString() {
        console.log(this.data);
    }
    _swap(i, j) {
        [this.data[i], this.data[j]] = [this.data[j], this.data[i]];
    }
    _parent(index) {
        return Math.floor((index - 1) / 2);
    }
    _leftChild(index) {
        return 2 * index + 1;
    }
    _rightChild(index) {
        return 2 * index + 2;
    }
    _siftUp(k) {
        // 上浮操作, 只要子元素优先级比父节点大, 父子交换位置, 一直向上直到根节点
        while (k > 0 && this.compare(this.data[k], this.data[this._parent(k)])) {
            this._swap(k, this._parent(k));
            k = this._parent(k);
        }
    }
    _siftDown(k) {
        // 存在左孩子的时候
        while (this._leftChild(k) < this.size) {
            let j = this._leftChild(k);
            // 存在右孩子而且右孩子比左孩子大
            if (this._rightChild(k) < this.size &&
                this.compare(this.data[this._rightChild(k)], this.data[j])) {
                j++;
            }
            if (this.compare(this.data[k], this.data[j]))
                return;
            // 父节点比子节点小, 交换位置
            this._swap(k, j);
            // 继续下沉

```

```
        k = j;
    }
}
```

## 11. 实现优先队列

有了最大堆作铺垫，实现优先队列就易如反掌，废话不多说，直接放上代码。

```
class PriorityQueue {
    // max 为优先队列的容量
    constructor(max, compare) {
        this.max = max;
        this.compare = compare;
        this.maxHeap = new MaxHeap([], compare);
    }

    getSize() {
        return this.maxHeap.getSize();
    }

    isEmpty() {
        return this.maxHeap.isEmpty();
    }

    getFront() {
        return this.maxHeap.findMax();
    }

    enqueue(e) {
        // 比当前最高的优先级的还要高，直接不处理
        if (this.getSize() === this.max) {
            if (this.compare(e, this.getFront())) return;
            this.dequeue();
        }
        return this.maxHeap.add(e);
    }

    dequeue() {
        if (this.getSize() === 0) return null;
        return this.maxHeap.extractMax();
    }
}
```

可能会有人问: 你怎么保证这个优先队列是正确的呢?

我们不妨来做一些测试:

```
let pq = new PriorityQueue(3);
pq.enqueue(1);
pq.enqueue(333);
console.log(pq.dequeue());
console.log(pq.dequeue());
pq.enqueue(3);
pq.enqueue(6);
pq.enqueue(62);
console.log(pq.dequeue());
console.log(pq.dequeue());
console.log(pq.dequeue());
```

结果如下:

```
333
1
62
6
3
```

可见，这个优先队列的功能初步满足了我们的预期。

## 12. 前 K 个高频元素

给定一个非空的整数数组，返回其中出现频率前 k 高的元素。

示例:

```
输入: nums = [1,1,1,2,2,3], k = 2
输出: [1,2]
```

说明:

- 可以假设给定的 k 总是合理的，且  $1 \leq k \leq$  数组中不相同的元素的个数。
- 算法的时间复杂度必须优于  $O(n \log n)$ ，n 是数组的大小。

### 思路

首先要做的肯定是统计频率，那之后如何来选取频率前 K 个元素同时又保证时间复杂度小于  $O(n \log n)$  呢？

当然，这是一道典型的考察优先队列的题，利用容量为 K 的优先队列每次踢出不符合条件的值，那么最后剩下的即为所求。整个时间复杂度成为  $O(n \log K)$ ，明显是小于  $O(n \log n)$  的。

既然是优先队列，就涉及到如何来定义优先级的的问题。

倘若我们以高频率为高优先级，那么队首始终是高频率的元素，因此每次出队是踢出出现频率最高的元素，假设优先队列容量为 K，那照这么做，剩下的是频率最低的 K 个元素，显然不符合题意。

因此，我们需要的是每次出队时踢出**频率最低的元素**，这样最后剩下下来的就是频率最高 K 个元素。

是不是我们为了踢出**频率最低的元素**，还要重新写一个小顶堆的实现呢？

完全不需要！就像我刚才所说的，合理地定义这个优先级的比较逻辑即可。接下来我们来具体实现一下。



```
var topKFrequent = function(nums, k) {
    let map = {};
    let pq = new PriorityQueue(k, (a, b) => map[a] - map[b] < 0);
    for(let i = 0; i < nums.length; i++) {
        if(!map[nums[i]]) map[nums[i]] = 1;
        else map[nums[i]] = map[nums[i]] + 1;
    }
    let arr = Array.from(new Set(nums));
    for(let i = 0; i < arr.length; i++) {
        pq.enqueue(arr[i]);
    }
    return pq.maxHeap.data;
};
```

## 13. 合并 K 个排序链表

合并 k 个排序链表，返回合并后的排序链表。请分析和描述算法的复杂度。

示例:

```
输入:
[
  1->4->5,
  1->3->4,
  2->6
]
输出: 1->1->2->3->4->4->5->6
```

这一题我们之前在链表实现过，殊不知，它也可以利用优先队列完美解决。

```
/**
 * @param {ListNode[]} lists
 * @return {ListNode}
 */
var mergeKLists = function(lists) {
    let dummyHead = p = new ListNode();
    // 定义优先级的函数，重要！
    let pq = new PriorityQueue(lists.length, (a, b) => a.val <= b.val);
    // 将头结点推入优先队列
    for(let i = 0; i < lists.length; i++)
        if(lists[i]) pq.enqueue(lists[i]);
    // 取出值最小的节点，如果 next 不为空，继续推入队列
    while(pq.getSize()) {
        let min = pq.dequeue();
        p.next = min;
        p = p.next;
        if(min.next) pq.enqueue(min.next);
    }
    return dummyHead.next;
};
```

## 14. 什么是双端队列？

双端队列是一种特殊的队列，首尾都可以添加或者删除元素，是一种加强版的队列。

JS 中的数组就是一种典型的双端队列。push、pop 方法分别从尾部添加和删除元素，unshift、shift 方法分别从首部添加和删除元素。

## 15. 滑动窗口最大值

给定一个数组 nums，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 k 个数字。滑动窗口每次只向右移动一位。

返回滑动窗口中的最大值。

示例：

输入：nums = [1,3,-1,-3,5,3,6,7]，和 k = 3  
输出：[3,3,5,5,6,7]  
解释：

| 滑动窗口的位置             | 最大值   |
|---------------------|-------|
| -----               | ----- |
| [1 3 -1] -3 5 3 6 7 | 3     |
| 1 [3 -1 -3] 5 3 6 7 | 3     |
| 1 3 [-1 -3 5] 3 6 7 | 5     |
| 1 3 -1 [-3 5 3] 6 7 | 5     |
| 1 3 -1 -3 [5 3 6] 7 | 6     |
| 1 3 -1 -3 5 [3 6 7] | 7     |

要求：时间复杂度应为线性。

### 思路

这是典型地使用双端队列求解的问题。

建立一个双端队列 window，每次 push 进来一个新的值，就将 window 中目前前面所有比它小的值都删除。利用双端队列的特性，可以从后往前遍历，遇到小的就删除之，否则停止。

这样可以保证队首始终是最大值，因此寻找最大值的时间复杂度可以降到  $O(1)$ 。由于 window 中会有越来越多的值被淘汰，因此整体的时间复杂度是线性的。

### 代码实现

代码非常的简洁，但是如果要写出 bug free 的代码还是有相当的难度的，希望你能自己独立实现一遍。

```
var maxSlidingWindow = function(nums, k) {  
  // 异常处理  
  if(nums.length === 0 || !k) return [];  
  let window = [], res = [];  
  for(let i = 0; i < nums.length; i++) {  
    // 先把滑动窗口之外的踢出  
    if(window[0] !== undefined && window[0] <= i - k) window.shift();  
    while(window.length > 0 && nums[i] >= nums[window[0]]) window.pop();  
    window.push(i);  
    if(i >= k - 1) res.push(nums[window[0]]);  
  }  
  return res;  
}
```

```

// 保证队首是最大的
while(nums>window.length - 1] <= nums[i]) window.pop();
window.push(i);
if(i >= k - 1) res.push(nums>window[0]))
}
return res;
};

```

## 16. 栈实现队列

使用栈实现队列的下列操作：

push(x) -- 将一个元素放入队列的尾部。 pop() -- 从队列首部移除元素。 peek() -- 返回队列首部的元素。 empty() -- 返回队列是否为空。

示例：

```

let queue = new MyQueue();

queue.push(1);
queue.push(2);
queue.peek(); // 返回 1
queue.pop(); // 返回 1
queue.empty(); // 返回 false

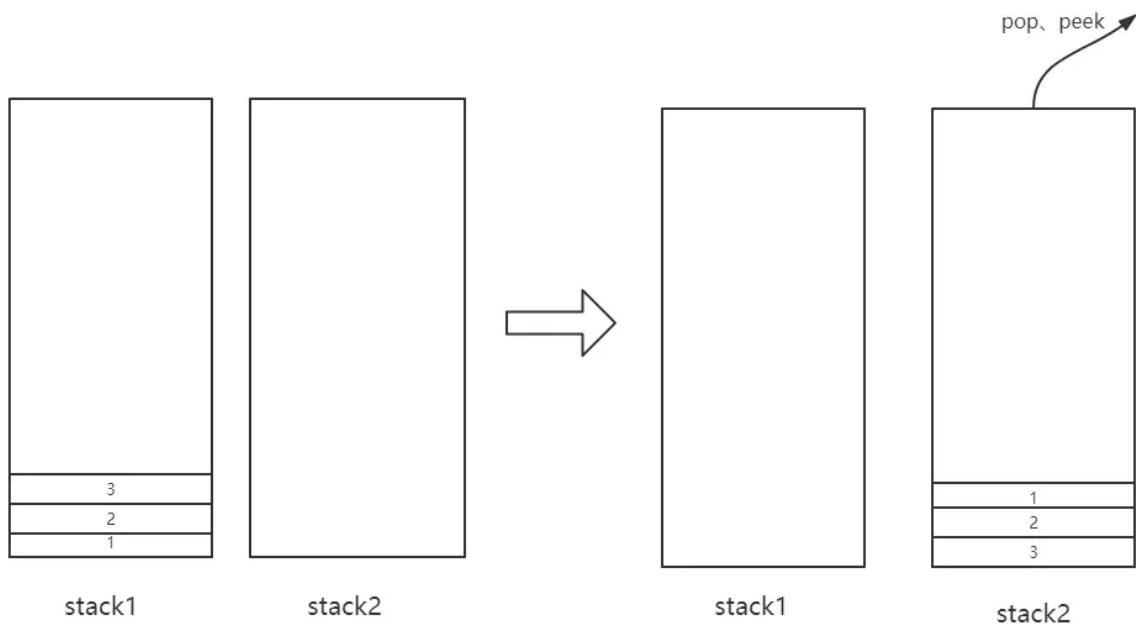
```

### 思路

既然栈是**先进后出**，要想得到**先进先出**的效果，我们不妨用两个栈。

当进行 push 操作时，push 到 stack1，而进行 pop 和 peek 的操作时，我们通过 stack2。

当然这其中有一个特殊情况，就是 stack2 是空，如何进行 pop 和 peek？很简单，把 stack1 中的元素依次 pop 并推入 stack2 中，然后正常地操作 stack2 即可，如下图所示：



这就就能保证先入先出的效果了。

## 代码实现

```
var MyQueue = function() {  
  this.stack1 = [];  
  this.stack2 = [];  
};  
  
MyQueue.prototype.push = function(x) {  
  this.stack1.push(x);  
};  
// 将 stack1 的元素转移到 stack2  
MyQueue.prototype.transform = function() {  
  while(this.stack1.length) {  
    this.stack2.push(this.stack1.pop());  
  }  
}  
  
MyQueue.prototype.pop = function() {  
  if(!this.stack2.length) this.transform();  
  return this.stack2.pop();  
};  
  
MyQueue.prototype.peek = function() {  
  if(!this.stack2.length) this.transform();  
  return this.stack2[this.stack2.length - 1];  
};  
  
MyQueue.prototype.empty = function() {  
  return !this.stack1.length && !this.stack2.length;  
};
```

## 17. 队列实现栈

和上一题的效果刚好相反，用队列先进先出的方式来实现先进后出的效果。

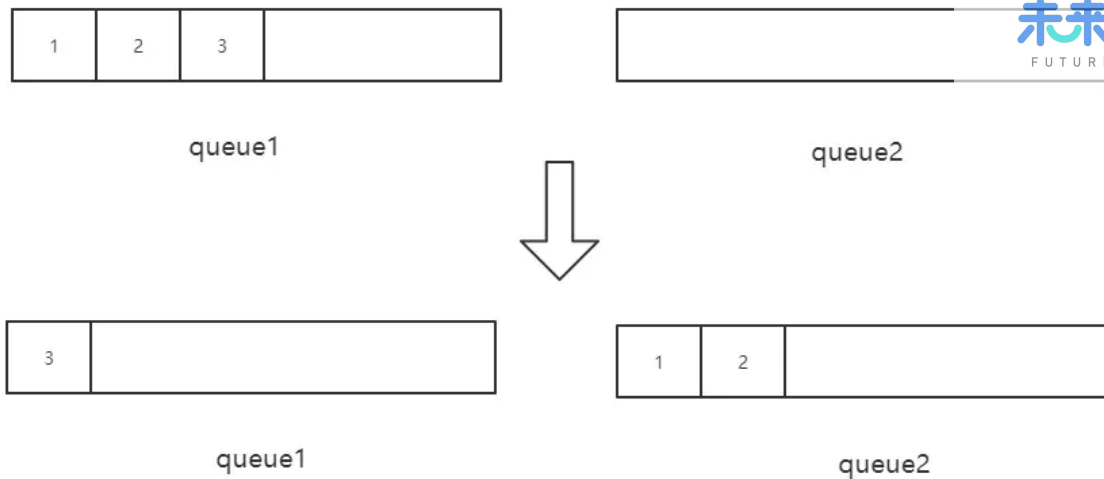
### 思路



queue1

以上面的队列为例，push 操作好说，直接从在队列末尾推入。但 pop 和 peek 呢？

回到我们的目标，我们的目标是拿到队尾的值，也就是 3。这就好办了，我们让前面的元素统统出队，只留队尾元素即可，剩下的元素让另外一个队列保存。



## 代码实现

实现过程中，值得注意的一点是，`queue1` 始终保存前面的元素，`queue2` 始终保存队尾元素（即栈顶元素）。

但是当 push 的时候有一个陷阱，就是当 `queue2` 已经有元素的时候，不能将新值 push 到 `queue1`，因为此时的**栈顶元素**应该更新。此时对于新的值来说，应先 push 到 `queue2`，然后将**旧的栈顶**从 `queue2` 出队，推入 `queue1`，这样就实现了**更新栈顶**的操作。

```
var MyStack = function() {  
  this.queue1 = [];  
  this.queue2 = [];  
};  
MyStack.prototype.push = function(x) {  
  if(!this.queue2.length) this.queue1.push(x);  
  else {  
    // queue2 已经有值  
    this.queue2.push(x);  
    // 旧的栈顶移到 queue1 中  
    this.queue1.push(this.queue2.shift());  
  }  
};  
MyStack.prototype.transform = function() {  
  while(this.queue1.length !== 1) {  
    this.queue2.push(this.queue1.shift())  
  }  
  // queue2 保存了前面的元素  
  // 让 queue1 和 queue2 交换  
  // 现在queue1 包含前面的元素，queue2 里面就只包含队尾的元素  
  let tmp = this.queue1;  
  this.queue1 = this.queue2;  
  this.queue2 = tmp;  
}  
MyStack.prototype.pop = function() {  
  if(!this.queue2.length) this.transform();  
  return this.queue2.shift();  
};  
MyStack.prototype.top = function() {  
  if(!this.queue2.length) this.transform();  
  return this.queue2[0];  
};
```

```
MyStack.prototype.empty = function() {  
    return !this.queue1.length && !this.queue2.length;  
};
```

## 二叉树

### 1. 前序遍历

示例:

示例:

输入: [1,null,2,3]

```
  1  
  \  
   2  
  \  
   3
```

输出: [1,2,3]

### 递归方式

```
/**  
 * @param {TreeNode} root  
 * @return {number[]}  
 */  
var preorderTraversal = function(root) {  
    let arr = [];  
    let traverse = (root) => {  
        if(root == null) return;  
        arr.push(root.val);  
        traverse(root.left);  
        traverse(root.right);  
    }  
    traverse(root);  
    return arr;  
};
```

### 非递归方式

```
var preorderTraversal = function(root) {
  if(root == null) return [];
  let stack = [], res = [];
  stack.push(root);
  while(stack.length) {
    let node = stack.pop();
    res.push(node.val);
    // 左孩子后进先出，进行先左后右的深度优先遍历
    if(node.right) stack.push(node.right);
    if(node.left) stack.push(node.left);
  }
  return res;
};
```

## 2. 中序遍历

给定一个二叉树，返回它的中序 遍历。

示例:

输入: [1,null,2,3]

```
  1
   \
    2
   /
  3
```

输出: [1,3,2]

递归方式:

```
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var inorderTraversal = function(root) {
  let arr = [];
  let traverse = (root) => {
    if(root == null) return;
    traverse(root.left);
    arr.push(root.val);
    traverse(root.right);
  }
  traverse(root);
  return arr;
};
```

非递归方式

```
var inorderTraversal = function(root) {
  if(root == null) return [];
```

```
let stack = [], res = [];
let p = root;
while(stack.length || p) {
    while(p) {
        stack.push(p);
        p = p.left;
    }
    let node = stack.pop();
    res.push(node.val);
    p = node.right;
}
return res;
};
```

### 3. 后序遍历

给定一个二叉树，返回它的 后序 遍历。

示例:

输入: [1,null,2,3]

```

  1
   \
    2
   /
  3
```

输出: [3,2,1]

#### 递归方式

```
/**
 * @param {TreeNode} root
 * @return {number[]}
 */
var postorderTraversal = function(root) {
    let arr = [];
    let traverse = (root) => {
        if(root == null) return;
        traverse(root.left);
        traverse(root.right);
        arr.push(root.val);
    }
    traverse(root);
    return arr;
};
```

#### 非递归方式

```
var postorderTraversal = function(root) {
```



```

if(root == null) return [];
let stack = [], res = [];
let visited = new Set();
let p = root;
while(stack.length || p) {
    while(p) {
        stack.push(p);
        p = p.left;
    }
    let node = stack[stack.length - 1];
    // 如果右孩子存在，而且右孩子未被访问
    if(node.right && !visited.has(node.right)) {
        p = node.right;
        visited.add(node.right);
    } else {
        res.push(node.val);
        stack.pop();
    }
}
return res;
};

```

## 4. 最大深度

给定一个二叉树，找出其最大深度。

二叉树的深度为根节点到最远叶子节点的最长路径上的节点数。

**说明:** 叶子节点是指没有子节点的节点。

**示例:** 给定二叉树 [3,9,20,null,null,15,7]:

```

    3
   / \
  9  20
 /  \
15   7

```

### 递归实现

实现非常简单，直接贴出代码:

```

/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxDepth = function(root) {
    // 递归终止条件
    if(root == null) return 0;
    return Math.max(maxDepth(root.left) + 1, maxDepth(root.right) + 1);
};

```

采用层序遍历的方式，非常好理解。

```
var maxDepth = function(root) {  
    if(root == null) return 0;  
    let queue = [root];  
    let level = 0;  
    while(queue.length) {  
        let size = queue.length;  
        while(size --) {  
            let front = queue.shift();  
            if(front.left) queue.push(front.left);  
            if(front.right) queue.push(front.right);  
        }  
        // level ++ 后的值代表着现在已经处理完了几层节点  
        level ++;  
    }  
    return level;  
};
```

## 5. 最小深度

给定一个二叉树，找出其最小深度。

最小深度是从根节点到最近叶子节点的最短路径上的节点数量。

**说明:** 叶子节点是指没有子节点的节点。

**示例:**

给定二叉树 [3,9,20,null,null,15,7]:

```
    3  
   / \  
  9  20  
 /  \  
15  7
```

返回它的最小深度 2.

## 递归实现

在实现的过程中，如果按照最大深度的方式来做会出现一个陷阱，即:

```
/**  
 * @param {TreeNode} root  
 * @return {number}  
 */  
var minDepth = function(root) {  
    // 递归终止条件  
    if(root == null) return 0;  
    return Math.min(minDepth(root.left) + 1, minDepth(root.right)+1);  
};
```

当 root 节点有一个孩子为空的时候，此时返回的是 1，但这是不对的，最小高度指的是根节点到最近叶子节点的最小路径，而不是到一个空节点的路径。

因此我们需要做如下的调整：

```
var minDepth = function(root) {  
  if(root == null) return 0;  
  // 左右孩子都不为空才能像刚才那样调用  
  if(root.left && root.right)  
    return Math.min(minDepth(root.left), minDepth(root.right)) + 1;  
  // 右孩子为空了，直接忽略之  
  else if(root.left)  
    return minDepth(root.left) + 1;  
  // 左孩子为空，忽略  
  else if(root.right)  
    return minDepth(root.right) + 1;  
  // 两个孩子都为空，说明到达了叶子节点，返回 1  
  else return 1;  
};
```

这样程序便能正常工作了。

## 非递归实现

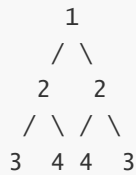
类似于最大高度问题，采用了层序遍历的方式，很容易理解。

```
var minDepth = function(root) {  
  if(root == null) return 0;  
  let queue = [root];  
  let level = 0;  
  while(queue.length) {  
    let size = queue.length;  
    while(size --) {  
      let front = queue.shift();  
      // 找到叶子节点  
      if(!front.left && !front.right) return level + 1;  
      if(front.left) queue.push(front.left);  
      if(front.right) queue.push(front.right);  
    }  
    // level ++ 后的值代表着现在已经处理完了几层节点  
    level ++;  
  }  
  return level;  
};
```

## 6. 对称二叉树

给定一个二叉树，检查它是否是镜像对称的。

例如，二叉树 [1,2,2,3,4,4,3] 是对称的。



但是下面这个 [1,2,2,null,3,null,3] 则不是镜像对称的:



## 递归实现

递归方式的代码是非常干练和优雅的，希望你先自己实现一遍，然后对比改进。

```

/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isSymmetric = function(root) {
  let help = (node1, node2) => {
    // 都为空
    if(!node1 && !node2) return true;
    // 一个为空一个不为空，或者两个节点值不相等
    if(!node1 || !node2 || node1.val !== node2.val) return false;
    return help(node1.left, node2.right) && help(node1.right, node2.left);
  }
  if(root == null) return true;
  return help(root.left, root.right);
};

```

## 非递归实现

用一个队列保存访问过的节点，每次取出两个节点，进行比较。

```

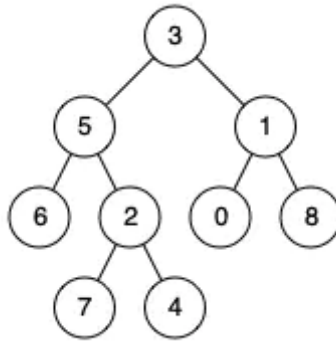
var isSymmetric = function(root) {
  if(root == null) return true;
  let queue = [root.left, root.right];
  let node1, node2;
  while(queue.length) {
    node1 = queue.shift();
    node2 = queue.shift();
    // 两节点均为空
    if(!node1 && !node2) continue;
    // 一个为空一个不为空，或者两个节点值不相等
    if(!node1 || !node2 || node1.val !== node2.val) return false;
    queue.push(node1.left);
    queue.push(node2.right);
    queue.push(node1.right);
  }
};

```

```
        queue.push(node2.left);
    }
    return true;
};
```

## 7. 二叉树的最近公共祖先

对于一个普通的二叉树: root = [3,5,1,6,2,0,8,null,null,7,4]



输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

输出: 3

解释: 节点 5 和节点 1 的最近公共祖先是节点 3。

### 思路分析

思路一: 首先遍历一遍二叉树, 记录下每个节点的父节点。然后对于题目给的 p 节点, 根据这个记录表不断的找 p 的上层节点, 直到根, 记录下 p 的**上层节点集合**。然后对于 q 节点, 根据记录不断向上找它的上层节点, 在寻找的过程中一旦发现**这个上层节点已经包含在刚刚的集合中**, 说明发现了最近公共祖先, 直接返回。

思路二: 深度优先遍历二叉树, 如果当前节点为 p 或者 q, 直接返回这个节点, 否则查看左右孩子, 左孩子中不包含 p 或者 q 则去找右孩子, 右孩子不包含 p 或者 q 就去找左孩子, 剩下的情况就是**左右孩子中都存在 p 或者 q**, 那么此时直接返回这个节点。

### 祖先节点集合法

```
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
    if(root == null || root == p || root == q) return root;
    let set = new Set();
    let map = new WeakMap();
    let queue = [];
    queue.push(root);
    // 层序遍历
```

```
while(queue.length) {
    let size = queue.length;
    while(size --) {
        let front = queue.shift();
        if(front.left) {
            queue.push(front.left);
            // 记录父亲节点
            map.set(front.left, front);
        }
        if(front.right) {
            queue.push(front.right);
            // 记录父亲节点
            map.set(front.right, front);
        }
    }
}
// 构造 p 的上层节点集合
while(p) {
    set.add(p);
    p = map.get(p);
}
while(q) {
    // 一旦发现公共节点重合，直接返回
    if(set.has(q)) return q;
    q = map.get(q);
}
};
```

可以看到整棵二叉树遍历了一遍，时间复杂度大致是  $O(n)$ ，但是由于哈希表的存在，空间复杂度比较高，接下来我们来用另一种遍历的方式，可以大大减少空间的开销。

### 深度优先遍历法

代码非常简洁、美观，不过更重要的是体会其中递归调用的过程，代码是自顶向下执行的，我建议大家用**自底向上**的方式来理解它，即从最左下的节点开始分析，相信你会很好的理解整个过程。

```
var lowestCommonAncestor = function(root, p, q) {
    if (root == null || root == p || root == q) return root;
    let left = lowestCommonAncestor(root.left, p, q);
    let right = lowestCommonAncestor(root.right, p, q);
    if(left == null) return right;
    else if(right == null) return left;
    return root;
};
```

## 8. 二叉搜索树的最近公共祖先

给定如下二叉搜索树: root = [6,2,8,0,4,7,9,null,null,3,5]

输入: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8  
 输出: 6  
 解释: 节点 2 和节点 8 的最近公共祖先是 6。

## 实现

二叉搜索树作为一种特殊的二叉树，当然是可以用上述的两种方式来实现的。

不过借助二叉搜索树有序的特性，我们也可以写出另外一个版本的深度优化遍历。

```
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @param {TreeNode} q
 * @return {TreeNode}
 */
var lowestCommonAncestor = function(root, p, q) {
    if(root == null || root == p || root == q) return root;
    // root.val 比 p 和 q 都大，找左孩子
    if(root.val > p.val && root.val > q.val)
        return lowestCommonAncestor(root.left, p, q);
    // root.val 比 p 和 q 都小，找右孩子
    if(root.val < p.val && root.val < q.val)
        return lowestCommonAncestor(root.right, p, q);
    else
        return root;
};
```

同时也可以采用非递归的方式：

```
var lowestCommonAncestor = function(root, p, q) {
    let node = root;
    while(node) {
        if(p.val > node.val && q.val > node.val)
            node = node.right;
        else if(p.val < node.val && q.val < node.val)
            node = node.left;
        else return node;
    }
};
```

是不是被二叉树精简而优雅的代码惊艳到了呢？希望你能好好体会其中遍历的过程，然后务必自己独立实现一遍，保证对这种数据结构足够熟悉，增强自己的编程内力。

## 9. 二叉树的直径

给定一棵二叉树，你需要计算它的直径长度。一棵二叉树的直径长度是任意两个结点路径长度中的最大值。这条路径可能穿过根结点。

示例：给定二叉树



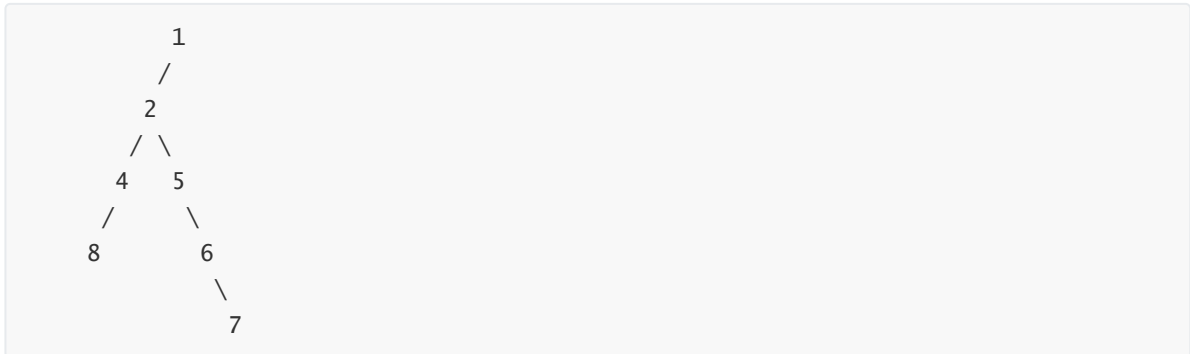
返回 3, 它的长度是路径 [4,2,1,3] 或者 [5,2,1,3]。

注意：两结点之间的路径长度是以它们之间边的数目表示。

## 思路

所谓的求直径, 本质上是求树中节点左右子树高度和的最大值。

注意, 这里我说的是树中节点, 并非根节点。因为会有这样一种情况:



那这个时候, 直径最大的路径是: 8 -> 4 -> 2 -> 5 -> 6 -> 7。交界的元素并不是根节点。这是这个问题特别需要注意的地方, 不然无解。

## 初步求解

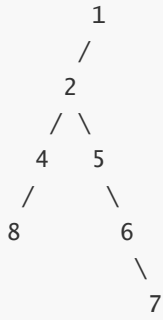
目标已经确定了, 求树中节点左右子树高度和的最大值。开干!

```
/**
 * @param {TreeNode} root
 * @return {number}
 */
var diameterOfBinaryTree = function(root) {
  // 求最大深度
  let maxDepth = (node) => {
    if(node == null) return 0;
    return Math.max(maxDepth(node.left) + 1, maxDepth(node.right) + 1);
  }
  let help = (node) => {
    if(node == null) return 0;
    let rootSum = maxDepth(node.left) + maxDepth(node.right);
    let childSum = Math.max(help(node.left), help(node.right));
    return Math.max(rootSum, childSum);
  }
  if(root == null) return 0;
  return help(root);
};
```

这样一段代码放到 LeetCode 是可以通过, 但时间上却不让人很满意, 为什么呢?

因为在反复调用 maxDepth 的过程, 对树中的一些节点增加了很多不必要的访问。比如:





我们看什么时候访问节点 8，maxDepth(节点 2)的时候访问，maxDepth(节点 4)的时候又会访问，如果节点层级更高，重复访问的次数更加频繁，剩下的节点 6、节点 7 都是同理。每一个节点访问的次数大概是  $O(\log K)$  (设当前节点在第  $K$  层)。那能不能把这个频率降到  $O(1)$  呢？

答案是肯定的，接下来我们来优化这个算法。

### 优化解法

```
var diameterOfBinaryTree = function(root) {  
  let help = (node) => {  
    if(node == null) return 0;  
    let left = node.left ? help(node.left) + 1: 0;  
    let right = node.right ? help(node.right) + 1: 0;  
    let cur = left + right;  
    if(cur > max) max = cur;  
    // 这个返回的操作相当关键  
    return Math.max(left, right);  
  }  
  let max = 0;  
  if(root == null) return 0;  
  help(root);  
  return max;  
};
```

在这个过程中设置了一个 `max` 全局变量，深度优先遍历这棵树，每遍历完一个节点就更新 `max`，并通过返回值的方式 **自底向上** 把当前节点左右子树的最大高度传给父函数使用，使得每个节点只需访问一次即可。

现在提交我们优化后的代码，时间消耗明显降低。

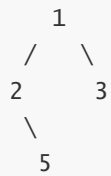
## 10. 二叉树的所有路径

给定一个二叉树，返回所有从根节点到叶子节点的路径。

**说明:** 叶子节点是指没有子节点的节点。

**示例:**

输入：



输出：["1->2->5", "1->3"]

**解释：**所有根节点到叶子节点的路径为：1->2->5, 1->3

## 递归解法

利用 DFS(深度优先遍历) 的方式进行遍历。

```

/**
 * @param {TreeNode} root
 * @return {string[]}
 */
var binaryTreePaths = function(root) {
    let path = [];
    let res = [];
    let dfs = (node) => {
        if(node == null) return;
        path.push(node);
        dfs(node.left);
        dfs(node.right);
        if(!node.left && !node.right)
            res.push(path.map(item => item.val).join('->'));
        // 注意每访问完一个节点记得把它从path中删除，达到回溯效果
        path.pop();
    }
    dfs(root);
    return res;
};
  
```

## 非递归解法

接下来我们通过 [非递归的后序遍历](#) 的方式来实现一下, 顺便复习一下后序遍历的实现。

```

var binaryTreePaths = function(root) {
    if(root == null) return [];
    let stack = [];
    let p = root;
    let set = new Set();
    res = [];
    while(stack.length || p) {
        while(p) {
            stack.push(p);
            p = p.left;
        }
        let node = stack[stack.length - 1];
  
```

```
// 叶子节点
if(!node.right && !node.left) {
    res.push(stack.map(item => item.val).join('->'));
}
// 右孩子存在, 且右孩子未被访问
if(node.right && !set.has(node.right)) {
    p = node.right;
    set.add(node.right);
} else {
    stack.pop();
}
}
return res;
};
```

## 11. 二叉树的最大路径和

给定一个非空二叉树，返回其最大路径和。

本题中，路径被定义为一条从树中任意节点出发，达到任意节点的序列。该路径至少包含一个节点，且不一定经过根节点。

示例:

输入: [-10,9,20,null,null,15,7]

```

-10
 / \
9  20
 / \
15 7
```

输出: 42

### 递归解

```
/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxPathSum = function(root) {
    let help = (node) => {
        if(node == null) return 0;
        let left = Math.max(help(node.left), 0);
        let right = Math.max(help(node.right), 0);
        let cur = left + node.val + right;
        // 如果发现某一个节点上的路径值比max还大, 则更新max
        if(cur > max) max = cur;
        // left 和 right 永远是"一根筋", 中间不会有转折
        return Math.max(left, right) + node.val;
    }
    let max = Number.MIN_SAFE_INTEGER;
    help(root);
}
```

```
    return max;
};
```

## 12. 验证二叉搜索树

给定一个二叉树，判断其是否是一个有效的二叉搜索树。

假设一个二叉搜索树具有如下特征：

节点的左子树只包含小于当前节点的数。 节点的右子树只包含大于当前节点的数。 所有左子树和右子树自身必须也是二叉搜索树。

**示例 1:**

```
输入：
    2
   / \
  1   3
输出：true
```

### 方法一: 中序遍历

通过中序遍历，保存前一个节点的值，扫描到当前节点时，和前一个节点的值比较，如果大于前一个节点，则满足条件，否则不是二叉搜索树。

```
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isValidBST = function(root) {
    let prev = null;
    const help = (node) => {
        if(node == null) return true;
        if(!help(node.left)) return false;
        if(prev !== null && prev >= node.val) return false;
        // 保存当前节点，为下一个节点的遍历做准备
        prev = node.val;
        return help(node.right);
    }
    return help(root);
};
```

### 方法二: 限定上下界进行DFS

二叉搜索树每一个节点的值，都有一个**上界和下界**，深度优先遍历的过程中，如果访问左孩子，则通过当前节点的值来**更新左孩子节点的上界**，同时访问右孩子，则**更新右孩子的下界**，只要出现节点值越界的情况，则不满足二叉搜索树的条件。

```
parent
 /   \
left  right
```

假设这是一棵巨大的二叉树的一个部分(parent、left、right都是实实在在的节点), 那么全部的节点排序一定是这样:

...left, parent, right...

可以看到左孩子的 最严格的上界 是该节点, 同时, 右孩子的 最严格的下界 也是该节点。我们按照这样的规则来进行更新上下界。

递归实现:

```
var isValidBST = function(root) {
  const help = (node, max, min) => {
    if(node == null) return true;
    if(node.val >= max || node.val <= min) return false;
    // 左孩子更新上界, 右孩子更新下界, 相当于边界要求越来越苛刻
    return help(node.left, node.val, min)
      && help(node.right, max, node.val);
  }
  return help(root, Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER);
};
```

非递归实现:

```
var isValidBST = function(root) {
  if(root == null) return true;
  let stack = [root];
  let min = Number.MIN_SAFE_INTEGER;
  let max = Number.MAX_SAFE_INTEGER;
  root.max = max; root.min = min;
  while(stack.length) {
    let node = stack.pop();
    if(node.val <= node.min || node.val >= node.max)
      return false;
    if(node.left) {
      stack.push(node.left);
      // 更新上下界
      node.left.max = node.val;
      node.left.min = node.min;
    }
    if(node.right) {
      stack.push(node.right);
      // 更新上下界
      node.right.max = node.max;
      node.right.min = node.val;
    }
  }
  return true;
};
```

### 13. 将有序数组转换为二叉搜索树

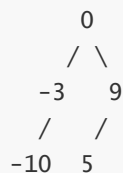
将一个按照升序排列的有序数组，转换为一棵高度平衡二叉搜索树。

本题中，一个高度平衡二叉树是指一个二叉树每个节点的左右两个子树的高度差的绝对值不超过 1。

示例:

给定有序数组: `[-10,-3,0,5,9]`,

一个可能的答案是: `[0,-3,9,-10,null,5]`, 它可以表示下面这个高度平衡二叉搜索树:



#### 递归实现

```
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
  let help = (start, end) => {
    if(start > end) return null;
    if(start === end) return new TreeNode(nums[start]);
    let mid = Math.floor((start + end) / 2);
    // 找出中点建立节点
    let node = new TreeNode(nums[mid]);
    node.left = help(start, mid - 1);
    node.right = help(mid + 1, end);
    return node;
  }
  return help(0, nums.length - 1);
};
```

递归程序比较好理解，不断地调用 help 完成整棵树的构建。那如何用非递归来解决呢？我觉得这是一个非常值得大家思考的问题。希望你能动手试一试，如果实在想不出来，可以参考下面我写的非递归版本。

其实思路跟递归的版本是一样的，只不过实现起来是用栈来实现 DFS 的效果。

```
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
  if(nums.length === 0) return null;
  let mid = Math.floor(nums.length / 2);
  let root = new TreeNode(nums[mid]);
  // 说明: 1. index 指的是当前元素在数组中的索引
  //       2. 每一个节点的值都是区间中点, 那么 start 属性就是这个区间的起点, end 为其终点
  root.index = mid; root.start = 0; root.end = nums.length - 1;
```

```

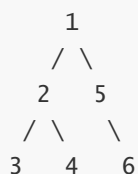
let stack = [root];
while(stack.length) {
    let node = stack.pop();
    // node 出来了，它本身包含了一个区间，[start, ..., index, ... end]
    // 下面判断[node.start, node.index - 1]之间是否还有开发的余地
    if(node.index - 1 >= node.start) {
        let leftMid = Math.floor((node.start + node.index)/2);
        let leftNode = new TreeNode(nums[leftMid]);
        node.left = leftNode;
        // 初始化新节点的区间起点、终点和索引
        leftNode.start = node.start;
        leftNode.end = node.index - 1;
        leftNode.index = leftMid;
        stack.push(leftNode);
    }
    // 中间夹着node.index，已经有元素了，这个位置不能再开发
    // 下面判断[node.index + 1, node.end]之间是否还有开发的余地
    if(node.end >= node.index + 1) {
        let rightMid = Math.floor((node.index + 1 + node.end)/2);
        let rightNode = new TreeNode(nums[rightMid]);
        node.right = rightNode;
        // 初始化新节点的区间起点、终点和索引
        rightNode.start = node.index + 1;
        rightNode.end = node.end;
        rightNode.index = rightMid;
        stack.push(rightNode);
    }
}
return root;
};

```

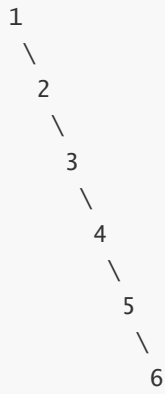
## 14. 二叉树展开为链表

给定一个二叉(搜索)树，原地将它展开为链表。

例如，给定二叉树

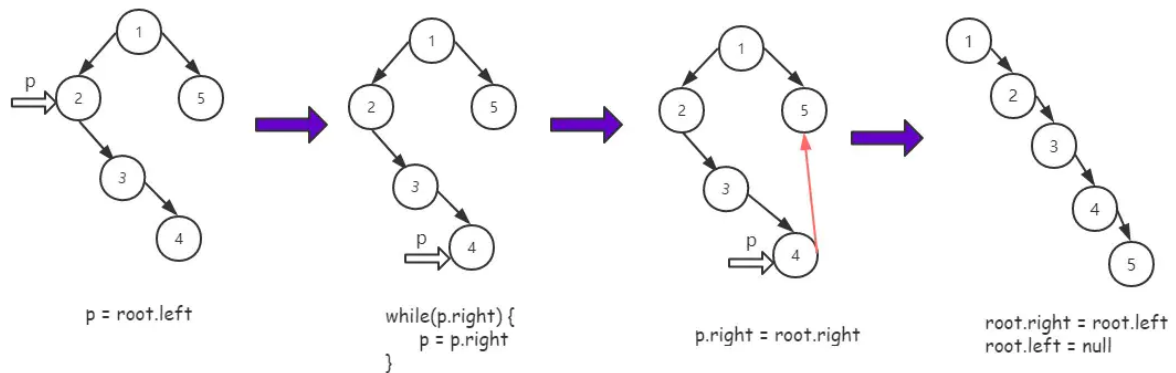


将其展开为：



## 递归方式

采用后序遍历，遍历完左右孩子我们要做些什么呢？用下面的图来演示一下：



```

/**
 * @param {TreeNode} root
 * @return {void} Do not return anything, modify root in-place instead.
 */
var flatten = function(root) {
  if(root == null) return;
  flatten(root.left);
  flatten(root.right);
  if(root.left) {
    let p = root.left;
    while(p.right) {
      p = p.right;
    }
    p.right = root.right;
    root.right = root.left;
    root.left = null;
  }
};

```

## 非递归方式

采用非递归的后序遍历方式，思路跟之前的完全一样。



```
var flatten = function(root) {
  if(root == null) return;
  let stack = [];
  let visited = new Set();
  let p = root;
  // 开始后序遍历
  while(stack.length || p) {
    while(p) {
      stack.push(p);
      p = p.left;
    }
    let node = stack[stack.length - 1];
    // 如果右孩子存在，而且右孩子未被访问
    if(node.right && !visited.has(node.right)) {
      p = node.right;
      visited.add(node.right);
    } else {
      // 以下为思路图中关键逻辑
      if(node.left) {
        let p = node.left;
        while(p.right) {
          p = p.right;
        }
        p.right = node.right;
        node.right = node.left;
        node.left = null;
      }
      stack.pop();
    }
  }
};
```

## 15. 不同的二叉搜索树II

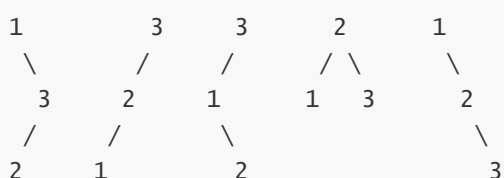
给定一个整数  $n$ ，生成所有由  $1 \dots n$  为节点所组成的二叉搜索树。

示例:

输入: 3  
输出:  
[  
 [1,null,3,2],  
 [3,2,null,1],  
 [3,1,null,null,2],  
 [2,1,3],  
 [1,null,2,null,3]  
]

解释:

以上的输出对应以下 5 种不同结构的二叉搜索树:



## 递归解法

### 递归创建子树

```
/**
 * @param {number} n
 * @return {TreeNode[]}
 */
var generateTrees = function(n) {
  let help = (start, end) => {
    if(start > end) return [null];
    if(start === end) return [new TreeNode(start)];
    let res = [];
    for(let i = start; i <= end; i++) {
      // 左孩子集
      let leftNodes = help(start, i - 1);
      // 右孩子集
      let rightNodes = help(i + 1, end);
      for(let j = 0; j < leftNodes.length; j++) {
        for(let k = 0; k < rightNodes.length; k++) {
          let root = new TreeNode(i);
          root.left = leftNodes[j];
          root.right = rightNodes[k];
          res.push(root);
        }
      }
    }
    return res;
  }
  if(n == 0) return [];
  return help(1, n);
};
```

## 非递归解法

```
var generateTrees = function(n) {
  let clone = (node, offset) => {
    if(node == null) return null;
    let newNode = new TreeNode(node.val + offset);
    newNode.left = clone(node.left, offset);
    newNode.right = clone(node.right, offset);
    return newNode;
  }
  if(n == 0) return [];
  let dp = [];
  dp[0] = [null];
  // i 是子问题中的节点个数，子问题：[1], [1,2], [1,2,3]...逐步递增，直到[1,2,3...,n]
  for(let i = 1; i <= n; i++) {
    dp[i] = [];
    for(let j = 1; j <= i; j++) {
      // 左子树集
      for(let leftNode of dp[j - 1]) {
        // 右子树集
```

```
        for(let rightNode of dp[i - j]) {  
            let node = new TreeNode(j);  
            // 左子树结构共享  
            node.left = leftNode;  
            // 右子树无法共享，但可以借用节点个数相同的树，每个节点增加一个偏移量  
            node.right = clone(rightNode, j);  
            dp[i].push(node);  
        }  
    }  
}  
return dp[n];  
};
```